🎯 Class 31 Goal

আজ SQL-এর পুরো course শুরু করছি না। আজ বুঝব:

1. SQL কী?
2. Database কী?
3. Table কী?
4. Row/Column কী?
5. Excel আর SQL-এর relationship
6. SELECT
7. WHERE
8. ORDER BY
9. LIMIT
10. Basic SQL thinking

1️⃣ SQL কী?

SQL = Structured Query Language

Database থেকে data:

- দেখতে
- filter করতে
- sort করতে
- calculate করতে
- update করতে
- insert করতে
- delete করতে

SQL ব্যবহার করা হয়।

উদাহরণ:

```
SELECT *
FROM employees;
```

মানে:

> employees table-এর সব data দেখাও।

2️⃣ Excel বনাম Database

Excel-এ:

ID	Name	Department	Salary
E001	Rahim	IT	45000
E002	Karim	HR	55000
E003	Hasan	IT	60000

Database-এও একই ধরনের structure থাকতে পারে:

```
employees
-----------------------------
id | name | department | salary
```

অর্থাৎ তুমি Excel-এ যে table concept শিখেছ, SQL database-এ সেই concept আরও structured এবং scalable ভাবে ব্যবহার করবে।

3️⃣ SQL Table

ধরি আমাদের table:

```
employees
```

Columns:

```
id
name
department
city
salary
```

Rows:

```
E001 | Rahim | IT | Dhaka | 45000
E002 | Karim | HR | Dhaka | 55000
E003 | Hasan | IT | Dhaka | 60000
E004 | Jamil | Finance | Sylhet | 70000
E005 | Nabil | Marketing | Dhaka | 40000
```

4️⃣ SELECT — Data দেখা

সব column:

```
SELECT *
FROM employees;
```

`*` মানে:

> সব columns

5️⃣ নির্দিষ্ট Column

শুধু Name এবং Salary:

```
SELECT name, salary
FROM employees;
```

Result:

name	salary
Rahim	45000
Karim	55000
Hasan	60000
Jamil	70000
Nabil	40000

6️⃣ SELECT-এর Mental Model

Excel-এ তুমি ভাবতে:

> কোন column দেখতে চাই?

SQL-এ:

```
SELECT column1, column2
```

মানে:

> এই columns আমাকে দেখাও।

7️⃣ WHERE — Filter

Excel-এ Filter ব্যবহার করেছ।

SQL-এ:

```
WHERE
```

ধরি শুধু IT employees:

```
SELECT *
FROM employees
WHERE department = 'IT';
```

Result:

id	name	department	salary
E001	Rahim	IT	45000
E003	Hasan	IT	60000

8️⃣ Salary Filter

Salary 50,000-এর বেশি:

```
SELECT *
FROM employees
WHERE salary > 50000;
```

Result:

```
Karim
Hasan
Jamil
```

9️⃣ Salary >= 50,000

```sql
SELECT *
FROM employees
WHERE salary >= 50000;
```

এখানে 50,000-ও include হবে।

🔟 Text Condition

Dhaka-এর employees:

```sql
SELECT *
FROM employees
WHERE city = 'Dhaka';
```

⚠️ Text সাধারণত single quote-এর মধ্যে:

```
'Dhaka'
'IT'
'Rahim'
```

1️⃣1️⃣ Multiple Conditions — AND

IT department এবং Dhaka:

```sql
SELECT *
FROM employees
WHERE department = 'IT'
AND city = 'Dhaka';
```

মানে:

```
department = IT
       AND
city = Dhaka
```

দুই condition-ই true হতে হবে।

1️⃣2️⃣ OR

IT অথবা HR:

```sql
SELECT *
FROM employees
WHERE department = 'IT'
OR department = 'HR';
```

এখানে যেকোনো একটি condition true হলেই হবে।

🧠 Excel বনাম SQL

তুমি আগে Excel-এ শিখেছ:

```
=IF(AND(B2="IT",C2="Dhaka"),"Yes","No")
```

SQL-এ filtering:

```sql
WHERE department = 'IT'
AND city = 'Dhaka'
```

Concept একই:

```
Excel Logic → SQL Logic
```

1️⃣3️⃣ ORDER BY

Excel-এ Sort করেছ।

SQL-এ:

```sql
ORDER BY
```

Salary ছোট থেকে বড়:

```sql
SELECT *
FROM employees
ORDER BY salary ASC;
```

`ASC` = Ascending

1️⃣4️⃣ বড় থেকে ছোট

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

`DESC` = Descending

Result:

```
Jamil    70000
Hasan    60000
Karim    55000
Rahim    45000
Nabil    40000
```

1️⃣5️⃣ Multiple Sorting

ধরি:

প্রথমে Department অনুযায়ী sort, তারপর Salary descending:

```
SELECT *
FROM employees
ORDER BY department ASC, salary DESC;
```

SQL প্রথমে department দেখবে।

একই department-এর মধ্যে salary অনুযায়ী sort করবে।

1️⃣6️⃣ LIMIT

ধরি শুধু top 3 highest-paid employees:

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Result:

```
Jamil
Hasan
Karim
```

🔥 খুব গুরুত্বপূর্ণ pattern:

```sql
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

মানে:

> Highest salary-এর top 3 employee।

1️⃣7️⃣ DISTINCT

Unique departments দেখতে চাই:

```sql
SELECT DISTINCT department
FROM employees;
```

Result:

```
IT
HR
Finance
Marketing
```

Excel-এর **Remove Duplicates / UNIQUE()** concept-এর সাথে তুলনা করতে পারো।

1️⃣8️⃣ COUNT

Employee কয়জন?

```sql
SELECT COUNT(*)
FROM employees;
```

Result:

```
5
```

1️⃣9️⃣ COUNT with WHERE

IT employee কতজন?

```sql
SELECT COUNT(*)
FROM employees
WHERE department = 'IT';
```

2️⃣0️⃣ SUM

সব employee-এর salary total:

```sql
SELECT SUM(salary)
FROM employees;
```

Excel-এর:

```
=SUM(EmployeeTable[Salary])
```

এর SQL equivalent concept:

```sql
SUM(salary)
```

2️⃣1️⃣ AVG

Average salary:

```sql
SELECT AVG(salary)
FROM employees;
```

Excel:

```
=AVERAGE(EmployeeTable[Salary])
```

SQL:

```
AVG(salary)
```

2️⃣2️⃣ MIN / MAX

Minimum salary:

```
SELECT MIN(salary)
FROM employees;
```

Maximum:

```
SELECT MAX(salary)
FROM employees;
```

🧠 Excel → SQL Cheat Sheet

Excel	SQL
Select columns	`SELECT`
Filter	`WHERE`
Sort	`ORDER BY`
Unique	`DISTINCT`
Top N	`LIMIT`
SUM	`SUM()`
Average	`AVG()`
Count	`COUNT()`
Minimum	`MIN()`
Maximum	`MAX()`
AND	`AND`
OR	`OR`

2️⃣3️⃣ একটি Complete Query

Question:

> Dhaka-এর IT employees-এর মধ্যে highest salary পাওয়া top 2 employee দেখাও।

SQL:

```sql
SELECT id, name, salary
FROM employees
WHERE city = 'Dhaka'
AND department = 'IT'
ORDER BY salary DESC
LIMIT 2;
```

এখানে আমরা একসাথে ব্যবহার করেছি:

```
SELECT
WHERE
AND
ORDER BY
DESC
LIMIT
```

এটাই SQL শেখার সবচেয়ে ভালো পদ্ধতি:

> **Business question → SQL query**

2️⃣4️⃣ আরেকটি Example

Question:

> Salary 50,000-এর বেশি employees-এর নাম ও salary দেখাও।

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;
```

2️⃣5️⃣ SQL Query পড়ার নিয়ম

এই query:

```sql
SELECT name, salary
FROM employees
WHERE department = 'IT'
ORDER BY salary DESC
LIMIT 3;
```

এভাবে পড়বে:

```
SELECT
↓
কোন columns?

FROM
↓
কোন table?

WHERE
↓
কোন rows?

ORDER BY
↓
কীভাবে sort?

LIMIT
↓
কতগুলো result?
```

🔥 SQL শেখার Golden Pattern

প্রায় প্রতিটি beginner query-তে এই structure মাথায় রাখো:

```sql
SELECT columns
FROM table
WHERE condition
ORDER BY column
LIMIT number;
```

সব অংশ সবসময় লাগবে না।

যেমন:

```sql
SELECT *
FROM employees;
```

এটাই valid।

🎯 Class 31 Practical Assignment

এই `employees` table নিয়ে নিচের queries নিজে লেখো।

Q1

সব employees দেখাও।

Q2

শুধু `name` এবং `salary` দেখাও।

Q3

শুধু IT employees দেখাও।

Q4

Salary 50,000-এর বেশি employees দেখাও।

Q5

Dhaka-এর employees দেখাও।

Q6

Dhaka-এর IT employees দেখাও।

Q7

IT অথবা Finance employees দেখাও।

Q8

Salary highest থেকে lowest sort করো।

Q9

Top 3 highest-paid employees দেখাও।

Q10

Unique departments দেখাও।

Q11

Total employees count করো।

Q12

Total salary বের করো।

Q13

Average salary বের করো।

Q14

Highest salary বের করো।

Q15

Lowest salary বের করো।

🧪 Class 31 Quiz

1. SQL-এর full form কী?

A. Simple Query Language
B. Structured Query Language
C. Standard Question Language
D. System Query Logic

2. Table-এর data দেখতে কোন command?

A. SELECT
B. SHOWDATA
C. GET
D. VIEW

3. Data filter করতে?

A. SORT
B. WHERE
C. FILTER
D. GROUP

4. Sort করতে?

A. ORDER BY
B. SORT BY
C. ARRANGE
D. ORDER

5. Highest থেকে lowest?

A. ASC
B. DESC
C. HIGH
D. TOP

6. Unique values পেতে?

A. UNIQUE
B. DISTINCT
C. DIFFERENT
D. ONLY

7. Employee count?

A. SUM()
B. COUNT()
C. TOTAL()
D. NUMBER()

8. Average salary?

A. MEAN()
B. AVG()
C. AVERAGE()
D. MID()

9. Top 5 records পেতে কোনটা useful?

A. TOP 5
B. LIMIT 5
C. FIRST 5
D. MAX 5

10. SQL শেখার সবচেয়ে ভালো approach কোনটি?

A. শুধু syntax মুখস্থ করা
B. Business question → SQL query
C. শুধু SELECT শেখা
D. শুধু database তৈরি করা

🚀 পরের Class 32

SQL Fundamentals — Database + Table Creation

সেখানে আমরা শিখব:

```
CREATE DATABASE
CREATE TABLE
Data Types
PRIMARY KEY
INSERT INTO
NULL
NOT NULL
DEFAULT
UNIQUE
CHECK
```

এবং Excel-এর **row/column** concept থেকে কীভাবে একটি proper relational database তৈরি হয়, সেটা হাতে-কলমে দেখব।

📘 Excel Mastery → SQL — Class 32

🗄️ Database & Table Creation Fundamentals

Class 31-এ আমরা শিখেছি:

```
SELECT
FROM
WHERE
ORDER BY
LIMIT
DISTINCT
COUNT
SUM
AVG
MIN
MAX
```

আজ থেকে আমরা SQL-এর **Database Design side** শুরু করব। এটা তোমার database designer হওয়ার জন্য খুব important।

আজ শিখব:

- Database
- DBMS
- CREATE DATABASE
- CREATE TABLE
- Data Types

- PRIMARY KEY
- NOT NULL
- UNIQUE
- DEFAULT
- CHECK
- INSERT INTO
- NULL
- Basic constraints

1️⃣ Database কী?

সহজভাবে:

> **Database হলো organized data-এর collection।**

ধরো একটি company-এর employee information আছে:

```
Employee ID
Name
Department
Salary
City
```

এই data সুন্দরভাবে store করার জন্য database ব্যবহার করা যায়।

2️⃣ DBMS কী?

DBMS = Database Management System

এটি database তৈরি, সংরক্ষণ, query এবং manage করার software।

Examples:

- PostgreSQL
- MySQL
- SQL Server
- Oracle Database
- SQLite

তুমি যেহেতু MySQL already ব্যবহার করেছ, আজকের SQL examples MySQL/PostgreSQL দুটোতেই সহজে বোঝা যাবে। কিছু syntax পরে আলাদা হতে পারে।

3️⃣ Database → Table → Row → Column

এই hierarchy মনে রাখো:

```
Database
   ↓
Tables
   ↓
Rows
   ↓
Columns
```

Example:

```
CompanyDB
   │
   ├── employees
   │       ├── id
   │       ├── name
   │       ├── department
   │       └── salary
   │
   ├── departments
   │
   └── projects
```

একটি database-এর ভিতরে অনেক table থাকতে পারে।

4️⃣ CREATE DATABASE

ধরি database-এর নাম:

```
company_db
```

SQL:

```
CREATE DATABASE company_db;
```

Database তৈরি হয়ে গেল।

5️⃣ Database ব্যবহার করা

MySQL

```
USE company_db;
```

এর অর্থ:

> এখন `company_db` database-এর ভিতরে কাজ করব।

PostgreSQL-এ database connection/select করার পদ্ধতি client অনুযায়ী হয়; `USE` command নেই।

6️⃣ CREATE TABLE

এখন employee table তৈরি করি।

```
CREATE TABLE employees (
    id INT,
    name VARCHAR(100),
    department VARCHAR(50),
    city VARCHAR(50),
    salary DECIMAL(10,2)
);
```

7️⃣ প্রতিটি অংশ বুঝি

```
id INT
```

মানে:

```
id → column name
INT → data type
```

```
name VARCHAR(100)
```

মানে:

```
name → column
VARCHAR(100) → maximum 100 characters-এর variable-length text
```

```
salary DECIMAL(10,2)
```

মানে:

> Decimal number; মোট precision 10 digits, যার মধ্যে decimal-এর পরে 2 digits।

উদাহরণ:

```
45000.50
```

8️⃣ Common SQL Data Types

Data Type	ব্যবহার
INT	পূর্ণ সংখ্যা
BIGINT	অনেক বড় পূর্ণ সংখ্যা
DECIMAL	টাকা/precise decimal
VARCHAR	Text
CHAR	Fixed-length text
TEXT	বড় text
DATE	Date
TIME	Time
DATETIME/TIMESTAMP	Date + Time
BOOLEAN	True/False

9️⃣ INT

পূর্ণ সংখ্যা:

```
age INT
```

Examples:

```
18
25
40
100
```

🔟 VARCHAR

Text:

```
name VARCHAR(100)
```

Examples:

```
Rahim
Karim
Hasan
```

1️⃣1️⃣ DECIMAL

Financial data-এর জন্য খুব useful:

```
salary DECIMAL(10,2)
```

উদাহরণ:

```
45000.00
```

💡 Money-এর ক্ষেত্রে সাধারণত floating-point-এর বদলে appropriate exact decimal type ব্যবহার করা ভালো।

1️⃣2️⃣ DATE

```
birth_date DATE
```

Example:

```
2001-05-20
```

SQL date সাধারণত:

```
YYYY-MM-DD
```

format-এ দেখা যায়।

1️⃣3️⃣ BOOLEAN

```
is_active BOOLEAN
```

Value:

```
TRUE
FALSE
```

1️⃣4️⃣ PRIMARY KEY 🔥

এটা খুব গুরুত্বপূর্ণ।

Primary Key = প্রতিটি row-কে uniquely identify করে এমন column বা column combination।

Example:

```
id
```

আমাদের:

```
E001
E002
E003
```

প্রতিটি আলাদা।

1️⃣5️⃣ Primary Key Example

```sql
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10,2)
);
```

এখন:

```
id
```

হচ্ছে Primary Key।

1️⃣6️⃣ Primary Key-এর Rules

একটি Primary Key:

✅ Unique হতে হবে

✅ NULL হতে পারবে না

✅ প্রতিটি row uniquely identify করবে

উদাহরণ:

```
id
---
1
2
3
```

ভালো।

কিন্তু:

```
1
1
2
```

Primary Key হিসেবে valid নয়।

1️⃣7️⃣ NOT NULL

ধরো employee-এর name অবশ্যই থাকতে হবে।

```
name VARCHAR(100) NOT NULL
```

এখন:

```
name = NULL
```

allowed নয়।

Complete example:

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    salary DECIMAL(10,2)
);
```

1️⃣8️⃣ NULL কী?

NULL মানে value জানা নেই / দেওয়া হয়নি / প্রযোজ্য নয়।

NULL:

```
0 নয়
```

NULL:

```
empty string '' নয়
```

NULL মানে:

> এখানে value নেই/unknown।

1️⃣9️⃣ UNIQUE

ধরো employee email unique হওয়া উচিত।

```
email VARCHAR(150) UNIQUE
```

তাহলে:

```
rahim@gmail.com
```

একবার থাকতে পারবে।

একই email আবার insert করলে constraint violation হবে।

2️⃣0️⃣ DEFAULT

ধরো নতুন employee-এর active status automatically:

```
TRUE
```

হবে।

```
is_active BOOLEAN DEFAULT TRUE
```

তাহলে value না দিলেও default value আসবে।

2️⃣1️⃣ CHECK

ধরো salary negative হতে পারবে না।

```
salary DECIMAL(10,2) CHECK (salary >= 0)
```

অথবা age:

```
age INT CHECK (age >= 18)
```

এটা database-level validation।

2️⃣2️⃣ Complete Employee Table 🔥

এবার সব একসাথে:

```
CREATE TABLE employees (
    id INT PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    age INT CHECK (age >= 18),
    department VARCHAR(50),
    city VARCHAR(50),
    salary DECIMAL(10,2) CHECK (salary >= 0),
    is_active BOOLEAN DEFAULT TRUE
);
```

এটা এখন অনেক বেশি professional table design।

2️⃣3️⃣ Excel বনাম SQL

তুমি Excel-এ শিখেছিলে Data Validation।

এখন compare করো:

Excel	Database
Unique validation	`UNIQUE`
Required value	`NOT NULL`
Numeric validation	`CHECK`
Default value	`DEFAULT`
Unique ID	`PRIMARY KEY`
Dropdown/valid related value	পরে `FOREIGN KEY`

এটাই Excel → Database transition।

2️⃣4️⃣ INSERT INTO

Table তৈরি হয়েছে।

এখন data ঢুকাতে হবে।

```
INSERT INTO employees
(id, name, email, age, department, city, salary)
VALUES
(1, 'Rahim', 'rahim@gmail.com', 25, 'IT', 'Dhaka', 45000);
```

2️⃣5️⃣ একসাথে Multiple Rows

```sql
INSERT INTO employees
(id, name, email, age, department, city, salary)
VALUES
(2, 'Karim', 'karim@gmail.com', 30, 'HR', 'Dhaka', 55000),
(3, 'Hasan', 'hasan@gmail.com', 28, 'IT', 'Chittagong', 60000),
(4, 'Jamil', 'jamil@gmail.com', 35, 'Finance', 'Sylhet', 70000);
```

2️⃣6️⃣ Data Check

এখন:

```sql
SELECT *
FROM employees;
```

Result:

id	name	email	age	department	city	salary
1	Rahim	rahim@gmail.com	25	IT	Dhaka	45000
2	Karim	karim@gmail.com	30	HR	Dhaka	55000
3	Hasan	hasan@gmail.com	28	IT	Chittagong	60000
4	Jamil	jamil@gmail.com	35	Finance	Sylhet	70000

2️⃣7️⃣ Primary Key Violation

ধরি আবার:

```sql
INSERT INTO employees
(id, name)
VALUES
(1, 'Nabil');
```

সমস্যা হবে।

কারণ:

```
id = 1
```

আগেই আছে।

Primary Key duplicate হতে দেয় না।

2️⃣8️⃣ NOT NULL Violation

আমাদের:

```
name VARCHAR(100) NOT NULL
```

তাই:

```
INSERT INTO employees
(id, email)
VALUES
(5, 'test@gmail.com');
```

এখানে `name` দেওয়া হয়নি।

Database reject করবে।

2️⃣9️⃣ UNIQUE Violation

ধরি:

```
rahim@gmail.com
```

আগেই আছে।

আবার:

```
INSERT INTO employees
(id, name, email)
VALUES
(5, 'Nabil', 'rahim@gmail.com');
```

করলে `UNIQUE` constraint violation হবে।

3️⃣0️⃣ CHECK Constraint

আমাদের:

```
salary DECIMAL(10,2) CHECK (salary >= 0)
```

তাই:

```
INSERT INTO employees
(id, name, salary)
VALUES
(5, 'Nabil', -5000);
```

এটা reject হওয়ার কথা।

কারণ:

```
-5000 < 0
```

3️⃣1️⃣ DEFAULT

আমাদের:

```
is_active BOOLEAN DEFAULT TRUE
```

তাই:

```
INSERT INTO employees
(id, name)
VALUES
(5, 'Nabil');
```

করলে `is_active` explicitly না দিলেও default value `TRUE` হতে পারে।

🧠 Constraint Mental Model

এটা মুখস্থ না করে logic বুঝো:

```
PRIMARY KEY
     ↓
Who is this row?
```

```
NOT NULL
     ↓
Must this value exist?

UNIQUE
     ↓
Can duplicates exist?

CHECK
     ↓
Is this value valid?

DEFAULT
     ↓
What should happen if value is omitted?
```

3️⃣2️⃣ Full Example

```sql
CREATE DATABASE company_db;
```

MySQL হলে:

```sql
USE company_db;
```

তারপর:

```sql
CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    age INT CHECK (age >= 18),
    department VARCHAR(50),
    city VARCHAR(50),
    salary DECIMAL(10,2) CHECK (salary >= 0),
    is_active BOOLEAN DEFAULT TRUE
);
```

Data:

```sql
INSERT INTO employees
(id, name, email, age, department, city, salary)
VALUES
(1, 'Rahim', 'rahim@gmail.com', 25, 'IT', 'Dhaka', 45000),
(2, 'Karim', 'karim@gmail.com', 30, 'HR', 'Dhaka', 55000),
```

```
(3, 'Hasan', 'hasan@gmail.com', 28, 'IT', 'Chittagong', 60000),
(4, 'Jamil', 'jamil@gmail.com', 35, 'Finance', 'Sylhet', 70000);
```

তারপর:

```
SELECT *
FROM employees;
```

🔥 Class 32-এর সবচেয়ে Important Concept

Database design মানে শুধু:

```
CREATE TABLE
```

লেখা নয়।

তোমাকে ভাবতে হবে:

> **এই data-এর rules কী?**

উদাহরণ:

Employee:

```
ID → unique
Name → required
Email → unique
Age → minimum 18
Salary → cannot be negative
Active → default TRUE
```

তারপর সেই business rules database constraint হিসেবে implement করবে।

এটাই **database design thinking**।

🎯 Class 32 Assignment

`company_db` তৈরি করে `employees` table বানাও।

Table columns:

```
id
name
email
age
department
city
salary
is_active
```

Rules:

- `id` → Primary Key
- `name` → NOT NULL
- `email` → UNIQUE
- `age` → >= 18
- `salary` → >= 0
- `is_active` → DEFAULT TRUE

তারপর অন্তত **10 employees** insert করো।

এরপর এই queries লেখো:

Q1

সব employee:

```sql
SELECT *
FROM employees;
```

Q2

শুধু name + salary।

Q3

IT employees।

Q4

Salary > 50,000।

Q5

Dhaka employees।

Q6

Salary highest → lowest।

Q7

Top 3 employees।

Q8

Unique departments।

Q9

Total salary।

Q10

Average salary।

🧪 Class 32 Quiz

1. Database কী?

A. শুধু একটি Excel file
B. Organized collection of data
C. শুধু একটি table
D. শুধু একটি query

2. Database manage করার software-কে কী বলে?

A. DBMS
B. HTML
C. IDE
D. API

3. Primary Key-এর প্রধান কাজ কী?

A. Data sort করা
B. Row uniquely identify করা
C. Table delete করা
D. Calculation করা

4. Primary Key কি NULL হতে পারে?

A. Yes
B. No
C. Sometimes
D. শুধু MySQL-এ

5. Duplicate value prevent করতে?

A. DEFAULT
B. CHECK
C. UNIQUE
D. NULL

6. Value অবশ্যই দিতে হবে এমন condition?

A. NOT NULL
B. UNIQUE
C. DEFAULT
D. ORDER BY

7. Salary negative prevent করতে কোনটি ভালো?

A. PRIMARY KEY
B. CHECK
C. UNIQUE
D. DEFAULT

8. Value না দিলে automatically value দেওয়ার জন্য?

A. CHECK
B. DEFAULT
C. NULL
D. DISTINCT

9. Data insert করতে?

A. ADD DATA
B. INSERT INTO
C. PUT
D. CREATE ROW

10. `VARCHAR(100)` কী বোঝায়?

A. 100 integer
B. Variable-length character/text field with declared maximum length 100
C. 100 rows
D. 100 tables

🚀 Next: Class 33

পরের class-এ আমরা যাব **SQL Data Manipulation & Filtering — INSERT, UPDATE, DELETE, NULL, LIKE, IN, BETWEEN, IS NULL**-এ।

বিশেষ করে শিখব:

```
UPDATE
DELETE
LIKE
IN
BETWEEN
IS NULL
IS NOT NULL
```

এবং **কীভাবে ভুল UPDATE/DELETE করে পুরো table-এর data নষ্ট হয়ে যেতে পারে এবং কীভাবে safe query লিখতে হয়**—এটা database designer-এর জন্য খুব গুরুত্বপূর্ণ।

📘 SQL — Class 33

✏️ INSERT, UPDATE, DELETE + LIKE, IN, BETWEEN, NULL

Class 32-এ আমরা শিখেছি:

```
CREATE DATABASE
CREATE TABLE
Data Types
PRIMARY KEY
NOT NULL
UNIQUE
CHECK
DEFAULT
INSERT INTO
```

আজ আমরা শিখব কীভাবে existing data **change, search এবং delete** করতে হয়।

1️⃣ INSERT INTO — আবার একটু Practice

ধরি আমাদের table:

```
employees
```

Columns:

```
id
name
email
```

```
age
department
city
salary
is_active
```

একজন employee:

```sql
INSERT INTO employees
(id, name, email, age, department, city, salary)
VALUES
(5, 'Nabil', 'nabil@gmail.com', 27, 'Marketing', 'Dhaka', 40000);
```

2️⃣ UPDATE — Existing Data পরিবর্তন

ধরি Rahim-এর salary:

```
45000
```

এখন salary 50000 করতে চাই।

```sql
UPDATE employees
SET salary = 50000
WHERE id = 1;
```

এখানে:

```
UPDATE → কোন table?
SET    → কী পরিবর্তন?
WHERE  → কোন row?
```

⚠️ 3️⃣ UPDATE-এর সবচেয়ে গুরুত্বপূর্ণ Rule

WHERE ভুলে যেও না।

❌ বিপজ্জনক:

```sql
UPDATE employees
SET salary = 50000;
```

এটা table-এর **সব employee-এর salary 50000** করে দিতে পারে।

সঠিক:

```
UPDATE employees
SET salary = 50000
WHERE id = 1;
```

4️⃣ Multiple Column Update

Rahim-এর:

```
salary = 50000
city = Dhaka
```

একসাথে:

```
UPDATE employees
SET salary = 50000,
    city = 'Dhaka'
WHERE id = 1;
```

5️⃣ Condition দিয়ে UPDATE

IT department-এর salary 10% increase:

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

এখানে:

```
salary * 1.10
```

মানে current salary + 10%।

Example:

```
50000 → 55000
60000 → 66000
```

6️⃣ DELETE — Row Delete

ধরি ID 5 employee delete করতে চাই:

```
DELETE FROM employees
WHERE id = 5;
```

⚠️ DELETE-এর ক্ষেত্রেও WHERE খুব গুরুত্বপূর্ণ

❌ বিপজ্জনক:

```
DELETE FROM employees;
```

এতে table-এর **সব rows delete** হয়ে যেতে পারে।

সঠিক:

```
DELETE FROM employees
WHERE id = 5;
```

🧠 UPDATE বনাম DELETE

Command	কাজ
INSERT	নতুন row যোগ
UPDATE	existing row পরিবর্তন
DELETE	row মুছে ফেলা
SELECT	data দেখা

Mental model:

```
INSERT → Add
SELECT → Read
UPDATE → Change
DELETE → Remove
```

এগুলোকে একসাথে অনেক সময় **CRUD** বলা হয়:

```
C → Create
R → Read
U → Update
D → Delete
```

7️⃣ LIKE — Pattern Search

ধরি এমন employee খুঁজতে চাই যাদের নাম `R` দিয়ে শুরু।

```sql
SELECT *
FROM employees
WHERE name LIKE 'R%';
```

এখানে `%` খুব গুরুত্বপূর্ণ।

8️⃣ `%` কী?

`%` মানে:

> Zero বা more characters

Example:

```sql
LIKE 'R%'
```

মানে:

```
Rahim
Rafi
Rakib
R
```

সব match করতে পারে।

9️⃣ Name শেষ হবে `n` দিয়ে

```sql
SELECT *
FROM employees
```

```
WHERE name LIKE '%n';
```

Match হতে পারে:

```
Rahman
Hasan
```

🔟 Name-এর মধ্যে `ha`

```
SELECT *
FROM employees
WHERE name LIKE '%ha%';
```

মানে name-এর যেকোনো জায়গায় `ha` থাকলে match করবে।

1️⃣1️⃣ `_` Wildcard

`_` মানে **exactly one character**।

Example:

```
WHERE name LIKE 'R_him';
```

এখানে:

```
R
_
h
i
m
```

`_` একটি character represent করছে।

🧠 `%` বনাম `_`

Symbol	Meaning
`%`	Zero or more characters

Symbol	Meaning
`_`	Exactly one character

1️⃣2️⃣ IN

ধরি IT অথবা HR অথবা Finance department-এর employees চাই।

এভাবে লিখতে পারো:

```sql
SELECT *
FROM employees
WHERE department = 'IT'
   OR department = 'HR'
   OR department = 'Finance';
```

কিন্তু আরও clean:

```sql
SELECT *
FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

`IN` মানে:

> এই list-এর মধ্যে যেকোনো একটি value।

1️⃣3️⃣ NOT IN

IT এবং HR বাদ দিয়ে:

```sql
SELECT *
FROM employees
WHERE department NOT IN ('IT', 'HR');
```

1️⃣4️⃣ BETWEEN

Salary 40,000 থেকে 60,000-এর মধ্যে:

```sql
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

সাধারণভাবে `BETWEEN` দুই boundary-ই include করে।

অর্থাৎ:

```
40000 ✓
50000 ✓
60000 ✓
```

1️⃣5️⃣ Date-এর সাথে BETWEEN

ধরি `join_date` column আছে।

January 2026-এর data:

```sql
SELECT *
FROM employees
WHERE join_date BETWEEN '2026-01-01' AND '2026-01-31';
```

Date/time column হলে boundary handling নিয়ে আরও সতর্ক হতে হয়; পরে আমরা proper date-range query শিখব।

1️⃣6️⃣ NULL

ধরি কিছু employee-এর email নেই:

```
id | name | email
-------------------------
1  | Rahim | rahim@gmail.com
2  | Karim | NULL
3  | Hasan | hasan@gmail.com
```

এখন যাদের email নেই তাদের খুঁজতে:

❌ এভাবে নয়:

```sql
WHERE email = NULL;
```

সঠিক:

```
SELECT *
FROM employees
WHERE email IS NULL;
```

1️⃣7️⃣ IS NOT NULL

যাদের email আছে:

```
SELECT *
FROM employees
WHERE email IS NOT NULL;
```

🔥 কেন `= NULL` কাজ করে না?

SQL-এ NULL একটি ordinary value নয়।

NULL বলতে বোঝায়:

> value unknown / missing / absent।

তাই SQL-এ:

```
email IS NULL
```

ব্যবহার করতে হয়।

1️⃣8️⃣ Practical Examples

Example 1 — Dhaka employees

```
SELECT *
FROM employees
WHERE city = 'Dhaka';
```

Example 2 — Salary 50K-এর বেশি

```sql
SELECT *
FROM employees
WHERE salary > 50000;
```

Example 3 — Salary 40K–60K

```sql
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 60000;
```

Example 4 — IT অথবা HR

```sql
SELECT *
FROM employees
WHERE department IN ('IT', 'HR');
```

Example 5 — Name starts with A

```sql
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Example 6 — Missing email

```sql
SELECT *
FROM employees
WHERE email IS NULL;
```

1️⃣9️⃣ একাধিক Condition

Dhaka-এর IT employees:

```sql
SELECT *
FROM employees
WHERE city = 'Dhaka'
AND department = 'IT';
```

Dhaka-এর IT অথবা HR:

```sql
SELECT *
FROM employees
```

```
WHERE city = 'Dhaka'
AND department IN ('IT', 'HR');
```

Salary 50K-এর বেশি এবং IT:

```
SELECT *
FROM employees
WHERE department = 'IT'
AND salary > 50000;
```

2️⃣0️⃣ UPDATE + LIKE

যাদের department Marketing:

```
UPDATE employees
SET is_active = FALSE
WHERE department = 'Marketing';
```

2️⃣1️⃣ UPDATE + IN

IT এবং HR employees-এর salary 5% increase:

```
UPDATE employees
SET salary = salary * 1.05
WHERE department IN ('IT', 'HR');
```

2️⃣2️⃣ DELETE + Condition

Marketing department-এর inactive employee delete:

```
DELETE FROM employees
WHERE department = 'Marketing'
AND is_active = FALSE;
```

⚠️ 2️⃣3️⃣ Safe UPDATE/DELETE Technique

Real database-এ সরাসরি UPDATE/DELETE করার আগে **SELECT দিয়ে একই condition test করা** ভালো অভ্যাস।

ধরি:

```
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'IT';
```

আগে:

```
SELECT *
FROM employees
WHERE department = 'IT';
```

দেখো কোন rows affected হবে।

তারপর UPDATE।

একইভাবে DELETE:

```
SELECT *
FROM employees
WHERE department = 'Marketing'
AND is_active = FALSE;
```

ঠিক rows হলে তারপর:

```
DELETE FROM employees
WHERE department = 'Marketing'
AND is_active = FALSE;
```

এটা খুব valuable database habit। 🔥

2️⃣4️⃣ CRUD পুরোটা

এখন তোমার CRUD understanding:

Create

```
INSERT INTO employees (...)
VALUES (...);
```

Read

```sql
SELECT *
FROM employees;
```

Update

```sql
UPDATE employees
SET salary = 50000
WHERE id = 1;
```

Delete

```sql
DELETE FROM employees
WHERE id = 1;
```

🧠 Excel → SQL Connection

তুমি Excel-এ already এগুলোর concept দেখেছ:

Excel	SQL
Add row	INSERT
Filter	WHERE
Search pattern	LIKE
Multiple choices	IN
Range condition	BETWEEN
Blank check	IS NULL
Sort	ORDER BY
Edit data	UPDATE
Remove row	DELETE

তাই SQL তোমার জন্য completely নতুন concept নয়।

🔥 Class 33 Query Challenge

ধরি:

```
employees
```

এর মধ্যে 10+ records আছে।

নিজে query লেখো:

Q1

ID 3-এর employee-এর salary 65,000 করো।

Q2

সব IT employee-এর salary 10% increase করো।

Q3

HR এবং Finance employees দেখাও।

Q4

Salary 45,000 থেকে 70,000-এর মধ্যে employees দেখাও।

Q5

যাদের name `A` দিয়ে শুরু তাদের দেখাও।

Q6

যাদের name `n` দিয়ে শেষ তাদের দেখাও।

Q7

যাদের email নেই তাদের দেখাও।

Q8

যাদের email আছে তাদের দেখাও।

Q9

IT এবং HR ছাড়া employees দেখাও।

Q10

ID 10-এর employee delete করো।

Q11

Dhaka-এর Marketing employees-এর salary 5% increase করো।

Q12

যাদের salary 60,000-এর বেশি তাদের inactive করো।

🧪 Class 33 Quiz

1. Existing data পরিবর্তন করতে কোন command?

A. CHANGE
B. UPDATE
C. MODIFY
D. EDIT

2. Row delete করতে?

A. REMOVE
B. DROP ROW
C. DELETE
D. CLEAR

3. `LIKE 'A%'` কী বোঝায়?

A. A দিয়ে শেষ
B. A দিয়ে শুরু
C. শুধু A
D. A নেই

4. `%` কী বোঝায়?

A. Exactly one character
B. Zero or more characters
C. Number
D. Space

5. `IN ('IT','HR')` কী বোঝায়?

A. IT এবং HR দুটোই একই row-তে
B. IT অথবা HR-এর মধ্যে যেকোনো একটি
C. IT বাদ
D. HR বাদ

6. 40,000 থেকে 60,000 range-এর জন্য?

A. RANGE
B. BETWEEN

C. WITHIN
D. INSIDE

7. NULL check করতে?

A. `= NULL`
B. `IS NULL`
C. `NULL =`
D. `CHECK NULL`

8. সব rows update করতে কোনটি dangerous?

A. UPDATE + WHERE
B. UPDATE without WHERE
C. UPDATE + ORDER BY
D. UPDATE + LIMIT

9. Safe DELETE-এর আগে কী করা ভালো?

A. SELECT দিয়ে condition verify
B. DROP DATABASE
C. Restart computer
D. CREATE TABLE

10. CRUD-এর U কী?

A. Upload
B. Update
C. Undo
D. Unique

📌 Class 33 Cheat Sheet

```sql
-- INSERT
INSERT INTO employees (...)
VALUES (...);

-- UPDATE
UPDATE employees
SET salary = 50000
WHERE id = 1;

-- DELETE
DELETE FROM employees
WHERE id = 1;
```

```sql
-- LIKE
WHERE name LIKE 'A%';

-- IN
WHERE department IN ('IT', 'HR');

-- BETWEEN
WHERE salary BETWEEN 40000 AND 60000;

-- NULL
WHERE email IS NULL;

-- NOT NULL
WHERE email IS NOT NULL;
```

⭐ আজকের সবচেয়ে গুরুত্বপূর্ণ ৩টি rule

```
1. UPDATE → WHERE carefully
2. DELETE → WHERE carefully
3. NULL → IS NULL / IS NOT NULL
```

Next Class 34: `GROUP BY + HAVING + Aggregate Functions` — এখানে Excel-এর **PivotTable + SUMIF/COUNTIF/AVERAGEIF** concept-কে SQL-এর powerful analysis query-তে convert করব।

📘 SQL — Class 34

📊 GROUP BY + HAVING + Aggregate Functions

Class 33-এ আমরা শিখেছি:

```
INSERT
UPDATE
DELETE
LIKE
IN
BETWEEN
IS NULL
IS NOT NULL
```

আজ থেকে আমরা SQL দিয়ে **real data analysis** শুরু করব।

আজকের topic খুব important কারণ Excel-এর:

- PivotTable

- SUMIF
- COUNTIF
- AVERAGEIF

— এগুলোর SQL equivalent বুঝতে শুরু করবে।

🎯 আজকের Learning Goals

আজ শিখবে:

1. Aggregate Functions
2. `COUNT()`
3. `SUM()`
4. `AVG()`
5. `MIN()`
6. `MAX()`
7. `GROUP BY`
8. `HAVING`
9. `WHERE` বনাম `HAVING`
10. Multiple-column grouping
11. Business questions → SQL

1️⃣ Aggregate Function কী?

Aggregate function অনেকগুলো row-এর data নিয়ে **একটি summary result** তৈরি করে।

উদাহরণ:

```
SELECT SUM(salary)
FROM employees;
```

অনেক employee-এর salary → একটি total।

2️⃣ COUNT()

Employee কতজন?

```sql
SELECT COUNT(*)
FROM employees;
```

ধরি 10 জন employee আছে:

```
COUNT
-----
10
```

3️⃣ COUNT(column)

```sql
SELECT COUNT(email)
FROM employees;
```

এখানে `email` যেসব row-তে NULL নয়, সেগুলো count হবে।

তাই:

```
COUNT(*)       → rows count
COUNT(email)   → non-NULL email count
```

এটা খুব important difference।

4️⃣ SUM()

সব employee-এর salary total:

```sql
SELECT SUM(salary)
FROM employees;
```

Excel equivalent:

```
=SUM(EmployeeTable[Salary])
```

5️⃣ AVG()

Average salary:

```sql
SELECT AVG(salary)
FROM employees;
```

Excel:

```
=AVERAGE(EmployeeTable[Salary])
```

6️⃣ MIN()

সবচেয়ে কম salary:

```sql
SELECT MIN(salary)
FROM employees;
```

7️⃣ MAX()

সবচেয়ে বেশি salary:

```sql
SELECT MAX(salary)
FROM employees;
```

8️⃣ একসাথে Multiple Aggregate

```sql
SELECT
    COUNT(*) AS total_employees,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM employees;
```

Result হতে পারে:

total_employees	total_salary	average_salary	minimum_salary	maximum_salary
10	580000	58000	35000	85000

9️⃣ `AS` কী?

```
SUM(salary) AS total_salary
```

`AS` দিয়ে column-এর temporary readable name দিচ্ছি।

Without:

```
SUM(salary)
```

With:

```
total_salary
```

এটাকে **alias** বলে।

🔥 1️⃣0️⃣ GROUP BY

এখন আসল topic।

ধরি আমাদের employee data:

Name	Department	Salary
Rahim	IT	45000
Hasan	IT	60000
Karim	HR	55000
Jamil	Finance	70000
Nabil	HR	50000

Question:

> প্রতিটি department-এ কয়জন employee আছে?

Excel-এ PivotTable করতে।

SQL:

```
SELECT
    department,
    COUNT(*) AS employee_count
```

```
FROM employees
GROUP BY department;
```

Result:

department	employee_count
IT	2
HR	2
Finance	1

1️⃣1️⃣ GROUP BY-এর সহজ ধারণা

`GROUP BY` বলছে:

> একই value-গুলোকে group করো।

যেমন:

```
IT
IT
HR
Finance
HR
```

Group করলে:

```
IT       → 2
HR       → 2
Finance  → 1
```

1️⃣2️⃣ Department-wise Total Salary

Question:

> প্রতিটি department-এর total salary কত?

```
SELECT
    department,
    SUM(salary) AS total_salary
```

```
FROM employees
GROUP BY department;
```

Result:

department	total_salary
IT	105000
HR	105000
Finance	70000

1️⃣3️⃣ Department-wise Average Salary

```
SELECT
    department,
    AVG(salary) AS average_salary
FROM employees
GROUP BY department;
```

1️⃣4️⃣ Department-wise Highest Salary

```
SELECT
    department,
    MAX(salary) AS highest_salary
FROM employees
GROUP BY department;
```

1️⃣5️⃣ Department-wise Lowest Salary

```
SELECT
    department,
    MIN(salary) AS lowest_salary
FROM employees
GROUP BY department;
```

1️⃣6️⃣ এক Query-তে সব

```sql
SELECT
    department,
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM employees
GROUP BY department;
```

এটা অনেকটা Excel PivotTable-এর মতো:

```
Rows → Department

Values:
Count of Employees
Sum of Salary
Average Salary
Minimum Salary
Maximum Salary
```

🔥 এই connectionটা খুব ভালোভাবে বুঝে রাখো।

1️⃣7️⃣ Excel PivotTable → SQL

Excel:

```
Rows:
Department

Values:
Count of ID
Sum of Salary
Average of Salary
```

SQL:

```sql
SELECT
    department,
    COUNT(*) AS employee_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary
```

```
FROM employees
GROUP BY department;
```

1️⃣8️⃣ GROUP BY City

Question:

> City অনুযায়ী employee count।

```
SELECT
    city,
    COUNT(*) AS employee_count
FROM employees
GROUP BY city;
```

1️⃣9️⃣ City-wise Average Salary

```
SELECT
    city,
    AVG(salary) AS average_salary
FROM employees
GROUP BY city;
```

2️⃣0️⃣ Multiple Columns GROUP BY

এখন ধরো:

> Department এবং City অনুযায়ী employee count।

```
SELECT
    department,
    city,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department, city;
```

এখানে group হবে:

```
IT + Dhaka
IT + Chittagong
HR + Dhaka
Finance + Sylhet
```

2️⃣1️⃣ WHERE + GROUP BY

ধরি শুধু Dhaka employees নিয়ে department-wise count চাই।

```sql
SELECT
    department,
    COUNT(*) AS employee_count
FROM employees
WHERE city = 'Dhaka'
GROUP BY department;
```

এখানে প্রথমে:

```
WHERE city = Dhaka
```

তারপর:

```
GROUP BY department
```

🧠 Query Processing-এর সহজ ধারণা

এই query:

```sql
SELECT
    department,
    AVG(salary)
FROM employees
WHERE city = 'Dhaka'
GROUP BY department;
```

Conceptually:

```
employees
    ↓
WHERE city = Dhaka
```

```
    ↓
Filtered rows
    ↓
GROUP BY department
    ↓
AVG(salary)
    ↓
Result
```

2️⃣2️⃣ HAVING

এখন ধরো:

> যেসব department-এ 2-এর বেশি employee আছে, শুধু সেগুলো দেখাও।

এখানে `WHERE` ব্যবহার করা যাবে না, কারণ আমরা **group-এর result** filter করছি।

তাই:

```sql
SELECT
    department,
    COUNT(*) AS employee_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

2️⃣3️⃣ WHERE বনাম HAVING 🔥

এটা খুব important।

WHERE

Rows filter করে।

```sql
WHERE salary > 50000
```

HAVING

Groups filter করে।

```sql
HAVING COUNT(*) > 2
```

Mental model:

```
WHERE
↓
individual rows

GROUP BY
↓
groups

HAVING
↓
groups filter
```

2️⃣4️⃣ Example

Question:

> যেসব department-এর average salary 60,000-এর বেশি, সেগুলো দেখাও।

```sql
SELECT
    department,
    AVG(salary) AS average_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

2️⃣5️⃣ WHERE + GROUP BY + HAVING

এখন আরও realistic query:

> Dhaka-এর employees নিয়ে এমন department দেখাও যেখানে employee count 2-এর বেশি।

```sql
SELECT
    department,
    COUNT(*) AS employee_count
FROM employees
WHERE city = 'Dhaka'
GROUP BY department
HAVING COUNT(*) > 2;
```

Concept:

```
সব employees
      ↓
শুধু Dhaka
      ↓
Department অনুযায়ী group
      ↓
যেখানে count > 2
```

2️⃣6️⃣ GROUP BY + ORDER BY

Department-wise total salary বের করে highest total salary আগে দেখাতে:

```sql
SELECT
    department,
    SUM(salary) AS total_salary
FROM employees
GROUP BY department
ORDER BY total_salary DESC;
```

Result:

```
Finance   150000
IT        130000
HR        110000
```

2️⃣7️⃣ Top Department

সবচেয়ে বেশি total salary-এর top department:

```sql
SELECT
    department,
    SUM(salary) AS total_salary
FROM employees
GROUP BY department
ORDER BY total_salary DESC
LIMIT 1;
```

🔥 Real-world analytics query।

2️⃣8️⃣ Department with Highest Average Salary

```sql
SELECT
    department,
    AVG(salary) AS average_salary
FROM employees
GROUP BY department
ORDER BY average_salary DESC
LIMIT 1;
```

2️⃣9️⃣ COUNT + HAVING

যেসব city-তে অন্তত 3 জন employee আছে:

```sql
SELECT
    city,
    COUNT(*) AS employee_count
FROM employees
GROUP BY city
HAVING COUNT(*) >= 3;
```

3️⃣0️⃣ SUM + HAVING

যেসব department-এর total salary 200,000-এর বেশি:

```sql
SELECT
    department,
    SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 200000;
```

3️⃣1️⃣ AVG + HAVING

Average salary 50,000-এর বেশি:

```sql
SELECT
    department,
    AVG(salary) AS average_salary
```

```
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

🔥 3️⃣2️⃣ SQL Query Structure

আজকের সবচেয়ে important pattern:

```
SELECT
    column,
    AGGREGATE_FUNCTION(column)
FROM table
WHERE condition
GROUP BY column
HAVING aggregate_condition
ORDER BY column
LIMIT number;
```

সব clause সবসময় লাগবে না।

3️⃣3️⃣ Example — Business Question

Question:

> Dhaka-এর কোন departments-এ 2-এর বেশি employee আছে এবং তাদের average salary 50,000-এর বেশি?

Query:

```
SELECT
    department,
    COUNT(*) AS employee_count,
    AVG(salary) AS average_salary
FROM employees
WHERE city = 'Dhaka'
GROUP BY department
HAVING COUNT(*) > 2
   AND AVG(salary) > 50000
ORDER BY average_salary DESC;
```

🔥 এই ধরনের query Data Analyst interview-এ খুব useful।

3️⃣4️⃣ Excel Function বনাম SQL

Excel	SQL
`SUM()`	`SUM()`
`AVERAGE()`	`AVG()`
`COUNT()`	`COUNT()`
`MIN()`	`MIN()`
`MAX()`	`MAX()`
PivotTable Rows	`GROUP BY`
PivotTable Values	Aggregate functions
Pivot filter on aggregate	`HAVING`
Normal Filter	`WHERE`
Sort	`ORDER BY`

🧠 একটি গুরুত্বপূর্ণ Difference

Excel:

```
SUMIF
COUNTIF
AVERAGEIF
```

SQL-এ অনেক সময়:

```
WHERE + GROUP BY + SUM()
WHERE + GROUP BY + COUNT()
WHERE + GROUP BY + AVG()
```

উদাহরণ:

Excel

```
=SUMIF(DepartmentRange,"IT",SalaryRange)
```

SQL

```sql
SELECT SUM(salary)
FROM employees
```

```
WHERE department = 'IT';
```

3️⃣5️⃣ GROUP BY vs DISTINCT

দুটোকে confuse করবে না।

Unique departments:

```
SELECT DISTINCT department
FROM employees;
```

এটা শুধু unique values দেয়।

Department-wise count:

```
SELECT
    department,
    COUNT(*)
FROM employees
GROUP BY department;
```

এখানে grouping + calculation হচ্ছে।

🎯 Class 34 Practical Assignment

এই `employees` table ব্যবহার করে query লেখো।

Q1

Total employee count।

Q2

Total salary।

Q3

Average salary।

Q4

Department-wise employee count।

Q5

Department-wise total salary।

Q6

Department-wise average salary।

Q7

City-wise employee count।

Q8

City-wise average salary।

Q9

Department + City অনুযায়ী employee count।

Q10

শুধু Dhaka employees নিয়ে department-wise average salary।

Q11

যেসব department-এ 3-এর বেশি employee আছে।

Q12

যেসব department-এর average salary 60,000-এর বেশি।

Q13

Department-wise total salary highest থেকে lowest।

Q14

Highest average salary-এর department।

Q15

যেসব city-তে 2-এর বেশি employee আছে এবং average salary 50,000-এর বেশি।

🧪 Class 34 Quiz

1. অনেক rows থেকে summary calculation করার function কী?

A. Aggregate Function
B. Filter Function
C. Search Function
D. Sort Function

2. Total বের করতে?

A. TOTAL()
B. SUM()
C. ADD()
D. PLUS()

3. Average বের করতে?

A. AVERAGE()
B. AVG()
C. MEAN()
D. MID()

4. Department-wise calculation করতে কোনটি ব্যবহার করব?

A. GROUP BY
B. ORDER BY
C. SORT BY
D. DISTINCT

5. Row filter করতে?

A. HAVING
B. WHERE
C. GROUP
D. FILTER BY

6. Group filter করতে?

A. WHERE
B. HAVING
C. GROUP
D. SELECT

7. `COUNT(*)` কী করে?

A. Salary যোগ করে
B. Rows count করে
C. Average বের করে
D. Duplicate delete করে

8. Department-এর total salary বের করতে?

A. `COUNT(salary)`
B. `AVG(salary)`
C. `SUM(salary)`
D. `MAX(salary)`

9. Highest total salary department প্রথমে দেখাতে?

A. `ORDER BY total_salary DESC`
B. `ORDER BY total_salary ASC`
C. `SORT total_salary`
D. `HAVING total_salary`

10. `HAVING COUNT(*) > 2` কী করে?

A. প্রতিটি row-এর count > 2 করে
B. যেসব group-এর count > 2 শুধু সেগুলো রাখে
C. 2 rows delete করে
D. 2 rows add করে

⭐ Class 34 Cheat Sheet

```
-- COUNT
SELECT COUNT(*)
FROM employees;

-- SUM
SELECT SUM(salary)
FROM employees;

-- AVG
SELECT AVG(salary)
FROM employees;

-- GROUP BY
SELECT department, COUNT(*)
FROM employees
GROUP BY department;

-- WHERE + GROUP BY
SELECT department, AVG(salary)
FROM employees
WHERE city = 'Dhaka'
GROUP BY department;

-- HAVING
SELECT department, COUNT(*)
```

```sql
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;

-- GROUP + ORDER
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
ORDER BY total_salary DESC;
```

🧠 আজকের Golden Rule

```
WHERE  → Row filter
GROUP BY → Rows → Groups
HAVING → Group filter
ORDER BY → Sort result
```

আর Excel-এর ভাষায়:

```
Excel PivotTable
      ↓
SQL GROUP BY

Excel Filter
      ↓
SQL WHERE

Pivot aggregate filter
      ↓
SQL HAVING
```

পরের Class 35: `SQL JOIN – INNER JOIN, LEFT JOIN, RIGHT JOIN` । এখানে আমরা প্রথমবার **দুই বা তার বেশি table একসাথে combine** করব—এটা database designer এবং Data Analyst দুজনের জন্যই অত্যন্ত গুরুত্বপূর্ণ।

📘 SQL — Class 35

🔗 JOIN — Multiple Tables একসাথে ব্যবহার

Class 34-এ আমরা শিখেছি:

```
COUNT()
SUM()
AVG()
```

```
MIN()
MAX()
GROUP BY
HAVING
ORDER BY
```

আজ SQL-এর **সবচেয়ে গুরুত্বপূর্ণ topic-এর একটি — JOIN** শুরু করছি।

Database designer বা Data Analyst হিসেবে JOIN না জানলে real database-এর data নিয়ে কাজ করা কঠিন।

🎯 আজকের Learning Goals

আজ শিখবে:

1. কেন multiple table লাগে
2. Primary Key
3. Foreign Key
4. Relationship
5. INNER JOIN
6. LEFT JOIN
7. RIGHT JOIN
8. JOIN + WHERE
9. JOIN + GROUP BY
10. JOIN-এর real-world use

1️⃣ কেন Multiple Table?

ধরো একটি company-তে customer এবং order আছে।

একটি বড় table বানালে:

Order_ID	Customer_ID	Customer_Name	City	Product	Price
O001	C001	Rahim	Dhaka	Laptop	50000
O002	C002	Karim	Chittagong	Mouse	1000
O003	C001	Rahim	Dhaka	Keyboard	2000

এখানে `Rahim`, `Dhaka` বারবার repeat হচ্ছে।

অনেক বড় database হলে redundancy বাড়বে।

তাই আলাদা table:

Customers

Customer_ID	Name	City
C001	Rahim	Dhaka
C002	Karim	Chittagong
C003	Hasan	Sylhet

Orders

Order_ID	Customer_ID	Product	Amount
O001	C001	Laptop	50000
O002	C002	Mouse	1000
O003	C001	Keyboard	2000

এখন দুই table-এর common column:

```
Customers.Customer_ID
Orders.Customer_ID
```

এই relationship ব্যবহার করে data combine করব।

2️⃣ Primary Key

Customers table:

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    city VARCHAR(50)
);
```

এখানে:

```
customer_id
```

হলো Primary Key।

প্রতিটি customer-এর unique ID।

3️⃣ Foreign Key

Orders table:

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product VARCHAR(100),
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

এখানে:

```
orders.customer_id
```

হলো Foreign Key।

এটি `customers.customer_id`-কে reference করছে।

🧠 Primary Key বনাম Foreign Key

Primary Key	Foreign Key
Row uniquely identify করে	অন্য table-এর row reference করে
সাধারণত unique	Repeat হতে পারে
NULL নয়	NULL হতে পারে, যদি relationship optional হয়
Parent/Referenced table-এ থাকে	Child/Referencing table-এ থাকে

Example:

```
customers
    customer_id ← PRIMARY KEY
          ↑
          |
          | FOREIGN KEY
          |
```

```
orders
    customer_id
```

4️⃣ Relationship

এখানে:

```
One Customer
     ↓
Many Orders
```

এটা:

> **One-to-Many Relationship**

একজন customer অনেক order করতে পারে।

5️⃣ JOIN কী?

সহজ ভাষায়:

> **JOIN দুই বা তার বেশি table-এর related data একসাথে দেখায়।**

Example:

Customers:

```
C001 → Rahim
C002 → Karim
```

Orders:

```
O001 → C001
O002 → C002
```

JOIN করলে:

```
O001 | Rahim | Laptop
O002 | Karim | Mouse
```

6️⃣ INNER JOIN 🔥

সবচেয়ে আগে `INNER JOIN`।

```
SELECT
    orders.order_id,
    customers.name,
    orders.product,
    orders.amount
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id;
```

7️⃣ `ON` কী?

```
ON orders.customer_id = customers.customer_id
```

এটাই বলে:

> কোন column-এর ভিত্তিতে দুই table match করবে?

অর্থাৎ:

```
orders.customer_id
        =
customers.customer_id
```

8️⃣ INNER JOIN-এর Result

Customers:

customer_id	name	city
1	Rahim	Dhaka
2	Karim	Chittagong
3	Hasan	Sylhet

Orders:

order_id	customer_id	product	amount
O001	1	Laptop	50000
O002	2	Mouse	1000
O003	1	Keyboard	2000

INNER JOIN:

order_id	name	product	amount
O001	Rahim	Laptop	50000
O002	Karim	Mouse	1000
O003	Rahim	Keyboard	2000

9️⃣ INNER JOIN Mental Model

```
Table A       Table B
   |             |
   └──── MATCH ──┘
          ↓
     Matching rows
```

অর্থাৎ:

> দুই table-এ match আছে এমন data।

🔟 LEFT JOIN

এবার ধরো:

Customers:

```
Rahim
Karim
Hasan
```

কিন্তু Hasan এখনো কোনো order করেনি।

Orders:

```
Rahim → Laptop
Karim → Mouse
```

আমরা চাই **সব customers**, order থাকুক বা না থাকুক।

তখন:

```sql
SELECT
    customers.customer_id,
    customers.name,
    orders.order_id,
    orders.product
FROM customers
LEFT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

1️⃣1️⃣ LEFT JOIN Result

customer_id	name	order_id	product
1	Rahim	O001	Laptop
2	Karim	O002	Mouse
3	Hasan	NULL	NULL

Hasan-এর order নেই।

তাই:

```
order_id = NULL
product = NULL
```

🧠 LEFT JOIN Mental Model

```
LEFT TABLE
    ↓
সব rows থাকবে
    +
RIGHT TABLE-এর matching rows
```

অর্থাৎ:

> **Left table-এর সব data রাখো।**

1️⃣2️⃣ RIGHT JOIN

RIGHT JOIN হলো LEFT JOIN-এর reverse concept।

```
SELECT
    customers.name,
    orders.order_id,
    orders.product
FROM customers
RIGHT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

এখানে:

> Right table-এর সব rows থাকবে।

1️⃣3️⃣ INNER vs LEFT vs RIGHT

JOIN	কী রাখে?
INNER JOIN	দুই table-এর matching rows
LEFT JOIN	Left table-এর সব rows + matching right rows
RIGHT JOIN	Right table-এর সব rows + matching left rows

Visual:

```
INNER JOIN

A ∩ B
```

```
LEFT JOIN

A + matching B
```

```
RIGHT JOIN

B + matching A
```

1️⃣4️⃣ LEFT JOIN কেন খুব Useful?

Question:

> কোন customers কোনো order করেনি?

```
SELECT
    customers.customer_id,
    customers.name
FROM customers
LEFT JOIN orders
    ON customers.customer_id = orders.customer_id
WHERE orders.order_id IS NULL;
```

Result:

```
Hasan
```

🔥 খুব common real-world query।

1️⃣5️⃣ JOIN + WHERE

Question:

> Dhaka-এর customers-এর orders দেখাও।

```
SELECT
    customers.name,
    customers.city,
    orders.product,
    orders.amount
FROM customers
INNER JOIN orders
    ON customers.customer_id = orders.customer_id
WHERE customers.city = 'Dhaka';
```

এখানে:

```
JOIN
 ↓
Related data
 ↓
WHERE
```

```
  ↓
Dhaka filter
```

1️⃣6️⃣ JOIN + ORDER BY

Highest order amount আগে:

```sql
SELECT
    customers.name,
    orders.product,
    orders.amount
FROM customers
INNER JOIN orders
    ON customers.customer_id = orders.customer_id
ORDER BY orders.amount DESC;
```

1️⃣7️⃣ JOIN + GROUP BY 🔥

এখন real analytics।

Question:

> প্রত্যেক customer মোট কত টাকা purchase করেছে?

```sql
SELECT
    customers.name,
    SUM(orders.amount) AS total_purchase
FROM customers
INNER JOIN orders
    ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name;
```

Result:

name	total_purchase
Rahim	52000
Karim	1000

1️⃣8️⃣ কেন GROUP BY-তে ID-ও দিলাম?

আমরা লিখেছি:

```
GROUP BY customers.customer_id, customers.name;
```

কারণ customer-এর unique identity হিসেবে ID ব্যবহার করা ভালো practice।

বিশেষ করে এমন database-এ যেখানে একই name-এর একাধিক customer থাকতে পারে।

1️⃣9️⃣ Customer Order Count

প্রতিটি customer কতটি order করেছে?

```
SELECT
    customers.name,
    COUNT(orders.order_id) AS order_count
FROM customers
LEFT JOIN orders
    ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name;
```

LEFT JOIN ব্যবহারের কারণে order না করা customer-ও result-এ থাকবে।

2️⃣0️⃣ `COUNT(*)` বনাম `COUNT(order_id)`

এটা খুব important।

LEFT JOIN-এর ক্ষেত্রে:

```
COUNT(*)
```

এবং

```
COUNT(orders.order_id)
```

এক জিনিস নয়।

Order না থাকা customer-এর জন্য:

```
orders.order_id = NULL
```

তাই:

```
COUNT(orders.order_id)
```

সাধারণত তার actual orders count করতে সাহায্য করে।

2️⃣1️⃣ Customers Without Orders

আরেকবার:

```
SELECT
    customers.name
FROM customers
LEFT JOIN orders
    ON customers.customer_id = orders.customer_id
WHERE orders.order_id IS NULL;
```

Mental model:

```
LEFT JOIN
    ↓
সব customer
    ↓
যাদের order নেই
    ↓
WHERE order_id IS NULL
```

2️⃣2️⃣ JOIN + HAVING

Question:

> যেসব customer 50,000-এর বেশি purchase করেছে।

```
SELECT
    customers.name,
    SUM(orders.amount) AS total_purchase
FROM customers
INNER JOIN orders
    ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name
HAVING SUM(orders.amount) > 50000;
```

এখানে:

```
WHERE
→ individual row filter

HAVING
→ grouped result filter
```

2️⃣3️⃣ তিনটি Table JOIN

Real database-এ দুইটির বেশি table join করা খুব common।

ধরি:

```
customers
products
orders
```

Orders:

```
order_id
customer_id
product_id
quantity
```

Products:

```
product_id
product_name
price
```

তাহলে:

```
SELECT
    customers.name,
    products.product_name,
    orders.quantity
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id
INNER JOIN products
    ON orders.product_id = products.product_id;
```

এখানে:

```
orders
   │
   ├──── customers
   │
   └──── products
```

2️⃣4️⃣ Three-Table Business Query

Question:

> Customer-এর নাম, product-এর নাম এবং order amount দেখাও।

```sql
SELECT
    customers.name AS customer_name,
    products.product_name,
    orders.amount
FROM orders
INNER JOIN customers
    ON orders.customer_id = customers.customer_id
INNER JOIN products
    ON orders.product_id = products.product_id;
```

2️⃣5️⃣ JOIN + Aggregate + GROUP BY

Question:

> Category-wise total sales কত?

ধরি products table-এ:

```
product_id
product_name
category
```

Query:

```sql
SELECT
    products.category,
    SUM(orders.amount) AS total_sales
FROM orders
```

```
INNER JOIN products
    ON orders.product_id = products.product_id
GROUP BY products.category;
```

🔥 এটা Data Analyst-এর real-world query।

2️⃣6️⃣ Excel-এর সাথে Connection

Excel-এ তোমার data যদি হয়:

```
Customers
Products
Sales
```

Power Pivot-এ relationship তৈরি করেছিলে।

SQL-এ সেই relationship-এর মাধ্যমে:

```
JOIN
```

ব্যবহার করি।

Excel → SQL Mapping

Excel / Power Pivot	SQL
Relationship	JOIN condition
Related tables	JOIN
Lookup	JOIN
PivotTable	GROUP BY
Slicer/filter	WHERE
Aggregate filter	HAVING
Dimension	Related descriptive table
Fact table	Transaction table

2️⃣7️⃣ JOIN-এর সবচেয়ে Important Rule

JOIN করার সময় নিজেকে প্রশ্ন করো:

> **কোন column দুই table-এর relationship তৈরি করছে?**

Example:

```
customers.customer_id
        ↕
orders.customer_id
```

তারপর:

```
ON customers.customer_id = orders.customer_id
```

2️⃣8️⃣ Common Beginner Mistake

ভুল:

```
SELECT *
FROM customers
JOIN orders;
```

Database বুঝবে না কীভাবে match করবে।

সাধারণত দরকার:

```
ON customers.customer_id = orders.customer_id
```

2️⃣9️⃣ Another Common Mistake — Wrong Relationship

ধরি:

```
customers.customer_id
products.product_id
```

এগুলো unrelated IDs।

তাই:

```
ON customers.customer_id = products.product_id
```

লেখা logically ভুল হতে পারে।

Relationship বুঝে JOIN করতে হবে।

3️⃣0️⃣ JOIN Summary

```
INNER JOIN
→ শুধু matching rows

LEFT JOIN
→ left-এর সব + matching right

RIGHT JOIN
→ right-এর সব + matching left
```

সবচেয়ে বেশি ব্যবহার করবে:

```
INNER JOIN
LEFT JOIN
```

🧠 Query Building Formula

Real-world JOIN query বানানোর সময়:

```
1. কোন data চাই?
        ↓
2. কোন table-এ আছে?
        ↓
3. কোন common key আছে?
        ↓
4. JOIN
        ↓
5. Filter দরকার?
        ↓
6. WHERE
        ↓
7. Summary দরকার?
        ↓
8. GROUP BY
        ↓
```

```
9. Group filter?
        ↓
10. HAVING
```

🎯 Class 35 Assignment

ধরি তিনটি table:

Customers

```
customer_id
name
city
```

Products

```
product_id
product_name
category
price
```

Orders

```
order_id
customer_id
product_id
quantity
amount
```

নিজে query লেখো:

Q1

Orders-এর সাথে customer name দেখাও।

Q2

Order-এর সাথে product name দেখাও।

Q3

Customer name + product name + amount দেখাও।

Q4

শুধু Dhaka customers-এর orders দেখাও।

Q5

প্রতিটি customer-এর total purchase বের করো।

Q6

প্রতিটি customer-এর order count বের করো।

Q7

যেসব customer কোনো order করেনি তাদের বের করো।

Q8

Category-wise total sales বের করো।

Q9

সবচেয়ে বেশি purchase করা customer বের করো।

Q10

50,000-এর বেশি purchase করা customers বের করো।

Q11

প্রতিটি product কত units sold হয়েছে বের করো।

Q12

প্রতিটি category-এর average order amount বের করো।

🧪 Class 35 Quiz

1. দুই table-এর related data combine করতে?

A. GROUP
B. JOIN
C. MERGE DATA
D. CONNECT

2. দুই table-এর matching rows?

A. INNER JOIN
B. LEFT JOIN

C. RIGHT JOIN
D. FULL JOIN

3. Left table-এর সব rows রাখতে?

A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. CROSS JOIN

4. JOIN-এর matching condition সাধারণত কোথায় লেখা হয়?

A. WHERE
B. ON
C. GROUP
D. MATCH

5. Primary key অন্য table-এ reference করলে সেটি কী?

A. Candidate Key
B. Foreign Key
C. Super Key
D. Alternate Key

6. Customer-এর order না থাকলেও customer দেখাতে চাইলে?

A. INNER JOIN
B. LEFT JOIN
C. RIGHT JOIN
D. None

7. Customer-wise total purchase বের করতে কোন combination useful?

A. JOIN + GROUP BY + SUM
B. JOIN + ORDER BY only
C. WHERE only
D. DISTINCT only

8. Group-এর result filter করতে?

A. WHERE
B. HAVING
C. ON
D. FILTER

9. `ON customers.id = orders.customer_id` **কী করছে?**

A. Sort
B. Join relationship define

C. Delete
D. Group

10. Real-world database-এ সবচেয়ে বেশি useful JOIN কোনগুলো?

A. INNER এবং LEFT
B. শুধু RIGHT
C. শুধু CROSS
D. শুধু FULL

⭐ Class 35 Cheat Sheet

```sql
-- INNER JOIN
SELECT *
FROM orders
INNER JOIN customers
ON orders.customer_id = customers.customer_id;
```

```sql
-- LEFT JOIN
SELECT *
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id;
```

```sql
-- Customer-wise total
SELECT
    customers.name,
    SUM(orders.amount) AS total_purchase
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name;
```

```sql
-- Customers without orders
SELECT customers.name
FROM customers
LEFT JOIN orders
ON customers.customer_id = orders.customer_id
WHERE orders.order_id IS NULL;
```

🔥 আজকের Golden Mental Model

```
PRIMARY KEY
     ↓
Unique identity

FOREIGN KEY
     ↓
Relationship

JOIN
     ↓
Combine related tables

GROUP BY
     ↓
Summarize

HAVING
     ↓
Filter summary
```

Next Class 36: আমরা JOIN আরও গভীরে যাব — **SELF JOIN, FULL OUTER JOIN concept, CROSS JOIN, multiple JOIN এবং JOIN-এর common mistakes**। এরপর আমরা **Subquery + CTE**-তে যাব, যা SQL interview এবং Data Analytics-এর জন্য খুব গুরুত্বপূর্ণ।

📘 SQL — Class 36

🔗 Advanced JOIN: SELF JOIN, FULL OUTER JOIN, CROSS JOIN & Multiple JOIN

Class 35-এ আমরা শিখেছি:

```
INNER JOIN
LEFT JOIN
RIGHT JOIN
PRIMARY KEY
FOREIGN KEY
ON
JOIN + GROUP BY
```

আজ JOIN-এর advanced concepts শিখব। এগুলো বুঝলে real-world relational database অনেক বেশি পরিষ্কার হবে।

🎯 আজকের Learning Goals

আজ শিখবে:

1. Multiple JOIN
2. SELF JOIN
3. FULL OUTER JOIN
4. CROSS JOIN
5. JOIN-এর common mistakes
6. JOIN + GROUP BY + HAVING
7. JOIN + aggregate
8. কখন কোন JOIN ব্যবহার করবে

1️⃣ Multiple JOIN

বাস্তব database-এ সাধারণত শুধু দুইটি table থাকে না।

ধরি:

```
customers
products
orders
```

Relationship:

```
customers
    |
    | customer_id
    ↓
orders
    |
    | product_id
    ↓
products
```

Example Tables

Customers

customer_id	name	city
1	Rahim	Dhaka
2	Karim	Chittagong
3	Hasan	Sylhet

Products

product_id	product_name	category	price
101	Laptop	Electronics	50000
102	Mouse	Accessories	1000
103	Monitor	Electronics	20000

Orders

order_id	customer_id	product_id	quantity
O001	1	101	2
O002	2	102	5
O003	1	103	1

2️⃣ Three Table JOIN

Question:

> কোন customer কোন product কিনেছে?

```
SELECT
    customers.name,
    products.product_name,
    orders.quantity
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
JOIN products
    ON orders.product_id = products.product_id;
```

Result:

name	product_name	quantity
Rahim	Laptop	2
Karim	Mouse	5
Rahim	Monitor	1

🧠 Query-টা কীভাবে চিন্তা করবে?

Start:

```
orders
```

তারপর:

```
orders → customers
```

তারপর:

```
orders → products
```

অর্থাৎ:

```
Orders
   │
   ├── Customer
   │
   └── Product
```

3️⃣ Multiple JOIN + Calculation

Question:

> প্রতিটি order-এর total price কত?

যেহেতু:

```
quantity × product price
```

তাই:

```sql
SELECT
    orders.order_id,
    customers.name,
    products.product_name,
    orders.quantity,
    products.price,
    orders.quantity * products.price AS total_amount
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
JOIN products
    ON orders.product_id = products.product_id;
```

Result:

order_id	name	product	qty	price	total_amount
O001	Rahim	Laptop	2	50000	100000
O002	Karim	Mouse	5	1000	5000
O003	Rahim	Monitor	1	20000	20000

🔥 এই ধরনের query Data Analytics-এ খুব common।

4️⃣ JOIN + GROUP BY

Question:

> Customer-wise total spending কত?

```sql
SELECT
    customers.customer_id,
    customers.name,
    SUM(orders.quantity * products.price) AS total_spending
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
JOIN products
    ON orders.product_id = products.product_id
GROUP BY
    customers.customer_id,
    customers.name;
```

Result:

customer	total_spending
Rahim	120000
Karim	5000

5️⃣ JOIN + GROUP BY + HAVING

Question:

> যেসব customer 50,000-এর বেশি spend করেছে তাদের দেখাও।

```
SELECT
    customers.customer_id,
    customers.name,
    SUM(orders.quantity * products.price) AS total_spending
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
JOIN products
    ON orders.product_id = products.product_id
GROUP BY
    customers.customer_id,
    customers.name
HAVING SUM(orders.quantity * products.price) > 50000;
```

এখানে:

```
JOIN
 ↓
Related data
 ↓
GROUP BY
 ↓
Customer-wise summary
 ↓
HAVING
 ↓
High-value customers
```

6️⃣ SELF JOIN 🔥

এখন একটু interesting concept।

SELF JOIN = একই table-কে নিজের সাথে JOIN করা।

কেন দরকার?

ধরো employee table:

id	name	manager_id
1	Rahim	NULL
2	Karim	1
3	Hasan	1
4	Jamil	2

এখানে:

```
Rahim
 ├── Karim
 └── Hasan

Karim
 └── Jamil
```

`manager_id` employee table-এরই অন্য employee-এর `id` reference করছে।

7️⃣ SELF JOIN Query

Question:

> Employee-এর নাম এবং তার manager-এর নাম দেখাও।

```sql
SELECT
    e.name AS employee,
    m.name AS manager
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.id;
```

এখানে:

```
e = employee
m = manager
```

দুটোই একই table-এর alias।

Result:

employee	manager
Rahim	NULL
Karim	Rahim
Hasan	Rahim
Jamil	Karim

8️⃣ Alias কেন ব্যবহার করলাম?

একই table দুইবার ব্যবহার করছি:

```
employees e
```

এবং:

```
employees m
```

এগুলো alias।

তাই:

```
e.name
```

মানে employee side-এর name।

আর:

```
m.name
```

মানে manager side-এর name।

🧠 SELF JOIN Mental Model

```
employees
   │
   ├── e = employee
```

```
    │
    └── m = manager
```

একই table:

```
employees → employees
```

9️⃣ SELF JOIN কোথায় ব্যবহার হয়?

Common examples:

- Employee → Manager
- Category → Parent Category
- Location → Parent Location
- Organization hierarchy
- Referral relationship
- Family tree type data

🔟 FULL OUTER JOIN

Concept:

> দুই table-এর **সব rows** রাখবে।

ধরি:

Table A:

```
1
2
3
```

Table B:

```
2
3
4
```

FULL OUTER JOIN result:

```
1
2
3
4
```

Matching হলে combine করবে।

1️⃣1️⃣ Important: MySQL

MySQL-এ native:

```
FULL OUTER JOIN
```

সরাসরি supported নয়।

PostgreSQL-এ:

```
FULL OUTER JOIN
```

supported।

1️⃣2️⃣ FULL OUTER JOIN Concept

PostgreSQL example:

```
SELECT
    customers.customer_id,
    customers.name,
    orders.order_id
FROM customers
FULL OUTER JOIN orders
    ON customers.customer_id = orders.customer_id;
```

এতে:

- order আছে এমন customer
- order নেই এমন customer
- customer-এর match নেই এমন order

সব side-এর rows preserve করার চেষ্টা করবে।

1️⃣3️⃣ CROSS JOIN

CROSS JOIN = প্রতিটি row-এর সাথে অন্য table-এর প্রতিটি row combine করে।

ধরি:

Table A:

```
Red
Blue
```

Table B:

```
Small
Large
```

CROSS JOIN:

```
Red   Small
Red   Large
Blue  Small
Blue  Large
```

Total:

```
2 × 2 = 4 rows
```

1️⃣4️⃣ CROSS JOIN Query

```sql
SELECT
    colors.color,
    sizes.size
FROM colors
CROSS JOIN sizes;
```

এখানে `ON` condition লাগে না।

1️⃣5️⃣ CROSS JOIN-এর Practical Use

ধরো:

Products:

```
Laptop
Mouse
```

Sizes:

```
Small
Large
```

সব possible combination:

```
Laptop → Small
Laptop → Large
Mouse  → Small
Mouse  → Large
```

Product variants generate করতে কাজে লাগতে পারে।

⚠️ কিন্তু বড় table-এর ক্ষেত্রে rows খুব দ্রুত বেড়ে যেতে পারে।

যেমন:

```
1000 × 1000
= 1,000,000 rows
```

তাই CROSS JOIN সাবধানে ব্যবহার করতে হয়।

1️⃣6️⃣ JOIN Types — Complete Picture

JOIN	Result
INNER JOIN	Matching rows
LEFT JOIN	Left-এর সব + matching right
RIGHT JOIN	Right-এর সব + matching left
FULL OUTER JOIN	দুই side-এর সব rows
CROSS JOIN	Every combination
SELF JOIN	Same table-এর সাথে নিজেই JOIN

1️⃣7️⃣ কোন JOIN কখন?

INNER JOIN

যখন শুধু matching data দরকার।

```
Customers + Orders
```

যাদের order আছে।

LEFT JOIN

যখন left table-এর **সব records** রাখতে হবে।

Example:

> সব customers, তাদের order থাকুক বা না থাকুক।

RIGHT JOIN

যখন right table-এর সব records রাখতে হবে।

তবে অনেক developer LEFT JOIN-কে বেশি prefer করে, কারণ table order বদলে একই logic clearer করা যায়।

FULL OUTER JOIN

যখন দুই side-এর unmatched data-ও দরকার।

SELF JOIN

যখন table-এর ভিতরেই hierarchy/relationship আছে।

CROSS JOIN

যখন intentionalভাবে সব possible combinations দরকার।

1️⃣8️⃣ JOIN-এর Common Mistake — Missing ON

ভুল:

```
SELECT *
FROM customers
JOIN orders;
```

সঠিক:

```
SELECT *
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id;
```

1️⃣9️⃣ Common Mistake — Wrong Column

ধরি:

```
customers.customer_id
orders.customer_id
```

তাহলে:

```
ON customers.customer_id = orders.customer_id
```

ঠিক।

কিন্তু:

```
ON customers.customer_id = orders.order_id
```

সাধারণত ভুল relationship।

2️⃣0️⃣ Common Mistake — Duplicate Rows

ধরো একজন customer-এর 10টি order আছে।

Customer-কে Orders-এর সাথে JOIN করলে customer-এর name **10 rows-এ দেখা যেতে পারে**।

এটা error নয়।

কারণ relationship:

```
1 Customer
   ↓
10 Orders
```

তাই result-এ 10 matching rows আসবে।

2️⃣1️⃣ JOIN করলে Row কেন বাড়ে?

ধরো:

Customers:

```
Rahim
```

Orders:

```
0001
0002
0003
```

JOIN:

```
Rahim | 0001
Rahim | 0002
Rahim | 0003
```

একজন customer তিনটি order করেছে।

তাই তিনটি result row।

2️⃣2️⃣ JOIN + DISTINCT

যদি শুধু unique customers চাই:

```
SELECT DISTINCT
    customers.customer_id,
    customers.name
FROM customers
```

```
JOIN orders
    ON customers.customer_id = orders.customer_id;
```

এতে একই customer-এর multiple order থাকলেও customer একবার আসবে।

2️⃣3️⃣ JOIN + WHERE + GROUP BY + HAVING

একটি full real-world example:

> Dhaka customers-এর মধ্যে যেসব customer 50,000-এর বেশি spend করেছে তাদের নাম ও total spending দেখাও।

```
SELECT
    customers.customer_id,
    customers.name,
    SUM(orders.quantity * products.price) AS total_spending
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
JOIN products
    ON orders.product_id = products.product_id
WHERE customers.city = 'Dhaka'
GROUP BY
    customers.customer_id,
    customers.name
HAVING SUM(orders.quantity * products.price) > 50000
ORDER BY total_spending DESC;
```

🔥 এই একটি query-তে:

```
JOIN
WHERE
GROUP BY
SUM
HAVING
ORDER BY
```

সব ব্যবহার হয়েছে।

2️⃣4️⃣ SQL Query Building Strategy

কোনো complex question দেখলে একসাথে query লিখতে যেও না।

এই order follow করো:

Step 1 — কোন table লাগবে?

```
customers
orders
products
```

Step 2 — Relationship কী?

```
customers.customer_id
        ↓
orders.customer_id

products.product_id
        ↓
orders.product_id
```

Step 3 — JOIN

Step 4 — Filter হলে WHERE

Step 5 — Calculation হলে SUM/AVG/COUNT

Step 6 — Group হলে GROUP BY

Step 7 — Group filter হলে HAVING

Step 8 — Sort হলে ORDER BY

Step 9 — Top N হলে LIMIT

🧠 Full SQL Mental Model

```
Tables
  ↓
JOIN
  ↓
WHERE
  ↓
GROUP BY
  ↓
HAVING
  ↓
SELECT
```

```
  ↓
ORDER BY
  ↓
LIMIT
```

এটা conceptual processing order-এর simplified learning model; SQL engine internally আরও sophisticatedভাবে optimize করতে পারে।

🎯 Class 36 Assignment

ধরি:

Employees

```
id
name
manager_id
department
salary
```

Customers

```
customer_id
name
city
```

Products

```
product_id
product_name
category
price
```

Orders

```
order_id
customer_id
product_id
quantity
```

নিজে query লেখো:

Q1

Customer + order information দেখাও।

Q2

Customer + product + quantity দেখাও।

Q3

Customer-এর total spending বের করো।

Q4

Category-wise total sales বের করো।

Q5

যেসব customer 50,000-এর বেশি spend করেছে।

Q6

যেসব customer কোনো order করেনি।

Q7

প্রতিটি product কত units sold হয়েছে।

Q8

সবচেয়ে বেশি units sold হওয়া product।

Q9

Employee এবং Manager-এর নাম দেখাও।
Hint: SELF JOIN

Q10

শুধু যেসব employee-এর manager আছে তাদের দেখাও।

Q11

Unique customers যারা অন্তত একটি order করেছে।

Q12

Dhaka customers-এর category-wise sales।

🧪 Class 36 Quiz

1. Same table-এর সাথে নিজেকেই JOIN করলে?

A. INNER JOIN
B. SELF JOIN
C. CROSS JOIN
D. FULL JOIN

2. Every possible combination কোন JOIN তৈরি করে?

A. LEFT
B. INNER
C. CROSS
D. SELF

3. FULL OUTER JOIN-এর মূল উদ্দেশ্য?

A. শুধু matching rows
B. শুধু left rows
C. দুই side-এর সব rows রাখা
D. সব rows delete

4. MySQL-এ native FULL OUTER JOIN আছে?

A. Yes
B. No

5. Employee-manager relationship-এর জন্য কোন JOIN useful?

A. SELF JOIN
B. CROSS JOIN
C. FULL JOIN
D. None

6. JOIN-এর relationship condition সাধারণত?

A. ON
B. HAVING
C. ORDER
D. LIMIT

7. একজন customer-এর 5 orders থাকলে normal JOIN result-এ customer কতবার আসতে পারে?

A. 1
B. 2
C. 5
D. 0

8. Unique customer list বের করতে?

A. DISTINCT
B. UNIQUE JOIN
C. ONLY
D. SINGLE

9. Product + Customer + Orders-এর জন্য?

A. Multiple JOIN
B. শুধু WHERE
C. শুধু GROUP BY
D. শুধু DISTINCT

10. বড় table-এর উপর CROSS JOIN কেন risky?

A. Data delete হয়
B. Result rows খুব দ্রুত বেড়ে যেতে পারে
C. Primary key automatically change হয়
D. JOIN কাজ করে না

⭐ Class 36 Cheat Sheet

```sql
-- Multiple JOIN
SELECT ...
FROM orders
JOIN customers
ON orders.customer_id = customers.customer_id
JOIN products
ON orders.product_id = products.product_id;
```

```sql
-- SELF JOIN
SELECT
    e.name AS employee,
    m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

```sql
-- CROSS JOIN
SELECT *
FROM colors
CROSS JOIN sizes;
```

```sql
-- JOIN + GROUP BY
SELECT
    customers.name,
    SUM(orders.amount) AS total
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name;
```

```sql
-- JOIN + HAVING
SELECT
    customers.name,
    SUM(orders.amount) AS total
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.name
HAVING SUM(orders.amount) > 50000;
```

🔥 আজকের Golden Mental Model

```
PRIMARY KEY
      ↓
FOREIGN KEY
      ↓
RELATIONSHIP
      ↓
JOIN
      ↓
GROUP BY
      ↓
HAVING
      ↓
BUSINESS INSIGHT
```

পরের ধাপে আমরা **Class 37: Subquery** শুরু করব—যেখানে একটি SQL query-এর result আরেকটি query-এর ভিতরে ব্যবহার করব। তারপর **CTE (`WITH`)**, Window Functions এবং Advanced SQL Analytics-এর দিকে যাব।

📘 SQL — Class 37

🧩 Subquery — Query-এর ভিতরে Query

Class 35–36-এ আমরা JOIN শিখেছি। আজ আমরা SQL-এর আরেকটি powerful concept শিখব:

> **Subquery = একটি SQL query-এর ভিতরে আরেকটি SQL query।**

এটা Data Analyst, Database Designer এবং SQL interview—তিন জায়গাতেই গুরুত্বপূর্ণ।

🎯 আজকের Learning Goals

আজ শিখবে:

1. Subquery কী
2. Single-row Subquery
3. Multiple-row Subquery
4. `IN` + Subquery
5. `>` `<` `=` + Subquery
6. Subquery with `AVG()`, `MAX()`, `MIN()`
7. Correlated Subquery-এর ধারণা
8. Subquery বনাম JOIN
9. Real-world business questions

1️⃣ Subquery কী?

সাধারণ query:

```sql
SELECT *
FROM employees;
```

Subquery হলে:

```sql
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

এখানে:

```sql
SELECT AVG(salary)
FROM employees
```

হলো **inner query / subquery**।

আর:

```
SELECT *
FROM employees
WHERE salary > (...);
```

হলো **outer query**।

2️⃣ সহজ Example

ধরি salaries:

Name	Salary
Rahim	40000
Karim	50000
Hasan	60000
Jamil	80000

Average:

```
57500
```

Question:

> Average salary-এর চেয়ে বেশি salary যাদের, তাদের দেখাও।

প্রথমে average বের করা যায়:

```
SELECT AVG(salary)
FROM employees;
```

তারপর:

```
SELECT *
FROM employees
WHERE salary > 57500;
```

কিন্তু manually `57500` লিখতে হবে না।

Subquery:

```sql
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

🔥 এটাই Subquery।

3️⃣ Query কীভাবে কাজ করছে?

ভেতরের query:

```sql
SELECT AVG(salary)
FROM employees;
```

ধরি result:

```
57500
```

তখন outer query conceptually হয়ে যায়:

```sql
SELECT *
FROM employees
WHERE salary > 57500;
```

4️⃣ Subquery-এর Basic Structure

```sql
SELECT ...
FROM ...
WHERE column operator (
    SELECT ...
    FROM ...
);
```

যেমন:

```sql
SELECT *
FROM employees
WHERE salary > (
```

```
    SELECT AVG(salary)
    FROM employees
);
```

5️⃣ Subquery + MAX()

Question:

> সবচেয়ে বেশি salary পাওয়া employee দেখাও।

প্রথমে:

```
SELECT MAX(salary)
FROM employees;
```

তারপর:

```
SELECT *
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

এখানে subquery maximum salary বের করছে।

6️⃣ Subquery + MIN()

সবচেয়ে কম salary:

```
SELECT *
FROM employees
WHERE salary = (
    SELECT MIN(salary)
    FROM employees
);
```

7️⃣ Subquery + COUNT()

Question:

> যেসব department-এ 3-এর বেশি employee আছে, তাদের department দেখাও।

একভাবে `GROUP BY + HAVING` দিয়ে করা যায়:

```
SELECT department
FROM employees
GROUP BY department
HAVING COUNT(*) > 3;
```

Subquery দিয়ে আরও complex situations-এ similar logic তৈরি করা যায়।

তবে **শুধু সহজ aggregation-এর জন্য অকারণে subquery ব্যবহার করা উচিত নয়**।

8️⃣ Multiple-Row Subquery

এখন গুরুত্বপূর্ণ বিষয়।

ধরি:

```
SELECT department
FROM employees
WHERE city = 'Dhaka';
```

এটা একাধিক department return করতে পারে:

```
IT
HR
Finance
```

এখন যদি query লিখি:

```
SELECT *
FROM employees
WHERE department = (
    SELECT department
    FROM employees
    WHERE city = 'Dhaka'
);
```

সমস্যা হতে পারে, কারণ `=` সাধারণত একটি single value expect করে।

এক্ষেত্রে `IN` ব্যবহার করা যায়।

9️⃣ IN + Subquery 🔥

```sql
SELECT *
FROM employees
WHERE department IN (
    SELECT department
    FROM employees
    WHERE city = 'Dhaka'
);
```

ভেতরের query:

```sql
SELECT department
FROM employees
WHERE city = 'Dhaka';
```

ধরি result:

```
IT
HR
Finance
```

Outer query:

```sql
department IN ('IT', 'HR', 'Finance')
```

🔟 Real Example — Customer Orders

ধরি:

Customers

customer_id	name	city
1	Rahim	Dhaka
2	Karim	Chittagong
3	Hasan	Dhaka

Orders

order_id	customer_id	amount
O001	1	50000
O002	2	1000
O003	3	70000

Question:

> Dhaka-এর customers-এর orders দেখাও।

Subquery:

```
SELECT *
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

1️⃣1️⃣ এখানে কী হচ্ছে?

Inner query:

```
SELECT customer_id
FROM customers
WHERE city = 'Dhaka';
```

Result:

```
1
3
```

Outer query:

```
SELECT *
FROM orders
WHERE customer_id IN (1, 3);
```

Result:

```
0001
0003
```

1️⃣2️⃣ Subquery + `>`

Question:

> Average order amount-এর চেয়ে বেশি orders দেখাও।

```sql
SELECT *
FROM orders
WHERE amount > (
    SELECT AVG(amount)
    FROM orders
);
```

1️⃣3️⃣ Subquery + `<`

Average-এর চেয়ে কম:

```sql
SELECT *
FROM orders
WHERE amount < (
    SELECT AVG(amount)
    FROM orders
);
```

1️⃣4️⃣ Subquery + `>=`

Average-এর সমান বা বেশি:

```sql
SELECT *
FROM employees
WHERE salary >= (
    SELECT AVG(salary)
    FROM employees
);
```

1️⃣5️⃣ Subquery + `NOT IN`

Dhaka customers ছাড়া অন্য customers-এর orders:

```sql
SELECT *
FROM orders
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

⚠️ `NOT IN` এবং `NULL` একসাথে থাকলে unexpected behavior হতে পারে। Real SQL-এ NULL handling সম্পর্কে সতর্ক থাকতে হয়।

1️⃣6️⃣ Subquery in SELECT

Subquery শুধু `WHERE`-এর মধ্যে নয়, `SELECT`-এর মধ্যেও থাকতে পারে।

Example:

```sql
SELECT
    name,
    salary,
    (SELECT AVG(salary) FROM employees) AS average_salary
FROM employees;
```

Result conceptually:

name	salary	average_salary
Rahim	40000	57500
Karim	50000	57500
Hasan	60000	57500
Jamil	80000	57500

1️⃣7️⃣ Salary Difference

এখন:

> প্রত্যেক employee-এর salary এবং company average salary-এর difference দেখাও।

```sql
SELECT
    name,
    salary,
    salary - (
        SELECT AVG(salary)
        FROM employees
    ) AS difference_from_average
FROM employees;
```

Result conceptually:

name	salary	difference
Rahim	40000	-17500
Karim	50000	-7500
Hasan	60000	2500
Jamil	80000	22500

🔥 এটা analytics-এ useful।

1️⃣8️⃣ Subquery in FROM

Subquery-কে temporary table-এর মতোও ব্যবহার করা যায়।

Example:

```sql
SELECT *
FROM (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_summary;
```

ভেতরের query:

```sql
SELECT
    department,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

এটা temporary result তৈরি করে।

তারপর outer query সেটার উপর কাজ করে।

1️⃣9️⃣ Subquery in FROM — Filter

ধরি department-wise average salary বের করলাম।

তারপর শুধু average salary > 50,000 departments চাই:

```
SELECT *
FROM (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_summary
WHERE avg_salary > 50000;
```

এখানে খুব সুন্দর pattern:

```
employees
   ↓
GROUP BY department
   ↓
average salary
   ↓
temporary result
   ↓
WHERE avg_salary > 50000
```

2️⃣0️⃣ Correlated Subquery 🔥

এটা একটু advanced।

সাধারণ subquery:

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
```

```
    FROM employees
);
```

Inner query independent।

কিন্তু **Correlated Subquery**-তে inner query outer query-এর current row-এর উপর depend করে।

Example:

> প্রত্যেক employee-এর department-এর average salary-এর চেয়ে বেশি salary যাদের, তাদের দেখাও।

```
SELECT
    e.name,
    e.department,
    e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

2️⃣1️⃣ এই Query কী করছে?

ধরি:

```
IT
Rahim   45000
Hasan   60000

HR
Karim   55000
Nabil   50000
```

Rahim-এর জন্য inner query:

```
IT average
```

Hasan-এর জন্যও:

```
IT average
```

Karim-এর জন্য:

```
HR average
```

অর্থাৎ inner query outer row-এর department অনুযায়ী change হচ্ছে।

🧠 Correlated Subquery Mental Model

```
Outer row
    ↓
Inner query
    ↓
Current row-এর সাথে related calculation
    ↓
Next outer row
    ↓
Inner query আবার
```

এজন্য correlated subquery অনেক ক্ষেত্রে বেশি expensive হতে পারে।

2️⃣2️⃣ Subquery বনাম JOIN

একই problem দুইভাবে solve করা যায়।

Subquery

```sql
SELECT *
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

JOIN

```sql
SELECT orders.*
FROM orders
JOIN customers
    ON orders.customer_id = customers.customer_id
WHERE customers.city = 'Dhaka';
```

দুটোই valid হতে পারে।

2️⃣3️⃣ কখন JOIN?

যখন:

> দুই বা তার বেশি table-এর columns একসাথে output-এ দরকার।

Example:

```
Customer Name
Product Name
Order Amount
```

তখন JOIN natural choice।

2️⃣4️⃣ কখন Subquery?

যখন:

> একটি query-এর result আরেকটি query-এর condition বা calculation হিসেবে দরকার।

Example:

> Average salary-এর চেয়ে বেশি salary।

```
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
)
```

2️⃣5️⃣ JOIN vs Subquery — Simple Rule

```
Need columns from another table?
        ↓
JOIN

Need result of another query?
        ↓
Subquery
```

তবে একই problem দুইভাবে solve করা সম্ভব হতে পারে। Performance ও readability অনুযায়ী choice করতে হয়।

2️⃣6️⃣ Business Question → Subquery

Question 1

> Average salary-এর বেশি salary কার?

```sql
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Question 2

> Highest salary কার?

```sql
SELECT name, salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

Question 3

> Lowest salary কার?

```sql
SELECT name, salary
FROM employees
WHERE salary = (
    SELECT MIN(salary)
    FROM employees
);
```

Question 4

> Dhaka customers-এর orders দেখাও।

```sql
SELECT *
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

Question 5

> Company average-এর চেয়ে বেশি salary পাওয়া employees-এর count কত?

```sql
SELECT COUNT(*)
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

2️⃣7️⃣ Important: `=` বনাম `IN`

Inner query এক value দিলে:

```sql
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
)
```

`AVG()` → একটি value।

তাই `>`, `<`, `=` ইত্যাদি ব্যবহার করা যায়।

Inner query multiple values দিলে:

```sql
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
)
```

এখানে multiple IDs আসতে পারে।

তাই `IN` useful।

2️⃣8️⃣ Common Mistakes

Mistake 1 — Multiple rows-এর সাথে `=`

ভুল হতে পারে:

```
WHERE department = (
    SELECT department
    FROM employees
    WHERE city = 'Dhaka'
);
```

যদি inner query multiple departments দেয়।

Better:

```
WHERE department IN (
    SELECT department
    FROM employees
    WHERE city = 'Dhaka'
);
```

Mistake 2 — Subquery-তে wrong column

যদি outer query:

```
customer_id
```

চায়, inner query-তে সেই compatible value দিতে হবে।

Mistake 3 — NULL with NOT IN

`NOT IN` এবং NULL থাকলে SQL three-valued logic-এর কারণে unexpected result হতে পারে।

এই ধরনের case-এ `NOT EXISTS` অনেক সময় safer pattern। এটা আমরা পরে শিখব।

2️⃣9️⃣ SQL Subquery Cheat Sheet

```sql
-- Above average
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

```sql
-- Maximum
SELECT *
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

```sql
-- Minimum
SELECT *
FROM employees
WHERE salary = (
    SELECT MIN(salary)
    FROM employees
);
```

```sql
-- Multiple values
SELECT *
FROM orders
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

```sql
-- Subquery in FROM
SELECT *
FROM (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_summary;
```

```sql
-- Correlated Subquery
SELECT e.name, e.department, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

🎯 Class 37 Assignment

নিজে query লেখো।

Q1

Average salary-এর চেয়ে বেশি salary পাওয়া employees।

Q2

Average salary-এর চেয়ে কম salary পাওয়া employees।

Q3

Highest salary পাওয়া employee।

Q4

Lowest salary পাওয়া employee।

Q5

Average salary-এর বেশি salary পাওয়া employee-এর count।

Q6

Dhaka customers-এর orders।

Q7

Dhaka customers ছাড়া অন্য customers-এর orders।

Q8

প্রতিটি employee-এর salary এবং company average salary দেখাও।

Q9

প্রতিটি employee-এর salary company average-এর তুলনায় কত বেশি/কম তা বের করো।

Q10

প্রতিটি department-এর average salary বের করে শুধু যেসব department-এর average > 50,000 সেগুলো দেখাও।

Q11 — ⭐ Advanced

Employee-এর department average salary-এর চেয়ে বেশি salary যাদের, তাদের বের করো।

Q12 — ⭐ Advanced

প্রতিটি customer-এর total spending বের করে company-wide average customer spending-এর চেয়ে বেশি spending করা customers বের করার চেষ্টা করো।

🧪 Class 37 Quiz

1. Query-এর ভিতরে query কী?

A. JOIN
B. Subquery
C. Function
D. Trigger

2. Average salary-এর চেয়ে বেশি salary বের করতে কোন pattern?

A. `salary > AVG()`
B. `salary > (SELECT AVG(...))`
C. `salary = GROUP BY`
D. `salary HAVING AVG`

3. Inner query একটি value return করলে কোন operator ব্যবহার করা যায়?

A. `=`
B. `>`
C. `<`
D. সবগুলো

4. Inner query multiple values return করলে কোনটি useful?

A. IN
B. =
C. ONLY
D. SAME

5. `MAX(salary)` সাধারণত কয়টি value return করে?

A. অনেক
B. একটি
C. শূন্য সবসময়
D. table-এর সমান

6. Correlated subquery-এর inner query কীসের উপর depend করে?

A. Database name
B. Outer query-এর current row
C. Table name only
D. ORDER BY

7. অন্য table-এর columns output-এ আনতে কোনটি সাধারণত natural?

A. JOIN
B. LIMIT
C. HAVING
D. DISTINCT

8. Subquery `FROM`-এর মধ্যে থাকলে তাকে কীভাবে ভাবা যায়?

A. Temporary result/table
B. Primary key
C. Foreign key
D. Index

9. `IN (subquery)` কেন ব্যবহার করা হয়?

A. Multiple possible values match করতে
B. Sorting করতে
C. Delete করতে
D. Table create করতে

10. `NOT IN` + NULL-এর ক্ষেত্রে কী হতে পারে?

A. কোনো সমস্যা নেই সবসময়
B. Unexpected/unknown results হতে পারে
C. Automatically NULL delete হয়
D. Table drop হয়

⭐ আজকের Golden Mental Model

```
SUBQUERY
   ↓
একটি query
```

```
    ↓
আরেকটি query-এর result তৈরি করে
    ↓
Outer query সেই result ব্যবহার করে
```

সবচেয়ে গুরুত্বপূর্ণ তিনটি pattern:

```sql
-- 1. Single value
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

```sql
-- 2. Multiple values
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE city = 'Dhaka'
);
```

```sql
-- 3. Correlated
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

🧠 মনে রাখো:

JOIN = related tables combine করার tool

Subquery = একটি query-এর result অন্য query-তে ব্যবহার করার tool

GROUP BY = data summarize করার tool

HAVING = summarized group filter করার tool

Next Class 38: CTE (`WITH`) + Subquery-এর cleaner alternative — complex SQL query কীভাবে ছোট ছোট logical steps-এ লেখা যায়, এবং `WITH` ব্যবহার করে readable, professional SQL তৈরি করা যায়।

📘 SQL — Class 38

🧩 CTE (`WITH`) — Clean & Professional SQL

Class 37-এ আমরা **Subquery** শিখেছি। আজ শিখব **CTE — Common Table Expression**।

CTE মূলত complex query-কে ছোট ছোট logical অংশে ভাগ করে **readable, maintainable এবং professional** করে।

🎯 আজকের Learning Goals

আজ শিখবে:

1. CTE কী
2. `WITH` syntax
3. Simple CTE
4. CTE + WHERE
5. CTE + GROUP BY
6. Multiple CTE
7. CTE + JOIN
8. CTE বনাম Subquery
9. CTE দিয়ে real-world analytics
10. Recursive CTE-এর introduction

1️⃣ CTE কী?

CTE = Common Table Expression

সহজ ভাষায়:

> একটি query-এর result-কে temporary named result হিসেবে তৈরি করে, তারপর পরের query-তে ব্যবহার করা।

Basic structure:

```
WITH cte_name AS (
    SELECT ...
    FROM ...
)
SELECT *
FROM cte_name;
```

এখানে:

```
WITH
 ↓
```

```
CTE তৈরি
 ↓
তারপর CTE ব্যবহার
```

2️⃣ সবচেয়ে সহজ Example

ধরি আমাদের employees table:

id	name	department	salary
1	Rahim	IT	45000
2	Karim	HR	55000
3	Hasan	IT	60000
4	Jamil	Finance	70000

Question:

> যাদের salary 50,000-এর বেশি তাদের দেখাও।

CTE:

```sql
WITH high_salary AS (
    SELECT *
    FROM employees
    WHERE salary > 50000
)
SELECT *
FROM high_salary;
```

Result:

id	name	department	salary
2	Karim	HR	55000
3	Hasan	IT	60000
4	Jamil	Finance	70000

3️⃣ CTE-এর Structure

```sql
WITH high_salary AS (
    SELECT *
    FROM employees
    WHERE salary > 50000
)
SELECT *
FROM high_salary;
```

ভেঙে দেখি:

Part 1

```sql
WITH high_salary AS (...)
```

নাম দিলাম:

```sql
high_salary
```

Part 2

```sql
(
    SELECT *
    FROM employees
    WHERE salary > 50000
)
```

এই query result তৈরি করছে।

Part 3

```sql
SELECT *
FROM high_salary;
```

এখন সেই result ব্যবহার করছি।

4️⃣ CTE কেন দরকার?

ধরি complex query:

```sql
SELECT *
FROM (
    SELECT
```

```
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_summary
WHERE avg_salary > 50000;
```

এটা কাজ করবে।

কিন্তু CTE version:

```
WITH dept_summary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_summary
WHERE avg_salary > 50000;
```

দ্বিতীয়টা অনেক easier to read।

5️⃣ CTE + GROUP BY

Question:

> Department-wise average salary বের করো।

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_salary;
```

Result:

department	avg_salary
IT	52500

department	avg_salary
HR	55000
Finance	70000

6️⃣ CTE Result-এর উপর Filter

এখন চাই:

> যেসব department-এর average salary 50,000-এর বেশি।

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_salary
WHERE avg_salary > 50000;
```

এখানে খুব clear workflow:

```
employees
   ↓
GROUP BY department
   ↓
AVG salary
   ↓
dept_salary CTE
   ↓
WHERE avg_salary > 50000
```

7️⃣ CTE + ORDER BY

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
```

```
)
SELECT *
FROM dept_salary
ORDER BY avg_salary DESC;
```

Highest average salary আগে আসবে।

8️⃣ CTE + LIMIT

Top 2 departments:

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_salary
ORDER BY avg_salary DESC
LIMIT 2;
```

9️⃣ Multiple CTE 🔥

একটি query-তে একাধিক CTE থাকতে পারে।

ধরি আমরা চাই:

1. Department-wise average salary
2. Department-wise employee count

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
),
dept_count AS (
    SELECT
        department,
        COUNT(*) AS employee_count
```

```
    FROM employees
    GROUP BY department
)
SELECT
    dept_salary.department,
    dept_salary.avg_salary,
    dept_count.employee_count
FROM dept_salary
JOIN dept_count
    ON dept_salary.department = dept_count.department;
```

Result:

department	avg_salary	employee_count
IT	52500	2
HR	55000	1
Finance	70000	1

🔟 Multiple CTE Mental Model

```
employees
   │
   ├──→ dept_salary
   │
   └──→ dept_count
          │
          ↓
        JOIN
          ↓
      Final Result
```

এটাই complex SQL organize করার একটি powerful technique।

1️⃣1️⃣ CTE + JOIN

ধরি:

customers

```
customer_id
name
city
```

orders

```
order_id
customer_id
amount
```

Question:

> Customer-wise total spending বের করো।

```sql
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(amount) AS total_spending
    FROM orders
    GROUP BY customer_id
)
SELECT
    customers.name,
    customers.city,
    customer_sales.total_spending
FROM customers
JOIN customer_sales
    ON customers.customer_id = customer_sales.customer_id;
```

1️⃣2️⃣ CTE + JOIN + HAVING Equivalent

Question:

> যেসব customer 50,000-এর বেশি spend করেছে তাদের দেখাও।

```sql
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(amount) AS total_spending
    FROM orders
    GROUP BY customer_id
)
SELECT
    customers.name,
```

```
    customer_sales.total_spending
FROM customers
JOIN customer_sales
    ON customers.customer_id = customer_sales.customer_id
WHERE customer_sales.total_spending > 50000;
```

এখানে aggregation আগে CTE-তে হয়ে গেছে।

1️⃣3️⃣ CTE বনাম Subquery

একই problem দুইভাবে করা যায়।

Subquery

```
SELECT *
FROM (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
) AS dept_summary
WHERE avg_salary > 50000;
```

CTE

```
WITH dept_summary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_summary
WHERE avg_salary > 50000;
```

Difference

Subquery	CTE
Query-এর ভিতরে থাকে	Query-এর শুরুতে named result
ছোট query-তে convenient	Complex query-তে readable
Nested হলে difficult হতে পারে	Multiple steps সহজ

Subquery	CTE
Reuse সীমিত	একই CTE query-এর মধ্যে reuse করা সহজ

1️⃣4️⃣ CTE কেন Professional SQL-এ Useful?

ধরি তোমার query 50 lines।

সব একসাথে লিখলে:

```
SELECT
JOIN
JOIN
WHERE
GROUP BY
CASE
SUBQUERY
...
```

বোঝা কঠিন।

CTE ব্যবহার করলে:

```
Step 1 → clean data
Step 2 → aggregate
Step 3 → calculate
Step 4 → filter
Step 5 → final result
```

এতে debugging-ও সহজ হয়।

1️⃣5️⃣ Real Data Analytics Example 🔥

Question:

> কোন customer-এর total purchase 50,000-এর বেশি এবং তাদের city কী?

```sql
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(amount) AS total_purchase
    FROM orders
    GROUP BY customer_id
```

```
)
SELECT
    c.name,
    c.city,
    cs.total_purchase
FROM customers c
JOIN customer_sales cs
    ON c.customer_id = cs.customer_id
WHERE cs.total_purchase > 50000
ORDER BY cs.total_purchase DESC;
```

এখানে:

```
Orders
 ↓
Customer-wise SUM
 ↓
customer_sales CTE
 ↓
JOIN Customers
 ↓
Filter > 50000
 ↓
Sort
```

🔥 এটা real-world analytics query-এর খুব common structure।

1️⃣6️⃣ CTE + CASE

CTE-এর ভিতরে calculated column-ও বানানো যায়।

```
WITH employee_status AS (
    SELECT
        name,
        department,
        salary,
        CASE
            WHEN salary >= 70000 THEN 'High'
            WHEN salary >= 50000 THEN 'Medium'
            ELSE 'Low'
        END AS salary_level
    FROM employees
)
SELECT *
FROM employee_status;
```

Result:

name	department	salary	salary_level
Rahim	IT	45000	Low
Karim	HR	55000	Medium
Hasan	IT	60000	Medium
Jamil	Finance	70000	High

1️⃣7️⃣ CTE + Window Function Preview

এখন একটা important preview।

ধরি আমরা department-wise salary ranking করতে চাই।

```
WITH employee_rank AS (
    SELECT
        name,
        department,
        salary,
        RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT *
FROM employee_rank;
```

এখানে আমরা নতুন concept দেখলাম:

```
RANK()
OVER()
PARTITION BY
```

🔥 এগুলো **Window Functions**।

এগুলো পরের advanced SQL phase-এর অত্যন্ত গুরুত্বপূর্ণ অংশ।

1️⃣8️⃣ Recursive CTE — Introduction

CTE-এর আরেকটি powerful type:

> **Recursive CTE**

যেখানে CTE নিজেকেই reference করতে পারে।

Hierarchy data-এর জন্য useful।

Example:

```
CEO
 ↓
Manager
 ↓
Team Lead
 ↓
Employee
```

Employee-manager hierarchy বের করতে Recursive CTE ব্যবহার করা যায়।

Basic structure:

```sql
WITH RECURSIVE employee_tree AS (
    -- Anchor
    SELECT
        id,
        name,
        manager_id
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive part
    SELECT
        e.id,
        e.name,
        e.manager_id
    FROM employees e
    JOIN employee_tree et
        ON e.manager_id = et.id
)
SELECT *
FROM employee_tree;
```

এটা এখন মুখস্থ করার দরকার নেই।

শুধু concept মনে রাখো:

```
Recursive CTE
      ↓
Hierarchy / Tree
      ↓
Parent → Child → Grandchild
```

আমরা পরে আলাদা করে এটা practice করব।

1️⃣9️⃣ CTE-এর Important Rules

Rule 1

CTE সাধারণত `WITH` দিয়ে শুরু হয়:

```
WITH name AS (...)
```

Rule 2

Multiple CTE comma দিয়ে আলাদা হয়:

```
WITH
cte1 AS (...),
cte2 AS (...),
cte3 AS (...)
SELECT ...
```

Rule 3

শেষে একটি main query থাকতে হয়।

```
WITH data AS (...)
SELECT *
FROM data;
```

Rule 4

CTE-কে table-এর মতো reference করা যায়:

```
FROM data
```

2️⃣0️⃣ CTE-এর Common Mistakes

❌ Mistake 1 — CTE-এর পরে main query না দেওয়া

```
WITH data AS (
    SELECT *
    FROM employees
);
```

সাধারণভাবে CTE definition-এর পরে consuming query লাগবে।

সঠিক:

```
WITH data AS (
    SELECT *
    FROM employees
)
SELECT *
FROM data;
```

❌ Mistake 2 — Multiple CTE-তে ভুল comma

সঠিক:

```
WITH cte1 AS (...),
cte2 AS (...)
SELECT ...
```

❌ Mistake 3 — CTE-এর column alias ভুলে যাওয়া

যেমন:

```
AVG(salary)
```

এর বদলে:

```
AVG(salary) AS avg_salary
```

দেওয়া অনেক বেশি readable।

2️⃣1️⃣ CTE Query Building Strategy

Complex problem দেখলে:

Step 1 — Raw data

```
employees
```

Step 2 — প্রথম calculation

```
department-wise salary
```

Step 3 — CTE

```
WITH dept_salary AS (...)
```

Step 4 — দ্বিতীয় calculation/filter

```
avg_salary > 50000
```

Step 5 — JOIN দরকার হলে JOIN

Step 6 — Final ORDER BY / LIMIT

🧠 Professional SQL Thinking

আগে তুমি হয়তো ভাবতে:

> "একটা huge query কীভাবে লিখব?"

এখন ভাববে:

> **"Problem-টাকে কয়েকটি logical step-এ কীভাবে ভাঙব?"**

উদাহরণ:

```
Business Question
      ↓
Raw Data
      ↓
CTE 1: Clean / Filter
      ↓
CTE 2: Aggregate
```

```
      ↓
CTE 3: Calculate
      ↓
JOIN
      ↓
Final Filter
      ↓
ORDER BY
```

এটাই professional SQL thinking-এর গুরুত্বপূর্ণ অংশ।

🎯 Class 38 Assignment

নিজে query লেখার চেষ্টা করো।

Q1

একটি CTE ব্যবহার করে salary > 50,000 employees দেখাও।

Q2

Department-wise average salary বের করে average > 50,000 departments দেখাও।

Q3

Department-wise employee count বের করে employee count > 3 departments দেখাও।

Q4

Customer-wise total spending বের করো CTE ব্যবহার করে।

Q5

50,000-এর বেশি spending করা customers দেখাও।

Q6

Customer-এর name, city এবং total spending দেখাও।

Q7

CTE ব্যবহার করে salary level তৈরি করো:

```
>=70000 → High
>=50000 → Medium
<50000 → Low
```

Q8 ⭐

Customer-wise total spending এবং customer city দেখাও, তারপর Dhaka customers filter করো।

Q9 ⭐

দুইটি CTE তৈরি করো:

```
CTE 1 → department average salary
CTE 2 → department employee count
```

তারপর JOIN করে একসাথে দেখাও।

Q10 ⭐

CTE + `RANK()` ব্যবহার করে department-এর ভিতরে salary ranking করার চেষ্টা করো।

🧪 Class 38 Quiz

1. CTE-এর পূর্ণরূপ কী?

A. Common Table Expression
B. Common Table Entity
C. Create Table Expression
D. Column Table Expression

2. CTE কোন keyword দিয়ে শুরু হয়?

A. `TABLE`
B. `WITH`
C. `CTE`
D. `TEMP`

3. CTE-এর result কীভাবে ব্যবহার করা যায়?

A. Table-এর মতো
B. শুধু INSERT-এর জন্য
C. শুধু DELETE-এর জন্য
D. ব্যবহার করা যায় না

4. Multiple CTE কীভাবে আলাদা করা হয়?

A. `AND`
B. `JOIN`
C. Comma `,`
D. `OR`

5. CTE complex query-তে কী improve করে?

A. Readability
B. Maintainability
C. Logical organization
D. সবগুলো

6. CTE এবং Subquery-এর মধ্যে কোনটি complex multi-step query-তে সাধারণত বেশি readable?

A. CTE
B. DELETE
C. INSERT
D. DISTINCT

7. Recursive CTE কোথায় useful?

A. Hierarchy
B. Tree structure
C. Parent-child relationship
D. সবগুলো

8. নিচের কোনটি CTE-এর সঠিক শুরু?

A. `CREATE CTE`
B. `WITH data AS (...)`
C. `START CTE`
D. `TABLE data AS`

9. Window function-এর সাথে CTE ব্যবহার করা যায়?

A. Yes
B. No

10. CTE কি database-এর permanent table?

A. Yes
B. No

⭐ Class 38 Cheat Sheet

Simple CTE

```
WITH data AS (
    SELECT *
    FROM employees
```

```
)
SELECT *
FROM data;
```

CTE + GROUP BY

```
WITH dept_salary AS (
    SELECT
        department,
        AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_salary
WHERE avg_salary > 50000;
```

Multiple CTE

```
WITH
dept_salary AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
),
dept_count AS (
    SELECT department, COUNT(*) AS employee_count
    FROM employees
    GROUP BY department
)
SELECT *
FROM dept_salary
JOIN dept_count
ON dept_salary.department = dept_count.department;
```

CTE + JOIN

```
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(amount) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.name,
    c.city,
```

```
    cs.total_sales
FROM customers c
JOIN customer_sales cs
ON c.customer_id = cs.customer_id;
```

🔥 Class 31 → 38 পর্যন্ত তোমার SQL Progress

```
Class 31 → SELECT, WHERE, ORDER BY
        ↓
Class 32 → CREATE TABLE, Constraints
        ↓
Class 33 → INSERT, UPDATE, DELETE, LIKE, IN, NULL
        ↓
Class 34 → GROUP BY, HAVING, Aggregation
        ↓
Class 35 → INNER/LEFT/RIGHT JOIN
        ↓
Class 36 → SELF/FULL/CROSS/MULTIPLE JOIN
        ↓
Class 37 → Subquery
        ↓
Class 38 → CTE
```

এখন তুমি **basic SQL থেকে intermediate SQL-এর দিকে চলে এসেছো।**

🚀 Next: Class 39 — Window Functions

এখানে আমরা শিখব:

```
ROW_NUMBER()
RANK()
DENSE_RANK()
OVER()
PARTITION BY
ORDER BY inside OVER()
LAG()
LEAD()
Running Total
Top N per Group
```

এটা **Data Analyst / Data Engineer-এর জন্য খুব গুরুত্বপূর্ণ SQL topic**, বিশেষ করে ranking, sales analysis, salary analysis এবং time-series analysis-এর ক্ষেত্রে।

📘 SQL — Class 39

🪟 Window Functions — `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`

Class 38-এ আমরা **CTE** শিখেছি। আজ থেকে SQL-এর একটি খুব important advanced topic শুরু করছি:

> **Window Functions**

Data Analyst, BI Analyst, Data Engineer এবং SQL interview—সব জায়গাতেই এটা গুরুত্বপূর্ণ।

🎯 আজকের Learning Goals

আজ শিখবে:

1. Window Function কী
2. `OVER()`
3. `ORDER BY` inside `OVER()`
4. `PARTITION BY`
5. `ROW_NUMBER()`
6. `RANK()`
7. `DENSE_RANK()`
8. তিনটির difference
9. Top N per Group
10. `LAG()`
11. `LEAD()`
12. Running Total
13. CTE + Window Function

1️⃣ Window Function কী?

সহজভাবে:

> **Window Function rows-কে group না করে, প্রতিটি row-এর পাশে অতিরিক্ত calculation result দেয়।**

এটাই সবচেয়ে গুরুত্বপূর্ণ ধারণা।

GROUP BY বনাম Window Function

ধরি:

name	department	salary
Rahim	IT	45000
Hasan	IT	60000
Karim	HR	55000
Jamil	HR	70000

GROUP BY

```
SELECT
    department,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Result:

department	avg_salary
IT	52500
HR	62500

Original employee rows আর থাকে না।

Window Function

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    ) AS department_avg
FROM employees;
```

Result:

name	department	salary	department_avg
Rahim	IT	45000	52500
Hasan	IT	60000	52500
Karim	HR	55000	62500
Jamil	HR	70000	62500

🔥 **Rows থাকল + calculation-এর result পাশে পেলাম।**

2️⃣ `OVER()` কী?

Window Function-এর মূল অংশ:

```
OVER()
```

Example:

```
SELECT
    name,
    salary,
    AVG(salary) OVER () AS company_avg
FROM employees;
```

এখানে পুরো table-এর average প্রতিটি row-তে দেখাবে।

3️⃣ `OVER()` + `ORDER BY`

এবার ranking করি।

```
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (
        ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

ধরি:

name	salary
Jamil	70000
Hasan	60000
Karim	55000
Rahim	45000

Result:

name	salary	row_num
Jamil	70000	1
Hasan	60000	2
Karim	55000	3
Rahim	45000	4

4️⃣ `ROW_NUMBER()`

`ROW_NUMBER()` প্রতিটি row-কে unique sequential number দেয়।

```
SELECT
    name,
    salary,
    ROW_NUMBER() OVER (
        ORDER BY salary DESC
    ) AS row_num
FROM employees;
```

Result:

```
70000 → 1
60000 → 2
55000 → 3
45000 → 4
```

5️⃣ Duplicate Salary হলে কী হবে?

ধরি:

name	salary
Jamil	70000
Hasan	60000
Karim	60000
Rahim	45000

`ROW_NUMBER()`:

name	salary	row_num
Jamil	70000	1
Hasan	60000	2
Karim	60000	3
Rahim	45000	4

দুইজনের salary একই হলেও number আলাদা।

6️⃣ `RANK()`

এখন `RANK()`।

```
SELECT
    name,
    salary,
    RANK() OVER (
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

Result:

name	salary	rank
Jamil	70000	1
Hasan	60000	2
Karim	60000	2
Rahim	45000	4

Notice:

```
1
2
2
4
```

3 বাদ গেছে।

7️⃣ DENSE_RANK()

```sql
SELECT
    name,
    salary,
    DENSE_RANK() OVER (
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

Result:

name	salary	dense_rank
Jamil	70000	1
Hasan	60000	2
Karim	60000	2
Rahim	45000	3

এখানে:

```
1
2
2
3
```

কোনো number skip হয়নি।

🧠 সবচেয়ে গুরুত্বপূর্ণ Difference

ধরি salary:

```
70000
60000
60000
45000
```

ROW_NUMBER

```
1
2
3
4
```

RANK

```
1
2
2
4
```

DENSE_RANK

```
1
2
2
3
```

⭐ সহজে মনে রাখো

Function	Duplicate হলে	Gap হয়?
`ROW_NUMBER()`	আলাদা number	না
`RANK()`	একই rank	**হ্যাঁ**
`DENSE_RANK()`	একই rank	**না**

8️⃣ `PARTITION BY`

এটা Window Function-এর সবচেয়ে important conceptগুলোর একটি।

> `PARTITION BY` data-কে logical groups-এ ভাগ করে, কিন্তু rows delete করে না।

Example

Department-wise salary ranking:

```
SELECT
    name,
    department,
    salary,
    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dept_rank
FROM employees;
```

Result:

name	department	salary	dept_rank
Hasan	IT	60000	1
Rahim	IT	45000	2
Jamil	HR	70000	1
Karim	HR	55000	2

এখানে IT এবং HR আলাদাভাবে rank হয়েছে।

9️⃣ `PARTITION BY` Mental Model

```
Employees
     ↓
PARTITION BY department
     ↓
 ┌──────────────┐
 │ IT           │
 │ Rahim        │
 │ Hasan        │
 └──────────────┘

 ┌──────────────┐
 │ HR           │
 │ Karim        │
 │ Jamil        │
 └──────────────┘
```

```
      ↓
প্রতিটি group-এর ভিতরে ranking
```

🔟 Top Employee Per Department 🔥

এটা interview-এর খুব common problem।

Question:

> প্রতিটি department-এর highest-paid employee বের করো।

প্রথমে:

```sql
WITH ranked_employees AS (
    SELECT
        name,
        department,
        salary,
        RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS dept_rank
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE dept_rank = 1;
```

Result:

```
IT → Hasan
HR → Jamil
```

🔥 এখানে আমরা ব্যবহার করেছি:

```
CTE
+
RANK()
+
PARTITION BY
```

1️⃣1️⃣ Top 2 Employees Per Department

```sql
WITH ranked_employees AS (
    SELECT
        name,
        department,
        salary,
        RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS dept_rank
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE dept_rank <= 2;
```

এটাই:

> **Top N per Group**

1️⃣2️⃣ `ROW_NUMBER()` দিয়ে Top 1

```sql
WITH ranked_employees AS (
    SELECT
        name,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rn
    FROM employees
)
SELECT *
FROM ranked_employees
WHERE rn = 1;
```

কিন্তু একটা গুরুত্বপূর্ণ difference:

যদি একই department-এ দুইজনের salary একই এবং দুজনই highest হয়:

`ROW_NUMBER()` → একজনকে 1, অন্যজনকে 2 দেবে।

`RANK()` → দুজনকেই 1 দেবে।

তাই:

```
একজন winner দরকার → ROW_NUMBER()
সব tied winner দরকার → RANK()
```

1️⃣3️⃣ LAG()

এবার ranking থেকে time-series analysis-এর দিকে যাই।

`LAG()`:

> বর্তমান row-এর আগের row-এর value দেয়।

ধরি sales:

month	sales
Jan	100000
Feb	120000
Mar	150000
Apr	130000

Query:

```sql
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales
FROM monthly_sales;
```

Result:

month	sales	previous_sales
Jan	100000	NULL
Feb	120000	100000
Mar	150000	120000
Apr	130000	150000

1️⃣4️⃣ LAG দিয়ে Growth

Current sales:

```
120000
```

Previous:

```
100000
```

Growth:

```
(120000 - 100000) / 100000
= 20%
```

SQL:

```sql
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales,
    ROUND(
        100.0 * (sales - LAG(sales) OVER (
            ORDER BY month
        ))
        / NULLIF(LAG(sales) OVER (
            ORDER BY month
        ), 0),
        2
    ) AS growth_percent
FROM monthly_sales;
```

প্রথম month-এর previous value নেই, তাই `NULL` থাকবে।

1️⃣5️⃣ `LEAD()`

`LEAD()` হলো `LAG()`-এর opposite।

> পরের row-এর value দেয়।

```sql
SELECT
    month,
    sales,
    LEAD(sales) OVER (
        ORDER BY month
    ) AS next_sales
FROM monthly_sales;
```

Result:

month	sales	next_sales
Jan	100000	120000
Feb	120000	150000
Mar	150000	130000
Apr	130000	NULL

🧠 LAG vs LEAD

```
LAG
← Previous row

Current row

LEAD
→ Next row
```

সহজে:

```
LAG  = পিছনে তাকাও
LEAD = সামনে তাকাও
```

1️⃣6️⃣ Running Total 🔥

এবার খুব useful analytics concept।

Sales:

Month	Sales
Jan	100
Feb	200
Mar	150
Apr	300

Running total:

```
Jan = 100
Feb = 300
Mar = 450
Apr = 750
```

SQL:

```sql
SELECT
    month,
    sales,
    SUM(sales) OVER (
        ORDER BY month
    ) AS running_total
FROM monthly_sales;
```

Result:

month	sales	running_total
Jan	100	100
Feb	200	300
Mar	150	450
Apr	300	750

🔥 Data Analytics-এ এটি খুব common।

1️⃣7️⃣ Department-wise Running Total

ধরি employee salary data।

```sql
SELECT
    name,
```

```
    department,
    salary,
    SUM(salary) OVER (
        PARTITION BY department
        ORDER BY salary
    ) AS running_salary
FROM employees;
```

এখানে প্রতিটি department আলাদাভাবে cumulative sum করবে।

1️⃣8️⃣ `AVG() OVER()`

Window Function শুধু ranking-এর জন্য নয়।

Company average:

```
SELECT
    name,
    salary,
    AVG(salary) OVER () AS company_avg
FROM employees;
```

প্রতিটি row-তে company average থাকবে।

1️⃣9️⃣ Department Average

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    ) AS department_avg
FROM employees;
```

এটা আগের GROUP BY-এর মতো aggregate করছে, কিন্তু employee rows রেখে দিচ্ছে।

2️⃣0️⃣ Salary vs Department Average

Question:

> Employee-এর salary তার department average-এর তুলনায় কত বেশি/কম?

```sql
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    ) AS department_avg,
    salary - AVG(salary) OVER (
        PARTITION BY department
    ) AS difference
FROM employees;
```

এটা খুব useful analytical query।

2️⃣1️⃣ Multiple Window Functions

একই query-তে অনেকগুলো window function ব্যবহার করা যায়।

```sql
SELECT
    name,
    department,
    salary,

    ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS row_num,

    RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS salary_rank,

    DENSE_RANK() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS dense_rank

FROM employees;
```

2️⃣2️⃣ `OVER()` -এর তিনটি Important Part

একটা window expression:

```
RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
)
```

এখানে:

`PARTITION BY`

Group তৈরি করে।

`ORDER BY`

Group-এর ভিতরে order ঠিক করে।

Function

```
RANK()
ROW_NUMBER()
SUM()
AVG()
LAG()
LEAD()
```

🧠 Window Function Formula

```
FUNCTION()
    ↓
OVER(
    PARTITION BY ...
    ORDER BY ...
)
```

সবসময় তিনটিই লাগবে না।

Example:

```
AVG(salary) OVER ()
```

এখানে partition/order নেই।

2️⃣3️⃣ GROUP BY vs Window Function

GROUP BY	Window Function
Rows collapse করে	Rows রাখে
প্রতি group-এ summary row	প্রতিটি original row থাকে
Aggregate analysis	Row-level + aggregate analysis
`GROUP BY`	`OVER()`
Department total	Employee + department total

Example

GROUP BY:

```
SELECT
    department,
    AVG(salary)
FROM employees
GROUP BY department;
```

Window:

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    )
FROM employees;
```

এই difference খুব ভালোভাবে বুঝে রাখো।

2️⃣4️⃣ Real-world Example — Top 3 Products Per Category

ধরি:

```
products
product_name
```

```
category
sales
```

Query:

```sql
WITH ranked_products AS (
    SELECT
        product_name,
        category,
        sales,
        RANK() OVER (
            PARTITION BY category
            ORDER BY sales DESC
        ) AS product_rank
    FROM products
)
SELECT *
FROM ranked_products
WHERE product_rank <= 3;
```

এটা বাস্তব Data Analyst-এর common query pattern।

2️⃣5️⃣ Real-world Example — Monthly Sales Growth

```sql
WITH sales_data AS (
    SELECT
        month,
        sales,
        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales
    FROM monthly_sales
)
SELECT
    month,
    sales,
    previous_sales,
    ROUND(
        100.0 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percent
FROM sales_data;
```

এখানে:

```
CTE
 ↓
LAG
 ↓
Previous Month
 ↓
Growth %
```

🔥 এটা তোমার Excel-এর monthly growth calculation-এর SQL version হিসেবে ভাবতে পারো।

2️⃣6️⃣ Excel → SQL Connection

তুমি যেহেতু Excel Mastery শেষ করে SQL শিখছো, mappingটা খুব useful:

Excel	SQL
Sort	ORDER BY
Filter	WHERE
PivotTable	GROUP BY
Pivot filter	HAVING
VLOOKUP/XLOOKUP	JOIN
Running Total	`SUM() OVER()`
Ranking	`RANK()`
Previous row	`LAG()`
Next row	`LEAD()`
Unique ranking	`ROW_NUMBER()`
Group-wise analysis	`PARTITION BY`

🎯 Class 39 Assignment

নিজে query লেখার চেষ্টা করো।

Q1

সব employees-কে salary অনুযায়ী descending `ROW_NUMBER()` দাও।

Q2

Salary অনুযায়ী `RANK()` বের করো।

Q3

Salary অনুযায়ী `DENSE_RANK()` বের করো।

Q4

Department-এর ভিতরে salary ranking করো।

Q5

প্রতিটি department-এর highest-paid employee বের করো।

Q6

প্রতিটি department-এর Top 2 employee বের করো।

Q7

প্রতিটি employee-এর department average salary দেখাও।

Q8

Employee salary এবং department average-এর difference বের করো।

Q9

Monthly sales-এর previous month sales বের করো `LAG()` দিয়ে।

Q10

Monthly sales-এর next month sales বের করো `LEAD()` দিয়ে।

Q11

Monthly running sales total বের করো।

Q12 ⭐

প্রতিটি category-এর Top 3 products বের করো।

Q13 ⭐

Monthly sales growth percentage বের করো।

Q14 ⭐

প্রতিটি department-এর salary ranking + department average একসাথে দেখাও।

Q15 🔥

CTE + `RANK()` ব্যবহার করে প্রতিটি department-এর highest-paid employee বের করো।

🧪 Class 39 Quiz

1. Window Function-এর মূল keyword কোনটি?

A. `WINDOW`
B. `OVER()`
C. `GROUP`
D. `PARTITION()`

2. `ROW_NUMBER()` duplicate value হলে কী করে?

A. একই number দেয়
B. Unique sequential number দেয়
C. NULL দেয়
D. Row delete করে

3. `RANK()` -এ tie হলে কী হয়?

A. Rank একই হয় এবং gap হতে পারে
B. সব rank unique
C. সবসময় gap হয় না
D. Row delete হয়

4. `DENSE_RANK()` এবং `RANK()` -এর প্রধান difference কী?

A. DENSE_RANK tie support করে না
B. DENSE_RANK gap skip করে না
C. RANK sort করতে পারে না
D. কোনো difference নেই

5. Group-wise ranking-এর জন্য কোনটি ব্যবহার করা হয়?

A. `PARTITION BY`
B. `GROUP BY` only
C. `LIMIT`
D. `DISTINCT`

6. Previous row-এর value পেতে?

A. LEAD
B. LAG
C. RANK
D. ROW

7. Next row-এর value পেতে?

A. LAG
B. LEAD
C. NEXT
D. FORWARD

8. Running total-এর জন্য কোন pattern useful?

A. `SUM() OVER (ORDER BY ...)`
B. `COUNT() ONLY`
C. `AVG() GROUP`
D. `SUM() WHERE`

9. Window Function ব্যবহার করলে original rows সাধারণত কী হয়?

A. Collapse হয়
B. থাকে
C. Delete হয়
D. Duplicate সবসময় হয়

10. Top 3 employees per department বের করার জন্য সবচেয়ে useful combination কোনটি?

A. `RANK() + PARTITION BY`
B. `DELETE + GROUP BY`
C. `DISTINCT + LIMIT 3`
D. `COUNT + HAVING`

⭐ Class 39 Cheat Sheet

1. Row Number

```
ROW_NUMBER() OVER (
    ORDER BY salary DESC
)
```

2. Rank

```
RANK() OVER (
    ORDER BY salary DESC
)
```

3. Dense Rank

```
DENSE_RANK() OVER (
    ORDER BY salary DESC
```

```
)
```

4. Department Ranking

```
RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
)
```

5. Previous Row

```
LAG(sales) OVER (
    ORDER BY month
)
```

6. Next Row

```
LEAD(sales) OVER (
    ORDER BY month
)
```

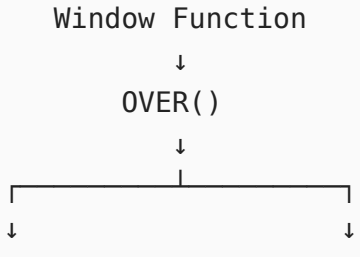
7. Running Total

```
SUM(sales) OVER (
    ORDER BY month
)
```

8. Group Average

```
AVG(salary) OVER (
    PARTITION BY department
)
```

🧠 সবচেয়ে গুরুত্বপূর্ণ Mental Model



```
        PARTITION BY          ORDER BY
        "কোন group?"          "কোন order?"
              ↓                    ↓
              └──────────┬─────────┘
                         ↓
              Calculation per row
```

আর এই তিনটা খুব ভালোভাবে মনে রাখো:

```
ROW_NUMBER → 1,2,3,4
RANK       → 1,2,2,4
DENSE_RANK → 1,2,2,3
```

Class 40-এ আমরা Window Functions-এর আরও powerful অংশে যাব: `FIRST_VALUE()`, `LAST_VALUE()`, `NTH_VALUE()`, **window frame (**`ROWS BETWEEN`**), moving average, cumulative percentage এবং advanced running calculations**।

📘 SQL — Class 40

🪟 Advanced Window Functions — Window Frame, `FIRST_VALUE()`, `LAST_VALUE()`, Moving Average & Cumulative Analysis

Class 39-এ আমরা শিখেছি:

```
ROW_NUMBER()
RANK()
DENSE_RANK()
PARTITION BY
LAG()
LEAD()
SUM() OVER()
```

আজ এগুলোর ওপর ভিত্তি করে **advanced analytics** করব।

🎯 আজকের Learning Goals

আজ শিখবে:

1. `FIRST_VALUE()`
2. `LAST_VALUE()`

3. `NTH_VALUE()`
4. Window Frame
5. `ROWS BETWEEN`
6. Running Total
7. Moving Average
8. Rolling Sum
9. Cumulative Percentage
10. Previous/Next comparison
11. Advanced CTE + Window Function

1️⃣ `FIRST_VALUE()`

`FIRST_VALUE()` একটি window-এর প্রথম row-এর value দেয়।

ধরি:

month	sales
Jan	100000
Feb	120000
Mar	150000
Apr	130000

Query:

```
SELECT
    month,
    sales,
    FIRST_VALUE(sales) OVER (
        ORDER BY month
    ) AS first_sales
FROM monthly_sales;
```

Result:

month	sales	first_sales
Jan	100000	100000
Feb	120000	100000
Mar	150000	100000
Apr	130000	100000

অর্থাৎ প্রথম month's sales প্রত্যেক row-তে দেখাবে।

2️⃣ Practical Use

Question:

> Current month's sales এবং year's first month's sales compare করো।

```
SELECT
    month,
    sales,
    FIRST_VALUE(sales) OVER (
        ORDER BY month
    ) AS first_month_sales
FROM monthly_sales;
```

তারপর difference:

```
SELECT
    month,
    sales,
    FIRST_VALUE(sales) OVER (
        ORDER BY month
    ) AS first_month_sales,
    sales - FIRST_VALUE(sales) OVER (
        ORDER BY month
    ) AS difference
FROM monthly_sales;
```

3️⃣ `LAST_VALUE()`

`LAST_VALUE()` শুনতে সহজ হলেও এখানে একটি **খুব important window-frame issue** আছে।

Basic example:

```
SELECT
    month,
    sales,
    LAST_VALUE(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
```

```
    ) AS last_sales
FROM monthly_sales;
```

Result:

month	sales	last_sales
Jan	100000	130000
Feb	120000	130000
Mar	150000	130000
Apr	130000	130000

এখানে সব row-তে final month's sales এসেছে।

⚠️ কেন `ROWS BETWEEN` দিলাম?

কারণ `LAST_VALUE()`-এর ক্ষেত্রে window-এর **বর্তমান frame** খুব গুরুত্বপূর্ণ।

আমরা explicitly বলেছি:

```
ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING
```

মানে:

```
শুরু থেকে
   ↓
পুরো window
   ↓
শেষ পর্যন্ত
```

তাই সত্যিকারের last row-এর value পাওয়া যায়।

4️⃣ Window Frame কী?

এটা Class 40-এর সবচেয়ে গুরুত্বপূর্ণ concept।

Window-এর ভিতরে কোন rows calculation-এর অংশ হবে, সেটা **window frame** দিয়ে নির্ধারণ করা যায়।

Example:

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

মানে:

```
আগের 2 rows
+
বর্তমান row
```

5️⃣ UNBOUNDED PRECEDING

মানে:

> Window-এর একদম শুরু থেকে।

```
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW
```

এটা running total-এর জন্য খুব useful।

6️⃣ CURRENT ROW

মানে:

> বর্তমান row।

Example:

```
ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW
```

7️⃣ UNBOUNDED FOLLOWING

মানে:

> Window-এর একদম শেষ পর্যন্ত।

```
ROWS BETWEEN CURRENT ROW
AND UNBOUNDED FOLLOWING
```

🧠 Window Frame Mental Model

ধরি:

```
Jan
Feb
Mar
Apr
May
```

যদি current row = Mar এবং:

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

তাহলে frame:

```
Jan ←
Feb ←
Mar ← Current
```

অর্থাৎ:

```
Jan + Feb + Mar
```

8️⃣ Running Total — Advanced Version

Class 39-এ আমরা লিখেছিলাম:

```
SELECT
    month,
    sales,
    SUM(sales) OVER (
        ORDER BY month
    ) AS running_total
FROM monthly_sales;
```

আরও explicit version:

```
SELECT
    month,
    sales,
```

```
    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS running_total
FROM monthly_sales;
```

Result:

month	sales	running_total
Jan	100	100
Feb	200	300
Mar	150	450
Apr	300	750

9️⃣ Moving Average 🔥

এটা Data Analytics-এর খুব important concept।

ধরি:

Month	Sales
Jan	100
Feb	200
Mar	150
Apr	300
May	250

আমরা **3-month moving average** চাই।

অর্থাৎ:

Jan

শুধু Jan:

```
100
```

Feb

```
(100 + 200) / 2 = 150
```

Mar

```
(100 + 200 + 150) / 3 = 150
```

Apr

```
(200 + 150 + 300) / 3
= 216.67
```

May

```
(150 + 300 + 250) / 3
= 233.33
```

SQL:

```sql
SELECT
    month,
    sales,
    AVG(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3
FROM monthly_sales;
```

🔥 এটাকে **3-period moving average** বলা হয়।

🔟 কেন Moving Average দরকার?

Real sales data:

```
100
500
120
600
150
700
```

এখানে ওঠানামা অনেক বেশি।

Moving average:

```
trend
──────→
```

দেখতে সাহায্য করে।

ব্যবহার:

- Sales trend
- Stock demand
- Website traffic
- Revenue forecasting-এর preliminary analysis
- Performance monitoring

1️⃣1️⃣ 3-Month Rolling Sum

Moving average-এর মতো rolling sum করা যায়।

```sql
SELECT
    month,
    sales,
    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_3_month_sales
FROM monthly_sales;
```

Example:

Month	Sales	Rolling 3 Month
Jan	100	100
Feb	200	300
Mar	150	450
Apr	300	650
May	250	700

1️⃣2️⃣ 2-Row Moving Average

```sql
SELECT
    month,
    sales,
    AVG(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) AS moving_avg_2
FROM monthly_sales;
```

Frame:

```
Previous row
+
Current row
```

1️⃣3️⃣ NTH_VALUE()

`NTH_VALUE()` window-এর নির্দিষ্ট position-এর value নিতে পারে।

Example:

```sql
SELECT
    month,
    sales,
    NTH_VALUE(sales, 2) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
    ) AS second_month_sales
FROM monthly_sales;
```

Result:

```
সব row-তে
second month's sales
```

ধরি February = 120000:

month	sales	second_month_sales
Jan	100000	120000
Feb	120000	120000
Mar	150000	120000

month	sales	second_month_sales
Apr	130000	120000

1️⃣4️⃣ `FIRST_VALUE()` vs `LAST_VALUE()` vs `NTH_VALUE()`

Function	কাজ
`FIRST_VALUE()`	প্রথম value
`LAST_VALUE()`	শেষ value
`NTH_VALUE()`	N-th value

Example:

```
100
200
300
400
```

```
FIRST_VALUE → 100
LAST_VALUE  → 400
NTH_VALUE(..., 3) → 300
```

1️⃣5️⃣ Cumulative Percentage 🔥

ধরি monthly sales:

Month	Sales
Jan	100
Feb	200
Mar	300
Apr	400

Total:

```
1000
```

Running total:

```
100
300
600
1000
```

Cumulative %:

```
10%
30%
60%
100%
```

SQL:

```sql
SELECT
    month,
    sales,
    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS running_total,

    100.0 *
    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    )
    /
    SUM(sales) OVER () AS cumulative_percentage

FROM monthly_sales;
```

1️⃣6️⃣ Percentage of Total

এটা Data Analytics-এ খুব useful।

Question:

> প্রতিটি product মোট sales-এর কত % করেছে?

```sql
SELECT
    product,
    sales,
    100.0 * sales / SUM(sales) OVER () AS percentage_of_total
FROM product_sales;
```

Example:

Product	Sales	% Total
Laptop	500	50%
Monitor	300	30%
Mouse	200	20%

1️⃣7️⃣ Department-wise Percentage

ধরি:

Department	Employee	Salary
IT	Rahim	45000
IT	Hasan	60000
HR	Karim	50000
HR	Jamil	70000

Question:

> Department-এর total salary-এর মধ্যে প্রত্যেক employee-এর salary কত %?

```sql
SELECT
    name,
    department,
    salary,
    100.0 * salary /
    SUM(salary) OVER (
        PARTITION BY department
    ) AS department_salary_percentage
FROM employees;
```

এখানে:

```
SUM(salary) OVER(PARTITION BY department)
```

প্রতিটি department-এর total salary বের করছে।

1️⃣8️⃣ Previous vs Current

`LAG()` দিয়ে:

```
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales
FROM monthly_sales;
```

তারপর difference:

```
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales,

    sales - LAG(sales) OVER (
        ORDER BY month
    ) AS sales_difference
FROM monthly_sales;
```

1️⃣9️⃣ Growth Percentage

আরও professional version:

```
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales,

    100.0 *
```

```
    (sales - LAG(sales) OVER (
        ORDER BY month
    ))
    / NULLIF(
        LAG(sales) OVER (
            ORDER BY month
        ),
        0
    ) AS growth_percentage

FROM monthly_sales;
```

`NULLIF()` কেন?

যদি previous sales = 0 হয়:

```
division by zero
```

problem হতে পারে।

তাই:

```
NULLIF(previous_sales, 0)
```

0 হলে NULL করে দেয়।

2️⃣0️⃣ LAG + CTE — Cleaner Version

Repeated `LAG()` না লিখে:

```
WITH sales_comparison AS (
    SELECT
        month,
        sales,
        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales
    FROM monthly_sales
)
SELECT
    month,
    sales,
    previous_sales,
    100.0 * (sales - previous_sales)
```

```
    / NULLIF(previous_sales, 0) AS growth_percentage
FROM sales_comparison;
```

এটা অনেক cleaner।

🔥 **CTE + Window Function** খুব powerful combination।

2️⃣1️⃣ `PARTITION BY` + Moving Average

ধরি আমাদের sales আছে:

Month	City	Sales
Jan	Dhaka	100
Feb	Dhaka	200
Mar	Dhaka	300
Jan	Chittagong	50
Feb	Chittagong	100
Mar	Chittagong	150

Question:

> City-wise 3-month moving average।

```
SELECT
    month,
    city,
    sales,
    AVG(sales) OVER (
        PARTITION BY city
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3
FROM city_sales;
```

এখানে Dhaka এবং Chittagong-এর calculations আলাদা হবে।

2️⃣2️⃣ Advanced Example — Top 3 + Percentage

প্রতিটি category-এর Top 3 products এবং category sales-এর percentage:

```sql
WITH product_analysis AS (
    SELECT
        product,
        category,
        sales,

        RANK() OVER (
            PARTITION BY category
            ORDER BY sales DESC
        ) AS product_rank,

        SUM(sales) OVER (
            PARTITION BY category
        ) AS category_total

    FROM product_sales
)
SELECT
    product,
    category,
    sales,
    product_rank,
    100.0 * sales / NULLIF(category_total, 0)
        AS category_percentage
FROM product_analysis
WHERE product_rank <= 3;
```

🔥 এই query-তে:

```
CTE
+
RANK
+
PARTITION BY
+
SUM OVER
+
Percentage
+
Filter
```

সব একসাথে আছে।

2️⃣3️⃣ Window Frame Cheat Sheet

শুরু থেকে বর্তমান পর্যন্ত

```
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW
```

➡️ Running Total

আগের 2 + Current

```
ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW
```

➡️ 3-row moving average/sum

Current + পরের 2

```
ROWS BETWEEN CURRENT ROW
AND 2 FOLLOWING
```

➡️ Forward-looking window

পুরো window

```
ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING
```

➡️ পুরো partition/window

2️⃣4️⃣ `ROWS` বনাম `RANGE` — Important Preview

দুটো দেখতে similar হলেও সবসময় একই নয়।

```
ROWS BETWEEN ...
```

সাধারণত physical rows-এর ভিত্তিতে frame তৈরি করে।

`RANGE` ordering value-এর peer/value range-এর ভিত্তিতে কাজ করতে পারে এবং duplicate ordering values থাকলে behavior আলাদা হতে পারে।

Beginner হিসেবে এখন:

> **Moving average / exact previous N rows → `ROWS` মনে রাখো।**

`RANGE` পরে advanced practice-এ আরও গভীরভাবে দেখব।

2️⃣5️⃣ GROUP BY বনাম Window — Final Understanding

GROUP BY

```
SELECT
    department,
    AVG(salary)
FROM employees
GROUP BY department;
```

Output:

```
IT → 52500
HR → 62500
```

Rows collapse হয়েছে।

Window

```
SELECT
    name,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    )
FROM employees;
```

Output:

```
Rahim → IT → 45000 → 52500
Hasan → IT → 60000 → 52500
```

```
Karim → HR → 55000 → 62500
Jamil → HR → 70000 → 62500
```

Rows আছে।

🔥 Golden Rule

> **GROUP BY = rows কমায়।**

> **Window Function = rows রেখে calculation যোগ করে।**

🎯 Class 40 Assignment

নিজে solve করার চেষ্টা করো।

Q1

প্রতিটি employee-এর সাথে company-এর first salary দেখাও।

Q2

প্রতিটি employee-এর সাথে company-এর last salary দেখাও।

Q3

প্রতিটি employee-এর সাথে second-highest salary/value related analysis করো `NTH_VALUE()` দিয়ে।

Q4

Monthly sales-এর running total বের করো।

Q5

Monthly sales-এর 3-month moving average বের করো।

Q6

Monthly sales-এর 3-month rolling sum বের করো।

Q7

প্রতিটি product-এর total sales-এর percentage বের করো।

Q8

প্রতিটি department-এর মধ্যে employee salary-এর percentage বের করো।

Q9

Previous month's sales এবং current month's sales difference বের করো।

Q10

Monthly sales growth percentage বের করো।

Q11 ⭐

প্রতিটি city-এর 3-month moving average বের করো।

Q12 ⭐

প্রতিটি department-এর Top 3 employees এবং department-এর total salary-এর মধ্যে তাদের percentage বের করো।

Q13 🔥

CTE + `LAG()` ব্যবহার করে monthly sales growth report বানাও:

```
Month
Current Sales
Previous Sales
Difference
Growth %
```

Q14 🔥

CTE + `RANK()` + `SUM() OVER()` ব্যবহার করে:

```
Category
Product
Sales
Rank
Category Total
Category %
```

এবং শুধু Top 3 products দেখাও।

🧪 Class 40 Quiz

1. `FIRST_VALUE()` কী দেয়?

A. প্রথম row-এর value
B. শেষ row-এর value

C. Average
D. Rank

2. `LAST_VALUE()` সঠিকভাবে পুরো window-এর শেষ value পেতে কোন concept গুরুত্বপূর্ণ?

A. `WHERE`
B. Window Frame
C. `GROUP BY`
D. `DISTINCT`

3. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` কতগুলো row পর্যন্ত include করতে পারে?

A. সর্বোচ্চ 2
B. বর্তমানসহ সর্বোচ্চ 3
C. 4
D. সব

4. Running Total-এর জন্য কোনটি useful?

A. `SUM() OVER()`
B. `AVG() ONLY`
C. `RANK()`
D. `LEAD()`

5. Previous row-এর value?

A. LEAD
B. LAG
C. LAST_VALUE
D. NTH_VALUE

6. Next row-এর value?

A. LAG
B. LEAD
C. FIRST_VALUE
D. RANK

7. 3-month moving average-এর জন্য কোন frame useful?

A. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`
B. `ROWS BETWEEN 10 FOLLOWING`
C. `GROUP BY month`
D. `LIMIT 3`

8. Percentage of total বের করতে কোন pattern useful?

```
A. value / SUM(value) OVER()
B. value / GROUP BY
C. value / ORDER BY
D. value / LIMIT
```

9. `PARTITION BY city` করলে কী হবে?

A. City অনুযায়ী logical window তৈরি হবে
B. Table delete হবে
C. City column remove হবে
D. সব data sort হবে

10. Window Function-এর সবচেয়ে বড় সুবিধা কী?

A. Original rows রেখে analytical calculation করা যায়
B. সব rows delete করা যায়
C. Table automatically normalize হয়
D. Database তৈরি হয়

⭐ Class 40 Cheat Sheet

First Value

```
FIRST_VALUE(sales) OVER (
    ORDER BY month
)
```

Last Value

```
LAST_VALUE(sales) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND UNBOUNDED FOLLOWING
)
```

N-th Value

```
NTH_VALUE(sales, 2) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND UNBOUNDED FOLLOWING
)
```

Running Total

```
SUM(sales) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW
)
```

Moving Average

```
AVG(sales) OVER (
    ORDER BY month
    ROWS BETWEEN 2 PRECEDING
    AND CURRENT ROW
)
```

Percentage of Total

```
100.0 * sales / SUM(sales) OVER ()
```

Previous Value

```
LAG(sales) OVER (
    ORDER BY month
)
```

Next Value

```
LEAD(sales) OVER (
    ORDER BY month
)
```

🧠 Class 31 → 40 পর্যন্ত তোমার SQL Journey

```
31 → SELECT / WHERE / ORDER BY
32 → CREATE TABLE / Constraints
33 → INSERT / UPDATE / DELETE
34 → GROUP BY / HAVING
35 → JOIN
36 → Advanced JOIN
37 → Subquery
38 → CTE
```

```
39 → Window Functions
40 → Advanced Window Functions
```

এখন তুমি SQL-এর **intermediate → advanced analytics** অংশে ঢুকে গেছো।

পরের Class 41-এ আমরা Window Functions-এর উপর একটি **full SQL Analytics Project** করব—Sales dataset নিয়ে `CTE + JOIN + GROUP BY + RANK + LAG + Running Total + Moving Average + Growth %` একসাথে ব্যবহার করে একটি বাস্তব Data Analyst-এর মতো analysis তৈরি করব।

📘 SQL — Class 41

🏆 Full SQL Analytics Project — Sales Analysis

আজ আমরা শুধু নতুন একটা function শিখব না। বরং **Class 31–40 পর্যন্ত শেখা SQL একসাথে ব্যবহার করে একটি real-world Sales Analytics Project** বানাব।

আজকের project-এ আমরা ব্যবহার করব:

```
SELECT
WHERE
JOIN
GROUP BY
HAVING
CTE
CASE
RANK()
LAG()
SUM() OVER()
AVG() OVER()
PARTITION BY
ROWS BETWEEN
```

🎯 Project Goal

ধরি, একটি কোম্পানির 3টি table আছে:

```
customers
products
orders
```

আমরা বের করব:

1. Total sales
2. Customer-wise sales
3. Product-wise sales
4. Category-wise sales
5. Top products
6. Top customers
7. Monthly sales
8. Previous month sales
9. Monthly growth %
10. Running total
11. 3-month moving average
12. Category-wise product ranking
13. Top 3 products per category
14. Customer ranking
15. Final business insights

1️⃣ Database Structure

Customers

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    city VARCHAR(50)
);
```

Data:

```sql
INSERT INTO customers VALUES
(1, 'Rahim', 'Dhaka'),
(2, 'Karim', 'Chittagong'),
(3, 'Hasan', 'Dhaka'),
(4, 'Jamil', 'Khulna'),
(5, 'Nayeem', 'Dhaka');
```

2️⃣ Products Table

```sql
CREATE TABLE products (
    product_id INT PRIMARY KEY,
```

```
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2)
);
```

Data:

```
INSERT INTO products VALUES
(101, 'Laptop', 'Electronics', 80000),
(102, 'Monitor', 'Electronics', 30000),
(103, 'Keyboard', 'Accessories', 3000),
(104, 'Mouse', 'Accessories', 1500),
(105, 'Chair', 'Furniture', 10000),
(106, 'Desk', 'Furniture', 20000);
```

3️⃣ Orders Table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    order_date DATE,
    quantity INT,

    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id),

    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

Data:

```
INSERT INTO orders VALUES
(1, 1, 101, '2026-01-05', 1),
(2, 2, 102, '2026-01-10', 2),
(3, 3, 103, '2026-01-15', 5),
(4, 1, 104, '2026-02-05', 4),
(5, 4, 105, '2026-02-12', 2),
(6, 5, 101, '2026-02-20', 1),
(7, 2, 106, '2026-03-03', 2),
(8, 3, 102, '2026-03-10', 1),
(9, 1, 103, '2026-03-15', 10),
(10, 5, 104, '2026-04-05', 5),
```

```
(11, 4, 106, '2026-04-12', 1),
(12, 2, 101, '2026-04-20', 1);
```

4️⃣ প্রথম কাজ — Full Sales Dataset

প্রথমে আমাদের তিনটা table JOIN করতে হবে।

```
SELECT
    o.order_id,
    o.order_date,
    c.customer_name,
    c.city,
    p.product_name,
    p.category,
    o.quantity,
    p.price,
    o.quantity * p.price AS total_amount
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN products p
    ON o.product_id = p.product_id;
```

এখন আমাদের কাছে একটি analytical dataset তৈরি হলো।

গুরুত্বপূর্ণ:

```
orders
   ↓
JOIN customers
   ↓
JOIN products
   ↓
Sales Analysis Dataset
```

5️⃣ Total Sales

```
SELECT
    SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id;
```

এখানে:

```
quantity × price
```

= Order Revenue

6️⃣ Customer-wise Sales

Question:

> কোন customer কত টাকা spend করেছে?

```sql
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name
ORDER BY total_sales DESC;
```

এখানে Class 34-এর:

```
GROUP BY
SUM()
ORDER BY
```

একসাথে ব্যবহার হয়েছে।

7️⃣ Customer Ranking 🔥

এখন প্রত্যেক customer-এর ranking চাই।

```sql
WITH customer_sales AS (
    SELECT
        c.customer_id,
        c.customer_name,
```

```
        SUM(o.quantity * p.price) AS total_sales
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        c.customer_id,
        c.customer_name
)

SELECT
    customer_name,
    total_sales,

    RANK() OVER (
        ORDER BY total_sales DESC
    ) AS customer_rank

FROM customer_sales;
```

এখানে structure:

```
JOIN
 ↓
GROUP BY
 ↓
CTE
 ↓
RANK()
```

8️⃣ Top 3 Customers

আগের query-কে CTE দিয়ে wrap করতে পারি:

```
WITH customer_sales AS (
    SELECT
        c.customer_id,
        c.customer_name,
        SUM(o.quantity * p.price) AS total_sales
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN products p
        ON o.product_id = p.product_id
```

```
    GROUP BY
        c.customer_id,
        c.customer_name
),

ranked_customers AS (
    SELECT
        customer_name,
        total_sales,
        RANK() OVER (
            ORDER BY total_sales DESC
        ) AS customer_rank
    FROM customer_sales
)

SELECT *
FROM ranked_customers
WHERE customer_rank <= 3;
```

🔥 এটা একটি very common Data Analyst pattern:

```
Raw Data
   ↓
Aggregation
   ↓
CTE
   ↓
Ranking
   ↓
Filter
```

9️⃣ Product-wise Sales

```
SELECT
    p.product_name,
    p.category,
    SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    p.product_name,
    p.category
ORDER BY total_sales DESC;
```

🔟 Category-wise Sales

```sql
SELECT
    p.category,
    SUM(o.quantity * p.price) AS category_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY category_sales DESC;
```

1️⃣1️⃣ Category-এর ভিতরে Product Ranking 🔥

Question:

> প্রতিটি category-এর মধ্যে কোন product #1?

```sql
WITH product_sales AS (
    SELECT
        p.product_id,
        p.product_name,
        p.category,
        SUM(o.quantity * p.price) AS total_sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        p.product_id,
        p.product_name,
        p.category
)

SELECT
    product_name,
    category,
    total_sales,

    RANK() OVER (
        PARTITION BY category
        ORDER BY total_sales DESC
    ) AS category_rank

FROM product_sales;
```

এখানে সবচেয়ে গুরুত্বপূর্ণ:

```
PARTITION BY category
```

মানে:

```
Electronics
   → ranking আলাদা

Accessories
   → ranking আলাদা

Furniture
   → ranking আলাদা
```

1️⃣2️⃣ Top 1 Product per Category

```sql
WITH product_sales AS (
    SELECT
        p.product_name,
        p.category,
        SUM(o.quantity * p.price) AS total_sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        p.product_name,
        p.category
),

ranked_products AS (
    SELECT
        product_name,
        category,
        total_sales,

        RANK() OVER (
            PARTITION BY category
            ORDER BY total_sales DESC
        ) AS category_rank

    FROM product_sales
)

SELECT *
```

```
FROM ranked_products
WHERE category_rank = 1;
```

1️⃣3️⃣ Monthly Sales

এবার date থেকে month বের করব।

MySQL:

```
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m') AS month,
    SUM(o.quantity * p.price) AS monthly_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

Result conceptually:

```
2026-01 → ...
2026-02 → ...
2026-03 → ...
2026-04 → ...
```

1️⃣4️⃣ Monthly Sales + Previous Month

এখন Class 39-এর `LAG()` ব্যবহার করব।

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
)

SELECT
```

```
    month,
    sales,

    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_month_sales

FROM monthly_sales;
```

1️⃣5️⃣ Monthly Growth %

এখন:

```
Current Sales
Previous Sales
Difference
Growth %
```

সব বের করব।

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
),

sales_comparison AS (
    SELECT
        month,
        sales,

        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales

    FROM monthly_sales
)

SELECT
    month,
```

```
    sales,
    previous_sales,

    sales - previous_sales AS difference,

    ROUND(
        100.0 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percentage

FROM sales_comparison;
```

🔥 এই query একটি real business report-এর মতো।

1️⃣6️⃣ Running Total

এখন cumulative sales:

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
)

SELECT
    month,
    sales,

    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS running_total

FROM monthly_sales;
```

Concept:

```
Jan       100
Feb       200
```

```
Mar       300

Running:
Jan       100
Feb       300
Mar       600
```

1️⃣7️⃣ 3-Month Moving Average

```sql
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
)

SELECT
    month,
    sales,

    ROUND(
        AVG(sales) OVER (
            ORDER BY month
            ROWS BETWEEN 2 PRECEDING
            AND CURRENT ROW
        ),
        2
    ) AS moving_average_3

FROM monthly_sales;
```

1️⃣8️⃣ Cumulative Percentage

Question:

> বছরের total sales-এর কত % এখন পর্যন্ত অর্জিত হয়েছে?

```sql
WITH monthly_sales AS (
    SELECT
```

```
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
)

SELECT
    month,
    sales,

    SUM(sales) OVER (
        ORDER BY month
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS running_total,

    ROUND(
        100.0 *
        SUM(sales) OVER (
            ORDER BY month
            ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
        )
        / SUM(sales) OVER (),
        2
    ) AS cumulative_percentage

FROM monthly_sales;
```

1️⃣9️⃣ Final Analytics Query 🔥🔥

এখন আমরা একটি বড় report বানাব:

```
Month
Sales
Previous Sales
Growth %
Running Total
Moving Average
Cumulative %
```

```sql
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        DATE_FORMAT(o.order_date, '%Y-%m')
),

analysis AS (
    SELECT
        month,
        sales,

        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales,

        SUM(sales) OVER (
            ORDER BY month
            ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
        ) AS running_total,

        AVG(sales) OVER (
            ORDER BY month
            ROWS BETWEEN 2 PRECEDING
            AND CURRENT ROW
        ) AS moving_average

    FROM monthly_sales
)

SELECT
    month,
    sales,
    previous_sales,

    sales - previous_sales AS difference,

    ROUND(
        100.0 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percentage,

    running_total,
```

```
    ROUND(moving_average, 2)
        AS moving_average,

    ROUND(
        100.0 * running_total
        / SUM(sales) OVER (),
        2
    ) AS cumulative_percentage

FROM analysis
ORDER BY month;
```

🔥🔥 **এটাই Class 41-এর মূল query।**

🧠 Query-টা কীভাবে চিন্তা করবে?

একসাথে পুরো query মুখস্থ করবে না।

এভাবে চিন্তা করবে:

```
Step 1
JOIN
↓
Sales data তৈরি

Step 2
GROUP BY
↓
Monthly Sales

Step 3
CTE
↓
Monthly result temporarily store

Step 4
LAG()
↓
Previous month

Step 5
SUM() OVER()
↓
Running Total

Step 6
```

```
AVG() OVER()
↓
Moving Average

Step 7
Calculation
↓
Growth %

Step 8
Final SELECT
↓
Business Report
```

⭐ Data Analyst-এর সবচেয়ে গুরুত্বপূর্ণ Pattern

এই pattern মনে রাখো:

```
Raw Tables
     ↓
JOIN
     ↓
GROUP BY
     ↓
CTE
     ↓
Window Functions
     ↓
Business Calculation
     ↓
Final Report
```

এটা বাস্তব SQL analytics-এ খুব common workflow।

📝 Class 41 Assignment

এবার তোমার practice।

Level 1

Q1. Customer-wise total sales বের করো।

Q2. Product-wise total sales বের করো।

Q3. Category-wise total sales বের করো।

Q4. Top 3 customers বের করো।

Level 2

Q5. প্রতিটি category-এর মধ্যে product ranking বের করো।

Q6. প্রতিটি category-এর Top 2 products বের করো।

Q7. Monthly sales বের করো।

Q8. Monthly sales-এর previous month বের করো।

Level 3

Q9. Monthly growth percentage বের করো।

Q10. Monthly running total বের করো।

Q11. 3-month moving average বের করো।

Q12. Cumulative percentage বের করো।

🔥 Challenge

একটি final query বানাও যার output হবে:

```
month
sales
previous_sales
growth_percentage
running_total
moving_average_3
cumulative_percentage
```

শর্ত: কমপক্ষে **2টি CTE + 3টি Window Function** ব্যবহার করতে হবে।

🧪 Class 41 Quiz

1. `LAG()` কী দেয়?

A. Next row
B. Previous row
C. First row
D. Average

2. `PARTITION BY category` কেন ব্যবহার করি?

A. Category delete করতে
B. Category অনুযায়ী আলাদা window তৈরি করতে
C. Category sort করতে
D. Category filter করতে

3. Running Total-এর জন্য কোনটি ব্যবহার করা হয়?

A. `SUM() OVER()`
B. `COUNT()`
C. `RANK()`
D. `LEAD()`

4. 3-month moving average-এর frame কোনটি?

A. `1 PRECEDING`
B. `2 PRECEDING AND CURRENT ROW`
C. `3 FOLLOWING`
D. `UNBOUNDED FOLLOWING`

5. `GROUP BY` করার পর ranking করতে কোন structure খুব useful?

A. CTE + Window Function
B. DELETE + JOIN
C. INSERT + LIMIT
D. UPDATE + WHERE

6. Customer-এর মোট sales ranking করতে কোনটি ব্যবহার করা যায়?

A. `RANK()`
B. `LAG()`
C. `LEAD()`
D. `FIRST_VALUE()`

7. `NULLIF(previous_sales, 0)` কেন ব্যবহার করা হয়?

A. Sorting-এর জন্য
B. Duplicate remove করতে
C. Division by zero এড়াতে
D. Ranking করতে

8. Cumulative percentage কী বোঝায়?

A. শুধু current row-এর percentage
B. এখন পর্যন্ত accumulated total-এর percentage
C. Maximum percentage
D. Average percentage

9. Category-এর ভিতরে product ranking-এর জন্য কোনটি সঠিক?

```
RANK() OVER (
    PARTITION BY _______
    ORDER BY sales DESC
)
```

A. product
B. category
C. sales
D. customer

10. আজকের project-এ সবচেয়ে গুরুত্বপূর্ণ workflow কোনটি?

A. `DELETE → UPDATE → INSERT`
B. `JOIN → GROUP BY → CTE → Window Function → Analysis`
C. `LIMIT → DISTINCT → DELETE`
D. `ORDER BY → DROP → CREATE`

হ্যাঁ, **আরও Class দরকার**—তবে এখন থেকে শুধু নতুন SQL syntax শেখানোর চেয়ে **real-world SQL + Database Design + Performance**-এর দিকে যাওয়া বেশি useful হবে।

তুমি যেহেতু **Database Designer / Data Analyst → Data Engineer** দিকে যেতে চাচ্ছো, আমি এইভাবে এগোতে বলব:

Class	Topic	গুরুত্ব
42	`CASE WHEN` + Advanced SQL Business Logic	⭐⭐⭐⭐⭐
43	`EXISTS`, `NOT EXISTS`, `IN`, `NOT IN`	⭐⭐⭐⭐⭐
44	Set Operations: `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`	⭐⭐⭐⭐
45	Date & Time Analytics	⭐⭐⭐⭐⭐
46	Advanced String & NULL Handling	⭐⭐⭐⭐
47	SQL Views	⭐⭐⭐⭐⭐
48	Stored Procedures & Functions	⭐⭐⭐⭐
49	Triggers	⭐⭐⭐⭐
50	Transactions + ACID + `COMMIT` / `ROLLBACK`	⭐⭐⭐⭐⭐

Class	Topic	গুরুত্ব
51	Indexing	⭐⭐⭐⭐⭐
52	Query Optimization + `EXPLAIN`	⭐⭐⭐⭐⭐
53	Database Normalization	⭐⭐⭐⭐⭐
54	Database Design + ERD	⭐⭐⭐⭐⭐
55	Fact & Dimension / Star Schema	⭐⭐⭐⭐⭐
56	SQL Interview Problems	⭐⭐⭐⭐⭐
57	Full SQL Data Analyst Project	⭐⭐⭐⭐⭐
58	Full Database Designer Project	⭐⭐⭐⭐⭐

এরপর কী?

Class 58-এর পর SQL-এর core অংশ মোটামুটি strong হয়ে যাবে।

তারপর তোমার roadmap হবে:

```
Excel ✅
   ↓
SQL ← এখন এখানে
   ↓
PostgreSQL
   ↓
Database Design
   ↓
Python
   ↓
Pandas
   ↓
Statistics
   ↓
Power BI
   ↓
ETL
   ↓
Airflow
   ↓
Docker
   ↓
Data Engineering
```

আমার recommendation

এখন **Class 42 থেকে চালিয়ে যাও**। বিশেষ করে `EXISTS → Transactions → Indexing → EXPLAIN → Normalization → Database Design` অংশগুলো তোমার জন্য খুব গুরুত্বপূর্ণ।

Class 42 হবে: `EXISTS, NOT EXISTS & Advanced Filtering` — কারণ এগুলো real-world SQL এবং interview দুটোতেই কাজে লাগে।

অবশ্যই। এবার **Class 42**-কে আরও structuredভাবে করি—বিশেষ করে `EXISTS`, `NOT EXISTS`, `IN`, `NOT IN`, এবং advanced filtering-এর পার্থক্যগুলো যেন পরিষ্কার হয়ে যায়।

📘 SQL Class 42

`EXISTS`, `NOT EXISTS` & Advanced Filtering

আজকের লক্ষ্য

Class শেষে তুমি পারবে:

- `EXISTS` ব্যবহার করতে
- `NOT EXISTS` ব্যবহার করতে
- `EXISTS` বনাম `IN` বুঝতে
- `NOT EXISTS` বনাম `NOT IN` বুঝতে
- correlated subquery বুঝতে
- `EXISTS + JOIN`
- `EXISTS + GROUP BY + HAVING`
- real-world filtering query লিখতে

---

1. আগে একটা Dataset

ধরি আমাদের ৩টি table:

`customers`

customer_id	name	city
1	Rahim	Dhaka
2	Karim	Chittagong
3	Hasan	Dhaka
4	Jamil	Khulna
5	Nayeem	Dhaka

`orders`

order_id	customer_id	amount
101	1	50000
102	1	30000
103	3	70000
104	5	20000

তাহলে:

```
Rahim  → 2 orders
Karim  → 0 orders
Hasan  → 1 order
Jamil  → 0 orders
Nayeem → 1 order
```

---

2. `EXISTS` কী?

সহজ ভাষায়:

> **কোনো matching row আছে কি না সেটা check করে।**

Syntax:

```sql
WHERE EXISTS (
    SELECT 1
    FROM table_name
    WHERE condition
)
```

---

3. Customers যাদের Order আছে

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

customer_id	name
1	Rahim
3	Hasan
5	Nayeem

কারণ এদের প্রত্যেকের অন্তত একটি order আছে।

---

4. `SELECT 1` কেন?

এখানে:

```
SELECT 1
```

এর অর্থ এই নয় যে আমরা `1` data চাইছি।

`EXISTS` শুধু জানতে চায়:

> matching row পাওয়া গেছে কি?

তাই:

```
SELECT 1
```

একটি common convention।

---

5. `EXISTS` কীভাবে চিন্তা করবে?

এই query:

```
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Rahim-এর জন্য:

```
Rahim customer_id = 1
        ↓
orders-এ customer_id = 1 আছে?
        ↓
YES
        ↓
Rahim রাখো
```

Karim:

```
Karim customer_id = 2
        ↓
orders-এ customer_id = 2 আছে?
        ↓
NO
        ↓
Karim বাদ
```

---

6. `NOT EXISTS`

এবার opposite।

> **Matching row নেই কি?**

```sql
SELECT *
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Result:

```
Karim
Jamil
```

মনে রাখো:

```
EXISTS
→ আছে?
```

```
NOT EXISTS
→ নেই?
```

---

7. `EXISTS` হলো Correlated Subquery

এই অংশটি লক্ষ্য করো:

```
o.customer_id = c.customer_id
```

ভেতরের query বাইরের query-এর `c.customer_id` ব্যবহার করছে।

এজন্য এটি:

> **Correlated Subquery**

```
Outer Query
customers c
     ↓
c.customer_id
     ↓
Inner Query
orders o
```

---

8. `IN` বনাম `EXISTS`

একই problem:

> যেসব customer order করেছে তাদের বের করো।

`IN`

```sql
SELECT *
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

`EXISTS`

```sql
SELECT *
FROM customers c
```

```
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

দুটোই এখানে একই ধরনের result দিতে পারে।

---

9. কখন `IN` ব্যবহার করব?

যদি তোমার চিন্তা হয়:

> "এই value কি এই list-এর মধ্যে আছে?"

তাহলে:

```
IN
```

Example:

```
SELECT *
FROM customers
WHERE city IN ('Dhaka', 'Khulna');
```

অর্থাৎ:

```
city কি Dhaka বা Khulna?
```

---

10. কখন `EXISTS` ব্যবহার করব?

যদি প্রশ্ন হয়:

> "এই row-এর সাথে matching record আছে?"

তাহলে `EXISTS` খুব natural।

Example:

```
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
```

```
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

---

11. `NOT IN` বনাম `NOT EXISTS` ⚠️

এটা খুব গুরুত্বপূর্ণ।

তাত্ত্বিকভাবে:

```
WHERE customer_id NOT IN (...)
```

দিয়ে missing customers বের করা যায়।

কিন্তু subquery-তে `NULL` থাকলে `NOT IN` unexpected result দিতে পারে।

তাই related record **নেই** check করার সময়:

```
NOT EXISTS
```

অনেক safer pattern।

Practical rule:

```
IN
→ list membership

EXISTS
→ existence

NOT EXISTS
→ absence
```

---

12. Product যেগুলো কখনো বিক্রি হয়নি

ধরি:

```
products.product_id
orders.product_id
```

Query:

```sql
SELECT *
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.product_id = p.product_id
);
```

এখানে আমরা খুঁজছি:

> Product-এর matching order নেই।

---

13. Product যেগুলো বিক্রি হয়েছে

```sql
SELECT *
FROM products p
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.product_id = p.product_id
);
```

---

14. Advanced Filtering: Amount

Question:

> যেসব customer অন্তত একটি `50000`-এর বেশি order করেছে।

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
      AND o.amount > 50000
);
```

এখানে:

```
customer match
+
```

```
amount > 50000
```

দুটো condition আছে।

---

15. Customer যারা 2026-এর March-এর পরে Order করেছে

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
      AND o.order_date >= '2026-03-01'
);
```

এটা real-world filtering-এর খুব common pattern।

---

16. `EXISTS + JOIN`

ধরি:

`products`

product_id	product_name	category
101	Laptop	Electronics
102	Monitor	Electronics
103	Chair	Furniture

Question:

> যেসব customer Electronics product কিনেছে।

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
```

```
      AND p.category = 'Electronics'
);
```

এখানে:

```
customers
    ↓
EXISTS
    ↓
orders
    ↓
JOIN
    ↓
products
    ↓
category = Electronics
```

---

17. Laptop কিনেছে এমন Customer

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.product_name = 'Laptop'
);
```

---

18. Electronics কখনো কেনেনি

```sql
SELECT *
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
```

```
      AND p.category = 'Electronics'
);
```

এখানে খুব সুন্দর একটা pattern:

```
NOT EXISTS
+
JOIN
+
condition
```

---

19. Electronics এবং Furniture দুটোই কিনেছে

এখানে দুইটি `EXISTS` ব্যবহার করা যায়।

```
SELECT *
FROM customers c

WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Electronics'
)

AND EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Furniture'
);
```

Logic:

```
Electronics?
   ↓ YES

Furniture?
   ↓ YES
```

```
Customer selected
```

---

20. `EXISTS + GROUP BY + HAVING`

এবার একটু advanced।

Question:

> যেসব customer কমপক্ষে 3টি order করেছে।

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    GROUP BY o.customer_id
    HAVING COUNT(*) >= 3
);
```

এখানে:

```
EXISTS
   +
GROUP BY
   +
HAVING
```

একসাথে ব্যবহার হচ্ছে।

---

21. Total Order Amount > 100000

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    GROUP BY o.customer_id
    HAVING SUM(o.amount) > 100000
);
```

অর্থাৎ:

> Customer-এর সব order-এর total > 100000 কি না।

---

22. `EXISTS` + CTE

CTE-এর সাথেও ব্যবহার করা যায়।

```
WITH recent_orders AS (
    SELECT *
    FROM orders
    WHERE order_date >= '2026-03-01'
)

SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM recent_orders r
    WHERE r.customer_id = c.customer_id
);
```

Workflow:

```
orders
   ↓
filter
   ↓
CTE
   ↓
EXISTS
   ↓
customers
```

---

23. JOIN বনাম EXISTS

একই প্রশ্ন:

> যেসব customer-এর order আছে।

JOIN:

```sql
SELECT DISTINCT
    c.customer_id,
    c.name
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id;
```

EXISTS:

```sql
SELECT
    c.customer_id,
    c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Main difference

JOIN:

> Matching rows combine করে।

EXISTS:

> Matching row আছে কি না check করে।

---

24. JOIN-এ `DISTINCT` কেন?

Rahim-এর যদি 3টা order থাকে:

```
Rahim
Rahim
Rahim
```

JOIN result-এ 3 rows আসতে পারে।

তাই:

```sql
DISTINCT
```

দিতে হতে পারে।

কিন্তু `EXISTS` শুধু existence check করে, তাই customer-এর row duplicate করে না।

---

25. Advanced Filtering-এর বড় Picture

SQL-এ filtering-এর জন্য এখন তোমার কাছে অনেক tools আছে:

```
WHERE
  ├── =
  ├── >
  ├── <
  ├── >=
  ├── <=
  ├── <>
  ├── AND
  ├── OR
  ├── IN
  ├── NOT IN
  ├── BETWEEN
  ├── LIKE
  ├── IS NULL
  ├── IS NOT NULL
  ├── EXISTS
  └── NOT EXISTS
```

আর grouped data-এর জন্য:

```
HAVING
```

🧠 সবচেয়ে গুরুত্বপূর্ণ পার্থক্য

Problem	Use
Value list-এর মধ্যে আছে?	`IN`
Value list-এর মধ্যে নেই?	`NOT IN`
Related row আছে?	`EXISTS`
Related row নেই?	`NOT EXISTS`
Pattern match	`LIKE`
Range	`BETWEEN`
NULL	`IS NULL`
Group filter	`HAVING`

🔥 Real-world Example

একজন Data Analyst-কে বলা হলো:

> "Find customers who have placed at least one order, have bought Electronics, but have never bought Furniture."

SQL:

```
SELECT *
FROM customers c

WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
)

AND EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Electronics'
)

AND NOT EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Furniture'
);
```

এখানে আমরা ব্যবহার করেছি:

```
EXISTS
+
EXISTS
+
NOT EXISTS
+
JOIN
```

```
+
AND
```

এটাই **advanced filtering**।

⭐ Golden Rules

Rule 1

```
EXISTS
```

মানে:

> **"আছে?"**

Rule 2

```
NOT EXISTS
```

মানে:

> **"নেই?"**

Rule 3

```
IN
```

মানে:

> **"এই list-এর মধ্যে আছে?"**

Rule 4

Missing related records খুঁজতে:

```
NOT EXISTS
```

খুব useful।

Rule 5

`NOT IN` + `NULL` থেকে সাবধান।

Rule 6

`EXISTS` সাধারণত **data return করার জন্য নয়; existence check করার জন্য।**

📝 Class 42 Assignment

Q1

যেসব customer অন্তত একটি order করেছে।

Q2

যেসব customer কোনো order করেনি।

Q3

যেসব product কখনো sell হয়নি।

Q4

যেসব product অন্তত একবার sell হয়েছে।

Q5

যেসব customer Electronics কিনেছে।

Q6

যেসব customer Electronics কখনো কেনেনি।

Q7

যেসব customer Laptop কিনেছে।

Q8

যেসব customer `2026-03-01` -এর পরে order করেছে।

Q9

যেসব customer অন্তত 3টি order করেছে।

Q10

যেসব customer-এর total order amount `100000` -এর বেশি।

🔥 Q11

যেসব customer **Electronics এবং Furniture দুটোই** কিনেছে।

🔥 Q12

যেসব customer **order করেছে কিন্তু Electronics কখনো কেনেনি।**

🧪 Quiz

1. `EXISTS` কী check করে?

A. Sorting
B. Matching row exists কি না
C. Duplicate
D. NULL

2. `NOT EXISTS` কী check করে?

A. Matching row আছে
B. Matching row নেই
C. Sorting
D. Ranking

3. `EXISTS` -এর ভিতরে common pattern কোনটি?

A. `SELECT 1`
B. `DELETE 1`
C. `UPDATE 1`
D. `DROP 1`

4. Customer-এর কোনো order নেই—কোনটি সবচেয়ে natural?

A. `EXISTS`
B. `NOT EXISTS`
C. `RANK()`
D. `LEAD()`

5. `NOT IN` -এর ক্ষেত্রে কোনটি বিশেষভাবে problematic হতে পারে?

A. JOIN
B. NULL
C. ORDER BY
D. LIMIT

6. `EXISTS + GROUP BY` -এর aggregate condition কোথায় দেওয়া যায়?

A. HAVING
B. ORDER BY

C. DISTINCT
D. LIMIT

7. Customer-এর অন্তত একটি order আছে কিনা check করতে সবচেয়ে direct approach কোনটি?

A. `EXISTS`
B. `RANK()`
C. `AVG()`
D. `CASE`

8. JOIN-এর ফলে duplicate customer rows এলে unique করার জন্য কী ব্যবহার করা যায়?

A. `DISTINCT`
B. `HAVING`
C. `LAG()`
D. `BETWEEN`

9. Missing related records-এর জন্য কোনটি সবচেয়ে useful?

A. `NOT EXISTS`
B. `ORDER BY`
C. `LIMIT`
D. `RANK()`

10. `EXISTS` কি subquery-এর actual returned value-এর ওপর মূলত নির্ভর করে?

A. Yes
B. No — matching row আছে কি না সেটাই মূল বিষয়

🎯 আজকের ৩টি Query অবশ্যই বুঝবে

Has orders

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Has no orders

```sql
SELECT *
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Bought Electronics

```sql
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    WHERE o.customer_id = c.customer_id
      AND p.category = 'Electronics'
);
```

🧠 One-line memory trick

> **`IN` = list-এর মধ্যে? | `EXISTS` = matching record আছে? | `NOT EXISTS` = matching record নেই?**

📘 Class 43 — SQL Set Operations

আজকে আমরা শিখব **Set Operations**:

- `UNION`
- `UNION ALL`
- `INTERSECT`
- `EXCEPT`
- Multiple Set Operations
- `ORDER BY` with Set Operations
- `JOIN vs UNION`
- Real-world Data Analytics examples
- Common mistakes
- Assignment + MCQ

1️⃣ Set Operation কী?

ধরো আমাদের দুইটা query আছে:

```
SELECT city FROM customers;
```

এবং

```
SELECT city FROM suppliers;
```

দুই query-এর result একসাথে combine করতে চাইলে **Set Operations** ব্যবহার করতে পারি।

সহজভাবে:

> **Set Operation = দুই বা তার বেশি SELECT query-এর result combine করা।**

2️⃣ UNION

`UNION` দুই query-এর result combine করে এবং **duplicate বাদ দেয়**।

Example

Customers:

name	city
Rahim	Dhaka
Karim	Chittagong
Hasan	Dhaka

Suppliers:

name	city
ABC	Dhaka
XYZ	Khulna

Query:

```
SELECT city
FROM customers
```

```
UNION

SELECT city
FROM suppliers;
```

Result:

city
Dhaka
Chittagong
Khulna

`Dhaka` দুইবার থাকলেও একবার এসেছে।

Golden Rule

> `UNION` = Combine + Remove Duplicate

3️⃣ UNION ALL

`UNION ALL` দুই result combine করে কিন্তু **duplicate রাখে**।

```
SELECT city
FROM customers

UNION ALL

SELECT city
FROM suppliers;
```

Result হতে পারে:

city
Dhaka
Chittagong
Dhaka
Khulna

এখানে Dhaka দুইবার এসেছে।

Golden Rule

> `UNION ALL` = Combine + Keep Duplicate

4️⃣ UNION vs UNION ALL

Feature	UNION	UNION ALL
Combine result	✅	✅
Duplicate remove	✅	❌
Duplicate রাখে	❌	✅
Usually faster	❌	✅
Use when	Unique result দরকার	সব records দরকার

Performance

যদি duplicate remove করার দরকার না থাকে:

```
UNION ALL
```

সাধারণত ভালো choice।

কারণ `UNION`-কে duplicate checking করতে হয়।

5️⃣ UNION-এর গুরুত্বপূর্ণ Rule

দুই query-তে **same number of columns** থাকতে হবে।

✅ Correct:

```
SELECT name, city
FROM customers

UNION

SELECT supplier_name, city
FROM suppliers;
```

দুই পাশে 2টি column।

❌ Incorrect:

```
SELECT name, city
FROM customers

UNION

SELECT supplier_name
FROM suppliers;
```

প্রথম query = 2 columns
দ্বিতীয় query = 1 column

তাই error হবে।

6️⃣ Column Data Type Compatible হতে হবে

যেমন:

```
SELECT customer_id
FROM customers

UNION

SELECT supplier_id
FROM suppliers;
```

দুইটাই integer → ভালো।

কিন্তু:

```
SELECT customer_id
FROM customers

UNION

SELECT supplier_name
FROM suppliers;
```

একটি integer, অন্যটি text।

এটা সাধারণত logical/type সমস্যা তৈরি করবে।

7️⃣ Column Name কোথা থেকে আসে?

`UNION` -এর final column name সাধারণত **প্রথম SELECT** থেকে আসে।

Example:

```
SELECT city AS location
FROM customers

UNION

SELECT city
FROM suppliers;
```

Result column:

```
location
```

দ্বিতীয় query-এর column name এখানে final name হবে না।

8️⃣ INTERSECT

`INTERSECT` দুই query-তে **common records** বের করে।

ধরো:

Customers:

```
Dhaka
Chittagong
Khulna
```

Suppliers:

```
Dhaka
Khulna
Rajshahi
```

Query:

```
SELECT city
FROM customers

INTERSECT
```

```sql
SELECT city
FROM suppliers;
```

Result:

```
Dhaka
Khulna
```

কারণ দুই জায়গাতেই আছে।

Golden Rule

> `INTERSECT` = Common records

9️⃣ Important: MySQL

এখানে একটি গুরুত্বপূর্ণ বিষয়।

MySQL-এ traditional `INTERSECT` support version-dependent/modern versions-এ available, কিন্তু অনেক পুরোনো MySQL environment-এ এটি ছিল না।

তুমি যেহেতু MySQL জানো, practical alternative হিসেবে `INNER JOIN` বা `EXISTS` ব্যবহার করতে পারো।

Example:

```sql
SELECT DISTINCT c.city
FROM customers c
JOIN suppliers s
ON c.city = s.city;
```

এটা common city বের করতে পারে।

🔟 EXCEPT

`EXCEPT` প্রথম query-তে আছে কিন্তু দ্বিতীয় query-তে নেই—এমন records দেয়।

Example:

Customers:

```
Dhaka
Chittagong
Khulna
```

Suppliers:

```
Dhaka
Khulna
Rajshahi
```

```sql
SELECT city
FROM customers

EXCEPT

SELECT city
FROM suppliers;
```

Result:

```
Chittagong
```

কারণ:

```
Customers → Dhaka, Chittagong, Khulna
Suppliers → Dhaka, Khulna

Difference → Chittagong
```

Golden Rule

> `EXCEPT` = First result − Second result

1️⃣1️⃣ MySQL-এ EXCEPT-এর Alternative

যদি তোমার MySQL environment-এ `EXCEPT` না থাকে, তাহলে `NOT EXISTS` ব্যবহার করতে পারো।

```sql
SELECT DISTINCT c.city
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM suppliers s
```

```
    WHERE s.city = c.city
);
```

এটা customers-এর এমন city বের করবে যেগুলো suppliers-এর মধ্যে নেই।

এখানে Class 42-এর `NOT EXISTS` concept আবার কাজে লাগছে। 🔥

1️⃣2️⃣ UNION + ORDER BY

খুব গুরুত্বপূর্ণ।

```
SELECT city
FROM customers

UNION

SELECT city
FROM suppliers

ORDER BY city;
```

সাধারণত পুরো combined result-এর শেষে `ORDER BY` দেবে।

ভুল approach

প্রতিটি SELECT-এর শেষে আলাদা `ORDER BY` দেওয়া unnecessary/সমস্যাজনক হতে পারে।

সাধারণ pattern:

```
SELECT ...
FROM ...

UNION

SELECT ...
FROM ...

ORDER BY ...;
```

1️⃣3️⃣ Real-World Example: Customer + Supplier Cities

ধরো company জানতে চায়:

> "আমাদের customer এবং supplier—দুই পক্ষের মোট কোন কোন city আছে?"

```
SELECT city
FROM customers

UNION

SELECT city
FROM suppliers

ORDER BY city;
```

এতে unique city list পাওয়া যাবে।

1️⃣4️⃣ UNION ALL — Real Business Example

ধরো আমাদের দুই বছরের sales table:

```
sales_2025
sales_2026
```

দুই table-এর সব sales একসাথে আনতে চাই।

```
SELECT order_id, customer_id, amount
FROM sales_2025

UNION ALL

SELECT order_id, customer_id, amount
FROM sales_2026;
```

এখানে `UNION ALL` ব্যবহার করা ভালো।

কারণ আমরা **সব sales record রাখতে চাই**।

1️⃣5️⃣ UNION কেন এখানে ভুল হতে পারে?

ধরো:

```
sales_2025
```

এবং

```
sales_2026
```

দুই table-এ একই ধরনের order record থাকতে পারে।

যদি লিখো:

```
UNION
```

তাহলে exact duplicate rows remove হয়ে যেতে পারে।

কিন্তু sales analytics-এ প্রতিটি transaction গুরুত্বপূর্ণ হতে পারে।

তাই:

```
UNION ALL
```

সাধারণত better।

1️⃣6️⃣ JOIN vs UNION

এটা খুব গুরুত্বপূর্ণ interview question।

JOIN

দুই table-এর **columns combine করে**।

Example:

```
customers
+
orders
```

```
SELECT c.name, o.amount
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id;
```

Result:

name	amount
Rahim	50000
Karim	30000

অর্থাৎ columns পাশাপাশি এসেছে।

UNION

দুই query-এর **rows combine করে**।

```
SELECT name
FROM customers

UNION

SELECT supplier_name
FROM suppliers;
```

Result:

name
Rahim
Karim
ABC
XYZ

🧠 Golden Difference

> **JOIN = Columns পাশে পাশে combine করে**
> **UNION = Rows উপর-নিচে combine করে**

Visual:

JOIN

```
Customer       Orders
--------       ------
Rahim    +     50000
Karim    +     30000
```

➡️ columns horizontally combine

UNION

```
Customers
---------
Rahim
Karim
  ↓
Suppliers
---------
ABC
XYZ
```

➡️ rows vertically combine

1️⃣7️⃣ Multiple UNION

তিনটি query-ও combine করা যায়।

```
SELECT city
FROM customers

UNION

SELECT city
FROM suppliers

UNION

SELECT city
FROM employees;
```

Result:

```
Dhaka
Chittagong
Khulna
Rajshahi
```

Duplicate থাকলে `UNION` remove করবে।

1️⃣8️⃣ UNION ALL + Multiple Tables

```
SELECT email
FROM customers
```

```sql
UNION ALL

SELECT email
FROM employees

UNION ALL

SELECT email
FROM suppliers;
```

এটা সব email একসাথে আনবে এবং duplicate রাখবে।

1️⃣9️⃣ Set Operation + WHERE

প্রতিটি query-তে নিজের `WHERE` থাকতে পারে।

```sql
SELECT city
FROM customers
WHERE city = 'Dhaka'

UNION

SELECT city
FROM suppliers
WHERE city = 'Dhaka';
```

Result:

```
Dhaka
```

2️⃣0️⃣ Set Operation + Aggregate

এটাও করা যায়।

```sql
SELECT COUNT(*) AS total_customers
FROM customers

UNION ALL

SELECT COUNT(*) AS total_suppliers
FROM suppliers;
```

Result:

total_customers
100
25

2️⃣1️⃣ UNION + Label

Analytics report-এ খুব useful।

```
SELECT
    'Customer' AS source,
    name
FROM customers

UNION ALL

SELECT
    'Supplier' AS source,
    supplier_name
FROM suppliers;
```

Result:

source	name
Customer	Rahim
Customer	Karim
Supplier	ABC
Supplier	XYZ

এটা data warehouse/reporting-এ খুব useful technique।

2️⃣2️⃣ NULL-এর সাথে UNION

ধরো দুই table-এর structure আলাদা।

Customers:

```
name
city
```

Suppliers:

```
supplier_name
city
```

তবুও combine করা যাবে:

```sql
SELECT
    name,
    city,
    NULL AS company
FROM customers

UNION ALL

SELECT
    supplier_name,
    city,
    'Supplier Company' AS company
FROM suppliers;
```

এখানে দুই query-তেই 3 columns আছে।

2️⃣3️⃣ Set Operations Cheat Sheet

Operator	Meaning
`UNION`	Combine + remove duplicates
`UNION ALL`	Combine + keep duplicates
`INTERSECT`	Common rows
`EXCEPT`	First − Second

মনে রাখার shortcut:

```
UNION       → OR
INTERSECT   → AND
EXCEPT      → DIFFERENCE
```

2️⃣4️⃣ Excel-এর সাথে Connection

তুমি Excel অনেক গভীরভাবে শিখেছো, তাই এভাবে মনে রাখো:

Excel	SQL
Append Queries	`UNION ALL`
Remove duplicates after append	`UNION`
Common records	`INTERSECT`
Difference	`EXCEPT`
Merge Queries	`JOIN`

বিশেষ করে Power Query-তে:

> **Append ≈ UNION ALL**

এটা খুব important connection।

2️⃣5️⃣ Common Beginner Mistakes

❌ Mistake 1: Column count different

```sql
SELECT name, city
FROM customers

UNION

SELECT supplier_name
FROM suppliers;
```

❌ ভুল।

❌ Mistake 2: JOIN আর UNION confuse করা

মনে রাখো:

```
JOIN  → columns combine
UNION → rows combine
```

❌ Mistake 3: UNION ALL-এর বদলে UNION

Transaction data combine করার সময় unnecessary duplicate removal performance কমাতে পারে।

❌ Mistake 4: EXCEPT direction ভুল বোঝা

```
A EXCEPT B
```

মানে:

> A-তে আছে কিন্তু B-তে নেই।

এটা:

```
A - B
```

🎯 Class 43 Assignment

ধরো তিনটি table:

```
customers
---------
customer_id
name
city

employees
---------
employee_id
name
city

suppliers
---------
supplier_id
name
city
```

Q1

Customers এবং employees-এর সব city combine করো, duplicate বাদ দিয়ে।

Q2

Customers এবং employees-এর সব city combine করো, duplicate সহ।

Q3

Customers এবং suppliers-এর common city বের করো।

Q4

Customers-এর এমন city বের করো যা suppliers-এ নেই।

Q5

Customers + employees + suppliers-এর unique city list বের করো।

Q6

Customers এবং employees-এর names একসাথে আনো এবং source column তৈরি করো:

```
source | name
```

Q7

`UNION` এবং `UNION ALL`-এর মধ্যে পার্থক্য লিখো।

Q8

`JOIN` এবং `UNION`-এর মূল পার্থক্য লিখো।

Q9

Sales 2025 + Sales 2026 combine করার জন্য `UNION` নাকি `UNION ALL` বেশি উপযুক্ত? কেন?

Q10

`A EXCEPT B` কী return করে?

🧠 Class 43 MCQ

1. `UNION` কী করে?

A. Duplicate রাখে
B. Duplicate remove করে
C. Table delete করে
D. Table join করে

2. `UNION ALL` কী করে?

A. Duplicate remove করে
B. Duplicate রাখে
C. শুধু common rows দেয়
D. শুধু NULL দেয়

3. Common records পেতে কোন operator?

A. UNION
B. EXCEPT
C. INTERSECT
D. JOIN

4. `A EXCEPT B` মানে কী?

A. A এবং B-এর common
B. B-তে আছে A-তে নেই
C. A-তে আছে B-তে নেই
D. সব records

5. JOIN মূলত কী combine করে?

A. Rows
B. Columns
C. Databases
D. Indexes

6. UNION মূলত কী combine করে?

A. Rows/results
B. Columns only
C. Indexes
D. Constraints

7. দুই SELECT-এর UNION করতে হলে কী গুরুত্বপূর্ণ?

A. একই table হতে হবে
B. Same number of columns
C. Same table name
D. Same primary key

8. Sales transactions combine করার ক্ষেত্রে সাধারণত কোনটি বেশি উপযুক্ত?

A. UNION
B. UNION ALL
C. INTERSECT
D. EXCEPT

9. `UNION`-এর duplicate removal-এর কারণে কোনটি হতে পারে?

A. Additional processing
B. Table deletion
C. Foreign key creation
D. Database creation

10. Power Query-এর Append-এর সবচেয়ে কাছাকাছি SQL operator কোনটি?

A. JOIN
B. UNION ALL
C. EXISTS
D. GROUP BY

🔥 আজকের সবচেয়ে গুরুত্বপূর্ণ ৪টি লাইন

```
UNION      = Combine + Remove Duplicate
UNION ALL  = Combine + Keep Duplicate
INTERSECT  = Common
EXCEPT     = First − Second
```

এবং:

```
JOIN  → Columns combine
UNION → Rows combine
```

Class 43-এর পরের logical topic: Class 44 — SQL Date & Time Analytics, যেখানে আমরা `YEAR()`, `MONTH()`, `DATEDIFF()`, monthly sales, monthly growth, date filtering, age/tenure এবং real-world date analysis করব।

📘 Class 44 — SQL Date & Time Analytics

আজকের class খুব important, কারণ **Data Analyst / Data Engineer / Database Designer** হিসেবে date নিয়ে কাজ প্রায় প্রতিদিনই করতে হবে।

আজ আমরা শিখব:

1. Date & Time data types
2. `YEAR()`, `MONTH()`, `DAY()`
3. `CURDATE()`, `NOW()`
4. Date filtering

5. `BETWEEN`
6. `DATEDIFF()`
7. Date addition/subtraction
8. Monthly/Yearly analysis
9. Monthly sales
10. Monthly growth
11. Age / Experience calculation
12. Date grouping
13. Common mistakes
14. Assignment + MCQ

1️⃣ SQL-এ Date কেন গুরুত্বপূর্ণ?

ধরো orders table:

order_id	order_date	amount
1	2026-01-05	50000
2	2026-01-15	30000
3	2026-02-10	70000
4	2026-03-20	40000

Business প্রশ্ন হতে পারে:

- January-তে কত sales?
- March-এ কত orders?
- গত 30 দিনে কত orders?
- দুই order-এর মধ্যে কত দিন?
- Customer কত বছর ধরে আছে?
- এই বছরের sales কত?
- Previous month থেকে growth কত?

এসবের জন্য Date & Time SQL জানা দরকার।

2️⃣ Common Date/Time Data Types

MySQL-এ প্রধানগুলো:

Type	Example	ব্যবহার
`DATE`	`2026-09-13`	শুধু date
`TIME`	`14:30:00`	শুধু time
`DATETIME`	`2026-09-13 14:30:00`	date + time
`TIMESTAMP`	`2026-09-13 14:30:00`	timestamp/time tracking

সাধারণ order date:

```
order_date DATE
```

Created time:

```
created_at DATETIME
```

3️⃣ DATE থেকে YEAR বের করা

```
SELECT
    order_date,
    YEAR(order_date) AS order_year
FROM orders;
```

Result:

order_date	order_year
2026-01-05	2026
2026-02-10	2026
2026-03-20	2026

4️⃣ MONTH()

```
SELECT
    order_date,
    MONTH(order_date) AS order_month
FROM orders;
```

Result:

order_date	order_month
2026-01-05	1
2026-02-10	2
2026-03-20	3

5️⃣ DAY()

```
SELECT
    order_date,
    DAY(order_date) AS order_day
FROM orders;
```

Result:

```
05
10
20
```

6️⃣ একসাথে YEAR + MONTH + DAY

```
SELECT
    order_date,
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    DAY(order_date) AS day
FROM orders;
```

এটা date analysis-এর basic foundation।

7️⃣ CURDATE()

আজকের date বের করতে:

```
SELECT CURDATE();
```

যেমন আজ যদি:

```
2026-09-13
```

হয়, result হবে:

```
2026-09-13
```

Use case

আজকের orders:

```sql
SELECT *
FROM orders
WHERE order_date = CURDATE();
```

8️⃣ NOW()

Current date + time:

```sql
SELECT NOW();
```

Example:

```
2026-09-13 09:30:25
```

Difference:

```
CURDATE() → শুধু date
NOW()     → date + time
```

9️⃣ Current Year-এর Orders

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = YEAR(CURDATE());
```

এখানে:

```
YEAR(order_date)
```

order-এর year বের করছে।

আর:

```
YEAR(CURDATE())
```

বর্তমান year বের করছে।

🔟 নির্দিষ্ট Year Filter

2026 সালের orders:

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

1️⃣1️⃣ নির্দিষ্ট Month

January 2026:

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026
  AND MONTH(order_date) = 1;
```

1️⃣2️⃣ Date Range

ধরো January 1 থেকে January 31:

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2026-01-01'
                     AND '2026-01-31';
```

মনে রাখবে

`BETWEEN` inclusive।

অর্থাৎ:

```
2026-01-01 ✅
2026-01-31 ✅
```

দুটিই included।

1️⃣3️⃣ `>=` এবং `<` দিয়ে Date Filtering

Production SQL-এ অনেক সময় এই pattern বেশি useful:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2026-02-01';
```

এখানে January পুরো month পাওয়া যাবে।

এটা বিশেষভাবে `DATETIME` column-এর ক্ষেত্রে গুরুত্বপূর্ণ।

1️⃣4️⃣ কেন `< next month` useful?

ধরো:

```
2026-01-31 23:59:59
```

তখন শুধু:

```
<= '2026-01-31'
```

ব্যবহার করলে `DATETIME` -এর ক্ষেত্রে unexpected result হতে পারে।

তাই:

```sql
WHERE created_at >= '2026-01-01'
  AND created_at < '2026-02-01'
```

এটা clean pattern।

Golden Rule ⭐

> Date range-এর ক্ষেত্রে অনেক production query-তে `>= start AND < next_period` pattern খুব useful।

1️⃣5️⃣ DATEDIFF()

দুই date-এর মধ্যে কত দিন difference তা বের করে।

```
SELECT DATEDIFF(
    '2026-09-13',
    '2026-09-01'
);
```

Result:

```
12
```

কারণ 1 September → 13 September = 12 days।

1️⃣6️⃣ Order কতদিন আগে হয়েছে?

```
SELECT
    order_id,
    order_date,
    DATEDIFF(CURDATE(), order_date) AS days_ago
FROM orders;
```

Result:

order_id	order_date	days_ago
1	2026-09-01	12
2	2026-09-05	8

1️⃣7️⃣ Last 30 Days

```sql
SELECT *
FROM orders
WHERE order_date >= CURDATE() - INTERVAL 30 DAY;
```

এখানে:

```sql
INTERVAL 30 DAY
```

মানে 30 দিন।

1️⃣8️⃣ Last 7 Days

```sql
SELECT *
FROM orders
WHERE order_date >= CURDATE() - INTERVAL 7 DAY;
```

1️⃣9️⃣ Next 7 Days

যদি future date থাকে:

```sql
SELECT *
FROM appointments
WHERE appointment_date
      BETWEEN CURDATE()
      AND CURDATE() + INTERVAL 7 DAY;
```

2️⃣0️⃣ DATE_ADD()

কোনো date-এর সাথে সময় যোগ করতে:

```sql
SELECT DATE_ADD(
    '2026-01-01',
    INTERVAL 10 DAY
);
```

Result:

```
2026-01-11
```

---

1 Month যোগ

```
SELECT DATE_ADD(
    '2026-01-01',
    INTERVAL 1 MONTH
);
```

Result:

```
2026-02-01
```

2️⃣1️⃣ DATE_SUB()

Date থেকে সময় কমাতে:

```
SELECT DATE_SUB(
    '2026-09-13',
    INTERVAL 7 DAY
);
```

Result:

```
2026-09-06
```

2️⃣2️⃣ YEAR-MONTH Analysis

এখন real analytics শুরু করি।

ধরো:

```
orders
```

এবং:

```
quantity
price
```

Monthly sales:

```sql
SELECT
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    YEAR(o.order_date),
    MONTH(o.order_date)
ORDER BY
    year,
    month;
```

Result:

year	month	total_sales
2026	1	110000
2026	2	90000
2026	3	120000
2026	4	150000

2️⃣3️⃣ Better Month Format

শুধু `1`, `2`, `3` না দেখিয়ে:

```
2026-01
2026-02
2026-03
```

দেখাতে পারি।

MySQL-এ:

```sql
DATE_FORMAT(order_date, '%Y-%m')
```

Example:

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    COUNT(*) AS total_orders
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

2️⃣4️⃣ Month Name

```
SELECT
    MONTHNAME(order_date) AS month_name,
    COUNT(*) AS total_orders
FROM orders
GROUP BY MONTHNAME(order_date);
```

Result:

```
January
February
March
```

⚠️ কিন্তু এখানে একটি problem আছে।

যদি multiple year থাকে:

```
January 2025
January 2026
```

দুটো January একসাথে group হয়ে যাবে।

তাই business analytics-এ:

```
DATE_FORMAT(order_date, '%Y-%m')
```

অনেক বেশি useful।

2️⃣5️⃣ Yearly Sales

```sql
SELECT
    YEAR(order_date) AS year,
    SUM(amount) AS total_sales
FROM orders
GROUP BY YEAR(order_date)
ORDER BY year;
```

2️⃣6️⃣ Quarterly Analysis

এক বছরে 4টি quarter:

```
Q1 → Jan-Mar
Q2 → Apr-Jun
Q3 → Jul-Sep
Q4 → Oct-Dec
```

MySQL-এ:

```sql
SELECT
    YEAR(order_date) AS year,
    QUARTER(order_date) AS quarter,
    SUM(amount) AS total_sales
FROM orders
GROUP BY
    YEAR(order_date),
    QUARTER(order_date)
ORDER BY
    year,
    quarter;
```

2️⃣7️⃣ Weekday Analysis

কোন দিনে বেশি orders?

```sql
SELECT
    DAYNAME(order_date) AS day_name,
    COUNT(*) AS total_orders
FROM orders
GROUP BY DAYNAME(order_date);
```

যেমন:

day_name	total_orders
Monday	15
Tuesday	22
Friday	30

2️⃣8️⃣ Weekday Number

```
SELECT
    DAYOFWEEK(order_date) AS day_number
FROM orders;
```

MySQL-এ সাধারণভাবে:

```
1 = Sunday
2 = Monday
...
7 = Saturday
```

2️⃣9️⃣ Age Calculation

ধরো customers:

```
date_of_birth
```

Age:

```
SELECT
    name,
    TIMESTAMPDIFF(
        YEAR,
        date_of_birth,
        CURDATE()
    ) AS age
FROM customers;
```

এটা age calculation-এর practical pattern।

3️⃣0️⃣ Employee Experience

ধরো:

```
joining_date
```

কত বছর কাজ করেছে:

```
SELECT
    name,
    TIMESTAMPDIFF(
        YEAR,
        joining_date,
        CURDATE()
    ) AS experience_years
FROM employees;
```

3️⃣1️⃣ Exact Months of Experience

```
SELECT
    name,
    TIMESTAMPDIFF(
        MONTH,
        joining_date,
        CURDATE()
    ) AS experience_months
FROM employees;
```

3️⃣2️⃣ Customer Retention-এর Basic Idea

Customer-এর first order date বের করা:

```
SELECT
    customer_id,
    MIN(order_date) AS first_order_date
FROM orders
GROUP BY customer_id;
```

Last order:

```sql
SELECT
    customer_id,
    MAX(order_date) AS last_order_date
FROM orders
GROUP BY customer_id;
```

এগুলো পরে **Customer Analytics / Retention / Cohort Analysis**-এ খুব useful হবে।

3️⃣3️⃣ Days Since Last Order

```sql
SELECT
    customer_id,
    MAX(order_date) AS last_order_date,
    DATEDIFF(
        CURDATE(),
        MAX(order_date)
    ) AS days_since_last_order
FROM orders
GROUP BY customer_id;
```

এখানে business প্রশ্ন:

> কোন customer অনেকদিন ধরে order করেনি?

3️⃣4️⃣ Inactive Customers

ধরো 90 দিন ধরে order নেই:

```sql
SELECT
    customer_id,
    MAX(order_date) AS last_order_date
FROM orders
GROUP BY customer_id
HAVING MAX(order_date)
       < CURDATE() - INTERVAL 90 DAY;
```

🔥 এখানে দেখো:

```
WHERE → individual rows
HAVING → grouped customer result
```

এটা Class 34 + Class 44-এর combination।

3️⃣5️⃣ Monthly Growth

এটা আমরা Window Function-এ আগেও করেছি।

প্রথমে monthly sales:

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(amount) AS sales
    FROM orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales
FROM monthly_sales;
```

Result:

month	sales	previous_sales
2026-01	100000	NULL
2026-02	120000	100000
2026-03	150000	120000

3️⃣6️⃣ Monthly Growth %

এখন calculation:

```
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(amount) AS sales
    FROM orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
),
```

```sql
comparison AS (
    SELECT
        month,
        sales,
        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales
    FROM monthly_sales
)
SELECT
    month,
    sales,
    previous_sales,
    ROUND(
        100.0 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percentage
FROM comparison;
```

3️⃣7️⃣ কেন `NULLIF()`?

ধরো previous sales:

```
0
```

তাহলে:

```sql
sales / 0
```

করলে division-by-zero problem হবে।

তাই:

```sql
NULLIF(previous_sales, 0)
```

0 হলে `NULL` করে দেয়।

Golden Rule

> Division করার সময় denominator 0 হওয়ার সম্ভাবনা থাকলে `NULLIF(..., 0)` useful।

3️⃣8️⃣ Date Analytics Full Workflow

একজন Data Analyst হিসেবে তুমি অনেক query এভাবে build করবে:

```
Raw Orders
    ↓
JOIN
    ↓
Date Extraction
    ↓
GROUP BY
    ↓
Monthly/Yearly Sales
    ↓
LAG()
    ↓
Growth %
    ↓
Business Insight
```

3️⃣9️⃣ Excel → SQL Mapping

তুমি Excel আগে শিখেছো, তাই connection:

Excel	SQL
YEAR()	`YEAR()`
MONTH()	`MONTH()`
TODAY()	`CURDATE()`
NOW()	`NOW()`
Date Difference	`DATEDIFF()`
Add days	`DATE_ADD()`
Subtract days	`DATE_SUB()`
Pivot by Month	`GROUP BY` date
Previous Month	`LAG()`
Monthly Growth	`LAG()` + calculation
Current Date	`CURDATE()`

⚠️ 4️⃣0️⃣ Common Beginner Mistakes

❌ Mistake 1

Date-কে string হিসেবে compare করা:

```
WHERE order_date = 'January';
```

ভুল।

Date column-এর actual format বুঝতে হবে।

❌ Mistake 2

Month-only grouping:

```
GROUP BY MONTH(order_date)
```

যদি multiple year থাকে, 2025 January এবং 2026 January একসাথে চলে আসবে।

Better:

```
GROUP BY
    YEAR(order_date),
    MONTH(order_date);
```

অথবা:

```
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

❌ Mistake 3

`DATETIME`-এ শুধু শেষ date ব্যবহার করা

```
WHERE created_at <= '2026-01-31'
```

এর ফলে January 31-এর দিনের পরের time অংশ বাদ পড়তে পারে।

Better:

```
WHERE created_at >= '2026-01-01'
  AND created_at < '2026-02-01';
```

❌ Mistake 4

Age = Year difference

শুধু:

```
YEAR(CURDATE()) - YEAR(date_of_birth)
```

ব্যবহার করলে birthday এখনো আসেনি—এমন ক্ষেত্রে age 1 বছর বেশি হতে পারে।

Practical age-এর জন্য:

```
TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())
```

ব্যবহার করা ভালো।

🧠 Class 44 Golden Rules

```
YEAR()        → Year বের করে
MONTH()       → Month বের করে
DAY()         → Day বের করে

CURDATE()     → আজকের date
NOW()         → current date + time

DATEDIFF()    → দুই date-এর day difference

DATE_ADD()    → date-এ যোগ
DATE_SUB()    → date থেকে বাদ

TIMESTAMPDIFF() → বছর/month/day difference

DATE_FORMAT()   → date সুন্দরভাবে format/group
```

সবচেয়ে important:

> **Date Analytics = Date Extraction + Filtering + Grouping + Difference + Window Functions**

📝 Class 44 Assignment

ধরো `orders` table:

```
order_id
customer_id
order_date
amount
```

Q1

সব order-এর year বের করো।

Q2

সব order-এর month বের করো।

Q3

2026 সালের সব orders বের করো।

Q4

January 2026-এর orders বের করো।

Q5

গত 30 দিনের orders বের করো।

Q6

প্রতিটি order আজ থেকে কতদিন আগে হয়েছে বের করো।

Q7

Year-wise total sales বের করো।

Q8

Month-wise total sales বের করো `YYYY-MM` format-এ।

Q9

Quarter-wise sales বের করো।

Q10

প্রতিটি customer-এর last order date বের করো।

Q11

প্রতিটি customer-এর last order থেকে কতদিন হয়েছে বের করো।

Q12

90 দিনের বেশি সময় ধরে order করেনি এমন customer বের করো।

Q13

Monthly sales এবং previous month sales বের করো।

Q14

Monthly growth percentage বের করো।

Q15 ⭐

একটি final query তৈরি করো যেখানে থাকবে:

```
month
sales
previous_sales
growth_percentage
running_total
```

🧪 Class 44 MCQ

1. শুধু current date পেতে কোন function?

A. `NOW()`
B. `CURDATE()`
C. `DATE_NOW()`
D. `CURRENT_YEAR()`

2. দুই date-এর day difference বের করতে কোন function?

A. `DATE_DIFF()`
B. `DATEDIFF()`
C. `DAYDIFF()`
D. `DIFF_DATE()`

3. Current date + time কোনটি?

A. `CURDATE()`
B. `NOW()`

C. `TODAY()`
D. `CURRENT_DAY()`

4. 2026-এর month বের করতে কোন function?

A. `YEAR()`
B. `MONTH()`
C. `DATE()`
D. `MONTHNAME()`

5. Date-এর সাথে 7 দিন যোগ করতে কোনটি?

A. `DATE_ADD()`
B. `DATE_PLUS()`
C. `ADD_DATE()`
D. `DATE_SUM()`

6. Customer-এর age calculation-এর জন্য কোনটি বেশি appropriate?

A. `YEAR(date)`
B. `TIMESTAMPDIFF()`
C. `MONTH()`
D. `DAY()`

7. Monthly sales বের করতে কোন combination useful?

A. `GROUP BY` + date
B. শুধু `ORDER BY`
C. শুধু `JOIN`
D. শুধু `WHERE`

8. Previous month's sales পেতে কোন window function?

A. `LEAD()`
B. `LAG()`
C. `RANK()`
D. `FIRST_VALUE()`

9. `DATE_FORMAT(order_date, '%Y-%m')` কী করতে সাহায্য করে?

A. Date delete
B. Date format/group
C. Table create
D. Row update

10. Multiple years থাকলে শুধু `MONTH(order_date)` দিয়ে group করলে কী সমস্যা হতে পারে?

A. Query চলবে না
B. সব date NULL হবে

C. Different years-এর একই month একসাথে group হবে
D. Table delete হবে

🎯 আজকের সবচেয়ে গুরুত্বপূর্ণ concept

```
DATE
 ↓
YEAR / MONTH / DAY
 ↓
WHERE / BETWEEN
 ↓
DATEDIFF
 ↓
GROUP BY Month
 ↓
LAG()
 ↓
Growth %
```

এটাই **real-world SQL Date Analytics pipeline**।

Next: Class 45 — Advanced String & NULL Handling: `COALESCE()`, `NULLIF()`, `IFNULL()`, `CASE`, text cleaning, email/phone extraction, NULL-safe calculations এবং real-world data cleaning।

📘 Class 45 — Advanced String & NULL Handling in SQL

আজকের class-এ আমরা SQL-এর **data cleaning + NULL handling** শিখব। Data Analyst এবং Database Designer—দুই ক্ষেত্রেই এগুলো খুব গুরুত্বপূর্ণ।

আজকের topics:

1. `NULL` আসলে কী?
2. `COALESCE()`
3. `IFNULL()`
4. `NULLIF()`
5. `CASE WHEN`
6. String functions
7. `CONCAT()`
8. `LOWER()` / `UPPER()`

9. `TRIM()`
10. `SUBSTRING()`
11. `LEFT()` / `RIGHT()`
12. `LENGTH()`
13. `REPLACE()`
14. Email/phone cleaning
15. NULL-safe calculation
16. Real-world data cleaning
17. Common mistakes
18. Assignment + MCQ

1️⃣ NULL কী?

`NULL` মানে:

> **Unknown / Missing / No value**

এটা `0` না।

এটা empty string `''` -ও না।

এমনকি:

```
NULL ≠ 0
NULL ≠ ''
NULL ≠ 'NULL'
```

উদাহরণ:

name	phone	salary
Rahim	01711111111	50000
Karim	NULL	60000
Hasan	01822222222	NULL

Karim-এর phone সম্পর্কে information নেই।

Hasan-এর salary সম্পর্কে information নেই।

2️⃣ NULL Check

❌ ভুল:

```sql
SELECT *
FROM employees
WHERE phone = NULL;
```

সঠিক:

```sql
SELECT *
FROM employees
WHERE phone IS NULL;
```

NULL নেই:

```sql
SELECT *
FROM employees
WHERE phone IS NOT NULL;
```

Golden Rule ⭐

> `NULL` check করতে `=` নয় → `IS NULL`

3️⃣ COALESCE()

`COALESCE()` খুব গুরুত্বপূর্ণ।

এটি **প্রথম non-NULL value** return করে।

```sql
SELECT
    COALESCE(phone, 'No Phone') AS phone
FROM employees;
```

যদি:

phone
01711111111
NULL
01822222222

Result:

phone
01711111111
No Phone
01822222222

4️⃣ Multiple Values with COALESCE()

```
SELECT
    COALESCE(phone, email, 'No Contact') AS contact
FROM customers;
```

Logic:

```
phone আছে?
   ↓
Yes → phone

No
 ↓
email আছে?
 ↓
Yes → email

No
 ↓
'No Contact'
```

অর্থাৎ:

> `COALESCE()` = প্রথম available value

5️⃣ IFNULL()

MySQL-এ:

```
IFNULL(value, replacement)
```

Example:

```sql
SELECT
    name,
    IFNULL(phone, 'No Phone') AS phone
FROM customers;
```

যদি phone NULL হয়:

```
NULL → No Phone
```

6️⃣ COALESCE vs IFNULL

Feature	COALESCE	IFNULL
Multiple values	✅	❌
দুই value	✅	✅
Standard SQL	✅	MySQL-specific
MySQL	✅	✅

Example:

```sql
COALESCE(phone, email, address, 'Unknown')
```

এখানে একাধিক fallback আছে।

`IFNULL()`:

```sql
IFNULL(phone, 'No Phone')
```

শুধু দুইটি value।

Beginner Rule

> MySQL-specific হলে `IFNULL()` বুঝবে।
> Portable SQL চাইলে `COALESCE()` বেশি useful।

7️⃣ NULL Calculation Problem

ধরো:

product	price	discount
Laptop	80000	5000
Monitor	30000	NULL

Query:

```
SELECT
    product,
    price - discount AS final_price
FROM products;
```

দ্বিতীয় row:

```
30000 - NULL = NULL
```

কারণ NULL-এর সাথে calculation করলে result সাধারণত NULL হয়।

8️⃣ COALESCE দিয়ে Fix

```
SELECT
    product,
    price - COALESCE(discount, 0) AS final_price
FROM products;
```

এখন:

```
Monitor
30000 - 0
= 30000
```

🔥 Real-world data cleaning-এ এটি খুব common।

9️⃣ NULLIF()

`NULLIF()` এর syntax:

```
NULLIF(value1, value2)
```

যদি দুটো value equal হয় → `NULL`

Example:

```
SELECT NULLIF(10, 10);
```

Result:

```
NULL
```

কিন্তু:

```
SELECT NULLIF(10, 5);
```

Result:

```
10
```

🔟 NULLIF() কেন দরকার?

সবচেয়ে common use:

> **Division by zero prevent করা**

ধরো:

```
SELECT
    sales / quantity
FROM orders;
```

যদি quantity = 0 হয় → সমস্যা।

তাই:

```
SELECT
    sales / NULLIF(quantity, 0) AS average_price
FROM orders;
```

যদি:

```
quantity = 0
```

তাহলে:

```
NULLIF(0,0)
→ NULL
```

এবং:

```
sales / NULL
→ NULL
```

Division-by-zero error এড়ানো যায়।

1️⃣1️⃣ COALESCE vs NULLIF

মনে রাখো:

```
COALESCE()
↓
NULL হলে alternative value

NULLIF()
↓
দুই value same হলে NULL
```

Example:

```
COALESCE(phone, 'Unknown')
```

মানে:

> Phone NULL হলে Unknown।

আর:

```
NULLIF(quantity, 0)
```

মানে:

> Quantity 0 হলে NULL করো।

1️⃣2️⃣ CASE WHEN

SQL-এর অন্যতম important logic feature।

Syntax:

```
CASE
    WHEN condition THEN result
    WHEN condition THEN result
    ELSE result
END
```

Example:

```
SELECT
    name,
    salary,
    CASE
        WHEN salary >= 80000 THEN 'High'
        WHEN salary >= 50000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_level
FROM employees;
```

1️⃣3️⃣ CASE-এর Flow

ধরো salary:

```
90000
60000
40000
```

Query:

```
CASE
    WHEN salary >= 80000 THEN 'High'
    WHEN salary >= 50000 THEN 'Medium'
    ELSE 'Low'
END
```

Result:

salary	salary_level
90000	High

salary	salary_level
60000	Medium
40000	Low

1️⃣4️⃣ CASE + NULL

```
SELECT
    name,
    CASE
        WHEN salary IS NULL THEN 'Salary Missing'
        WHEN salary >= 50000 THEN 'Good'
        ELSE 'Below Target'
    END AS status
FROM employees;
```

এখানে NULL-এর জন্য আলাদা logic।

1️⃣5️⃣ CASE দিয়ে Data Classification

ধরো order amount:

```
SELECT
    order_id,
    amount,
    CASE
        WHEN amount >= 100000 THEN 'Large'
        WHEN amount >= 50000 THEN 'Medium'
        ELSE 'Small'
    END AS order_category
FROM orders;
```

এটা Data Analytics-এ খুব common।

1️⃣6️⃣ CASE + Aggregate

ধরো আমরা জানতে চাই:

> কতজন employee IT department-এ?

```sql
SELECT
    COUNT(
        CASE
            WHEN department = 'IT' THEN 1
        END
    ) AS it_employees
FROM employees;
```

আরেকটি pattern:

```sql
SELECT
    SUM(
        CASE
            WHEN department = 'IT' THEN 1
            ELSE 0
        END
    ) AS it_employees
FROM employees;
```

1️⃣7️⃣ Conditional SUM

ধরো IT department-এর total salary:

```sql
SELECT
    SUM(
        CASE
            WHEN department = 'IT'
            THEN salary
            ELSE 0
        END
    ) AS it_salary
FROM employees;
```

এটা খুব useful analytics technique।

1️⃣8️⃣ String Functions

এখন text cleaning শুরু করি।

Common functions:

Function	কাজ
`UPPER()`	uppercase
`LOWER()`	lowercase
`TRIM()`	extra spaces remove
`LENGTH()`	string length
`LEFT()`	left characters
`RIGHT()`	right characters
`SUBSTRING()`	অংশ বের করা
`REPLACE()`	text replace
`CONCAT()`	text combine

1️⃣9️⃣ UPPER()

```
SELECT
    UPPER(name) AS name_upper
FROM customers;
```

`rahim` →

```
RAHIM
```

2️⃣0️⃣ LOWER()

```
SELECT
    LOWER(email) AS email
FROM customers;
```

Example:

```
RAHIM@GMAIL.COM
```

→

```
rahim@gmail.com
```

Email cleaning-এ useful।

2️⃣1️⃣ TRIM()

ধরো:

```
'   Rahim   '
```

Query:

```
SELECT TRIM('   Rahim   ');
```

Result:

```
Rahim
```

Database cleaning:

```
SELECT
    TRIM(name) AS clean_name
FROM customers;
```

2️⃣2️⃣ LEFT()

String-এর বাম দিক থেকে characters:

```
SELECT LEFT('Bangladesh', 4);
```

Result:

```
Bang
```

2️⃣3️⃣ RIGHT()

```
SELECT RIGHT('Bangladesh', 4);
```

Result:

```
desh
```

2️⃣4️⃣ SUBSTRING()

মাঝখান থেকে অংশ নিতে:

```
SELECT SUBSTRING('Bangladesh', 1, 4);
```

Result:

```
Bang
```

Syntax:

```
SUBSTRING(text, starting_position, length)
```

2️⃣5️⃣ Email Username বের করা

ধরো:

```
atiar@gmail.com
```

`@`-এর আগের অংশ চাই।

MySQL-এ:

```
SELECT
    SUBSTRING_INDEX(email, '@', 1) AS username
FROM customers;
```

Result:

```
atiar
```

এটা practical string analytics-এর example।

2️⃣6️⃣ Email Domain বের করা

```
SELECT
    SUBSTRING_INDEX(email, '@', -1) AS domain
FROM customers;
```

Result:

```
gmail.com
```

2️⃣7️⃣ REPLACE()

Text-এর একটি অংশ অন্য text দিয়ে replace:

```
SELECT
    REPLACE(phone, '-', '') AS clean_phone
FROM customers;
```

যদি:

```
017-123-45678
```

হয়:

```
01712345678
```

2️⃣8️⃣ Multiple REPLACE

ধরো phone:

```
017-123 45678
```

Hyphen এবং space দুটোই remove করতে:

```
SELECT
    REPLACE(
        REPLACE(phone, '-', ''),
        ' ',
        ''
```

```
    ) AS clean_phone
FROM customers;
```

Result:

```
01712345678
```

2️⃣9️⃣ CONCAT()

দুই বা তার বেশি text combine করতে:

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name
FROM customers;
```

Example:

```
first_name = Md.
last_name = Atiar
```

Result:

```
Md. Atiar
```

3️⃣0️⃣ CONCAT + NULL Problem

এখানে একটি important issue আছে।

```
CONCAT(first_name, ' ', last_name)
```

যদি কোনো value NULL হয়, MySQL-এর `CONCAT()` -এ পুরো result NULL হতে পারে।

তাই:

```
SELECT
    CONCAT(
        COALESCE(first_name, ''),
        ' ',
        COALESCE(last_name, '')
```

```
    ) AS full_name
FROM customers;
```

এটা safer।

3️⃣1️⃣ LENGTH()

String-এর length:

```
SELECT
    name,
    LENGTH(name) AS name_length
FROM customers;
```

Example:

```
Rahim → 5
Karim → 5
```

3️⃣2️⃣ Data Cleaning Example

ধরো messy customer data:

name	email	phone
Rahim	RAHIM@GMAIL.COM	017-111-11111
karim	KARIM@YAHOO.COM	018 222 22222

Clean করার query:

```
SELECT
    TRIM(name) AS clean_name,

    LOWER(TRIM(email)) AS clean_email,

    REPLACE(
        REPLACE(TRIM(phone), '-', ''),
        ' ',
        ''
    ) AS clean_phone
```

```
FROM customers;
```

🔥 এটা real-world data cleaning-এর খুব practical pattern।

3️⃣3️⃣ Missing Data Label

```
SELECT
    name,
    COALESCE(phone, 'Missing') AS phone_status
FROM customers;
```

আরও useful:

```
SELECT
    name,
    CASE
        WHEN phone IS NULL THEN 'Missing'
        ELSE 'Available'
    END AS phone_status
FROM customers;
```

3️⃣4️⃣ NULL vs Empty String

খুব গুরুত্বপূর্ণ।

এগুলো আলাদা:

```
NULL
''
'   '
```

`TRIM()` ব্যবহার করে spaces clean করা যায়:

```
TRIM(phone)
```

কিন্তু empty string:

```
''
```

NULL নয়।

3️⃣5️⃣ Empty String-কে NULL করা

ধরো:

```
phone = ''
```

এটাকে NULL করতে:

```
NULLIF(TRIM(phone), '')
```

এখানে:

```
TRIM(phone)
```

প্রথমে spaces remove করবে।

তারপর:

```
NULLIF(..., '')
```

empty string হলে NULL করবে।

Example:

```
SELECT
    NULLIF(TRIM(phone), '') AS clean_phone
FROM customers;
```

🔥 এটি খুব useful data-cleaning pattern।

3️⃣6️⃣ Powerful Cleaning Pattern

একটি phone field clean করতে:

```
SELECT
    COALESCE(
        NULLIF(
            REPLACE(
                REPLACE(TRIM(phone), '-', ''),
```

```
                ' ',
                ''
            ),
            ''
        ),
        'No Phone'
    ) AS clean_phone
FROM customers;
```

Flow:

```
Raw phone
   ↓
TRIM
   ↓
REPLACE "-"
   ↓
REPLACE " "
   ↓
NULLIF empty string
   ↓
COALESCE missing value
   ↓
Clean phone
```

এটাই real-world SQL data cleaning-এর style।

3️⃣7️⃣ CASE + COALESCE

```sql
SELECT
    name,
    CASE
        WHEN COALESCE(salary, 0) = 0
            THEN 'Salary Missing'
        WHEN salary >= 50000
            THEN 'Good'
        ELSE 'Low'
    END AS salary_status
FROM employees;
```

3️⃣8️⃣ Data Quality Check

ধরো আমরা দেখতে চাই কোন employee-এর email missing:

```sql
SELECT
    name,
    CASE
        WHEN email IS NULL
             OR TRIM(email) = ''
        THEN 'Invalid/Missing'
        ELSE 'Valid'
    END AS email_status
FROM employees;
```

এখানে আমরা দুই ধরনের missing value ধরছি:

```
NULL
''
```

3️⃣9️⃣ SQL Data Cleaning Workflow

তুমি Data Analyst হিসেবে এই workflow মনে রাখতে পারো:

```
Raw Data
   ↓
TRIM()
   ↓
UPPER()/LOWER()
   ↓
REPLACE()
   ↓
NULLIF()
   ↓
COALESCE()
   ↓
CASE
   ↓
Validation
   ↓
Clean Data
```

4️⃣0️⃣ Excel → SQL Connection

তুমি Excel-এর Data Cleaning already শিখেছো।

Excel	SQL
`TRIM()`	`TRIM()`
`UPPER()`	`UPPER()`
`LOWER()`	`LOWER()`
`LEFT()`	`LEFT()`
`RIGHT()`	`RIGHT()`
`MID()`	`SUBSTRING()`
`SUBSTITUTE()`	`REPLACE()`
`CONCAT()`	`CONCAT()`
`IF()`	`CASE WHEN`
IFERROR-like NULL handling	`COALESCE()` / `IFNULL()`
Missing check	`IS NULL`

তাই Excel-এর skill SQL-এ সরাসরি কাজে লাগছে। 🔥

⚠️ Common Beginner Mistakes

❌ Mistake 1

```
WHERE salary = NULL
```

সঠিক:

```
WHERE salary IS NULL
```

❌ Mistake 2

```
salary + bonus
```

যদি bonus NULL হয়, result NULL হতে পারে।

Better:

```
salary + COALESCE(bonus, 0)
```

❌ Mistake 3

`NULL` এবং empty string একই মনে করা।

```
NULL ≠ ''
```

❌ Mistake 4

`NOT IN` + NULL-এর danger ভুলে যাওয়া।

যেমন:

```
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM orders
);
```

Subquery-তে NULL থাকলে unexpected result হতে পারে।

এই ধরনের missing-related logic-এ:

```
NOT EXISTS
```

অনেক ক্ষেত্রে safer।

🧠 Class 45 Golden Cheat Sheet

```
IS NULL
    → NULL check

COALESCE()
    → প্রথম non-NULL value

IFNULL()
    → NULL হলে replacement

NULLIF()
    → দুই value same হলে NULL

CASE WHEN
    → SQL-এর IF/ELSE logic
```

```
TRIM()
    → extra spaces remove

UPPER()
    → uppercase

LOWER()
    → lowercase

REPLACE()
    → text replace

SUBSTRING()
    → text-এর অংশ বের করা

CONCAT()
    → text combine

LENGTH()
    → text length
```

সবচেয়ে important তিনটি:

> **COALESCE = NULL হলে alternative**

> **NULLIF = equal হলে NULL**

> **CASE WHEN = condition-based business logic**

📝 Class 45 Assignment

ধরো `customers` table:

```
customer_id
first_name
last_name
email
phone
city
```

এবং `employees` :

```
employee_id
name
salary
```

```
bonus
department
```

Q1

NULL phone-কে `'No Phone'` হিসেবে দেখাও।

Q2

NULL bonus-কে `0` ধরে salary + bonus calculate করো।

Q3

সব customer name uppercase করো।

Q4

সব email lowercase করো।

Q5

Customer name-এর extra spaces remove করো।

Q6

Phone থেকে `-` এবং spaces remove করো।

Q7

Email থেকে username বের করো।

Q8

Email থেকে domain বের করো।

Q9

Salary classify করো:

```
>= 80000 → High
>= 50000 → Medium
< 50000  → Low
```

Q10

NULL বা empty email identify করো।

Q11

`NULLIF(TRIM(phone), '')` কেন useful—নিজের ভাষায় explain করো।

Q12 ⭐

একটি clean customer query লিখো যেখানে থাকবে:

```
customer_id
clean_name
clean_email
clean_phone
email_status
```

Q13 ⭐

Employee-এর জন্য:

```
name
salary
bonus
total_salary
salary_level
```

calculate করো।

🧪 Class 45 MCQ

1. NULL check করার সঠিক syntax কোনটি?

A. `= NULL`
B. `IS NULL`
C. `== NULL`
D. `NULL =`

2. `COALESCE(a,b,c)` কী return করে?

A. সব value
B. শেষ NULL value
C. প্রথম non-NULL value
D. শুধু NULL

3. `NULLIF(10,10)` কী return করবে?

A. 10
B. 0

C. NULL
D. Error

4. Division by zero prevent করতে কোনটি useful?

A. `COALESCE()`
B. `NULLIF()`
C. `TRIM()`
D. `UPPER()`

5. SQL-এর IF/ELSE logic কোনটি?

A. CASE WHEN
B. SWITCH
C. IF ELSE ONLY
D. LOOP

6. Extra spaces remove করতে কোন function?

A. `CLEAN()`
B. `TRIM()`
C. `SPACE()`
D. `REMOVE()`

7. String lowercase করতে কোনটি?

A. `LOWER()`
B. `SMALL()`
C. `DOWN()`
D. `MIN()`

8. Text replace করতে কোন function?

A. `CHANGE()`
B. `REPLACE()`
C. `UPDATE_TEXT()`
D. `SUB()`

9. Email-এর `@`-এর আগের অংশ বের করতে MySQL-এ কোনটি useful?

A. `SUBSTRING_INDEX()`
B. `LEFT_ONLY()`
C. `EMAIL_PART()`
D. `SPLIT_TEXT()`

10. `NULL` এবং empty string সম্পর্কে কোনটি সঠিক?

A. একই
B. NULL = 0

C. NULL এবং `''` আলাদা
D. NULL = `'NULL'`

🎯 আজকের Core Skill

Class 45 শেষ করার পর তোমার এই query pattern খুব ভালোভাবে জানা উচিত:

```
SELECT
    TRIM(name) AS clean_name,

    LOWER(TRIM(email)) AS clean_email,

    COALESCE(
        NULLIF(
            REPLACE(REPLACE(TRIM(phone), '-', ''), ' ', ''),
            ''
        ),
        'No Phone'
    ) AS clean_phone,

    CASE
        WHEN email IS NULL OR TRIM(email) = ''
            THEN 'Missing'
        ELSE 'Available'
    END AS email_status

FROM customers;
```

এখানে একসাথে ব্যবহার হচ্ছে:

TRIM + LOWER + REPLACE + NULLIF + COALESCE + CASE

এটাই আজকের সবচেয়ে practical SQL data-cleaning pattern।

📘 Class 46 — SQL Views

আজ থেকে আমরা SQL-এর **Database Object** এবং production-level database concepts-এর দিকে যাচ্ছি।

আজকের topic:

1. View কী?
2. কেন View ব্যবহার করি?
3. `CREATE VIEW`

4. View থেকে `SELECT`
5. View + JOIN
6. View + GROUP BY
7. View + WHERE
8. `CREATE OR REPLACE VIEW`
9. `DROP VIEW`
10. View vs Table
11. View vs CTE
12. Real-world reporting
13. Security-এর জন্য View
14. Common mistakes
15. Assignment + MCQ

1️⃣ View কী?

সহজভাবে:

> **View হলো একটি saved SQL query, যেটাকে আমরা table-এর মতো ব্যবহার করতে পারি।**

ধরো প্রতিবার তোমাকে এই query লিখতে হচ্ছে:

```
SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    SUM(o.quantity * p.price) AS total_spending
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name,
    c.city;
```

Query অনেক বড়।

আমরা এটাকে একটি View হিসেবে save করতে পারি।

2️⃣ CREATE VIEW

```sql
CREATE VIEW customer_sales AS
SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    SUM(o.quantity * p.price) AS total_spending
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name,
    c.city;
```

এখন `customer_sales` হলো একটি View।

3️⃣ View থেকে Data দেখা

এখন:

```sql
SELECT *
FROM customer_sales;
```

তুমি এমন result পাবে:

customer_id	customer_name	city	total_spending
1	Rahim	Dhaka	89000
2	Karim	Chittagong	120000
3	Hasan	Dhaka	73000

এখানে মনে হবে যেন `customer_sales` একটি table।

কিন্তু এটি **সাধারণ table নয়**।

4️⃣ View-এর মূল ধারণা

```
Original Tables
      ↓
```

```
      JOIN
       ↓
    GROUP BY
       ↓
    SQL Query
       ↓
      VIEW
       ↓
SELECT / WHERE / ORDER BY
```

Golden Rule ⭐

> **View = Saved SQL Query + Table-like interface**

5️⃣ View কেন ব্যবহার করব?

ধরো company-তে একই report বারবার লাগে:

```
Customer Sales Report
```

প্রতিবার 20 লাইনের query না লিখে:

```sql
SELECT *
FROM customer_sales;
```

এটাই যথেষ্ট।

Benefits

- Query reuse
- Complex query সহজ করা
- Reporting সহজ
- Security improve করা যায়
- Application code সহজ করা
- Consistent business logic

6️⃣ View + WHERE

View-এর উপর আবার filtering করা যায়।

```sql
SELECT *
FROM customer_sales
WHERE total_spending > 100000;
```

মানে:

> যেসব customer-এর total spending 100000-এর বেশি।

7️⃣ View + ORDER BY

```sql
SELECT *
FROM customer_sales
ORDER BY total_spending DESC;
```

সবচেয়ে বেশি spending করা customer উপরে আসবে।

8️⃣ View + LIMIT

```sql
SELECT *
FROM customer_sales
ORDER BY total_spending DESC
LIMIT 5;
```

Top 5 customers।

9️⃣ View + JOIN

View-কে অন্য table-এর সাথে JOIN করা যায়।

ধরো:

```sql
SELECT
    cs.customer_name,
    cs.total_spending,
    c.email
FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id;
```

অর্থাৎ:

```
View
 +
Table
 ↓
JOIN
```

🔟 Simple View

সব View complex হতে হবে এমন নয়।

```sql
CREATE VIEW active_customers AS
SELECT
    customer_id,
    customer_name,
    city
FROM customers
WHERE is_active = TRUE;
```

তারপর:

```sql
SELECT *
FROM active_customers;
```

1️⃣1️⃣ View + JOIN Example

ধরো sales report দরকার:

```sql
CREATE VIEW sales_report AS
SELECT
    o.order_id,
    o.order_date,
    c.customer_name,
    c.city,
    p.product_name,
    p.category,
    o.quantity,
    p.price,
    o.quantity * p.price AS total_amount
FROM orders o
JOIN customers c
```

```
    ON o.customer_id = c.customer_id
JOIN products p
    ON o.product_id = p.product_id;
```

এখন:

```
SELECT *
FROM sales_report;
```

Result:

order_id	customer	product	quantity	total_amount
1	Rahim	Laptop	1	80000
2	Karim	Monitor	2	60000

এখন এই View-কে আমরা reporting layer হিসেবে ব্যবহার করতে পারি।

1️⃣2️⃣ View থেকে Specific Columns

```
SELECT
    customer_name,
    product_name,
    total_amount
FROM sales_report;
```

View-এর সব column নিতে হবে না।

1️⃣3️⃣ View + WHERE

```
SELECT *
FROM sales_report
WHERE category = 'Electronics';
```

শুধু Electronics sales।

1️⃣4️⃣ View + Date Filter

```sql
SELECT *
FROM sales_report
WHERE order_date >= '2026-01-01'
  AND order_date < '2026-02-01';
```

January sales।

1️⃣5️⃣ View + Aggregate

ধরো `sales_report` আছে।

এখন category-wise sales:

```sql
SELECT
    category,
    SUM(total_amount) AS total_sales
FROM sales_report
GROUP BY category;
```

Result:

category	total_sales
Electronics	265000
Furniture	70000
Accessories	45000

🔥 এখানে একটি interesting concept:

আমরা প্রথমে complex JOIN View বানিয়েছি।

তারপর View-এর উপর analytics করেছি।

1️⃣6️⃣ View দিয়ে Business Logic Centralize করা

ধরো company-এর rule:

```
Revenue = Quantity × Price
```

যদি 5টা application একই calculation করে, তাহলে inconsistency হতে পারে।

View:

```
CREATE VIEW sales_report AS
SELECT
    order_id,
    quantity,
    price,
    quantity * price AS revenue
FROM orders;
```

এখন সবাই:

```
SELECT revenue
FROM sales_report;
```

ব্যবহার করতে পারে।

Golden Rule

> **View business logic centralize করতে সাহায্য করে।**

1️⃣7️⃣ CREATE OR REPLACE VIEW

View-এর definition পরিবর্তন করতে:

```
CREATE OR REPLACE VIEW active_customers AS
SELECT
    customer_id,
    customer_name,
    city,
    email
FROM customers
WHERE is_active = TRUE;
```

আগের View থাকলে definition replace/update করা যায়।

1️⃣8️⃣ DROP VIEW

View আর দরকার না হলে:

```
DROP VIEW active_customers;
```

তারপর View থাকবে না।

1️⃣9️⃣ DROP VIEW IF EXISTS

Production script-এ safe approach:

```
DROP VIEW IF EXISTS active_customers;
```

View না থাকলেও unnecessary error এড়ানো যায়।

2️⃣0️⃣ View vs Table

এটা interview-এর জন্য খুব important।

Feature	Table	View
Physical table	✅	সাধারণত ❌
Data store করে	✅	সাধারণত query result represent করে
`CREATE TABLE`	✅	❌
`CREATE VIEW`	❌	✅
Complex query save	❌	✅
JOIN-based report	Possible	Very useful
Direct data storage	✅	❌
Query reuse	Limited	✅

সহজভাবে:

> **Table = Data রাখে**

> **View = Query-এর result-এর interface দেয়**

2️⃣1️⃣ View কি Data Copy করে?

সাধারণ View-এর ক্ষেত্রে:

না।

যেমন:

```sql
CREATE VIEW active_customers AS
SELECT *
FROM customers
WHERE is_active = TRUE;
```

এটা সাধারণত customer data-এর নতুন physical copy তৈরি করে না।

View-এর underlying query থেকে data আসে।

তাই original table update হলে View-তে নতুন data দেখা যেতে পারে।

2️⃣2️⃣ Example

প্রথমে:

```
customers

Rahim → Active
Karim → Active
Hasan → Inactive
```

View:

```sql
CREATE VIEW active_customers AS
SELECT *
FROM customers
WHERE is_active = TRUE;
```

Result:

```
Rahim
Karim
```

পরে Hasan active হলে:

```sql
UPDATE customers
SET is_active = TRUE
WHERE customer_id = 3;
```

তারপর:

```sql
SELECT *
```

```
FROM active_customers;
```

Hasan-ও দেখা যাবে।

অর্থাৎ View সাধারণত current underlying data-এর উপর query করে।

2️⃣3️⃣ View vs CTE

এটা খুব গুরুত্বপূর্ণ কারণ তুমি Class 38-এ CTE শিখেছো।

CTE

```
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(amount) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_sales;
```

CTE সাধারণত **একটি query-এর জন্য**।

View

```
CREATE VIEW customer_sales AS
SELECT
    customer_id,
    SUM(amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

তারপর অনেক query-তে ব্যবহার করা যায়:

```
SELECT * FROM customer_sales;
```

```
SELECT *
FROM customer_sales
WHERE total_sales > 100000;
```

```sql
SELECT *
FROM customer_sales
ORDER BY total_sales DESC;
```

Golden Rule ⭐

> **CTE = Temporary query-level result**

> **View = Saved database-level query object**

2️⃣4️⃣ View vs Subquery

Subquery:

```sql
SELECT *
FROM (
    SELECT
        customer_id,
        SUM(amount) AS total_sales
    FROM orders
    GROUP BY customer_id
) AS x
WHERE total_sales > 100000;
```

এখানে query-এর মধ্যেই subquery আছে।

View:

```sql
CREATE VIEW customer_sales AS
SELECT
    customer_id,
    SUM(amount) AS total_sales
FROM orders
GROUP BY customer_id;
```

তারপর:

```sql
SELECT *
FROM customer_sales
WHERE total_sales > 100000;
```

মনে রাখো:

```
Subquery → one query-এর ভিতরে
CTE      → one query-এর শুরুতে
View     → database-এ saved query
```

2️⃣5️⃣ Security-এর জন্য View

এটা Database Designer হিসেবে তোমার জন্য খুব important।

ধরো employees table:

```
employee_id
name
email
salary
department
nid
address
```

HR application-এর সাধারণ employee user-কে salary/NID দেখাতে চাই না।

তাহলে:

```sql
CREATE VIEW employee_public AS
SELECT
    employee_id,
    name,
    email,
    department
FROM employees;
```

Application:

```sql
SELECT *
FROM employee_public;
```

এতে sensitive columns expose না করেই data access করা যায়।

Important

View security-এর জন্য useful হলেও **View একাই complete security system নয়**।

Proper permissions/privileges-ও দরকার।

2️⃣6️⃣ Real-World Architecture

Production application-এ conceptually:

```
              Database Tables
                     ↓
               JOIN / Logic
                     ↓
                  VIEW
                     ↓
          Reporting / API Layer
                     ↓
            Frontend / Dashboard
```

যেমন:

```
customers
products
orders
    ↓
sales_report VIEW
    ↓
Django / DRF
    ↓
React / Power BI
```

তোমার Django + DRF background-এর সাথে এই concept খুব useful।

2️⃣7️⃣ View + Window Function

Advanced example:

```sql
CREATE VIEW customer_ranking AS
SELECT
    customer_id,
    customer_name,
    total_spending,
    RANK() OVER (
        ORDER BY total_spending DESC
    ) AS customer_rank
FROM customer_sales;
```

এখন:

```sql
SELECT *
FROM customer_ranking
WHERE customer_rank <= 3;
```

Top 3 customers।

🔥 এখানে আমরা ব্যবহার করছি:

```
View
+
Window Function
```

2️⃣8️⃣ View + CASE

```sql
CREATE VIEW customer_segment AS
SELECT
    customer_id,
    customer_name,
    total_spending,
    CASE
        WHEN total_spending >= 100000
            THEN 'Premium'
        WHEN total_spending >= 50000
            THEN 'Regular'
        ELSE 'Low'
    END AS segment
FROM customer_sales;
```

তারপর:

```sql
SELECT *
FROM customer_segment
WHERE segment = 'Premium';
```

2️⃣9️⃣ View-এর Limitation

View সবসময় magic solution নয়।

কিছু limitation:

- Complex View performance-heavy হতে পারে

- অনেক JOIN থাকলে query expensive হতে পারে
- সব View সহজে update করা যায় না
- Nested Views complexity তৈরি করতে পারে
- Underlying table structure পরিবর্তন করলে View break করতে পারে

3️⃣0️⃣ Updatable View

কিছু simple View-এর মাধ্যমে underlying table update করা সম্ভব হতে পারে।

Example:

```
CREATE VIEW active_customers AS
SELECT
    customer_id,
    customer_name,
    city
FROM customers
WHERE is_active = TRUE;
```

কিছু DBMS পরিস্থিতিতে:

```
UPDATE active_customers
SET city = 'Dhaka'
WHERE customer_id = 1;
```

underlying `customers` table-এ change করতে পারে।

কিন্তু complex View—বিশেষ করে:

```
GROUP BY
Aggregate
DISTINCT
UNION
Multiple complex JOIN
```

থাকলে updateability সীমিত বা unavailable হতে পারে।

Beginner Rule

> **View primarily reporting/read abstraction হিসেবে ব্যবহার করো; update করার আগে DBMS-specific rules বুঝবে।**

3️⃣1️⃣ Materialized View — ধারণা

Normal View:

```
Query definition saved
        ↓
Data usually calculated when queried
```

Materialized View:

```
Query result physically stored
        ↓
Faster reads
        ↓
Needs refresh
```

এটা large analytics system-এ useful হতে পারে।

তবে MySQL/PostgreSQL implementation details আলাদা।

এখন শুধু concept মনে রাখো।

3️⃣2️⃣ Normal View vs Materialized View

	View	Materialized View
Query definition	Save	Save
Result physically store	সাধারণত না	হ্যাঁ
Freshness	সাধারণত current underlying data	Refresh-এর উপর নির্ভর
Read performance	Query-dependent	Often faster
Storage	কম	বেশি
Analytics	✅	✅

3️⃣3️⃣ Real-World Sales View

এবার পুরো একটি practical example:

```
CREATE VIEW sales_report AS
SELECT
    o.order_id,
```

```
    o.order_date,

    c.customer_id,
    c.customer_name,
    c.city,

    p.product_id,
    p.product_name,
    p.category,

    o.quantity,
    p.price,

    o.quantity * p.price AS revenue

FROM orders o

JOIN customers c
    ON o.customer_id = c.customer_id

JOIN products p
    ON o.product_id = p.product_id;
```

এখন business analyst:

```
SELECT *
FROM sales_report;
```

Monthly sales:

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    SUM(revenue) AS monthly_sales
FROM sales_report
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Category sales:

```
SELECT
    category,
    SUM(revenue) AS total_sales
FROM sales_report
GROUP BY category
ORDER BY total_sales DESC;
```

Top customers:

```sql
SELECT
    customer_name,
    SUM(revenue) AS total_spending
FROM sales_report
GROUP BY customer_id, customer_name
ORDER BY total_spending DESC
LIMIT 5;
```

🔥 একটি View থেকে অনেক ধরনের report তৈরি করা যাচ্ছে।

3️⃣4️⃣ Common Beginner Mistakes

❌ Mistake 1: View-কে physical table মনে করা

View সাধারণত data-এর নতুন copy নয়।

❌ Mistake 2: View এবং CTE একই মনে করা

```
CTE → query-level
View → database-level saved object
```

❌ Mistake 3: View বানালেই performance সবসময় বাড়বে

না।

View complex হলে underlying query আবার expensive হতে পারে।

❌ Mistake 4: Security সম্পূর্ণ View-এর উপর ছেড়ে দেওয়া

View data exposure কমাতে সাহায্য করতে পারে, কিন্তু database permissions-ও দরকার।

❌ Mistake 5: সব query-এর জন্য View বানানো

একবার ব্যবহার করা ছোট query-এর জন্য View বানানোর দরকার নেই।

Repeated / reusable / reporting logic-এর জন্য View বেশি useful।

🧠 Class 46 Cheat Sheet

```
CREATE VIEW
    ↓
Saved SQL Query

SELECT FROM view
    ↓
View ব্যবহার

CREATE OR REPLACE VIEW
    ↓
View definition update

DROP VIEW
    ↓
View delete

View
    ↓
Reusable query

CTE
    ↓
Temporary query-level result

Table
    ↓
Actual stored data

Materialized View
    ↓
Stored query result
```

⭐ Golden Rules

> **Table = Data**

> **View = Saved Query**

> **CTE = Temporary Query Result**

> **Materialized View = Stored Query Result**

📝 Class 46 Assignment

ধরো tables:

```
customers
products
orders
```

Q1

একটি `active_customers` View তৈরি করো।

Q2

একটি `sales_report` View তৈরি করো যেখানে থাকবে:

```
order_id
order_date
customer_name
city
product_name
category
quantity
price
revenue
```

Q3

`sales_report` View থেকে category-wise sales বের করো।

Q4

`sales_report` View থেকে monthly sales বের করো।

Q5

`sales_report` View থেকে top 5 customers বের করো।

Q6

একটি `customer_sales` View তৈরি করো যেখানে থাকবে:

```
customer_id
customer_name
total_spending
```

Q7

এই View থেকে `total_spending > 100000` customer বের করো।

Q8

Customer ranking-এর জন্য View তৈরি করো যেখানে `RANK()` থাকবে।

Q9

`employee_public` View তৈরি করো যেখানে salary এবং sensitive fields থাকবে না।

Q10 ⭐

নিজের ভাষায় explain করো:

```
Table vs View vs CTE
```

🧪 Class 46 MCQ

1. View কী?

A. Physical storage only
B. Saved SQL query
C. Index
D. Database

2. View তৈরি করতে কোন command?

A. `CREATE TABLE`
B. `CREATE QUERY`
C. `CREATE VIEW`
D. `MAKE VIEW`

3. View থেকে data নিতে?

A. `SELECT FROM view`
B. `GET VIEW`
C. `READ VIEW`
D. `OPEN VIEW`

4. CTE-এর তুলনায় View-এর প্রধান সুবিধা কী?

A. Query-এর মধ্যে শুধু একবার ব্যবহার করা যায়
B. Database-এ reusable object হিসেবে থাকে
C. সব data copy করে
D. Index তৈরি করে

5. View-এর underlying table update হলে সাধারণ View-তে কী দেখা যেতে পারে?

A. নতুন data reflect হতে পারে
B. সব data delete হবে
C. View automatically table হবে
D. Database বন্ধ হবে

6. View security-তে কীভাবে সাহায্য করতে পারে?

A. Password generate করে
B. নির্দিষ্ট columns expose করে
C. Database encrypt করে
D. Backup তৈরি করে

7. View delete করতে?

A. `DELETE VIEW`
B. `DROP VIEW`
C. `REMOVE VIEW`
D. `CLEAR VIEW`

8. Materialized View-এর বৈশিষ্ট্য কী?

A. Result physically store করতে পারে
B. কখনো refresh লাগে না
C. সবসময় faster
D. Table-এর synonym

9. View কি সবসময় performance improve করে?

A. হ্যাঁ
B. না
C. শুধু JOIN থাকলে
D. শুধু MySQL-এ

10. কোনটি সঠিক?

A. Table = Saved Query
B. View = Physical Data Storage
C. CTE = Query-level temporary result
D. View = Index

🎯 আজকের মূল concept

```
TABLE
  ↓
```

```
Actual Data

CTE
  ↓
Temporary Query Result

VIEW
  ↓
Reusable Saved Query

MATERIALIZED VIEW
  ↓
Stored Query Result
```

পরের ধাপ: Class 47 — Stored Procedures & Functions, যেখানে আমরা শিখব database-এর ভিতরে reusable logic কীভাবে তৈরি করা হয়, `CREATE PROCEDURE`, parameters, `CALL`, functions, `IF`, variables এবং real-world business logic।

📘 Class 47 — SQL Stored Procedures & Functions

আজকে আমরা SQL-এর **Reusable Database Logic** শিখব।

এখন পর্যন্ত তুমি শিখেছো:

```
SELECT
WHERE
GROUP BY
JOIN
Subquery
CTE
Window Function
EXISTS
Date Analytics
NULL Handling
View
```

এখন আমরা শিখব:

> **Database-এর ভিতরেই reusable logic কীভাবে তৈরি করা যায়।**

আজকের topics:

1. Stored Procedure কী?
2. Procedure কেন ব্যবহার করি?
3. `CREATE PROCEDURE`

4. `CALL`
5. Parameters
6. `IN`, `OUT`, `INOUT`
7. Variables
8. `IF`
9. Procedure + SELECT
10. Procedure + UPDATE
11. Stored Function
12. Procedure vs Function
13. Real-world examples
14. Common mistakes
15. Assignment + MCQ

1️⃣ Stored Procedure কী?

সহজভাবে:

> **Stored Procedure হলো database-এর ভিতরে save করা এক বা একাধিক SQL statement-এর reusable group।**

ধরো তুমি বারবার এই কাজ করো:

```
SELECT *
FROM employees
WHERE department = 'IT';
```

এটা একটি procedure হিসেবে save করা যায়।

তারপর:

```
CALL get_it_employees();
```

করলেই query execute হবে।

2️⃣ Procedure কেন ব্যবহার করব?

ধরো একটি banking/application system-এ:

```
Transfer Money
     ↓
```

```
Check balance
    ↓
Deduct money
    ↓
Add money
    ↓
Record transaction
```

এই ধরনের multiple SQL operation database-এর একটি procedure-এর মধ্যে রাখা যেতে পারে।

Benefits:

- Logic reuse
- Centralized business logic
- Application code কমানো
- Database-side processing
- Parameters ব্যবহার
- Multiple SQL statements একসাথে execute করা

3️⃣ Basic Stored Procedure

MySQL example:

```sql
DELIMITER //

CREATE PROCEDURE get_employees()
BEGIN
    SELECT *
    FROM employees;
END //

DELIMITER ;
```

এখানে:

```
CREATE PROCEDURE
        ↓
Procedure তৈরি
```

এবং:

```
BEGIN
...
```

```
END
```

এর মধ্যে procedure-এর statements থাকে।

4️⃣ DELIMITER কেন?

Normally MySQL:

```
;
```

দেখলেই statement শেষ মনে করে।

কিন্তু procedure-এর ভিতরে অনেকগুলো statement থাকতে পারে।

তাই আমরা temporarily delimiter পরিবর্তন করি:

```
DELIMITER //
```

তারপর:

```
END //
```

শেষে আবার:

```
DELIMITER ;
```

মনে রাখবে

> `DELIMITER` SQL logic-এর অংশ নয়; MySQL client-কে statement boundary বোঝাতে ব্যবহৃত হয়।

5️⃣ Procedure Execute করা

Procedure তৈরি করার পর:

```
CALL get_employees();
```

এতে:

```
SELECT *
FROM employees;
```

execute হবে।

6️⃣ Procedure with WHERE

```
DELIMITER //

CREATE PROCEDURE get_it_employees()
BEGIN
    SELECT *
    FROM employees
    WHERE department = 'IT';
END //

DELIMITER ;
```

Run:

```
CALL get_it_employees();
```

7️⃣ Parameter কী?

Procedure-কে বাইরে থেকে value পাঠাতে চাইলে parameter ব্যবহার করি।

যেমন:

```
department = 'IT'
```

hardcode না করে user যেন department পাঠাতে পারে।

8️⃣ IN Parameter

সবচেয়ে common parameter:

```
IN
```

Example:

```
DELIMITER //

CREATE PROCEDURE get_employees_by_department(
    IN dept_name VARCHAR(50)
)
BEGIN
    SELECT *
    FROM employees
    WHERE department = dept_name;
END //

DELIMITER ;
```

এখন:

```
CALL get_employees_by_department('IT');
```

অথবা:

```
CALL get_employees_by_department('HR');
```

একই procedure, different input।

9️⃣ Procedure Flow

```
CALL procedure('IT')
        ↓
dept_name = 'IT'
        ↓
SELECT employees
WHERE department = 'IT'
        ↓
Result
```

🔟 Multiple Parameters

ধরো city এবং minimum salary দুটোই দিতে চাই।

```
DELIMITER //

CREATE PROCEDURE search_employees(
    IN emp_city VARCHAR(50),
    IN min_salary DECIMAL(10,2)
)
BEGIN
    SELECT *
    FROM employees
    WHERE city = emp_city
      AND salary >= min_salary;
END //

DELIMITER ;
```

Call:

```
CALL search_employees('Dhaka', 50000);
```

1️⃣1️⃣ Procedure + Date

```
DELIMITER //

CREATE PROCEDURE get_orders_after(
    IN start_date DATE
)
BEGIN
    SELECT *
    FROM orders
    WHERE order_date >= start_date;
END //

DELIMITER ;
```

Call:

```
CALL get_orders_after('2026-03-01');
```

1️⃣2️⃣ Procedure + UPDATE

Procedure শুধু SELECT-এর জন্য নয়।

Employee salary increase:

```
DELIMITER //

CREATE PROCEDURE increase_salary(
    IN dept_name VARCHAR(50),
    IN increase_percent DECIMAL(5,2)
)
BEGIN
    UPDATE employees
    SET salary = salary * (1 + increase_percent / 100)
    WHERE department = dept_name;
END //

DELIMITER ;
```

Call:

```
CALL increase_salary('IT', 10);
```

IT department-এর salary 10% increase হবে।

1️⃣3️⃣ Procedure + INSERT

নতুন customer insert:

```
DELIMITER //

CREATE PROCEDURE add_customer(
    IN p_name VARCHAR(100),
    IN p_city VARCHAR(50)
)
BEGIN
    INSERT INTO customers(customer_name, city)
    VALUES(p_name, p_city);
END //

DELIMITER ;
```

Call:

```
CALL add_customer('Rahim', 'Dhaka');
```

1️⃣4️⃣ Procedure + DELETE

```
DELIMITER //

CREATE PROCEDURE delete_customer(
    IN p_customer_id INT
)
BEGIN
    DELETE FROM customers
    WHERE customer_id = p_customer_id;
END //

DELIMITER ;
```

Call:

```
CALL delete_customer(5);
```

⚠️ Production system-এ delete করার আগে foreign key এবং business rules carefully handle করতে হবে।

1️⃣5️⃣ Variable

Procedure-এর ভিতরে local variable তৈরি করা যায়।

Syntax:

```
DECLARE variable_name data_type;
```

Example:

```
DELIMITER //

CREATE PROCEDURE test_variable()
BEGIN
    DECLARE total INT DEFAULT 0;

    SET total = 100;

    SELECT total;
END //

DELIMITER ;
```

Call:

```
CALL test_variable();
```

Result:

```
100
```

1️⃣6️⃣ Variable + SELECT INTO

Database থেকে value এনে variable-এ রাখা যায়।

```
DELIMITER //

CREATE PROCEDURE get_employee_count()
BEGIN
    DECLARE total_employees INT;

    SELECT COUNT(*)
    INTO total_employees
    FROM employees;

    SELECT total_employees AS employee_count;
END //

DELIMITER ;
```

Call:

```
CALL get_employee_count();
```

1️⃣7️⃣ IF Statement

Procedure-এর ভিতরে condition ব্যবহার করা যায়।

```
DELIMITER //

CREATE PROCEDURE check_salary(
    IN emp_salary DECIMAL(10,2)
)
BEGIN
```

```
    IF emp_salary >= 50000 THEN
        SELECT 'Good Salary' AS result;
    ELSE
        SELECT 'Below Target' AS result;
    END IF;
END //

DELIMITER ;
```

Call:

```
CALL check_salary(60000);
```

Result:

```
Good Salary
```

1️⃣8️⃣ IF / ELSEIF / ELSE

```
DELIMITER //

CREATE PROCEDURE salary_level(
    IN emp_salary DECIMAL(10,2)
)
BEGIN
    IF emp_salary >= 80000 THEN
        SELECT 'High' AS level;

    ELSEIF emp_salary >= 50000 THEN
        SELECT 'Medium' AS level;

    ELSE
        SELECT 'Low' AS level;
    END IF;
END //

DELIMITER ;
```

এখানে:

```
>= 80000 → High
>= 50000 → Medium
< 50000  → Low
```

1️⃣9️⃣ IN vs OUT vs INOUT

Stored Procedure-এর parameters তিন ধরনের হতে পারে:

```
IN
OUT
INOUT
```

`IN`

Input নেওয়ার জন্য।

```
IN p_id INT
```

`OUT`

Procedure থেকে result বাইরে পাঠাতে।

```
OUT total INT
```

`INOUT`

Input নেয় এবং পরিবর্তিত value output করতে পারে।

```
INOUT value INT
```

2️⃣0️⃣ OUT Parameter Example

Employee count:

```
DELIMITER //

CREATE PROCEDURE get_employee_count(
    OUT total_employees INT
)
```

```
BEGIN
    SELECT COUNT(*)
    INTO total_employees
    FROM employees;
END //

DELIMITER ;
```

Call:

```
CALL get_employee_count(@total);
```

তারপর:

```
SELECT @total;
```

Result:

```
15
```

এখানে:

```
@total
```

একটি MySQL session variable।

2️⃣1️⃣ OUT Parameter Flow

```
CALL procedure
      ↓
Database calculates
      ↓
OUT parameter
      ↓
@total
      ↓
SELECT @total
```

2️⃣2️⃣ INOUT Example

ধরো input:

```
100
```

Procedure 10 add করবে।

```
DELIMITER //

CREATE PROCEDURE add_ten(
    INOUT value INT
)
BEGIN
    SET value = value + 10;
END //

DELIMITER ;
```

Call:

```
SET @number = 100;

CALL add_ten(@number);

SELECT @number;
```

Result:

```
110
```

2️⃣3️⃣ Stored Function কী?

এখন আসি **Function**-এ।

> **Stored Function হলো database-এর ভিতরে save করা reusable logic যা একটি value return করে।**

Procedure:

```
CALL procedure()
```

Function:

```
SELECT function()
```

2️⃣4️⃣ Simple Function

ধরো Celsius → Fahrenheit:

```
DELIMITER //

CREATE FUNCTION celsius_to_fahrenheit(
    celsius DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN (celsius * 9 / 5) + 32;
END //

DELIMITER ;
```

Use:

```
SELECT celsius_to_fahrenheit(30);
```

Result:

```
86
```

2️⃣5️⃣ Function in SELECT

Employee salary bonus:

```
SELECT
    name,
    salary,
    salary * 0.10 AS bonus
FROM employees;
```

এটা সরাসরি query-তে করা যায়।

কিন্তু reusable complex logic হলে function ব্যবহার করা যেতে পারে।

2️⃣6️⃣ Custom Function Example

```
DELIMITER //

CREATE FUNCTION salary_with_bonus(
    salary DECIMAL(10,2),
    bonus_percent DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN salary + (salary * bonus_percent / 100);
END //

DELIMITER ;
```

Use:

```
SELECT
    name,
    salary,
    salary_with_bonus(salary, 10) AS final_salary
FROM employees;
```

2️⃣7️⃣ Procedure vs Function

Interview-এর জন্য খুব important।

Feature	Procedure	Function
`CALL`	✅	❌
`SELECT`-এ directly	সাধারণত ❌	✅
Return value	একাধিক OUT/result হতে পারে	একটি return value
Main purpose	Perform operation	Calculate/return value
INSERT/UPDATE/DELETE	✅	Function restrictions apply
Parameters	IN/OUT/INOUT	সাধারণত input parameters

সহজভাবে:

> **Procedure = কাজ করো**

> **Function = value return করো**

2️⃣8️⃣ Real-World Example

ধরো e-commerce system।

Procedure:

```
Place Order
    ↓
Create Order
    ↓
Reduce Inventory
    ↓
Create Payment Record
    ↓
Create Order History
```

এখানে Procedure useful।

Function:

```
Calculate Discount
Calculate Tax
Calculate Final Price
Calculate Customer Score
```

এখানে Function useful।

2️⃣9️⃣ Procedure + Business Logic

ধরো customer-এর total spending:

```sql
DELIMITER //

CREATE PROCEDURE get_customer_spending(
    IN p_customer_id INT
)
BEGIN
    SELECT
        c.customer_id,
        c.customer_name,
        COALESCE(SUM(o.amount), 0) AS total_spending
```

```
    FROM customers c
    LEFT JOIN orders o
        ON c.customer_id = o.customer_id
    WHERE c.customer_id = p_customer_id
    GROUP BY
        c.customer_id,
        c.customer_name;
END //

DELIMITER ;
```

Call:

```
CALL get_customer_spending(1);
```

3️⃣0️⃣ Procedure + CASE

```
DELIMITER //

CREATE PROCEDURE customer_segment(
    IN p_customer_id INT
)
BEGIN
    DECLARE total_spending DECIMAL(12,2);

    SELECT
        COALESCE(SUM(amount), 0)
    INTO total_spending
    FROM orders
    WHERE customer_id = p_customer_id;

    IF total_spending >= 100000 THEN
        SELECT 'Premium' AS segment;

    ELSEIF total_spending >= 50000 THEN
        SELECT 'Regular' AS segment;

    ELSE
        SELECT 'Basic' AS segment;
    END IF;
END //

DELIMITER ;
```

এখানে একসাথে:

```
Procedure
+
Variable
+
SELECT INTO
+
COALESCE
+
IF
```

🔥 এটি production-style database logic-এর foundation।

3️⃣1️⃣ Procedure vs View

তুমি Class 46-এ View শিখেছো।

View:

```
Saved Query
↓
Mostly data reading/reporting
```

Procedure:

```
Saved SQL Logic
↓
Can perform operations
```

Example:

```
View
→ Customer Sales Report

Procedure
→ Increase Customer Credit
```

3️⃣2️⃣ Procedure vs CTE

```
CTE
→ Temporary query structure
```

```
View
→ Saved query

Procedure
→ Saved executable logic

Function
→ Saved calculation returning value
```

এটা চারটা concept একসাথে মনে রাখো।

3️⃣3️⃣ Common Beginner Mistakes

❌ Mistake 1 — `DELIMITER` ভুলে যাওয়া

MySQL client-এ multi-statement procedure তৈরি করার সময় delimiter ঠিকভাবে manage করতে হবে।

❌ Mistake 2 — Parameter name conflict

যেমন:

```
WHERE department = department;
```

এতে ambiguity হতে পারে।

Better:

```
WHERE department = p_department;
```

Naming convention:

```
p_ → parameter
v_ → local variable
```

যেমন:

```
p_customer_id
p_salary
v_total
```

❌ Mistake 3 — Procedure আর Function একই মনে করা

মনে রাখো:

```
Procedure → CALL
Function  → RETURN
```

❌ Mistake 4 — সব business logic database-এ রাখা

Application এবং database-এর responsibility carefully design করতে হবে।

সব logic Stored Procedure-এ ঢুকিয়ে দিলে application architecture unnecessarily complex হতে পারে।

🧠 Class 47 Golden Cheat Sheet

```
CREATE PROCEDURE
    ↓
Reusable SQL operation

CALL
    ↓
Procedure execute

IN
    ↓
Input

OUT
    ↓
Output

INOUT
    ↓
Input + Output

DECLARE
    ↓
Local variable

SET
    ↓
Variable value set
```

```
SELECT ... INTO
    ↓
Query result → variable

IF / ELSEIF / ELSE
    ↓
Conditional logic

CREATE FUNCTION
    ↓
Reusable calculation

RETURN
    ↓
Function result
```

সবচেয়ে গুরুত্বপূর্ণ:

> **Procedure = Perform an operation**

> **Function = Return a value**

> **View = Saved query**

> **CTE = Temporary query result**

📝 Class 47 Assignment

ধরো তোমার `employees` table:

```
employee_id
name
department
salary
city
```

এবং `orders`:

```
order_id
customer_id
order_date
amount
```

Q1

একটি procedure তৈরি করো যেটা সব employees দেখাবে।

Q2

একটি procedure তৈরি করো যেটা `department` parameter নিয়ে employees দেখাবে।

Q3

`city` এবং `minimum_salary` parameter নিয়ে employee search procedure তৈরি করো।

Q4

একটি procedure তৈরি করো যেটা একটি department-এর salary 10% increase করবে।

Q5

একটি procedure তৈরি করো যেটা customer ID নিয়ে তার সব orders দেখাবে।

Q6

একটি procedure তৈরি করো যেটা customer ID নিয়ে total order amount বের করবে।

Q7

`OUT` parameter ব্যবহার করে total employee count return করো।

Q8

একটি function তৈরি করো:

```
salary + 10% bonus
```

Q9

একটি function তৈরি করো যা salary থেকে level return করবে:

```
>= 80000 → High
>= 50000 → Medium
< 50000 → Low
```

Q10 ⭐

নিজের ভাষায় explain করো:

```
Procedure
Function
View
CTE
```

কখন কোনটি ব্যবহার করবে?

🧪 Class 47 MCQ

1. Stored Procedure কী?

A. Temporary table
B. Saved executable SQL logic
C. Index
D. Database

2. Procedure execute করতে কোন command?

A. `RUN`
B. `EXECUTE ONLY`
C. `CALL`
D. `START`

3. `IN` parameter কী?

A. Output
B. Input
C. Table
D. Variable only

4. `OUT` parameter কী করে?

A. Input দেয়
B. Result বাইরে দেয়
C. Table তৈরি করে
D. Query delete করে

5. Local variable declare করতে?

A. `DECLARE`
B. `DEFINE ONLY`
C. `VAR`
D. `CREATE VARIABLE`

6. Function-এর প্রধান বৈশিষ্ট্য কী?

A. একটি value return করে
B. Table delete করে
C. Database তৈরি করে
D. শুধু SELECT করে

7. Function result ফেরত দিতে কোন keyword?

A. `OUTPUT`
B. `RETURN`
C. `SEND`
D. `RESULT`

8. কোনটি বেশি সঠিক?

A. Procedure = calculation only
B. Function = operation only
C. Procedure = operation, Function = return value
D. দুটো সম্পূর্ণ একই

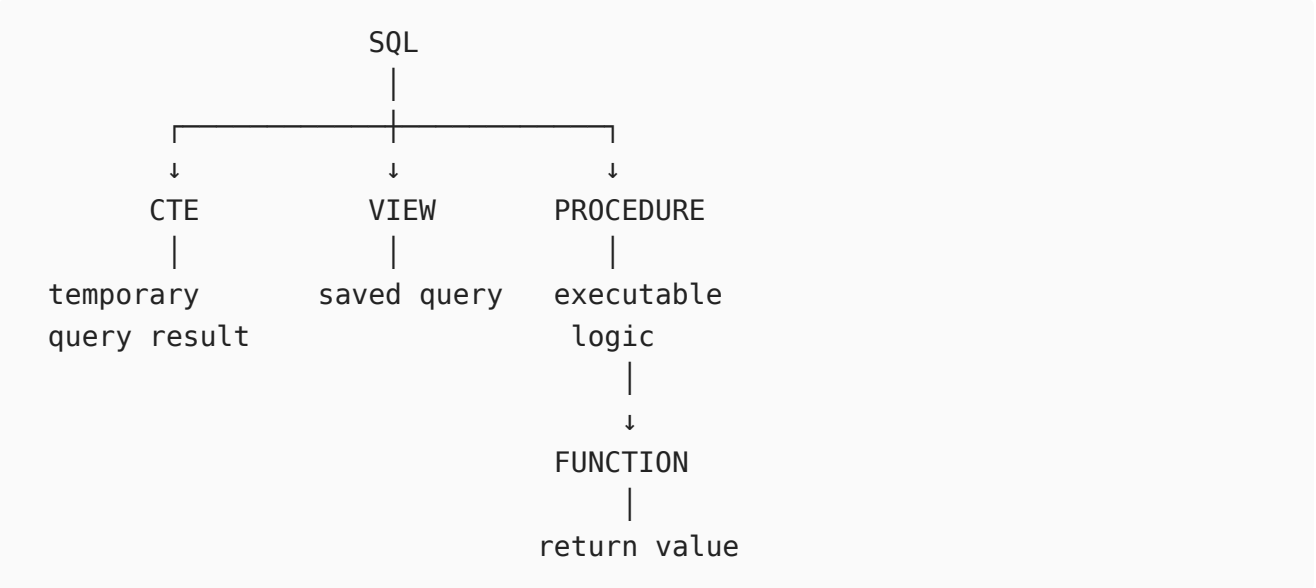
9. MySQL procedure-এ `BEGIN ... END` কেন ব্যবহার করি?

A. Multiple statements group করতে
B. Table create করতে
C. Index করতে
D. Database delete করতে

10. `DELIMITER` মূলত কী সাহায্য করে?

A. Data type change
B. Statement boundary manage
C. Index create
D. NULL remove

🎯 আজকের পুরো SQL Object Map



আর Database Designer হিসেবে এই distinction খুব ভালোভাবে মাথায় রাখো:

```
TABLE
→ Data

VIEW
→ Reusable Query

CTE
→ Temporary Query Structure

PROCEDURE
→ Reusable Operation

FUNCTION
→ Reusable Calculation
```

Next: Class 48 — SQL Transactions + ACID + `COMMIT`, `ROLLBACK`, `SAVEPOINT`। এটা তোমার DBMS preparation-এর সাথেও সরাসরি connect করবে এবং banking/e-commerce-এর মতো systems কীভাবে safe database operations করে সেটা বুঝতে সাহায্য করবে।

📘 Class 48 — SQL Transactions & ACID

আজকের classটা **খুবই important**—বিশেষ করে তুমি যেহেতু **Database Designer** হতে চাও।

আজ আমরা শিখব:

1. Transaction কী?
2. কেন Transaction দরকার?
3. `COMMIT`
4. `ROLLBACK`
5. `SAVEPOINT`
6. ACID Properties
7. Atomicity
8. Consistency
9. Isolation
10. Durability
11. Banking example
12. E-commerce example
13. Transaction-এর common mistakes
14. Practice + MCQ

1️⃣ Transaction কী?

সহজ ভাষায়:

> **Transaction হলো database-এর এক বা একাধিক operation-এর এমন একটি logical unit, যেটা সম্পূর্ণ সফল হবে অথবা পুরোপুরি বাতিল হবে।**

ধরো:

Rahim → Karim = ৳1000 transfer

এখানে দুইটি operation:

```
1. Rahim-এর account থেকে 1000 কমানো
2. Karim-এর account-এ 1000 যোগ করা
```

দুটো operation একসাথে সফল হওয়া দরকার।

যদি:

```
Rahim -1000 ✅
Karim +1000 ❌
```

হয়, তাহলে database-এর data ভুল হয়ে যাবে।

তাই transaction ব্যবহার করি।

2️⃣ Transaction-এর Basic Flow

```
BEGIN TRANSACTION
        ↓
Operation 1
        ↓
Operation 2
        ↓
সব ঠিক?
    ↙       ↘
 YES         NO
  ↓           ↓
COMMIT     ROLLBACK
  ↓           ↓
Save       Undo
```

এটাই আজকের সবচেয়ে important flow।

3️⃣ Transaction শুরু করা

MySQL-এ:

```
START TRANSACTION;
```

অথবা:

```
BEGIN;
```

তারপর SQL statements।

4️⃣ COMMIT কী?

`COMMIT` মানে:

> **Transaction-এর changes permanently save করো।**

Example:

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

এখানে `COMMIT` করার পরে changes save হয়ে যাবে।

5️⃣ ROLLBACK কী?

`ROLLBACK` মানে:

> **Transaction-এর uncommitted changes বাতিল করে আগের অবস্থায় ফিরে যাও।**

Example:

```sql
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

ROLLBACK;
```

Result:

```
আগের balance ফিরে আসবে।
```

6️⃣ COMMIT vs ROLLBACK

Command	কাজ
`START TRANSACTION`	Transaction শুরু
`COMMIT`	Changes permanently save
`ROLLBACK`	Uncommitted changes undo
`SAVEPOINT`	Transaction-এর ভিতরে checkpoint

সহজভাবে:

```
COMMIT   → Save
ROLLBACK → Undo
```

7️⃣ Banking Example

ধরো:

```
Account A = 10,000
Account B = 5,000
```

A থেকে B-তে 2,000 transfer।

```sql
START TRANSACTION;

UPDATE accounts
SET balance = balance - 2000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 2000
WHERE account_id = 2;

COMMIT;
```

Final:

```
A = 8,000
B = 7,000
```

Total:

```
10,000 + 5,000 = 15,000
```

এখন transaction-এর মাধ্যমে total money consistent থাকে।

8️⃣ মাঝখানে Error হলে?

ধরো:

```sql
START TRANSACTION;

UPDATE accounts
SET balance = balance - 2000
WHERE account_id = 1;

-- এখানে কোনো error হলো

UPDATE accounts
SET balance = balance + 2000
WHERE account_id = 2;

ROLLBACK;
```

Rollback করলে প্রথম update-টাও undo হবে।

তাই:

```
A থেকে টাকা কমেছে ❌
B-তে টাকা যোগ হয়নি ❌

ROLLBACK
↓
দুটো change বাতিল
```

9️⃣ SAVEPOINT কী?

Transaction-এর ভিতরে **checkpoint** তৈরি করতে `SAVEPOINT` ব্যবহার করি।

Example:

```sql
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

SAVEPOINT point1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;
```

এখন চাইলে:

```sql
ROLLBACK TO point1;
```

করতে পারি।

এতে পুরো transaction বাতিল হবে না।

`point1` -এর পরের changes rollback হবে।

🔟 SAVEPOINT Flow

```
START TRANSACTION
        ↓
Update A
        ↓
SAVEPOINT P1
```

```
        ↓
Update B
        ↓
ROLLBACK TO P1
        ↓
Update B cancelled
        ↓
Update A remains
```

শেষে:

```
COMMIT;
```

করলে remaining changes save হবে।

1️⃣1️⃣ Multiple SAVEPOINT

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

SAVEPOINT p1;

UPDATE accounts
SET balance = balance + 500
WHERE account_id = 2;

SAVEPOINT p2;

UPDATE accounts
SET balance = balance + 500
WHERE account_id = 3;

ROLLBACK TO p2;

COMMIT;
```

এখানে `p2` -এর পরের changes rollback হবে।

1️⃣2️⃣ এখন আসি ACID-এ 🔥

Database transaction-এর চারটি গুরুত্বপূর্ণ property:

```
A → Atomicity
C → Consistency
I → Isolation
D → Durability
```

অর্থাৎ:

> **ACID**

এটা DBMS exam + interview দুটোর জন্যই অত্যন্ত গুরুত্বপূর্ণ।

1️⃣3️⃣ A — Atomicity

Atomicity মানে:

> **Transaction-এর সব operation সফল হবে অথবা কোনো operation-ই final হবে না।**

অর্থাৎ:

```
ALL or NOTHING
```

Bank transfer:

```
Debit ✅
Credit ❌
```

এভাবে হওয়া যাবে না।

Transaction fail করলে:

```
Debit ❌
Credit ❌
```

Rollback হবে।

Golden Rule:

> **Atomicity = All or Nothing**

1️⃣4️⃣ C — Consistency

Consistency মানে:

> **Transaction-এর আগে এবং পরে database valid state-এ থাকতে হবে।**

ধরো:

```
A = 10,000
B = 5,000
Total = 15,000
```

A → B 2,000 transfer:

```
A = 8,000
B = 7,000
Total = 15,000
```

Database-এর defined rules/constraints ভাঙা যাবে না।

যেমন:

```
balance < 0
```

যদি business rule অনুযায়ী allowed না হয়, তাহলে transaction fail করতে হবে।

Golden Rule:

> **Consistency = Valid state → Valid state**

1️⃣5️⃣ Atomicity vs Consistency

এই দুইটা অনেকেই confuse করে।

Atomicity:

```
সব operation হবে
অথবা
কোনোটাই হবে না
```

Consistency:

```
Database rules/constraints ঠিক থাকবে
```

Example:

```
Atomicity
→ Debit + Credit দুটো একসাথে হবে

Consistency
→ Transfer-এর পর total balance এবং constraints valid থাকবে
```

1️⃣6️⃣ I — Isolation

Isolation মানে:

> **একটি transaction-এর intermediate/uncommitted changes অন্য concurrent transaction যেন অনাকাঙ্ক্ষিতভাবে দেখতে বা প্রভাবিত করতে না পারে।**

ধরো:

```
Transaction A
        ↓
Balance update করছে

Transaction B
        ↓
একই balance পড়ছে
```

A এখনো `COMMIT` করেনি।

B যেন A-এর incomplete state-এর কারণে ভুল result না পায়—এই ধরনের সমস্যা control করতে isolation ব্যবহৃত হয়।

1️⃣7️⃣ Isolation কেন দরকার?

একাধিক user একই সময়ে database ব্যবহার করে।

যেমন:

```
User A → Bank Account
User B → Same Bank Account
User C → Same Bank Account
```

সবাই একই সময়ে operation করতে পারে।

Isolation না থাকলে concurrency problems হতে পারে।

1️⃣8️⃣ Common Isolation Problems

চারটি নাম মনে রাখো:

```
Dirty Read
Non-Repeatable Read
Phantom Read
Lost Update
```

আজ এগুলোর basic concept বুঝি।

Dirty Read

Transaction A data পরিবর্তন করেছে কিন্তু এখনো commit করেনি।

Transaction B সেই uncommitted data পড়ে ফেলল।

```
A → UPDATE
      ↓
   uncommitted
      ↓
B → reads it ❌
```

এটাই Dirty Read।

1️⃣9️⃣ Non-Repeatable Read

একই transaction-এর মধ্যে একই row দুইবার read করা হলো।

প্রথমবার:

```
salary = 50,000
```

অন্য transaction update করল।

দ্বিতীয়বার:

```
salary = 60,000
```

একই transaction-এ একই row-এর value বদলে গেল।

এটাই Non-Repeatable Read।

2️⃣0️⃣ Phantom Read

একই condition দিয়ে query দুইবার চালানো হলো।

প্রথমবার:

```
5 employees
```

অন্য transaction নতুন matching row insert করল।

দ্বিতীয়বার:

```
6 employees
```

নতুন row যেন "phantom" হিসেবে দেখা দিল।

এটাই Phantom Read।

2️⃣1️⃣ Lost Update

দুই transaction একই data update করতে গিয়ে একজনের update অন্যজন overwrite করে ফেললে Lost Update হতে পারে।

```
Initial salary = 50,000

A reads → 50,000
B reads → 50,000

A → 55,000
B → 52,000

Final → 52,000
```

A-এর update হারিয়ে গেল।

2️⃣2️⃣ D — Durability

Durability মানে:

> **COMMIT হয়ে গেলে transaction-এর changes database failure হলেও হারিয়ে যাওয়া উচিত নয়।**

ধরো:

```
Transaction
    ↓
COMMIT
    ↓
Server crash
    ↓
Restart
```

তারপরও committed data থাকার কথা।

Database সাধারণত logging/recovery mechanisms ব্যবহার করে durability support করে।

Golden Rule:

> **Durability = Committed data stays saved**

2️⃣3️⃣ ACID এক লাইনে

Property	সহজ অর্থ
Atomicity	All or Nothing
Consistency	Valid state বজায় রাখা
Isolation	Concurrent transactions-এর interference control
Durability	Commit-এর পর data স্থায়ী

Shortcut:

```
A → All or Nothing
C → Correct/Valid State
I → Independent/Isolated execution
D → Data stays after commit
```

2️⃣4️⃣ ACID Real-Life Example

ধরো ATM থেকে টাকা তুলছো।

```
ATM Withdrawal
      ↓
Check balance
```

```
      ↓
Deduct money
      ↓
Record transaction
      ↓
Complete
```

Atomicity

টাকা deduction এবং transaction operation মাঝপথে ভেঙে গেলে partial result হওয়া উচিত নয়।

Consistency

Invalid balance state তৈরি করা যাবে না।

Isolation

একই account নিয়ে concurrent transactions safely handle করতে হবে।

Durability

Successful transaction commit হওয়ার পর record হারানো যাবে না।

2️⃣5️⃣ Transaction + Constraints

Transaction শুধু rollback-এর জন্য নয়।

Database constraints-এর সাথে কাজ করে:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
CHECK
NOT NULL
```

যদি transaction-এর কোনো operation constraint violate করে এবং application/database logic failure handle করে rollback করে, তাহলে database invalid state থেকে রক্ষা পেতে পারে।

2️⃣6️⃣ Transaction-এর একটি গুরুত্বপূর্ণ Rule

ধরো:

```
START TRANSACTION;
```

```sql
UPDATE accounts
SET balance = balance - 1000
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

`COMMIT` করার আগে changes সাধারণত transaction-এর অংশ।

কিন্তু `COMMIT` করার পরে:

```sql
ROLLBACK;
```

দিয়ে committed transaction undo করা যায় না।

মনে রাখো:

> **ROLLBACK works on uncommitted changes.**

2️⃣7️⃣ Autocommit

MySQL-এ সাধারণত autocommit enabled থাকতে পারে।

মানে একটি statement সফলভাবে execute হলে সেটি automatically commit হতে পারে।

তাই multiple statements-এর transaction control করতে:

```sql
START TRANSACTION;
```

ব্যবহার করা গুরুত্বপূর্ণ।

2️⃣8️⃣ Transaction কখন ব্যবহার করব?

বিশেষ করে যখন multiple operations **একসাথে সফল হওয়া দরকার**।

Examples:

Banking

```
Debit
+
Credit
```

E-commerce

```
Create Order
+
Reduce Stock
+
Create Payment
```

Hotel

```
Create Booking
+
Reserve Room
+
Create Payment
```

Ticket System

```
Reserve Seat
+
Create Booking
+
Payment Record
```

2️⃣9️⃣ Database Designer হিসেবে ভাবো

ধরো Hotel Management System:

```
Customer books room
        ↓
Check availability
        ↓
Create booking
        ↓
Update room availability
        ↓
Create payment
```

এই কাজগুলো আলাদা আলাদা SQL statement হতে পারে।

কিন্তু booking operation-এর logical unit হিসেবে transaction ব্যবহার করা যেতে পারে।

```
START TRANSACTION
       ↓
Check/Reserve
       ↓
Booking Insert
       ↓
Room Update
       ↓
Payment Record
       ↓
COMMIT
```

কোনো critical step fail হলে:

```
ROLLBACK
```

3️⃣0️⃣ Transaction + Stored Procedure

Class 47-এর সাথে connect করি।

Stored Procedure-এর ভিতরেও transaction logic থাকতে পারে।

Conceptually:

```
CALL place_order()
       ↓
START TRANSACTION
       ↓
Create Order
       ↓
Reduce Stock
       ↓
Create Payment
       ↓
COMMIT
```

Error হলে:

```
ROLLBACK
```

এটা real-world backend/database design-এর গুরুত্বপূর্ণ pattern।

3️⃣1️⃣ Common Beginner Mistakes

❌ Mistake 1

শুধু:

```
START TRANSACTION;
```

লিখে `COMMIT` / `ROLLBACK` না করা।

❌ Mistake 2

মনে করা `ROLLBACK` সব historical data undo করতে পারে।

না।

> Rollback মূলত current transaction-এর uncommitted changes-এর জন্য।

❌ Mistake 3

Atomicity এবং Consistency একই মনে করা।

```
Atomicity
→ All or Nothing

Consistency
→ Valid State
```

❌ Mistake 4

Isolation মানে "database-এর সব transaction একটার পর একটা চলবে" মনে করা।

Isolation-এর উদ্দেশ্য concurrency control—সব transaction necessarily serially execute করানো নয়।

❌ Mistake 5

প্রতিটি SELECT-এর জন্য transaction ব্যবহার করা।

সব query-তে complex transaction দরকার হয় না। Business operation অনুযায়ী transaction boundary design করতে হয়।

🧠 Class 48 Golden Cheat Sheet

```
TRANSACTION
↓
Logical unit of work

START TRANSACTION
↓
Begin

COMMIT
↓
Save permanently

ROLLBACK
↓
Undo uncommitted changes

SAVEPOINT
↓
Create checkpoint
```

ACID:

```
A = Atomicity
    All or Nothing

C = Consistency
    Valid State

I = Isolation
    Concurrent transaction control

D = Durability
    Committed data remains
```

📝 Class 48 Assignment

Q1

একটি transaction লিখো যেখানে:

```
Account 1 থেকে 500 টাকা কমবে
Account 2-তে 500 টাকা যোগ হবে
```

তারপর `COMMIT` করবে।

Q2

উপরের transaction-এ দ্বিতীয় operation fail করলে কীভাবে `ROLLBACK` করবে?

Q3

`SAVEPOINT` ব্যবহার করে:

```
Update A
SAVEPOINT
Update B
ROLLBACK TO SAVEPOINT
COMMIT
```

লিখো।

Q4

নিজের ভাষায় explain করো:

```
Atomicity
Consistency
Isolation
Durability
```

Q5 ⭐

Bank transfer-এর ক্ষেত্রে কেন transaction দরকার?

Q6 ⭐

এই চারটির difference explain করো:

```
COMMIT
ROLLBACK
SAVEPOINT
ROLLBACK TO SAVEPOINT
```

🧪 Class 48 MCQ

1. Transaction কী?

A. Table
B. Logical unit of work
C. Index
D. Column

2. Permanentভাবে transaction changes save করতে?

A. SAVE
B. COMMIT
C. STORE
D. ACCEPT

3. Uncommitted changes বাতিল করতে?

A. DELETE
B. DROP
C. ROLLBACK
D. CLEAR

4. ACID-এর A কী?

A. Access
B. Atomicity
C. Accuracy
D. Authorization

5. "All or Nothing" কোন property?

A. Consistency
B. Isolation
C. Atomicity
D. Durability

6. Database valid state বজায় রাখার property?

A. Atomicity
B. Consistency
C. Isolation
D. Durability

7. Concurrent transactions-এর interference control-এর সাথে কোনটি সম্পর্কিত?

A. Isolation
B. Atomicity
C. Durability
D. Backup

8. Commit-এর পরে data persistence কোন property?

A. Atomicity
B. Consistency
C. Isolation
D. Durability

9. Transaction-এর ভিতরে checkpoint তৈরি করতে?

A. CHECKPOINT ONLY
B. SAVEPOINT
C. MARK
D. HOLD

10. `ROLLBACK TO p1` কী করে?

A. পুরো database delete করে
B. পুরো transaction permanently commit করে
C. নির্দিষ্ট savepoint-এ ফিরে যায়
D. Table drop করে

🎯 আজকের সবচেয়ে গুরুত্বপূর্ণ ৫টি লাইন

```
Transaction = Logical Unit of Work

COMMIT = Save
```

```
ROLLBACK = Undo uncommitted changes

SAVEPOINT = Checkpoint

ACID = Atomicity + Consistency + Isolation + Durability
```

Next → Class 49: SQL Triggers — `BEFORE/AFTER INSERT`, `UPDATE`, `DELETE`, audit log, automatic validation এবং real-world database automation।

📘 Class 49 — SQL Triggers

আজকে আমরা শিখব **SQL Trigger**।

তুমি Class 47-এ শিখেছো **Stored Procedure**, Class 48-এ **Transaction**। এখন Trigger শিখলে database-এর **automatic actions** বুঝতে পারবে।

1️⃣ Trigger কী?

সহজভাবে:

> **Trigger হলো database-এর এমন একটি automatic program, যা কোনো নির্দিষ্ট database event ঘটলে নিজে থেকেই execute হয়।**

তুমি manually:

```
CALL procedure();
```

করবে না।

বরং:

```
INSERT / UPDATE / DELETE
        ↓
   Event ঘটলো
        ↓
    TRIGGER
        ↓
Automatic Action
```

Example

কেউ employee-এর salary update করল:

```
Salary changed
      ↓
Trigger automatically runs
      ↓
Audit table-এ old/new salary save
```

2️⃣ Trigger কেন ব্যবহার করি?

Real-world database-এ Trigger ব্যবহার হতে পারে:

- Audit log
- Automatically update related data
- Data validation
- History maintain
- Inventory update
- Change tracking
- Automatic timestamp
- Delete history
- Business rule enforcement

3️⃣ Trigger-এর Basic Structure

MySQL:

```
DELIMITER //

CREATE TRIGGER trigger_name
BEFORE INSERT
ON table_name
FOR EACH ROW
BEGIN

    -- SQL statements

END //

DELIMITER ;
```

এখানে:

```
BEFORE
↓
কখন?

INSERT
↓
কোন event?

ON table_name
↓
কোন table?

FOR EACH ROW
↓
কোন row-এর জন্য?
```

4️⃣ Trigger-এর তিনটি Main Event

সবচেয়ে গুরুত্বপূর্ণ:

```
INSERT
UPDATE
DELETE
```

অর্থাৎ:

```
INSERT → নতুন row
UPDATE → row পরিবর্তন
DELETE → row মুছে ফেলা
```

5️⃣ BEFORE vs AFTER

Trigger event-এর আগে অথবা পরে execute হতে পারে।

```
BEFORE
AFTER
```

তাই basic combination:

```
BEFORE INSERT
AFTER INSERT
```

```
BEFORE UPDATE
AFTER UPDATE

BEFORE DELETE
AFTER DELETE
```

6️⃣ BEFORE INSERT

ধরো employee insert করার আগে salary validation করতে চাই।

```
DELIMITER //

CREATE TRIGGER check_salary
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN

    IF NEW.salary < 0 THEN
        SET NEW.salary = 0;
    END IF;

END //

DELIMITER ;
```

এখানে:

```
NEW.salary
```

মানে হচ্ছে:

> যে নতুন row insert হতে যাচ্ছে, তার salary।

7️⃣ `NEW` কী?

Trigger-এ `NEW` খুব গুরুত্বপূর্ণ।

ধরো:

```
INSERT INTO employees
(name, salary)
```

```
VALUES
('Rahim', 50000);
```

Trigger-এর ভিতরে:

```
NEW.name
```

হবে:

```
Rahim
```

আর:

```
NEW.salary
```

হবে:

```
50000
```

Golden Rule

> **NEW = নতুন value**

8️⃣ `OLD` কী?

`OLD` মানে:

> **আগের value**

বিশেষ করে UPDATE এবং DELETE trigger-এ বেশি ব্যবহৃত হয়।

ধরো salary আগে:

```
50000
```

Update হয়ে:

```
60000
```

তাহলে:

```
OLD.salary = 50000
NEW.salary = 60000
```

9️⃣ NEW vs OLD

Event	NEW	OLD
INSERT	✅	❌
UPDATE	✅	✅
DELETE	❌	✅

এটা **খুব গুরুত্বপূর্ণ MCQ**।

মনে রাখো:

```
INSERT → NEW
UPDATE → OLD + NEW
DELETE → OLD
```

🔟 BEFORE INSERT — Real Example

ধরো user email lowercase করতে চাই।

```
DELIMITER //

CREATE TRIGGER normalize_email
BEFORE INSERT
ON customers
FOR EACH ROW
BEGIN
    SET NEW.email = LOWER(TRIM(NEW.email));
END //

DELIMITER ;
```

এখন:

```
INSERT INTO customers
(name, email)
```

```
VALUES
('Rahim', '  RAHIM@GMAIL.COM  ');
```

Trigger automatically করবে:

```
"  RAHIM@GMAIL.COM  "
          ↓
"rahim@gmail.com"
```

1️⃣1️⃣ AFTER INSERT

কখনো নতুন row insert হওয়ার **পরে** অন্য কাজ করতে চাই।

ধরো audit table:

```
CREATE TABLE customer_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    action VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

এখন customer insert হলে automatically log:

```
DELIMITER //

CREATE TRIGGER customer_insert_log
AFTER INSERT
ON customers
FOR EACH ROW
BEGIN

    INSERT INTO customer_logs
    (customer_id, action)
    VALUES
    (NEW.customer_id, 'CUSTOMER_CREATED');

END //

DELIMITER ;
```

এখন:

```sql
INSERT INTO customers
(customer_name, city)
VALUES
('Karim', 'Dhaka');
```

তখন automatically:

```
customers
   ↓
New customer

customer_logs
   ↓
CUSTOMER_CREATED
```

1️⃣2️⃣ BEFORE UPDATE

ধরো salary negative হতে দিতে চাই না।

```sql
DELIMITER //

CREATE TRIGGER validate_salary_update
BEFORE UPDATE
ON employees
FOR EACH ROW
BEGIN

    IF NEW.salary < 0 THEN
        SET NEW.salary = 0;
    END IF;

END //

DELIMITER ;
```

যদি:

```sql
UPDATE employees
SET salary = -5000
WHERE employee_id = 1;
```

তাহলে Trigger:

```
NEW.salary = -5000
       ↓
Check
       ↓
< 0
       ↓
Set = 0
```

1️⃣3️⃣ AFTER UPDATE

এবার salary change history রাখি।

History table:

```sql
CREATE TABLE salary_history (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Trigger:

```sql
DELIMITER //

CREATE TRIGGER salary_change_log
AFTER UPDATE
ON employees
FOR EACH ROW
BEGIN

    IF OLD.salary <> NEW.salary THEN

        INSERT INTO salary_history
        (
            employee_id,
            old_salary,
            new_salary
        )
        VALUES
        (
            NEW.employee_id,
            OLD.salary,
            NEW.salary
```

```
        );

    END IF;

END //

DELIMITER ;
```

এখন:

```
UPDATE employees
SET salary = 60000
WHERE employee_id = 1;
```

Automatically history তৈরি হবে।

1️⃣4️⃣ কেন `OLD` এবং `NEW` দুটো দরকার?

কারণ আমরা জানতে চাই:

```
আগে কত ছিল?
↓
OLD.salary

এখন কত?
↓
NEW.salary
```

Example:

```
OLD.salary = 50000
NEW.salary = 60000
```

Audit system-এর জন্য এটা খুব useful।

1️⃣5️⃣ AFTER DELETE

ধরো employee delete করলে তার history রাখতে চাই।

```
CREATE TABLE deleted_employees (
    employee_id INT,
    name VARCHAR(100),
```

```
    deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Trigger:

```
DELIMITER //

CREATE TRIGGER employee_delete_log
AFTER DELETE
ON employees
FOR EACH ROW
BEGIN

    INSERT INTO deleted_employees
    (employee_id, name)
    VALUES
    (OLD.employee_id, OLD.name);

END //

DELIMITER ;
```

এখন:

```
DELETE FROM employees
WHERE employee_id = 5;
```

Delete হওয়ার পর:

```
employees
   ↓
row deleted

deleted_employees
   ↓
old row saved
```

এখানে:

```
OLD.employee_id
OLD.name
```

ব্যবহার করেছি।

কারণ DELETE-এর পরে `NEW` row নেই।

1️⃣6️⃣ BEFORE DELETE

কখনো delete হওয়ার আগে restriction দিতে পারি।

Example:

```
DELIMITER //

CREATE TRIGGER prevent_employee_delete
BEFORE DELETE
ON employees
FOR EACH ROW
BEGIN

    IF OLD.department = 'Management' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Management employee cannot be deleted';

    END IF;

END //

DELIMITER ;
```

এখন management employee delete করতে গেলে error হবে।

1️⃣7️⃣ `SIGNAL` কী?

MySQL-এ custom error তৈরি করতে:

```
SIGNAL SQLSTATE '45000'
```

ব্যবহার করা যায়।

Example:

```
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Invalid salary';
```

Result:

```
Error: Invalid salary
```

এটা database-level validation-এর জন্য useful।

1️⃣8️⃣ Inventory Example 🔥

এখন একটি খুব important real-world example।

ধরো:

```
products
---------
product_id
name
stock
price
```

Order তৈরি হলে stock automatically কমবে।

Order insert:

```
INSERT INTO orders
(product_id, quantity)
VALUES
(10, 3);
```

Trigger:

```
DELIMITER //

CREATE TRIGGER reduce_stock
AFTER INSERT
ON orders
FOR EACH ROW
BEGIN

    UPDATE products
    SET stock = stock - NEW.quantity
    WHERE product_id = NEW.product_id;

END //

DELIMITER ;
```

এখন:

```
Order quantity = 3
        ↓
Trigger
        ↓
Product stock - 3
```

1️⃣9️⃣ কিন্তু এখানে একটি Problem আছে ⚠️

শুধু stock কমালেই হবে না।

ধরো:

```
Current stock = 2
Order quantity = 5
```

তাহলে:

```
2 - 5 = -3
```

Stock negative হয়ে যাবে।

তাই validation দরকার।

2️⃣0️⃣ BEFORE INSERT Stock Validation

```
DELIMITER //

CREATE TRIGGER check_stock
BEFORE INSERT
ON orders
FOR EACH ROW
BEGIN

    DECLARE current_stock INT;

    SELECT stock
    INTO current_stock
    FROM products
    WHERE product_id = NEW.product_id;

    IF current_stock < NEW.quantity THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient stock';

    END IF;

END //

DELIMITER ;
```

Flow:

```
New Order
   ↓
Check stock
   ↓
Enough?
 ↙      ↘
YES      NO
 ↓        ↓
Insert   Error
```

2️⃣1️⃣ Trigger + Transaction

Class 48-এর সাথে connect করি।

ধরো:

```
Create Order
+
Reduce Stock
+
Payment
```

Transaction:

```
START TRANSACTION
       ↓
INSERT order
       ↓
Trigger runs
       ↓
Stock update
       ↓
Payment
```

```
        ↓
COMMIT
```

কোনো critical operation fail করলে transaction rollback করা যায়।

2️⃣2️⃣ Trigger vs Procedure

খুব important comparison:

Feature	Trigger	Procedure
Manually call	❌	✅ `CALL`
Automatically runs	✅	❌
Event based	✅	সাধারণত ❌
INSERT/UPDATE/DELETE event	✅	Logic-এর উপর নির্ভর
Parameters	Trigger parameters নেই	IN/OUT/INOUT
Audit	Excellent	Possible
Business operation	Limited/Use carefully	Excellent

সহজভাবে:

> **Procedure = তুমি ডাকো**

> **Trigger = Database নিজে ডাকে**

2️⃣3️⃣ Trigger vs Function

Trigger	Function
Event-এর কারণে automatically execute	Query-তে call করা যায়
INSERT/UPDATE/DELETE event	Value return করে
Audit/automation	Calculation
`NEW` / `OLD`	Parameters
Directly `CALL` নয়	`SELECT function()`

2️⃣4️⃣ Trigger vs View

```
VIEW
→ Data দেখানোর reusable query

TRIGGER
→ Event ঘটলে automatic action

PROCEDURE
→ Explicitly call করা reusable operation

FUNCTION
→ Reusable calculation
```

এখন তোমার SQL object knowledge:

```
TABLE
↓
Actual Data

VIEW
↓
Saved Query

CTE
↓
Temporary Query Structure

FUNCTION
↓
Return Value

PROCEDURE
↓
Executable Logic

TRIGGER
↓
Automatic Event-Based Logic
```

2️⃣5️⃣ Trigger কখন ব্যবহার করা ভালো?

ভালো use case:

```
Audit Log
History
Automatic timestamp
```

```
Data validation
Automatic related update
Inventory synchronization
```

2️⃣6️⃣ Trigger কখন Avoid করা উচিত?

Trigger powerful হলেও বেশি ব্যবহার করলে application বুঝতে কঠিন হয়ে যেতে পারে।

ধরো:

```
INSERT INTO orders ...
```

তুমি ভাবলে শুধু order insert হচ্ছে।

কিন্তু database-এর ভিতরে:

```
Trigger 1
↓
Update stock

Trigger 2
↓
Create log

Trigger 3
↓
Update customer points

Trigger 4
↓
Create history
```

তখন debugging কঠিন হতে পারে।

Golden Rule:

> **Trigger ব্যবহার করো যখন automatic database-level behavior সত্যিই দরকার।**

সব business logic trigger-এ ঢুকিয়ে দিও না।

2️⃣7️⃣ Common Beginner Mistakes

❌ Mistake 1

`INSERT` trigger-এ `OLD` ব্যবহার করা।

ভুল:

```
OLD.salary
```

INSERT-এর ক্ষেত্রে `OLD` নেই।

❌ Mistake 2

DELETE trigger-এ `NEW` ব্যবহার করা।

DELETE-এর ক্ষেত্রে:

```
OLD ✅
NEW ❌
```

❌ Mistake 3

একই table-এর জন্য duplicate trigger name তৈরি করা।

Trigger name database/schema অনুযায়ী unique রাখতে হবে।

❌ Mistake 4

Trigger-এর ভিতরে ভুলভাবে recursive operation করা।

একটি trigger অন্য trigger-কে indirectly activate করলে logic complex হতে পারে।

❌ Mistake 5

Trigger দিয়ে সব business logic করা।

এতে debugging এবং maintenance কঠিন হতে পারে।

2️⃣8️⃣ Trigger দেখার Command

MySQL-এ:

```
SHOW TRIGGERS;
```

নির্দিষ্ট trigger-এর definition:

```
SHOW CREATE TRIGGER trigger_name;
```

2️⃣9️⃣ Trigger Delete

Trigger আর দরকার না হলে:

```
DROP TRIGGER trigger_name;
```

Safe version:

```
DROP TRIGGER IF EXISTS trigger_name;
```

🧠 Class 49 Golden Cheat Sheet

```
TRIGGER
↓
Automatic database action
```

Events:

```
INSERT
UPDATE
DELETE
```

Timing:

```
BEFORE
AFTER
```

NEW/OLD:

```
INSERT → NEW

UPDATE → OLD + NEW
```

```
DELETE → OLD
```

সবচেয়ে গুরুত্বপূর্ণ:

```
Procedure
→ Manually CALL

Trigger
→ Automatically fires

Function
→ Returns value

View
→ Saved query
```

📝 Class 49 Assignment

ধরো তোমার database:

`products`

```
product_id
name
stock
price
```

`orders`

```
order_id
product_id
quantity
order_date
```

`price_history`

```
history_id
product_id
old_price
new_price
changed_at
```

Task 1

একটি `BEFORE INSERT` trigger তৈরি করো যাতে:

```
quantity <= 0
```

হলে error দেখায়।

Task 2

একটি `AFTER INSERT` trigger তৈরি করো যাতে order তৈরি হলে product stock automatically কমে যায়।

Task 3 ⭐

একটি `BEFORE INSERT` trigger তৈরি করো যাতে insufficient stock হলে:

```
Insufficient stock
```

error দেয়।

Task 4

Product-এর price পরিবর্তন হলে `price_history` table-এ:

```
old_price
new_price
```

automatically save করো।

Task 5

নিজের ভাষায় explain করো:

```
NEW
OLD
```

```
BEFORE
AFTER
```

Task 6 ⭐

এই চারটির difference লিখো:

```
Procedure
Function
Trigger
View
```

🧪 Class 49 MCQ

1. Trigger কী?

A. Saved table
B. Automatically executed database logic
C. Index
D. Database

2. Trigger কোন event-এর উপর কাজ করতে পারে?

A. INSERT
B. UPDATE
C. DELETE
D. All of the above

3. INSERT-এর ক্ষেত্রে কোনটি available?

A. OLD
B. NEW
C. OLD + NEW
D. কোনোটিই নয়

4. DELETE-এর ক্ষেত্রে কোনটি available?

A. NEW
B. OLD
C. OLD + NEW
D. Parameter

5. UPDATE-এর ক্ষেত্রে?

A. শুধু OLD
B. শুধু NEW
C. OLD + NEW
D. কোনোটিই নয়

6. Trigger manually `CALL` করা হয়?

A. Yes
B. No

7. Custom MySQL error তৈরি করতে কোনটি ব্যবহার করা যায়?

A. SIGNAL
B. ERROR()
C. THROW ONLY
D. STOP

8. Audit log-এর জন্য কোনটি useful?

A. Trigger
B. Index only
C. Database name
D. Column alias

9. `BEFORE INSERT` trigger কখন execute হয়?

A. Insert-এর আগে
B. Insert-এর পরে
C. Database বন্ধ হওয়ার পরে
D. SELECT-এর সময়

10. কোন statement trigger delete করে?

A. DELETE TRIGGER
B. REMOVE TRIGGER
C. DROP TRIGGER
D. CLEAR TRIGGER

🎯 আজকের ৬টা Golden Line

```
1. Trigger = Automatic database logic

2. BEFORE = Event-এর আগে

3. AFTER = Event-এর পরে
```

```
4. INSERT → NEW

5. UPDATE → OLD + NEW

6. DELETE → OLD
```

Next → Class 50: SQL Indexing — Primary Index, Secondary Index, Composite Index, Unique Index, B-Tree concept, কেন index query দ্রুত করে, কখন index ব্যবহার করা উচিত/উচিত নয়, এবং `EXPLAIN` দিয়ে query performance বোঝার foundation।

📘 Class 50 — SQL Indexing

আজকের classটা **Database Designer** হওয়ার জন্য খুবই গুরুত্বপূর্ণ।

এখন পর্যন্ত তুমি SQL-এর query writing, JOIN, CTE, Window Function, View, Procedure, Transaction, Trigger শিখেছো।

আজ থেকে আমরা query **কীভাবে দ্রুত এবং efficiently execute করানো যায়** সেটা শিখব।

আজকের topics:

1. Index কী?
2. কেন Index দরকার?
3. Without Index vs With Index
4. `CREATE INDEX`
5. Unique Index
6. Composite Index
7. Primary Key Index
8. Foreign Key Index
9. Index কখন ব্যবহার করবে?
10. Index কখন ব্যবহার করবে না?
11. B-Tree concept
12. Composite Index-এর column order
13. Covering Index concept
14. `EXPLAIN` introduction
15. Common mistakes
16. Assignment + MCQ

1️⃣ Index কী?

সহজভাবে:

> **Index হলো database-এর একটি data structure, যা নির্দিষ্ট data দ্রুত খুঁজে পেতে সাহায্য করে।**

একটা বই চিন্তা করো।

তুমি যদি বইয়ের মধ্যে:

```
Database
```

শব্দটা খুঁজতে চাও, তাহলে পুরো 500 page পড়বে না।

বইয়ের শেষে:

```
Index

Database ............ Page 120
Database Design ..... Page 250
Database Security ... Page 380
```

থাকলে সরাসরি relevant page-এ যেতে পারো।

Database index একই ধরনের ধারণা।

2️⃣ Without Index কী হয়?

ধরো:

```sql
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

এবং table-এ:

```
10,000,000 rows
```

থাকে।

যদি suitable index না থাকে, database-কে অনেক row পরীক্ষা করতে হতে পারে।

Conceptually:

```
1
2
3
4
```

```
5
...
10,000,000
```

3️⃣ With Index

যদি:

```
CREATE INDEX idx_customer_email
ON customers(email);
```

থাকে, database email lookup-এর জন্য index ব্যবহার করতে পারে।

Conceptually:

```
Index
  ↓
Find email
  ↓
Find row location
  ↓
Read row
```

তাই search অনেক efficient হতে পারে।

4️⃣ Index-এর মূল উদ্দেশ্য

```
Index
  ↓
Faster Data Retrieval
```

বিশেষ করে:

```
WHERE
JOIN
ORDER BY
GROUP BY
```

এর কিছু ক্ষেত্রে index খুব useful হতে পারে।

তবে **সব query automatically faster হবে না**।

5️⃣ Basic Index তৈরি

```
CREATE INDEX idx_customer_email
ON customers(email);
```

Structure:

```
CREATE INDEX index_name
ON table_name(column_name);
```

6️⃣ Index List দেখা

MySQL:

```
SHOW INDEX FROM customers;
```

এতে table-এর indexes দেখতে পারবে।

7️⃣ Index Delete

Index আর দরকার না হলে:

```
DROP INDEX idx_customer_email
ON customers;
```

8️⃣ Primary Key কি Index?

হ্যাঁ।

যেমন:

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100)
);
```

`PRIMARY KEY` সাধারণত একটি index structure তৈরি করে।

তাই আলাদাভাবে:

```
CREATE INDEX idx_customer_id
ON customers(customer_id);
```

করার সাধারণত দরকার নেই।

9️⃣ UNIQUE Index

ধরো customer email unique হতে হবে।

```
CREATE UNIQUE INDEX idx_unique_email
ON customers(email);
```

এতে একই email duplicate insert করা যাবে না।

Example:

```
rahim@gmail.com ✅

rahim@gmail.com ❌
```

তবে যদি column-এ আগে থেকেই:

```
email VARCHAR(255) UNIQUE
```

থাকে, database ইতিমধ্যে uniqueness enforce করার জন্য appropriate index তৈরি করতে পারে। তাই duplicate index তৈরি করা উচিত নয়।

🔟 Index + WHERE

সবচেয়ে common use case:

```
SELECT *
FROM employees
WHERE department = 'IT';
```

যদি এই ধরনের query খুব frequently চলে:

```sql
CREATE INDEX idx_employee_department
ON employees(department);
```

তাহলে optimizer পরিস্থিতি অনুযায়ী index ব্যবহার করতে পারে।

1️⃣1️⃣ Index + JOIN

ধরো:

```sql
SELECT *
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id;
```

এখানে:

```sql
orders.customer_id
```

একটি important join column।

তাই appropriate index useful হতে পারে।

```sql
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

তবে actual performance `EXPLAIN` দিয়ে verify করা উচিত।

1️⃣2️⃣ Foreign Key এবং Index

ধরো:

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

`customer_id`-তে index থাকা সাধারণত useful, বিশেষ করে JOIN এবং parent-child operations-এর জন্য।

কিছু MySQL/InnoDB পরিস্থিতিতে foreign key constraint-এর জন্য প্রয়োজনীয় index automatically তৈরি/ব্যবহৃত হতে পারে।

তাই blindly duplicate index তৈরি না করে:

```
SHOW INDEX FROM orders;
```

দিয়ে আগে check করবে।

1️⃣3️⃣ Composite Index

এখন important topic।

ধরো query:

```
SELECT *
FROM employees
WHERE department = 'IT'
  AND city = 'Dhaka';
```

দুই column-এর উপর একসাথে index তৈরি করা যায়:

```
CREATE INDEX idx_dept_city
ON employees(department, city);
```

এটাকে বলে:

> **Composite Index / Multi-column Index**

1️⃣4️⃣ Composite Index-এর Structure

```
(department, city)
```

এখানে:

```
1st column → department
2nd column → city
```

Column order গুরুত্বপূর্ণ।

1️⃣5️⃣ Leftmost Prefix Rule 🔥

এটা খুব গুরুত্বপূর্ণ।

Index:

```
CREATE INDEX idx_dept_city
ON employees(department, city);
```

এই index ভালোভাবে কাজে লাগতে পারে যখন query শুরু করছে:

```
WHERE department = 'IT'
```

অথবা:

```
WHERE department = 'IT'
AND city = 'Dhaka'
```

কিন্তু শুধু:

```
WHERE city = 'Dhaka'
```

এর ক্ষেত্রে এই composite index সাধারণত পুরোপুরি equivalent সুবিধা দেবে না।

কারণ index order:

```
department → city
```

1️⃣6️⃣ কেন Order গুরুত্বপূর্ণ?

ধরো telephone directory:

```
Last Name
   ↓
First Name
```

যদি তুমি শুধু first name জানো:

```
Rahim
```

তাহলে last name না জেনে পুরো structure থেকে efficiently locate করা কঠিন হতে পারে।

একইভাবে:

```
(department, city)
```

এবং:

```
(city, department)
```

এক জিনিস নয়।

1️⃣7️⃣ Composite Index Example

Query:

```sql
SELECT *
FROM orders
WHERE customer_id = 10
  AND order_date >= '2026-01-01';
```

Potential index:

```sql
CREATE INDEX idx_customer_date
ON orders(customer_id, order_date);
```

এখানে:

```
customer_id → first
order_date  → second
```

এই query pattern-এর সাথে ভালোভাবে match করে।

1️⃣8️⃣ Single vs Composite Index

ধরো:

```
WHERE department = ?
AND city = ?
```

Possible:

Option A

```
CREATE INDEX idx_department
ON employees(department);

CREATE INDEX idx_city
ON employees(city);
```

Option B

```
CREATE INDEX idx_department_city
ON employees(department, city);
```

কোনটা better?

> **Query pattern, data distribution, workload এবং optimizer plan-এর উপর নির্ভর করে।**

শুধু "দুইটা condition আছে, তাই composite index"—এমন fixed rule নেই।

1️⃣9️⃣ Index + ORDER BY

ধরো:

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Frequently used workload হলে:

```
CREATE INDEX idx_salary
ON employees(salary);
```

optimizer পরিস্থিতি অনুযায়ী index ব্যবহার করতে পারে।

2️⃣0️⃣ Index + WHERE + ORDER BY

আরও interesting:

```
SELECT *
FROM employees
```

```
WHERE department = 'IT'
ORDER BY salary DESC;
```

Potential index:

```
CREATE INDEX idx_department_salary
ON employees(department, salary);
```

এখানে index query-এর filtering এবং ordering pattern-এর সাথে match করতে পারে।

তবে actual execution plan অবশ্যই check করবে।

2️⃣1️⃣ Index-এর Cost আছে ⚠️

অনেকে ভাবে:

> "অনেক index দিলে database আরো fast হবে।"

❌ ভুল।

Index-এর সুবিধা আছে, কিন্তু cost-ও আছে।

কারণ:

```
INSERT
UPDATE
DELETE
```

করলে index-ও maintain/update করতে হয়।

তাই:

```
More Indexes
     ↓
Faster Reads
     +
More Storage
     +
More Write Maintenance
```

2️⃣2️⃣ INSERT কেন slower হতে পারে?

ধরো table-এ:

```
1 million rows
```

এবং:

```
5 indexes
```

আছে।

নতুন row insert হলে শুধু table-এ data রাখলেই হবে না।

Relevant index structures-ও update করতে হবে।

তাই অনেক unnecessary index থাকলে write performance খারাপ হতে পারে।

2️⃣3️⃣ UPDATE-এর ক্ষেত্রেও

যদি indexed column update করো:

```sql
UPDATE employees
SET department = 'HR'
WHERE employee_id = 10;
```

তাহলে `department` indexed হলে index structure-ও maintain করতে হবে।

2️⃣4️⃣ DELETE-এর ক্ষেত্রেও

```sql
DELETE FROM employees
WHERE employee_id = 10;
```

Row delete হওয়ার সাথে সাথে relevant indexes থেকেও entry remove/update করতে হবে।

2️⃣5️⃣ কখন Index ব্যবহার করব?

সাধারণত consider করবে যখন column:

1. Frequently searched

```sql
WHERE email = ?
```

2. Frequently joined

```
ON orders.customer_id = customers.customer_id
```

3. Frequently sorted

```
ORDER BY created_at
```

4. Frequently filtered

```
WHERE status = 'active'
```

5. High-cardinality column

যেমন:

```
email
user_id
order_id
```

এগুলোতে অনেক unique/distinct value থাকতে পারে।

2️⃣6️⃣ Low Cardinality Column

ধরো:

```
gender
```

শুধু:

```
Male
Female
```

অথবা:

```
status
```

শুধু:

```
Active
```

```
Inactive
```

এ ধরনের low-cardinality column-এ index সব query-তে খুব effective নাও হতে পারে।

কিন্তু!

এটা absolute rule নয়।

যদি table বিশাল হয় এবং query pattern appropriate হয়, low-cardinality column-এও index useful হতে পারে।

তাই:

> **Cardinality + table size + query pattern + execution plan—সব consider করবে।**

2️⃣7️⃣ B-Tree Index কী?

এখন একটু ভিতরে যাই।

Relational databases-এ common index structure হলো:

> **B-Tree / B+Tree family**

সহজভাবে:

```
            [50]
          /      \
      [20]        [80]
     /   \       /   \
  [10] [30]  [60] [90]
```

Data sorted structure-এ organized থাকে।

তাই database পুরো table scan না করে search space দ্রুত narrow করতে পারে।

2️⃣8️⃣ Binary Search-এর সাথে সম্পর্ক

ধরো sorted data:

```
10
20
30
40
50
```

```
60
70
80
90
```

তুমি `80` খুঁজছো।

Sequentially:

```
10 → 20 → 30 → ...
```

না খুঁজে tree/search structure ব্যবহার করে search space দ্রুত ছোট করা যায়।

Database index-এর internal structure অনেক বেশি sophisticated এবং disk/page oriented, কিন্তু basic intuition হলো:

> **Organized search structure → fewer data pages/entries examined**

2️⃣9️⃣ Clustered vs Secondary Index — Concept

এটা Database Designer হিসেবে জানা ভালো।

Clustered Index

Table data-এর physical/logical organization index order-এর সাথে closely associated।

Secondary Index

Table data-এর আলাদা index structure থাকে, যা মূল data row-তে locate করতে সাহায্য করে।

MySQL InnoDB-তে:

> **Primary key হলো clustered index-এর ভিত্তি।**

আর অন্যান্য indexes সাধারণত secondary indexes।

3️⃣0️⃣ InnoDB-এর Important Concept

MySQL InnoDB table-এ:

```
Primary Key
     ↓
Clustered Index
```

```
      ↓
Actual row data
```

Secondary index-এর leaf entries primary key value-ও carry করে, যাতে corresponding clustered row locate করা যায়।

এ কারণে primary key design গুরুত্বপূর্ণ।

3️⃣1️⃣ Primary Key ছোট রাখা কেন ভালো?

ধরো:

```
PRIMARY KEY = BIGINT
```

এবং table-এর অনেক secondary indexes আছে।

InnoDB secondary index entries-এ primary key value ব্যবহার হওয়ায় খুব বড় primary key হলে index size বাড়তে পারে।

তাই সাধারণভাবে:

> **Primary key ছোট, stable এবং appropriate type-এর রাখা ভালো।**

3️⃣2️⃣ Covering Index কী?

Advanced concept।

ধরো query:

```
SELECT customer_id, order_date
FROM orders
WHERE customer_id = 10;
```

যদি index:

```
CREATE INDEX idx_customer_date
ON orders(customer_id, order_date);
```

থাকে, তাহলে index-এর মধ্যেই query-এর প্রয়োজনীয় columns থাকতে পারে।

এমন situation-কে সাধারণভাবে:

> **Covering Index**

বলা হয়।

তখন database-কে মূল table row-এ ফিরে যেতে নাও হতে পারে।

এতে কিছু query আরও efficient হতে পারে।

3️⃣3️⃣ EXPLAIN — আজকের সবচেয়ে Important Tool 🔥

Index তৈরি করলেই কাজ শেষ নয়।

আমাদের দেখতে হবে:

> Database আসলে index ব্যবহার করছে কি না?

এর জন্য:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

3️⃣4️⃣ EXPLAIN কী করে?

`EXPLAIN` query-এর execution plan সম্পর্কে information দেয়।

অর্থাৎ database query execute করার জন্য কী strategy নিতে পারে তা বুঝতে সাহায্য করে।

3️⃣5️⃣ EXPLAIN-এ Important Columns

MySQL-এ তুমি এগুলো দেখতে পারো:

```
type
possible_keys
key
key_len
rows
Extra
```

সব এখন মুখস্থ করার দরকার নেই।

আজ important:

`possible_keys`

কোন index ব্যবহার করা যেতে পারে।

`key`

Actually কোন index choose করা হয়েছে।

`rows`

Optimizer আনুমানিক কত rows examine করতে হতে পারে।

`type`

Access strategy সম্পর্কে ধারণা দেয়।

3️⃣6️⃣ Example

Index:

```
CREATE INDEX idx_email
ON customers(email);
```

তারপর:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

যদি:

```
possible_keys → idx_email
key           → idx_email
```

দেখো, তাহলে optimizer index ব্যবহার করছে।

3️⃣7️⃣ Full Table Scan

যদি index না থাকে:

```
Table
↓
```

```
Read many/all rows
↓
Check condition
```

এটাকে broadly:

> **Full Table Scan**

বলা হয়।

EXPLAIN-এ MySQL-এ `type = ALL` অনেক সময় full table scan নির্দেশ করে।

কিন্তু `ALL` দেখলেই সবসময় query খারাপ—এমন নয়।

যদি table ছোট হয়, full scan কখনো index lookup-এর চেয়ে efficient হতে পারে।

3️⃣8️⃣ Index Scan সবসময় Fast?

❌ না।

এটা খুব important।

ধরো table:

```
100 rows
```

এবং query:

```
SELECT *
FROM employees;
```

এখানে index ব্যবহার করার কোনো দরকার নেই।

Database সরাসরি table পড়তে পারে।

3️⃣9️⃣ Selectivity

একটি index কতটা useful হতে পারে সেটা বোঝার জন্য **selectivity** গুরুত্বপূর্ণ।

ধরো:

```
1,000,000 rows
```

Email:

```
999,000 different values
```

খুব selective।

কিন্তু:

```
status = active/inactive
```

শুধু 2 values।

কম selective।

তবে আবার বলি:

> Low selectivity ≠ useless index.

Actual workload এবং execution plan দেখতে হবে।

4️⃣0️⃣ Index Design-এর Golden Rule

❌ ভুল approach:

```
"এই table-এ 20টা column আছে,
সব column-এ index বানাই।"
```

✅ ভালো approach:

```
Query Pattern
      ↓
WHERE?
JOIN?
ORDER BY?
GROUP BY?
      ↓
Candidate Index
      ↓
EXPLAIN
      ↓
Measure
      ↓
Keep / Change / Remove
```

4️⃣1️⃣ Real-world Example

ধরো তোমার hotel management system:

```
rooms
---------
room_id
hotel_id
room_type
price
status
```

Frequently query:

```sql
SELECT *
FROM rooms
WHERE hotel_id = 10
  AND status = 'available';
```

Potential index:

```sql
CREATE INDEX idx_rooms_hotel_status
ON rooms(hotel_id, status);
```

তারপর:

```sql
EXPLAIN
SELECT *
FROM rooms
WHERE hotel_id = 10
  AND status = 'available';
```

দিয়ে verify করবে।

4️⃣2️⃣ Database Designer Mindset 🔥

একজন beginner ভাবে:

> "Query কাজ করছে।"

একজন database designer ভাবে:

> "Query 10 rows-এর উপর কাজ করছে, নাকি 10 million rows scan করছে?"

আর production database-এ এই difference বিশাল।

4️⃣3️⃣ Indexing + Your Data Engineer Goal

তুমি যেহেতু Database Designer/Data Engineering-এর দিকে যেতে চাচ্ছো, এই flowটা মনে রাখো:

```
Database Design
      ↓
Tables
      ↓
Relationships
      ↓
Queries
      ↓
Indexes
      ↓
EXPLAIN
      ↓
Optimization
```

এটাই আমাদের পরের class-এর foundation।

4️⃣4️⃣ Common Beginner Mistakes

❌ 1. সব column index করা

Index-এর storage এবং write cost আছে।

❌ 2. Duplicate index

যেমন:

```
INDEX(email)
UNIQUE INDEX(email)
```

একই উদ্দেশ্যে unnecessary duplicate structure তৈরি করা উচিত নয়।

❌ 3. Composite index-এর order ignore করা

```
(department, city)
```

এবং:

```
(city, department)
```

এক নয়।

❌ 4. EXPLAIN না দেখা

Index তৈরি করে ধরে নেওয়া:

> "অবশ্যই database এটা ব্যবহার করছে।"

ভুল।

❌ 5. Small table-এ blindly index

Small table-এর full scan কখনো faster হতে পারে।

❌ 6. Index মানেই সব query fast

না।

> **Correct index + correct query pattern + good execution plan = performance improvement**

🧠 Class 50 Golden Cheat Sheet

```
INDEX
↓
Fast data lookup

CREATE INDEX
↓
Create normal index

UNIQUE INDEX
↓
```

```
Uniqueness + index

COMPOSITE INDEX
↓
Multiple columns

PRIMARY KEY
↓
Indexed key; InnoDB uses it as clustered index

NEW/OLD
↓
Trigger concept

EXPLAIN
↓
Check query execution plan
```

Most Important:

```
More Index ≠ Always Better

Good Index
=
Query Pattern
+
Data Distribution
+
Workload
+
EXPLAIN
```

📝 Class 50 Assignment

ধরো:

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(255),
    city VARCHAR(50),
    status VARCHAR(20),
    created_at DATE
);
```

Q1

`email` -এর উপর একটি index তৈরি করো।

Q2

`email` unique করার জন্য Unique Index তৈরি করো।

Q3

`city` এবং `status` নিয়ে একটি composite index তৈরি করো।

Q4

নিচের query-এর জন্য suitable index design করো:

```
SELECT *
FROM customers
WHERE city = 'Dhaka'
AND status = 'active';
```

Q5 ⭐

এই query-এর execution plan দেখাও:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

এবং explain করো:

```
possible_keys
key
rows
type
```

Q6 ⭐

কেন excessive indexes `INSERT`, `UPDATE`, `DELETE` slow করতে পারে?

🧪 Class 50 MCQ

1. Index-এর প্রধান উদ্দেশ্য কী?

A. Data delete করা
B. Query/data retrieval দ্রুত করতে সাহায্য করা
C. Table তৈরি করা
D. Backup নেওয়া

2. Index তৈরি করতে কোন command?

A. `MAKE INDEX`
B. `CREATE INDEX`
C. `BUILD INDEX ONLY`
D. `NEW INDEX`

3. Composite Index কী?

A. Multiple tables
B. Multiple databases
C. Multiple columns-এর index
D. Multiple users

4. `(department, city)` composite index-এর ক্ষেত্রে কোন column first?

A. city
B. department
C. দুটোই একই
D. কোনোটিই নয়

5. Query execution plan দেখতে?

A. `PLAN`
B. `EXPLAIN`
C. `SHOW QUERY`
D. `CHECK QUERY`

6. কোন operation-এর উপর excessive indexes performance cost তৈরি করতে পারে?

A. INSERT
B. UPDATE
C. DELETE
D. All of the above

7. `INSERT` trigger-এ কোনটি available?

A. OLD
B. NEW
C. OLD + NEW
D. None

8. InnoDB-তে primary key-এর সাথে কোন concept strongly associated?

A. Clustered index
B. Trigger
C. View
D. CTE

9. কোনটি true?

A. More indexes always means faster database
B. Index has no storage cost
C. Index can improve reads but adds maintenance cost
D. Index only works for DELETE

10. `WHERE city = 'Dhaka'` query-এর জন্য `(department, city)` index সম্পর্কে কোন statement সবচেয়ে সঠিক?

A. অবশ্যই পুরো index ব্যবহার করবে
B. সাধারণত leading column না থাকায় এর সুবিধা সীমিত হতে পারে
C. এটি city index-এর সমান
D. এটি অবশ্যই query-কে slower করবে

🎯 আজকের ৫টা Golden Line

```
1. Index = Faster data retrieval-এর জন্য data structure

2. Primary Key সাধারণত indexed

3. Composite Index-এ column order গুরুত্বপূর্ণ

4. More Index ≠ Always Better

5. EXPLAIN = Query কীভাবে execute হচ্ছে তা বোঝার প্রধান tool
```

Next → Class 51: SQL Query Optimization + `EXPLAIN` Deep Dive — `type`, `key`, `rows`, `Extra`, Full Table Scan, Index Scan, Composite Index optimization, slow query diagnosis এবং practical performance tuning।

📘 Class 51 — SQL Query Optimization + EXPLAIN Deep Dive

আজকের ক্লাসটা **Database Designer / Data Engineer** হওয়ার জন্য খুব important।

Class 50-এ আমরা শিখেছি **Index**।
আজ শিখব:

> **Query slow কেন? কোন index ব্যবহার করছে? Database কীভাবে query execute করছে? কীভাবে query fast করব?**

---

1. Query Optimization কী?

ধরো তোমার database-এ **10 million rows** আছে।

তুমি লিখলে:

```sql
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

Database-এর সামনে দুইটা রাস্তা থাকতে পারে:

Option A — Full Table Scan

```
Row 1 → check
Row 2 → check
Row 3 → check
...
Row 10,000,000 → check
```

খুব slow হতে পারে।

Option B — Index ব্যবহার

যদি `email` -এ index থাকে:

```sql
CREATE INDEX idx_customer_email
ON customers(email);
```

তাহলে database index ব্যবহার করে দ্রুত matching row খুঁজতে পারে।

```
Index
  ↓
rahim@gmail.com
  ↓
Customer row
```

এটাই Query Optimization-এর basic idea।

2. EXPLAIN কী?

SQL query চালানোর আগে আমরা database-কে জিজ্ঞেস করতে পারি:

> "এই query তুমি কীভাবে execute করবে?"

এর জন্য:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

এটা query-এর **execution plan** দেখাবে।

---

3. EXPLAIN কেন গুরুত্বপূর্ণ?

ধরো query:

```
SELECT *
FROM orders
WHERE customer_id = 10;
```

তুমি ভাবতে পারো index ব্যবহার হচ্ছে।

কিন্তু database হয়তো বলছে:

```
Full Table Scan
```

অর্থাৎ index থাকলেও optimizer সেটি ব্যবহার না-ও করতে পারে।

তাই:

```
Query লিখলাম
      ↓
EXPLAIN
      ↓
Execution Plan দেখলাম
      ↓
Problem identify
      ↓
Index / Query optimize
```

4. EXPLAIN-এর গুরুত্বপূর্ণ Columns

MySQL-এ `EXPLAIN` করলে অনেক column দেখা যায়।

সবচেয়ে important:

Column	Meaning
id	Query block identifier
select_type	Query type
table	কোন table access করছে
partitions	Partition information
type	কী ধরনের access হচ্ছে
possible_keys	কোন index ব্যবহার করা সম্ভব
key	কোন index actually ব্যবহার করছে
key_len	index-এর কত অংশ ব্যবহার করছে
ref	কোন value/column-এর সাথে match করছে
rows	আনুমানিক কত row examine করবে
filtered	কত % row filter pass করবে
Extra	অতিরিক্ত execution information

আজ সবচেয়ে important:

> `type`, `possible_keys`, `key`, `rows`, `Extra`

---

5. `type` — সবচেয়ে গুরুত্বপূর্ণ

`type` দেখে আমরা বুঝতে পারি database কী ধরনের access করছে।

সাধারণভাবে ভালো থেকে খারাপের দিকে:

```
system
const
eq_ref
ref
range
index
ALL
```

সবসময় এই ranking blindly apply করবে না, কিন্তু basic diagnosis-এর জন্য useful।

---

6. `type = ALL`

এটা সাধারণত:

> **Full Table Scan**

উদাহরণ:

```
SELECT *
FROM customers
WHERE city = 'Dhaka';
```

যদি `city`-তে index না থাকে, database অনেক row scan করতে পারে।

EXPLAIN:

```
type = ALL
```

মানে:

```
পুরো table scan করার সম্ভাবনা
```

⚠️ তবে ছোট table হলে `ALL` দেখালেই panic করার দরকার নেই।

যেমন:

```
customers = 50 rows
```

৫০টা row scan করা অনেক সময় index lookup-এর চেয়েও practical হতে পারে।

---

7. `type = const`

খুব efficient access।

সাধারণত primary key বা unique indexed value দিয়ে একটি row খুঁজলে।

```
SELECT *
FROM customers
WHERE customer_id = 10;
```

যদি:

```
customer_id PRIMARY KEY
```

হয়, optimizer খুব দ্রুত row locate করতে পারে।

Concept:

```
Primary Key
     ↓
customer_id = 10
     ↓
One row
```

---

8. `type = eq_ref`

এটা বিশেষ করে JOIN-এর ক্ষেত্রে দেখা যায়।

উদাহরণ:

```
SELECT *
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id;
```

যদি `customers.customer_id` primary key হয়, প্রতিটি order-এর জন্য একটি নির্দিষ্ট customer row পাওয়া যায়।

এটা সাধারণত efficient JOIN access।

---

9. `type = ref`

Non-unique index দিয়ে matching rows খোঁজার ক্ষেত্রে `ref` দেখা যায়।

ধরো:

```
CREATE INDEX idx_customer_city
ON customers(city);
```

তারপর:

```sql
SELECT *
FROM customers
WHERE city = 'Dhaka';
```

একাধিক customer Dhaka-তে থাকতে পারে।

তাই:

```
city index
   ↓
Dhaka
   ↓
multiple rows
```

এখানে `ref` দেখা যেতে পারে।

---

10. `type = range`

Range condition-এর জন্য।

যেমন:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2026-02-01';
```

অথবা:

```sql
WHERE price BETWEEN 1000 AND 5000
```

Index থাকলে:

```
range scan
```

হতে পারে।

---

11. `possible_keys`

এটা বলে:

> এই query-এর জন্য কোন কোন index ব্যবহার করা সম্ভব?

উদাহরণ:

```
possible_keys:
idx_customer_email
```

কিন্তু গুরুত্বপূর্ণ বিষয়:

possible_keys মানেই index ব্যবহার করেছে না।

---

12. `key`

এটা আরও important।

`key` বলে:

> Database শেষ পর্যন্ত কোন index ব্যবহার করেছে?

ধরো:

```
possible_keys = idx_email
key           = idx_email
```

তাহলে index actually ব্যবহার করেছে।

কিন্তু:

```
possible_keys = idx_email
key           = NULL
```

মানে index available ছিল, কিন্তু optimizer ব্যবহার করেনি।

---

13. `possible_keys` বনাম `key`

মনে রাখো:

```
possible_keys
     ↓
Could use

key
```

```
      ↓
Actually used
```

এটা interview-তে খুব useful।

---

14. `rows`

`rows` হলো optimizer-এর estimated number of rows যা examine করতে হতে পারে।

ধরো:

```
rows = 1000000
```

তাহলে query potentially অনেক rows examine করছে।

আর:

```
rows = 1
```

সাধারণত ভালো sign।

⚠️ এটা exact result count নয়।

এটা **optimizer-এর estimate**।

---

15. `Extra`

`Extra` অনেক useful information দেয়।

Common values:

```
Using where
Using index
Using temporary
Using filesort
```

---

16. `Using where`

Example:

```sql
SELECT *
FROM customers
WHERE city = 'Dhaka';
```

Database additional filtering করছে।

এটা নিজে থেকেই bad না।

---

17. `Using index`

এটা অনেক সময় ভালো sign।

মানে query-এর প্রয়োজনীয় data index থেকেই পাওয়া যাচ্ছে।

এটাকে **covering index** situation-এর সাথে সম্পর্কিত করা যায়।

Example:

```sql
CREATE INDEX idx_city_name
ON customers(city, customer_name);
```

Query:

```sql
SELECT customer_name
FROM customers
WHERE city = 'Dhaka';
```

Index-এর মধ্যেই দরকারি information থাকতে পারে।

---

18. `Using filesort`

নাম দেখে ভয় পাওয়ার দরকার নেই।

এটা সাধারণত বলে database-কে sorting-এর জন্য additional work করতে হচ্ছে।

Example:

```sql
SELECT *
FROM employees
ORDER BY salary DESC;
```

যদি appropriate index না থাকে, sorting করতে extra work হতে পারে।

Large dataset-এ এটি expensive হতে পারে।

---

19. `Using temporary`

কিছু query intermediate temporary structure ব্যবহার করতে পারে।

বিশেষ করে complex:

```
GROUP BY
ORDER BY
DISTINCT
```

এর combination-এ দেখা যেতে পারে।

Example:

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
ORDER BY COUNT(*) DESC;
```

`Using temporary` দেখালেই query অবশ্যই bad—এমন না।

---

20. সবচেয়ে common optimization mistake

ধরো:

```
SELECT *
FROM customers
WHERE YEAR(created_at) = 2026;
```

এখানে `created_at` indexed হলেও function ব্যবহার করার কারণে index-এর সুবিধা কমে যেতে পারে।

Better:

```
SELECT *
FROM customers
WHERE created_at >= '2026-01-01'
  AND created_at < '2027-01-01';
```

এটাকে বলা হয় **Sargability**-এর basic idea।

21. Sargability কী?

সহজভাবে:

> এমনভাবে condition লেখা যাতে database index efficiently ব্যবহার করতে পারে।

Less index-friendly

```
WHERE YEAR(order_date) = 2026
```

More index-friendly

```
WHERE order_date >= '2026-01-01'
  AND order_date < '2027-01-01'
```

Golden Rule:

> **Indexed column-এর উপর unnecessary function প্রয়োগ না করাই ভালো।**

---

22. আরেকটি common mistake — `SELECT *`

ধরো:

```
SELECT *
FROM customers
WHERE email = 'a@gmail.com';
```

কিন্তু তোমার দরকার শুধু:

```
customer_id
customer_name
```

তাহলে:

```
SELECT customer_id, customer_name
FROM customers
WHERE email = 'a@gmail.com';
```

Better।

কারণ:

```
SELECT *
    ↓
unnecessary columns
    ↓
more data transfer
    ↓
more I/O
```

বিশেষ করে large table এবং application/API response-এর ক্ষেত্রে গুরুত্বপূর্ণ।

---

23. WHERE দিয়ে আগে Filter করা

ধরো:

```sql
SELECT
    c.city,
    SUM(o.quantity * p.price)
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY c.city;
```

যদি তোমার দরকার শুধু 2026 সালের orders:

```sql
SELECT
    c.city,
    SUM(o.quantity * p.price)
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
WHERE o.order_date >= '2026-01-01'
  AND o.order_date < '2027-01-01'
GROUP BY c.city;
```

এতে unnecessary rows processing কমতে পারে।

---

24. JOIN Optimization

ধরো:

```
SELECT *
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id;
```

Important columns:

```
orders.customer_id
customers.customer_id
```

সঠিক indexing JOIN দ্রুত করতে সাহায্য করতে পারে।

বিশেষ করে:

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

এবং `customers.customer_id` যদি primary key হয়, সেখানে already index থাকে।

---

25. Composite Index Optimization

আগের class-এ শিখেছিলাম:

```
CREATE INDEX idx_customer_date
ON orders(customer_id, order_date);
```

এখানে:

```
1st → customer_id
2nd → order_date
```

এখন:

Query A

```
WHERE customer_id = 10
```

✅ ভালো candidate

Query B

```
WHERE customer_id = 10
AND order_date >= '2026-01-01'
```

✅ আরও ভালো use case

Query C

```
WHERE order_date >= '2026-01-01'
```

⚠️ এই index-এর জন্য সাধারণত ideal use case নয়, কারণ প্রথম column `customer_id` বাদ গেছে।

এটাই **Leftmost Prefix Rule**-এর basic idea।

---

26. ORDER BY Optimization

Query:

```
SELECT *
FROM orders
ORDER BY order_date DESC
LIMIT 10;
```

যদি appropriate index থাকে:

```
CREATE INDEX idx_order_date
ON orders(order_date);
```

তাহলে database sorting work কমাতে পারে।

এখানে:

```
LIMIT 10
```

খুব useful হতে পারে।

কারণ তোমার দরকার মাত্র ১০টা row।

---

27. LIMIT Optimization

ধরো:

```sql
SELECT *
FROM products
ORDER BY price DESC;
```

তুমি সব products চাও না।

শুধু top 10:

```sql
SELECT *
FROM products
ORDER BY price DESC
LIMIT 10;
```

এতে application-এর জন্য unnecessary rows return করা কমে।

---

28. Query Optimization-এর একটা Real Example

ধরো table:

```
orders
```

Rows:

```
10,000,000
```

Query:

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

Problem

`YEAR()` function ব্যবহার করছি।

Better

```sql
SELECT order_id, customer_id, order_date, quantity
FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2027-01-01';
```

আর index:

```
CREATE INDEX idx_orders_date
ON orders(order_date);
```

তারপর:

```
EXPLAIN
SELECT order_id, customer_id, order_date, quantity
FROM orders
WHERE order_date >= '2026-01-01'
  AND order_date < '2027-01-01';
```

তারপর execution plan দেখে সিদ্ধান্ত নেব।

---

29. Query Optimization Workflow

একজন Database Engineer হিসেবে workflow হবে:

```
Slow Query
    ↓
EXPLAIN
    ↓
Check type
    ↓
Check possible_keys
    ↓
Check key
    ↓
Check rows
    ↓
Check Extra
    ↓
Identify bottleneck
    ↓
Optimize Query / Index
    ↓
EXPLAIN Again
    ↓
Compare
```

এটা খুব important workflow।

---

30. শুধু Index দিলেই হবে?

❌ না।

ধরো:

```
CREATE INDEX idx_name
ON customers(name);
```

তারপর:

```
WHERE age = 25
```

এই index কোনো সাহায্য করবে না।

কারণ query condition:

```
age
```

কিন্তু index:

```
name
```

---

31. বেশি Index দিলেই Query Fast?

❌ না।

ধরো:

```
10টা index
20টা index
30টা index
```

প্রতিটা index-এর:

```
Storage cost
+
INSERT cost
+
UPDATE cost
+
DELETE maintenance
```

আছে।

তাই:

> **Index strategically তৈরি করতে হয়।**

---

32. Query Optimization-এর Golden Rules ⭐

Rule 1

```
EXPLAIN first
```

Guess করো না।

Rule 2

```
possible_keys ≠ actual key
```

`key` দেখো।

Rule 3

```
rows কমানো important
```

Rule 4

Unnecessary:

```
SELECT *
```

avoid করো।

Rule 5

Indexed column-এর উপর unnecessary function avoid করো।

Rule 6

Composite index-এর column order গুরুত্বপূর্ণ।

Rule 7

সব query-তে index দরকার নেই।

Rule 8

Optimization করার পরে আবার:

```
EXPLAIN
```

করো।

🧠 Class 51 Cheat Sheet

```
EXPLAIN
   ↓
Execution Plan

type
   ↓
Access method

possible_keys
   ↓
Possible indexes

key
   ↓
Actually used index

rows
   ↓
Estimated rows examined

Extra
   ↓
Additional execution information
```

Important `type`

```
const
eq_ref
ref
range
index
ALL
```

বিশেষভাবে মনে রাখবে:

```
ALL = Full Table Scan
range = Range scan
ref = Indexed lookup with multiple possible matches
const = Very efficient single-row lookup
```

🎯 Practice Assignment

ধরো:

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2)
);
```

Task 1

`customer_id` দিয়ে search করার জন্য index তৈরি করো।

Task 2

`order_date` দিয়ে search করার জন্য index তৈরি করো।

Task 3

এই query-এর EXPLAIN লিখো:

```sql
SELECT *
FROM orders
WHERE customer_id = 100;
```

Task 4

এই query optimize করো:

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

Task 5

এই query-এর জন্য suitable composite index চিন্তা করো:

```sql
SELECT *
FROM orders
WHERE customer_id = 100
AND order_date >= '2026-01-01'
AND order_date < '2026-02-01';
```

📝 MCQ

1. `EXPLAIN` কী দেখায়?

A. Table data
B. Query execution plan
C. Database password
D. Backup

2. `key` কী নির্দেশ করে?

A. Possible index
B. Actually used index
C. Primary key only
D. Foreign key

3. `type = ALL` সাধারণত কী বোঝায়?

A. Index lookup
B. Full table scan
C. Unique lookup
D. Range lookup

4. কোনটি সাধারণত index-friendly?

A.

```sql
WHERE YEAR(order_date) = 2026
```

B.

```sql
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
```

5. Composite index:

```sql
(customer_id, order_date)
```

এর ক্ষেত্রে কোন query সবচেয়ে natural use case?

A.

```
WHERE order_date = '2026-01-01'
```

B.

```
WHERE customer_id = 10
AND order_date = '2026-01-01'
```

C. কোনোটাই না

⭐ আজকের সবচেয়ে গুরুত্বপূর্ণ concept

```
Index তৈরি করা
      ≠
Query optimized হওয়া

EXPLAIN
      ↓
Database কী করছে দেখো
      ↓
Problem identify করো
      ↓
Query + Index optimize করো
      ↓
EXPLAIN আবার চালাও
```

Class 52 হবে → Database Normalization (1NF, 2NF, 3NF, BCNF) — Database Designer হওয়ার জন্য খুব গুরুত্বপূর্ণ ক্লাস।

📘 Class 52 — Database Normalization

আজকের ক্লাসটা **Database Designer** হওয়ার জন্য সবচেয়ে গুরুত্বপূর্ণ topics-এর একটি।

আমরা শিখব:

- Normalization কী?
- কেন দরকার?
- Data redundancy

- Update/Insert/Delete anomaly
- Functional Dependency
- 1NF
- 2NF
- 3NF
- BCNF
- Normalization কীভাবে step-by-step করতে হয়
- Real-world example
- Common mistakes
- Assignment + MCQ

---

1. Normalization কী?

সহজ ভাষায়:

> **Database-এর data এমনভাবে organize করা যাতে unnecessary duplication কমে এবং data consistency বজায় থাকে—এটাই Normalization।**

ধরো আমাদের table:

StudentID	StudentName	Course	Teacher	TeacherPhone
1	Atiar	Database	Rahim	01711
2	Karim	Database	Rahim	01711
3	Nila	SQL	Hasan	01822

এখানে:

```
Database → Rahim → 01711
```

বারবার duplicate হচ্ছে।

আর:

```
SQL → Hasan → 01822
```

এটাও repeat হতে পারে।

এটাই **data redundancy**।

---

2. Data Redundancy কী?

একই information unnecessarily বারবার store হওয়া।

Example:

```
Student 1 → Database → Rahim → 01711
Student 2 → Database → Rahim → 01711
Student 3 → Database → Rahim → 01711
Student 4 → Database → Rahim → 01711
```

`Rahim` এবং `01711` বারবার এসেছে।

Problem:

```
More duplicate data
        ↓
More storage
        ↓
More update problems
        ↓
Inconsistency
```

---

3. Normalization কেন দরকার?

মূল উদ্দেশ্য:

1. Redundancy কমানো

2. Data consistency বজায় রাখা

3. Update anomaly কমানো

4. Insert anomaly কমানো

5. Delete anomaly কমানো

---

4. Anomaly কী?

Database-এ ভুল/অস্বাভাবিক data operation-এর সমস্যা।

তিনটি famous anomaly:

```
1. Update Anomaly
2. Insert Anomaly
3. Delete Anomaly
```

এগুলো exam-এ খুব common।

---

5. Update Anomaly

ধরো:

Student	Course	Teacher	Phone
Atiar	DBMS	Rahim	01711
Karim	DBMS	Rahim	01711
Nila	DBMS	Rahim	01711

Rahim-এর phone number পরিবর্তন হলো:

```
01711 → 01999
```

তাহলে ৩টি row update করতে হবে।

যদি ভুল করে:

```
Row 1 → 01999
Row 2 → 01999
Row 3 → 01711
```

হয়ে যায়?

একই teacher-এর দুইটা phone number হয়ে গেল!

এটাই **Update Anomaly**।

---

6. Insert Anomaly

ধরো Course table-এর তথ্য:

Student	Course	Teacher
Atiar	DBMS	Rahim

Student	Course	Teacher
Karim	DBMS	Rahim

এখন নতুন course:

```
Python
Teacher = Hasan
```

কিন্তু এখনো কোনো student Python course নেয়নি।

আমাদের table-এ Student column required হলে কীভাবে শুধু:

```
Python → Hasan
```

insert করব?

পারছি না।

অর্থাৎ:

> নতুন information insert করতে অন্য unrelated information প্রয়োজন হচ্ছে।

এটাই **Insert Anomaly**।

---

7. Delete Anomaly

ধরো:

Student	Course	Teacher
Atiar	DBMS	Rahim
Karim	DBMS	Rahim
Nila	Python	Hasan

এখন Nila Python course থেকে চলে গেল।

আমরা Nila-এর row delete করলাম।

```
Nila → Python → Hasan
```

তাহলে Python course সম্পর্কে একমাত্র information-ও চলে গেল।

অর্থাৎ:

> একটি unrelated record delete করার কারণে দরকারি information হারিয়ে গেল।

এটাই **Delete Anomaly**।

---

8. Normalization-এর Main Idea

আমরা বড় table:

```
One giant table
```

কে logically ছোট ছোট table-এ ভাগ করি।

Example:

Students

student_id	student_name
1	Atiar
2	Karim

Courses

course_id	course_name	teacher_id
101	DBMS	1
102	Python	2

Teachers

teacher_id	teacher_name	phone
1	Rahim	01711
2	Hasan	01822

Enrollment

student_id	course_id
1	101
2	101

student_id	course_id
1	102

এখন duplicate কমেছে।

---

9. Normal Forms

Normalization-এর বিভিন্ন level আছে:

```
UNF
 ↓
1NF
 ↓
2NF
 ↓
3NF
 ↓
BCNF
```

সবচেয়ে commonly discussed:

> **1NF → 2NF → 3NF**

---

10. 1NF — First Normal Form

1NF-এর basic rule:

> **প্রতিটি cell-এ atomic/single value থাকতে হবে।**

অর্থাৎ এক cell-এ multiple values রাখা যাবে না।

❌ Not 1NF

StudentID	Name	Phone
1	Atiar	01711, 01822

একটি cell-এ:

```
01711, 01822
```

দুইটা value।

এটা atomic না।

✅ 1NF

StudentID	Name	Phone
1	Atiar	01711
1	Atiar	01822

প্রতিটি cell-এ single value।

---

11. আরেকটি 1NF Example

❌

Student	Courses
Atiar	DBMS, SQL, Python

Courses cell-এ multiple values।

1NF করতে:

Student	Course
Atiar	DBMS
Atiar	SQL
Atiar	Python

⭐ 1NF Golden Rule

```
One Cell
   ↓
```

```
One Atomic Value
```

---

12. কিন্তু 1NF-এর সমস্যা কী?

1NF করলাম:

StudentID	CourseID	StudentName	CourseName	Teacher
1	101	Atiar	DBMS	Rahim
1	102	Atiar	SQL	Hasan
2	101	Karim	DBMS	Rahim

এখন:

```
Atiar → Atiar
DBMS → DBMS
Rahim → Rahim
```

আবার repeat হচ্ছে।

এখন আসবে **2NF**।

---

13. Functional Dependency

2NF বোঝার আগে Functional Dependency বুঝতে হবে।

ধরো:

```
StudentID → StudentName
```

এর অর্থ:

> StudentID জানা থাকলে StudentName uniquely determine করা যায়।

Example:

```
StudentID = 1
      ↓
StudentName = Atiar
```

তাই:

```
StudentID → StudentName
```

---

14. Functional Dependency Example

ধরো:

StudentID	StudentName
1	Atiar
2	Karim

এখানে:

```
StudentID → StudentName
```

কিন্তু:

```
StudentName → StudentID
```

সবসময় valid নাও হতে পারে।

কারণ দুইজন student-এর নাম একই হতে পারে।

---

15. Determinant কী?

Functional Dependency:

```
A → B
```

এখানে:

> `A` হলো **determinant**।

Example:

```
StudentID → StudentName
```

এখানে:

```
StudentID = Determinant
```

---

16. 2NF — Second Normal Form

2NF-এর জন্য table-কে:

Condition 1

1NF হতে হবে।

Condition 2

Partial Dependency থাকতে পারবে না।

এখন প্রশ্ন:

> Partial Dependency কী?

---

17. Composite Primary Key

ধরো Enrollment table:

StudentID	CourseID	StudentName	CourseName	Grade
1	101	Atiar	DBMS	A
1	102	Atiar	SQL	B
2	101	Karim	DBMS	A

Primary key:

```
(StudentID, CourseID)
```

কারণ:

```
StudentID alone
```

একটা enrollment uniquely identify করতে পারে না।

এবং:

```
CourseID alone
```

এটাও পারে না।

কিন্তু:

```
StudentID + CourseID
```

একটি enrollment uniquely identify করতে পারে।

---

18. Partial Dependency

এখন দেখো:

```
(StudentID, CourseID) → Grade
```

ঠিক আছে।

কিন্তু:

```
StudentID → StudentName
```

এখানে `StudentName` পুরো composite key-এর উপর depend করছে না।

শুধু:

```
StudentID
```

এর উপর depend করছে।

এটাই:

> **Partial Dependency**

---

19. 2NF Violation

আমাদের table:

```
(StudentID, CourseID, StudentName, CourseName, Grade)
```

Dependencies:

```
StudentID → StudentName

CourseID → CourseName

(StudentID, CourseID) → Grade
```

তাই:

```
StudentName
CourseName
```

partial dependency তৈরি করছে।

❌ Table 2NF-এ নেই।

---

20. 2NF করতে কী করব?

Table ভেঙে দেব।

Students

StudentID	StudentName
1	Atiar
2	Karim

Courses

CourseID	CourseName
101	DBMS
102	SQL

Enrollment

StudentID	CourseID	Grade
1	101	A
1	102	B

StudentID	CourseID	Grade
2	101	A

এখন:

```
StudentID → StudentName
CourseID → CourseName
(StudentID, CourseID) → Grade
```

Partial dependency নেই।

✅ 2NF।

⭐ 2NF Golden Rule

> **1NF + No Partial Dependency**

আরেকটা important shortcut:

> **2NF problem সাধারণত composite primary key থাকলেই দেখা যায়।**

যদি primary key single column হয়, partial dependency সাধারণত থাকে না।

---

21. 3NF — Third Normal Form

3NF-এর জন্য:

Condition 1

Table অবশ্যই 2NF হতে হবে।

Condition 2

Transitive Dependency থাকতে পারবে না।

এখন Transitive Dependency কী?

---

22. Transitive Dependency

ধরো:

```
StudentID → DepartmentID
DepartmentID → DepartmentName
```

তাহলে indirectly:

```
StudentID → DepartmentName
```

হচ্ছে।

অর্থাৎ:

```
StudentID
    ↓
DepartmentID
    ↓
DepartmentName
```

এটাই **Transitive Dependency**।

---

23. 3NF Example

ধরো:

StudentID	StudentName	DepartmentID	DepartmentName
1	Atiar	10	CSE
2	Karim	10	CSE
3	Nila	20	EEE

Dependencies:

```
StudentID → StudentName
StudentID → DepartmentID

DepartmentID → DepartmentName
```

তাই:

```
StudentID → DepartmentID → DepartmentName
```

এখানে `DepartmentName` student-এর উপর directly depend করছে না।

এটা DepartmentID-এর উপর depend করছে।

❌ Transitive dependency।

---

24. 3NF করতে কী করব?

দুইটি table:

Students

StudentID	StudentName	DepartmentID
1	Atiar	10
2	Karim	10
3	Nila	20

Departments

DepartmentID	DepartmentName
10	CSE
20	EEE

এখন:

```
DepartmentID → DepartmentName
```

Department information আলাদা table-এ।

✅ 3NF।

⭐ 3NF Golden Rule

```
2NF
 +
No Transitive Dependency
 =
3NF
```

---

25. 1NF vs 2NF vs 3NF

Normal Form	Main Problem
1NF	Multiple/non-atomic values
2NF	Partial dependency
3NF	Transitive dependency

Shortcut:

```
1NF → Atomic

2NF → No Partial Dependency

3NF → No Transitive Dependency
```

এটা অবশ্যই মুখস্থ রাখবে। ⭐

---

26. Complete Example

ধরো initial table:

StudentID	StudentName	CourseID	CourseName	TeacherID	TeacherName
1	Atiar	101	DBMS	T1	Rahim
1	Atiar	102	SQL	T2	Hasan
2	Karim	101	DBMS	T1	Rahim

Primary key:

```
(StudentID, CourseID)
```

Dependencies:

```
StudentID → StudentName

CourseID → CourseName
CourseID → TeacherID

TeacherID → TeacherName
```

```
(StudentID, CourseID) → Enrollment information
```

Step 1 — 1NF

সব cell atomic।

তাই:

✅ 1NF

Step 2 — 2NF

Problem:

```
StudentID → StudentName
```

এবং:

```
CourseID → CourseName
CourseID → TeacherID
```

এগুলো composite key-এর অংশের উপর depend করছে।

তাই table split:

Students

```
StudentID
StudentName
```

Courses

```
CourseID
CourseName
TeacherID
```

Enrollment

```
StudentID
CourseID
```

Step 3 — 3NF

Courses table:

```
CourseID → TeacherID

TeacherID → TeacherName
```

তাই:

```
CourseID → TeacherID → TeacherName
```

Transitive dependency আছে।

তাই:

Courses

CourseID	CourseName	TeacherID

Teachers

TeacherID	TeacherName

Final structure:

```
Students
   ↓
Enrollment
   ↓
Courses
   ↓
Teachers
```

এটাই normalized relational design-এর basic pattern।

---

27. Normalization এবং JOIN

অনেকে ভাবে:

> "Table ভেঙে ফেললে তো অনেক JOIN লাগবে!"

হ্যাঁ, JOIN লাগতে পারে।

কিন্তু normalization-এর লক্ষ্য:

```
Data integrity
+
Less redundancy
+
Better maintainability
```

তারপর প্রয়োজন অনুযায়ী JOIN করা হয়।

Example:

```sql
SELECT
    s.student_name,
    c.course_name,
    t.teacher_name
FROM enrollment e
JOIN students s
    ON e.student_id = s.student_id
JOIN courses c
    ON e.course_id = c.course_id
JOIN teachers t
    ON c.teacher_id = t.teacher_id;
```

---

28. Normalization কি সবসময় যত বেশি সম্ভব করা উচিত?

❌ না।

Real-world database design-এ কখনো কখনো **denormalization** করা হয়।

কারণ অতিরিক্ত normalization করলে:

```
Too many tables
      ↓
Too many JOINs
      ↓
Complex queries
      ↓
Potential performance cost
```

তাই practical database design-এ balance দরকার।

29. Normalization vs Denormalization

Normalization	Denormalization
Redundancy কমায়	কিছু redundancy intentionally রাখে
Data integrity ভালো	Read performance improve করতে পারে
More tables হতে পারে	Fewer tables হতে পারে
JOIN বেশি হতে পারে	JOIN কম হতে পারে
OLTP systems-এ common	Analytics/read-heavy systems-এ useful হতে পারে

---

30. BCNF — Boyce-Codd Normal Form

3NF-এর থেকেও stronger normal form হলো:

> **BCNF**

Basic rule:

> প্রতিটি non-trivial functional dependency `X → Y`-এর ক্ষেত্রে `X` অবশ্যই a **superkey** হতে হবে।

সহজভাবে:

```
Every determinant
      ↓
Must be a candidate/super key
```

BCNF একটু advanced topic।

Exam-এর জন্য basic definition মনে রাখো।

🧠 Normalization Master Cheat Sheet

```
Normalization
      ↓
Reduce redundancy
      ↓
Improve consistency
      ↓
Avoid anomalies
```

Anomalies

```
Update
Insert
Delete
```

Normal Forms

```
1NF
 ↓
Atomic values

2NF
 ↓
1NF + No Partial Dependency

3NF
 ↓
2NF + No Transitive Dependency

BCNF
 ↓
Every determinant is a superkey
```

⭐ সবচেয়ে গুরুত্বপূর্ণ Example

মনে রাখো:

```
StudentID → StudentName
```

এটা Functional Dependency।

```
(StudentID, CourseID) → Grade
```

Composite key-এর উপর full dependency।

```
StudentID → StudentName
```

যদি key হয়:

```
(StudentID, CourseID)
```

তাহলে এটা **Partial Dependency**।

আর:

```
StudentID → DepartmentID
DepartmentID → DepartmentName
```

তাহলে:

```
StudentID → DepartmentName
```

হলো **Transitive Dependency**।

📝 Class 52 Assignment

নিচের table দেখো:

OrderID	ProductID	ProductName	CustomerID	CustomerName	CustomerCity
1	P1	Laptop	C1	Atiar	Dhaka
2	P2	Mouse	C1	Atiar	Dhaka
3	P1	Laptop	C2	Karim	Chittagong

তোমার কাজ:

1. এখানে কোন কোন data redundant?

2. Primary key কী হতে পারে?

3. Functional dependencies লিখো।

4. এখানে Partial Dependency আছে কি?

5. Transitive Dependency আছে কি?

6. Table-টি 3NF পর্যন্ত normalize করো।

7. Final tables-এর মধ্যে Primary Key এবং Foreign Key identify করো।

🎯 MCQ

1. Normalization-এর প্রধান উদ্দেশ্য কী?

A. Data redundancy বাড়ানো
B. Data redundancy কমানো
C. Database delete করা
D. Query বন্ধ করা

2. 1NF-এর প্রধান requirement কী?

A. No transitive dependency
B. No partial dependency
C. Atomic values
D. Foreign key বাধ্যতামূলক

3. 2NF কী remove করে?

A. Atomicity
B. Partial dependency
C. Transitive dependency
D. Primary key

4. 3NF কী remove করে?

A. Partial dependency
B. Transitive dependency
C. Foreign key
D. Composite key

5. কোনটি Transitive Dependency?

A.

```
StudentID → StudentName
```

B.

```
StudentID → DepartmentID
DepartmentID → DepartmentName
```

C.

```
CourseID → CourseName
```

D.

```
OrderID → OrderDate
```

6. কোন anomaly-তে data update করার সময় inconsistent data হতে পারে?

A. Insert
B. Delete
C. Update
D. Select

🔥 Class 52-এর 5-line Revision

```
1NF → Atomic values

2NF → 1NF + No Partial Dependency

3NF → 2NF + No Transitive Dependency

Normalization → Redundancy + Anomaly কমানো

BCNF → Every determinant should be a superkey
```

পরের ধাপে **Class 53 — Database Design + ER Diagram + Entity + Attribute + Relationship + Cardinality + Primary/Foreign Key** শেখা হবে।

📘 Class 53 — Database Design + ER Diagram

আজকের ক্লাসটা তোমার **Database Designer** goal-এর জন্য খুব গুরুত্বপূর্ণ।

Class 52-এ আমরা শিখেছি **Normalization**।
আজ শিখব কীভাবে requirement থেকে actual database design তৈরি করতে হয়।

আজকের flow:

```
Business Requirement
        ↓
Entities
        ↓
Attributes
        ↓
Relationships
        ↓
Cardinality
        ↓
Primary Key
        ↓
Foreign Key
        ↓
```

```
ER Diagram
        ↓
Normalized Tables
        ↓
SQL Tables
```

---

1. Database Design কী?

সহজভাবে:

> **কোন data কোথায় থাকবে, কোন table হবে, tableগুলোর মধ্যে কী relationship থাকবে—এগুলো পরিকল্পনা করাই Database Design।**

ধরো তুমি একটা **Hotel Management System** বানাবে।

Requirement:

```
Customer hotel room booking করবে।
একজন customer অনেক booking করতে পারবে।
একটি room বিভিন্ন সময়ে অনেকবার booked হতে পারে।
প্রতিটি booking-এর একটি payment থাকতে পারে।
```

এখন সরাসরি SQL লিখে ফেললে ভুল হওয়ার সম্ভাবনা আছে।

আগে design:

```
Customer
   ↓
Booking
   ↓
Room
```

তারপর:

```
Booking → Payment
```

---

2. Entity কী?

> **যে real-world object বা concept সম্পর্কে আমরা data store করতে চাই, সেটাই Entity।**

Hotel system-এ:

```
Customer
Room
Booking
Payment
Hotel
```

এগুলো Entity হতে পারে।

---

3. Entity-এর Real-world Example

ধরো University system:

```
Student
Teacher
Course
Department
Enrollment
```

E-commerce:

```
Customer
Product
Order
Payment
Category
Supplier
```

Hospital:

```
Patient
Doctor
Appointment
Prescription
Department
```

---

4. Entity বনাম Table

Conceptually:

```
Entity
   ↓
```

```
Database Table
```

যেমন:

```
Customer
   ↓
customers table
```

তবে Entity আর table **একই concept নয়**।

Entity হলো design-level concept।

Table হলো database implementation।

---

5. Attribute কী?

> Entity-এর properties/information হলো Attribute।

Customer entity:

```
Customer
 ├── customer_id
 ├── name
 ├── email
 ├── phone
 └── city
```

এখানে:

```
customer_id
name
email
phone
city
```

সবগুলো attribute।

---

6. Entity এবং Attribute

উদাহরণ:

Student

Attribute
student_id
name
email
phone
date_of_birth

Entity:

```
Student
```

Attributes:

```
student_id
name
email
phone
date_of_birth
```

---

7. Attribute-এর Types

Important types:

```
1. Simple Attribute
2. Composite Attribute
3. Single-valued Attribute
4. Multi-valued Attribute
5. Derived Attribute
6. Key Attribute
```

---

8. Simple Attribute

যে attribute আর ছোট meaningful অংশে ভাঙার প্রয়োজন নেই।

Example:

```
Age
Salary
```

```
Gender
```

---

9. Composite Attribute

একটি attribute-কে logical sub-parts-এ ভাগ করা যায়।

Example:

```
Name
 ├── First Name
 └── Last Name
```

Address:

```
Address
 ├── Street
 ├── City
 ├── District
 └── Country
```

---

10. Single-valued Attribute

একজন student-এর:

```
date_of_birth
```

সাধারণত একটি value থাকবে।

```
Student → DOB = 2001-05-10
```

এটা single-valued।

---

11. Multi-valued Attribute

একজন customer-এর একাধিক phone থাকতে পারে:

```
Customer
   ↓
Phone
```

```
    ├── 01711...
    ├── 01822...
    └── 01933...
```

এটা multi-valued attribute-এর concept।

⚠️ Relational database-এ সাধারণত comma-separated phone না রেখে আলাদা related table করা ভালো।

যেমন:

```
customers
customer_phones
```

---

12. Derived Attribute

যে value অন্য data থেকে calculate করা যায়।

Example:

```
date_of_birth
       ↓
     Age
```

Age stored না করেও calculate করা যায়।

আরেকটি:

```
quantity × unit_price
        ↓
      total
```

`total` derived হতে পারে।

---

13. Key Attribute

যে attribute entity-কে uniquely identify করে।

Student:

```
student_id
```

Customer:

```
customer_id
```

Product:

```
product_id
```

সাধারণত এগুলো Primary Key হিসেবে ব্যবহার করা হয়।

---

14. Primary Key

> **একটি table-এর প্রতিটি row uniquely identify করে এমন key হলো Primary Key।**

Example:

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
```

এখানে:

```
student_id = Primary Key
```

---

15. Primary Key-এর Properties

Primary key:

- Unique
- NULL হতে পারে না
- একটি row uniquely identify করে
- একটি table-এ একটি primary key constraint থাকে
- Primary key single বা composite হতে পারে

---

16. Composite Primary Key

কখনো একাধিক column মিলে row uniquely identify করে।

Enrollment:

```
student_id
course_id
```

একসাথে:

```
(student_id, course_id)
```

Primary Key হতে পারে।

SQL:

```sql
CREATE TABLE enrollment (
    student_id INT,
    course_id INT,
    grade CHAR(2),

    PRIMARY KEY (student_id, course_id)
);
```

---

17. Foreign Key

> একটি table-এর column যখন অন্য table-এর primary key-কে reference করে, তখন সেটি Foreign Key।

Example:

Customers

```
customer_id PK
```

Orders

```
order_id PK
customer_id FK
```

Relationship:

```
customers
    |
```

```
    | customer_id
    ↓
orders
```

SQL:

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,

    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

---

18. Relationship কী?

> দুই বা ততোধিক entity-এর মধ্যে logical connection হলো Relationship।

Example:

```
Customer ─── places ─── Order
```

মানে:

> Customer order places করে।

আর:

```
Student ─── enrolls ─── Course
```

---

19. Relationship-এর Degree

Relationship কতগুলো entity involve করছে তার উপর degree নির্ভর করে।

Unary

একটি entity:

```
Employee
   ↕
manages
```

Employee নিজেকেই relate করছে।

Binary

দুইটি entity:

```
Student ─── enrolls ─── Course
```

সবচেয়ে common।

Ternary

তিনটি entity:

```
Supplier
    \
     Supply
    /     \
Product   Project
```

তিন entity এক relationship-এ।

---

20. Cardinality কী?

এটা খুব important।

> **একটি entity-এর কতগুলো instance অন্য entity-এর কতগুলো instance-এর সাথে related হতে পারে—এটাই Cardinality।**

Main types:

```
1 : 1
1 : N
N : 1
M : N
```

---

21. One-to-One — 1:1

একজন person-এর একটি passport।

```
Person ──────── Passport
  1               1
```

Example:

```
Person 1 → Passport 1
Passport 1 → Person 1
```

---

22. One-to-Many — 1:N

একজন customer অনেক order করতে পারে।

```
Customer
   1
   |
   |
   N
 Order
```

অর্থাৎ:

```
One Customer
      ↓
Many Orders
```

Database:

customers

```
customer_id PK
```

orders

```
order_id PK
customer_id FK
```

---

23. Many-to-One — N:1

একই relationship অন্য দিক থেকে দেখলে:

```
Orders
   N
   |
   |
   1
Customer
```

অর্থাৎ:

```
Many Orders
     ↓
One Customer
```

তাই:

```
1:N = N:1
```

শুধু viewpoint আলাদা।

---

24. Many-to-Many — M:N

একজন student অনেক course নিতে পারে।

একটি course অনেক student নিতে পারে।

```
Student
   M
   |
   |
   N
 Course
```

এটা M:N।

কিন্তু relational database-এ সরাসরি M:N relationship সাধারণত table দিয়ে implement করি না।

আমরা junction/bridge table ব্যবহার করি।

---

25. M:N কীভাবে Table-এ Convert করব?

Original:

```
Student M:N Course
```

Convert:

```
Students
    ↓
Enrollment
    ↓
Courses
```

Enrollment:

```
student_id
course_id
```

Example:

students

student_id	name
1	Atiar
2	Karim

courses

course_id	name
101	DBMS
102	SQL

enrollment

student_id	course_id
1	101
1	102
2	101

এখানে `Enrollment` হলো **junction/associative table**।

26. ER Diagram কী?

ER = **Entity Relationship**

> Entity, Attribute এবং Relationship visually represent করার diagram হলো ER Diagram।

Basic structure:

```
[Student] ————————< [Enrollment] >———————— [Course]
```

আরেকভাবে:

```
Student 1 ——— N Enrollment N ——— 1 Course
```

---

27. ER Diagram-এর Common Symbols

Traditional ER notation:

Symbol	Meaning
Rectangle	Entity
Oval	Attribute
Diamond	Relationship
Double Oval	Multivalued Attribute
Dashed Oval	Derived Attribute
Underlined Attribute	Key Attribute
Double Rectangle	Weak Entity
Double Diamond	Identifying Relationship

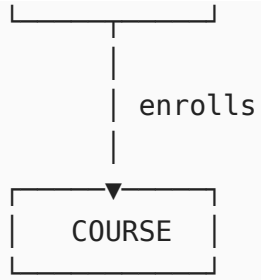
Exam-এর জন্য এগুলো খুব important।

---

28. Example ER Diagram Concept

University:

```
                    ┌──────────────┐
                    │   STUDENT    │
```



আর যদি Enrollment-এর নিজের attributes থাকে:

```
Student
   │
   │
Enrollment
   │
   │
Course

Enrollment:
 ├── enrollment_date
 └── grade
```

এটা important।

---

29. Relationship-এর Attribute

কখনো relationship-এরও নিজস্ব information থাকে।

Example:

```
Student ─── enrolls ─── Course
```

`enrollment_date` কার property?

Student-এর নয়।

Course-এরও নয়।

বরং:

```
Student + Course
       ↓
Enrollment
       ↓
```

```
enrollment_date
grade
```

তাই এগুলো relationship/junction entity-তে রাখা ভালো।

---

30. Optionality

সব relationship mandatory নয়।

Example:

```
Customer → Order
```

একজন customer হয়তো এখনো কোনো order করেনি।

তাই:

```
Customer → 0 or many Orders
```

অন্যদিকে একটি order-এর অবশ্যই একজন customer থাকতে পারে:

```
Order → exactly 1 Customer
```

এখানে minimum/maximum cardinality গুরুত্বপূর্ণ।

---

31. `0..1`, `1..1`, `0..N`, `1..N`

এগুলো practical database design-এ খুব useful।

`0..1`

Zero অথবা one।

Example:

```
Employee → ParkingSpot
```

কেউ parking spot নাও পেতে পারে।

`1..1`

Exactly one।

`0..N`

Zero বা many।

Customer:

```
0..N Orders
```

`1..N`

At least one, potentially many।

---

32. ER Design থেকে SQL

ধরো requirement:

> একজন customer অনেক order করতে পারে।

Design:

```
Customer 1 ──── N Order
```

Tables:

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100)
);
```

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,

    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

এখানে relationship implement হয়েছে:

```
customers.customer_id
        ↑
        | FK
orders.customer_id
```

---

33. Database Design-এর Complete Process

একজন Database Designer সাধারণত এভাবে কাজ করবে:

Step 1 — Requirement collect

```
Business কী চায়?
```

Step 2 — Entities identify

```
Customer
Product
Order
Payment
```

Step 3 — Attributes identify

```
Customer:
id
name
email
phone
```

Step 4 — Primary Keys

```
customer_id
product_id
order_id
```

Step 5 — Relationships

```
Customer → Order
Order → Product
Order → Payment
```

Step 6 — Cardinality

```
Customer 1:N Order
Order M:N Product
Order 1:1 Payment
```

Step 7 — Normalize

```
1NF
2NF
3NF
```

Step 8 — Foreign Keys

Step 9 — Constraints

```
NOT NULL
UNIQUE
CHECK
DEFAULT
FK
```

Step 10 — Indexing

Query pattern অনুযায়ী।

---

34. Real-world E-commerce Design

একটা ছোট e-commerce system:

```
Customer
Product
Category
Order
OrderItem
Payment
```

Relationships:

```
Customer 1 ─── N Order

Order 1 ─── N OrderItem

Product 1 ─── N OrderItem
```

```
Category 1 ─── N Product

Order 1 ─── 1 Payment
```

এখানে:

```
Order
   ↓
OrderItem
   ↓
Product
```

কারণ এক order-এ multiple products থাকতে পারে।

---

35. কেন OrderItem দরকার?

ধরো Order #100:

```
Laptop × 1
Mouse × 2
Keyboard × 1
```

একটা order-এর মধ্যে multiple products।

তাই:

```
orders
```

এবং:

```
products
```

এর মধ্যে M:N relationship।

আমরা bridge table বানাই:

```
order_items
```

যেখানে:

```
order_id
product_id
```

```
quantity
unit_price
```

থাকতে পারে।

---

36. Order-এর Final Structure

```
customers
-----------
customer_id PK
name
email


orders
-----------
order_id PK
customer_id FK
order_date


products
-----------
product_id PK
product_name
price


order_items
-----------
order_id FK
product_id FK
quantity
unit_price
```

Composite PK:

```
PRIMARY KEY (order_id, product_id)
```

হতে পারে, অথবা আলাদা `order_item_id` ব্যবহার করা যায়—business requirements অনুযায়ী।

---

37. Database Designer-এর সবচেয়ে গুরুত্বপূর্ণ চিন্তা

Database design করার সময় শুধু:

> "কোন কোন table বানাব?"

ভাববে না।

বরং:

```
1. What data?
2. Who owns the data?
3. What uniquely identifies it?
4. How are entities related?
5. How many can relate?
6. Is relationship optional?
7. Will data repeat?
8. What constraints are needed?
9. What queries will be common?
10. What indexes will be needed?
```

⭐ Class 53 Golden Rules

Rule 1

```
Entity → Thing
Attribute → Property
Relationship → Connection
```

Rule 2

```
1:N
```

relationship implement করার সময় সাধারণত **N-side table-এ FK** থাকে।

Example:

```
Customer 1:N Order

Order.customer_id → Customer.customer_id
```

Rule 3

```
M:N
```

relationship → **junction/bridge table**

Rule 4

Relationship-এর নিজস্ব attributes থাকলে associative entity/table লাগতে পারে।

Rule 5

Design আগে, SQL পরে।

```
Requirement
 ↓
ERD
 ↓
Normalization
 ↓
Tables
 ↓
Constraints
 ↓
Indexes
```

🧠 Class 53 Cheat Sheet

```
Entity
↓
Real-world object/concept

Attribute
↓
Entity-এর property

Primary Key
↓
Unique row identification

Foreign Key
↓
Another table-এর key reference

Relationship
↓
Entity-এর connection

1:1
↓
```

```
One-to-One

1:N
↓
One-to-Many

M:N
↓
Many-to-Many
↓
Junction Table

ERD
↓
Entity + Attribute + Relationship-এর visual design
```

📝 Assignment

ধরো তুমি একটি **University Management System** design করবে।

Requirements:

1. একজন student অনেক course নিতে পারে।
2. একটি course অনেক student নিতে পারে।
3. একজন teacher অনেক course teach করতে পারে।
4. একটি course একজন teacher-এর অধীনে থাকবে।
5. Student-এর `student_id`, `name`, `email` থাকবে।
6. Course-এর `course_id`, `course_name`, `credit` থাকবে।
7. Enrollment-এর `enrollment_date` এবং `grade` থাকবে।

তোমার কাজ:

1. সব Entity identify করো।

2. প্রতিটি Entity-এর Attributes লিখো।

3. Primary Key identify করো।

4. Relationships identify করো।

5. Cardinality লিখো।

6. M:N relationship কোথায় আছে?

7. Junction table-এর নাম কী হবে?

8. কোন table-এ কোন Foreign Key থাকবে?

9. Final tables-এর structure লিখো।

🎯 MCQ

1. Entity কী?

A. Table-এর row
B. Real-world object/concept
C. SQL query
D. Constraint

2. Attribute কী?

A. Entity-এর property
B. Database server
C. Relationship
D. Foreign key only

3. Customer এবং Order-এর relationship সাধারণত কোনটি?

A. 1:1
B. 1:N
C. N:1 only
D. M:N

4. M:N relationship relational database-এ সাধারণত কী দিয়ে implement করা হয়?

A. Index
B. Trigger
C. Junction table
D. View

5. 1:N relationship-এ সাধারণত Foreign Key কোথায় থাকে?

A. 1-side
B. N-side
C. দুই side-এই বাধ্যতামূলক
D. কোথাও না

6. `enrollment_date` **কোনটির attribute হতে পারে?**

A. শুধু Student
B. শুধু Course

C. Enrollment relationship/entity
D. Database

7. কোনটি Primary Key-এর বৈশিষ্ট্য?

A. Duplicate হতে পারে
B. NULL হতে পারে
C. Row uniquely identify করে
D. সবসময় VARCHAR হতে হবে

🔥 আজকের সবচেয়ে গুরুত্বপূর্ণ বিষয়

একজন **Database Designer** হিসেবে এই pattern মাথায় রাখো:

```
Business Requirement
        ↓
Identify Entities
        ↓
Identify Attributes
        ↓
Choose Primary Keys
        ↓
Identify Relationships
        ↓
Determine Cardinality
        ↓
Resolve M:N with Junction Table
        ↓
Normalize to 3NF
        ↓
Add Foreign Keys & Constraints
        ↓
Add Indexes based on query patterns
        ↓
Implement SQL
```

Class 54 → Fact Table + Dimension Table + Star Schema + Snowflake Schema — এখান থেকে আমরা Database Design থেকে Data Warehouse/Analytics Design-এর দিকে যাব।

📘 Class 54 — Fact Table, Dimension Table & Star Schema

আজকের ক্লাস থেকে আমরা **Database Design → Data Warehouse / Analytics Database Design**-এ ঢুকছি।

Class 53-এ ছিল:

```
Entity
Attribute
Relationship
ERD
Primary Key
Foreign Key
Normalization
```

আজ শিখব:

```
Fact Table
Dimension Table
Measure
Grain
Star Schema
Snowflake Schema
OLTP vs Data Warehouse
Star Schema কীভাবে design করতে হয়
```

---

1. Data Warehouse কী?

সহজভাবে:

> **বিভিন্ন source থেকে data সংগ্রহ করে analytics/reporting-এর জন্য organize করা বড় data store হলো Data Warehouse।**

OLTP database-এর লক্ষ্য:

```
Transaction
```

Data Warehouse-এর লক্ষ্য:

```
Analysis
Reporting
BI
Dashboard
```

উদাহরণ:

Hotel Management

OLTP:

```
Customer books room
Payment করা
Booking cancel করা
```

Data Warehouse:

```
কোন মাসে বেশি booking?
কোন room type সবচেয়ে popular?
কোন city থেকে সবচেয়ে বেশি customer?
মাসভিত্তিক revenue কত?
```

---

2. OLTP বনাম Data Warehouse

OLTP	Data Warehouse
Transaction	Analytics
INSERT/UPDATE/DELETE বেশি	SELECT/aggregation বেশি
Current data	Historical data
Highly normalized	Often dimensional model
Application-এর জন্য	Reporting/BI-এর জন্য
Example: Django backend DB	Example: Analytics warehouse

সহজ shortcut:

```
OLTP → Business runs
DW   → Business analysis
```

---

3. Dimensional Modeling কী?

Data Warehouse design করার একটি জনপ্রিয় পদ্ধতি হলো:

> **Dimensional Modeling**

এর মূল দুই ধরনের table:

```
Fact Table
Dimension Table
```

---

4. Fact Table কী?

> **Business event বা measurable transaction-এর data রাখে Fact Table।**

E-commerce:

```
Sales
```

Hotel:

```
Bookings
```

Bank:

```
Transactions
```

Hospital:

```
Medical visits
```

---

5. Fact Table-এর মধ্যে কী থাকে?

সাধারণত:

1. Foreign Keys

এগুলো dimension table-এর দিকে point করে।

2. Measures

যে numerical values আমরা analyze করতে চাই।

Example:

```
quantity
sales_amount
```

```
discount
profit
```

---

6. Dimension Table কী?

> **Business event-এর context বা description দেয় Dimension Table।**

Example:

Sales-এর ক্ষেত্রে:

```
DimCustomer
DimProduct
DimDate
DimStore
```

এগুলো answer করে:

```
Who?
What?
When?
Where?
```

---

7. Fact + Dimension Example

ধরো:

```
Customer Atiar
Laptop
Dhaka
2026-09-13
Quantity = 2
Sales = 150000
```

এখানে:

Dimensions

```
Customer → Atiar
Product → Laptop
```

```
Location → Dhaka
Date → 2026-09-13
```

Fact

```
Quantity = 2
Sales = 150000
```

---

8. Dimension Table-এর Example

`dim_customer`

customer_key	customer_id	name	city
1	C001	Atiar	Dhaka
2	C002	Karim	Chittagong

এখানে:

```
customer_key
customer_id
name
city
```

Customer-এর descriptive information।

---

9. Product Dimension

`dim_product`

product_key	product_id	product_name	category	brand
1	P001	Laptop	Electronics	Dell
2	P002	Mouse	Accessories	Logitech

---

10. Date Dimension

Data Warehouse-এ Date Dimension খুব important।

`dim_date`

date_key	full_date	year	month	month_name	quarter
20260913	2026-09-13	2026	9	September	Q3

এখানে আমরা date-এর অনেক analytical information রাখতে পারি:

```
year
quarter
month
month_name
week
day
day_name
is_weekend
```

---

11. Fact Table Example

`fact_sales`

date_key	customer_key	product_key	quantity	sales_amount
20260913	1	1	2	150000
20260913	2	2	5	5000

এখানে:

Foreign Keys

```
date_key
customer_key
product_key
```

Measures

```
quantity
sales_amount
```

---

12. Fact Table + Dimension Table

Relationship:



Fact table হলো center।

Dimensions চারপাশে।

এটাই আমাদের নিয়ে যায়:

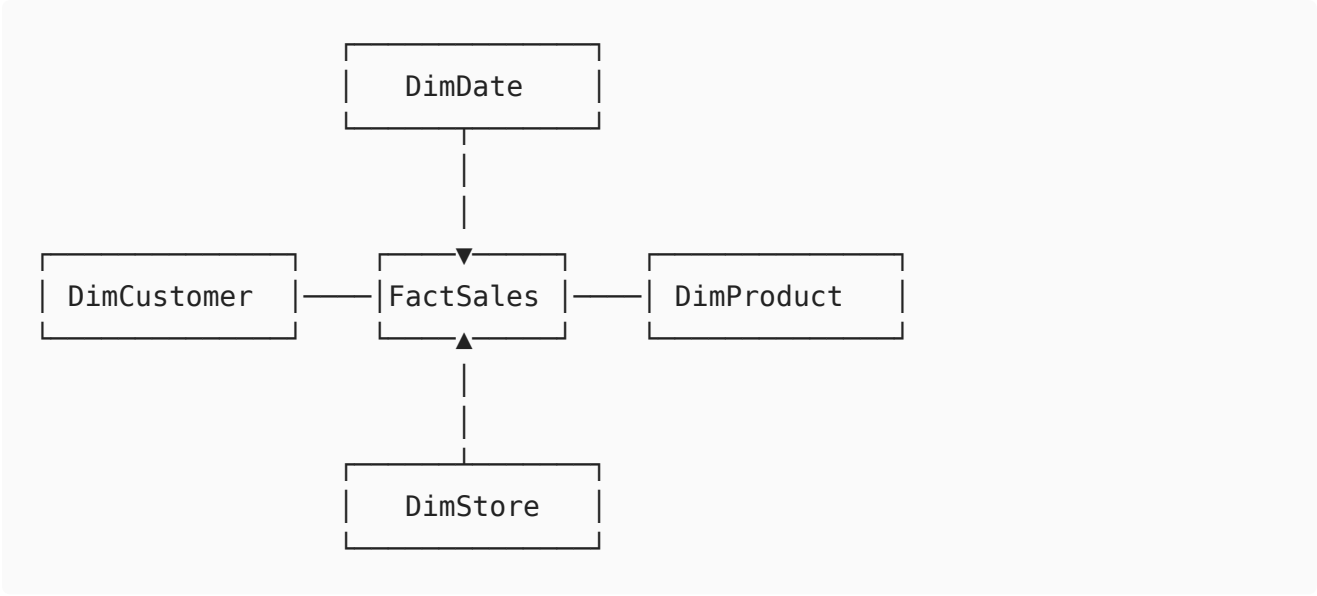
⭐ Star Schema

---

13. Star Schema কী?

> **একটি central Fact Table-এর চারপাশে multiple Dimension Table থাকলে তাকে Star Schema বলে।**

Visual:



দেখতে star-এর মতো।

তাই নাম:

> **Star Schema**

---

14. Star Schema-এর সুবিধা

1. সহজ structure

বোঝা সহজ।

2. Analytics-friendly

Dashboard/report query সহজ হয়।

3. JOIN তুলনামূলক straightforward

Fact-এর সাথে dimensions join করা হয়।

4. BI tools-এর জন্য ভালো

Power BI, Tableau ইত্যাদির dimensional analytics-এর জন্য খুব common pattern।

---

15. Fact Table-এর Grain

আজকের সবচেয়ে important conceptগুলোর একটি:

⭐ Grain

> **Fact table-এর একটি row ঠিক কী represent করে—এটাই Grain।**

ধরো:

```
fact_sales
```

এর grain যদি হয়:

> **One row = One product sold in one order**

তাহলে:

```
Order 100
 ├── Laptop
 ├── Mouse
 └── Keyboard
```

হলে তিনটি fact row হতে পারে।

---

16. Grain আগে কেন নির্ধারণ করতে হয়?

যদি grain পরিষ্কার না থাকে, fact table ভুল design হতে পারে।

Example:

তুমি যদি বলো:

```
fact_sales
```

কিন্তু জানো না row কী represent করে:

```
One order?
One product?
One customer per day?
One store per day?
```

তাহলে:

```
quantity
sales
profit
```

এর meaning-ও ambiguous হয়ে যাবে।

---

17. Golden Rule ⭐

> **Fact Table design করার আগে Grain define করো।**

Example:

```
Grain:
One row = One product line in one order
```

তারপর columns:

```
order_id
product_key
customer_key
date_key
quantity
sales_amount
```

```
discount
profit
```

---

18. Measures কী?

Fact table-এর numerical values যেগুলো aggregate/analyze করা যায়।

Example:

```
quantity
sales_amount
discount_amount
cost
profit
```

SQL:

```sql
SELECT
    SUM(sales_amount),
    AVG(sales_amount),
    SUM(quantity)
FROM fact_sales;
```

---

19. Additive Measure

যে measure সব dimension বরাবর SUM করা যায়।

Example:

```
sales_amount
quantity
```

ধরো:

```
Dhaka = 10000
Chittagong = 15000
```

Total:

```
25000
```

এগুলো সাধারণত **Additive Measures**।

---

20. Semi-Additive Measure

কিছু dimension-এ sum করা যায়, কিন্তু সব dimension-এ নয়।

Classic example:

```
Account Balance
```

সময় অনুযায়ী:

```
Monday = 10,000
Tuesday = 12,000
Wednesday = 15,000
```

এগুলো যোগ করে:

```
37,000
```

বললে meaningful নাও হতে পারে।

কিন্তু customer বা branch dimension অনুযায়ী কিছু aggregation সম্ভব হতে পারে।

তাই:

> **Semi-additive**

---

21. Non-Additive Measure

যে measure সাধারণভাবে SUM করা যায় না।

Example:

```
Percentage
Ratio
Average
```

যেমন:

```
Conversion Rate = 25%
```

দুই campaign-এর:

```
25%
40%
```

যোগ করে:

```
65%
```

করলে অর্থপূর্ণ নয়।

---

22. Dimension-এর Surrogate Key

Data Warehouse-এ আমরা অনেক সময় natural/business key-এর বদলে:

> **Surrogate Key**

ব্যবহার করি।

Example:

Business key:

```
customer_id = C1001
```

Warehouse key:

```
customer_key = 501
```

Dimension:

```
customer_key PK
customer_id
name
city
```

Fact table:

```
customer_key FK
```

---

23. Surrogate Key কেন?

কারণ source system-এর ID সবসময় stable/unique across systems নাও হতে পারে।

ধরো:

```
CRM:
customer_id = 101

Website:
customer_id = 101
```

দুটো 101 একই customer নাও হতে পারে।

Warehouse-এ:

```
customer_key = 1
customer_key = 2
```

দিয়ে আলাদা করা যায়।

---

24. Natural Key বনাম Surrogate Key

Natural Key	Surrogate Key
Business meaning আছে	Artificial/generated
Example: customer_id	Example: customer_key
Source system থেকে আসে	Warehouse তৈরি করে
Business change হতে পারে	সাধারণত stable
Integration-এ সমস্যা হতে পারে	Warehouse design-এ useful

---

25. Star Schema-এর Full Example

ধরো Sales Data Warehouse।

Dimension Tables

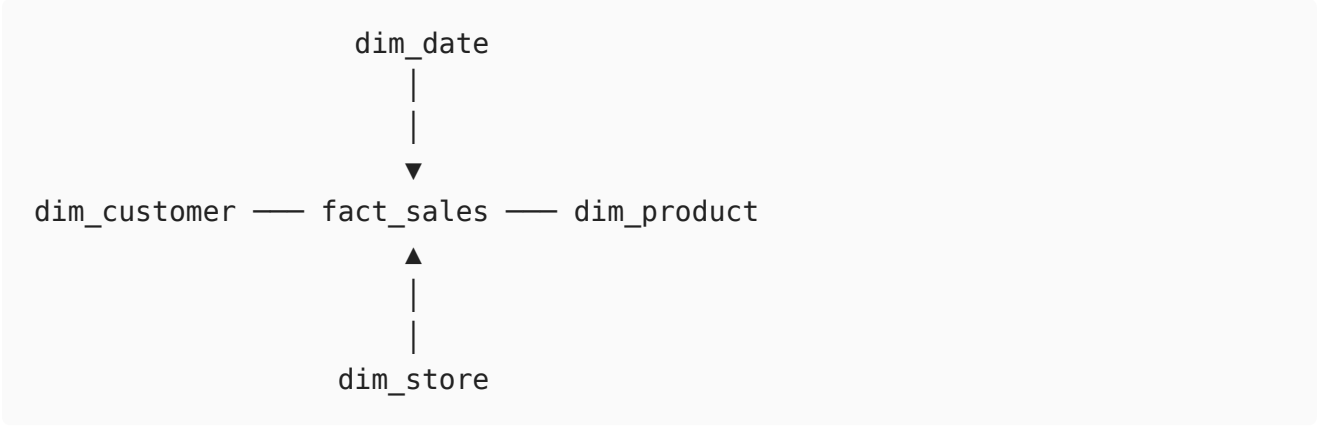
```
dim_date
dim_customer
```

```
dim_product
dim_store
```

Fact Table

```
fact_sales
```

Structure:



---

26. SQL Design

Date Dimension

```sql
CREATE TABLE dim_date (
    date_key INT PRIMARY KEY,
    full_date DATE,
    year INT,
    month INT,
    month_name VARCHAR(20),
    quarter INT
);
```

Customer Dimension

```sql
CREATE TABLE dim_customer (
    customer_key INT PRIMARY KEY,
    customer_id VARCHAR(50),
    customer_name VARCHAR(100),
    city VARCHAR(100)
);
```

Product Dimension

```sql
CREATE TABLE dim_product (
    product_key INT PRIMARY KEY,
    product_id VARCHAR(50),
    product_name VARCHAR(100),
    category VARCHAR(100)
);
```

Fact

```sql
CREATE TABLE fact_sales (
    date_key INT,
    customer_key INT,
    product_key INT,

    quantity INT,
    sales_amount DECIMAL(12,2),
    discount_amount DECIMAL(12,2),

    FOREIGN KEY (date_key)
        REFERENCES dim_date(date_key),

    FOREIGN KEY (customer_key)
        REFERENCES dim_customer(customer_key),

    FOREIGN KEY (product_key)
        REFERENCES dim_product(product_key)
);
```

---

27. Star Schema Query

এখন প্রশ্ন:

> 2026 সালে category-wise sales কত?

```sql
SELECT
    p.category,
    SUM(f.sales_amount) AS total_sales
FROM fact_sales f
JOIN dim_date d
    ON f.date_key = d.date_key
JOIN dim_product p
    ON f.product_key = p.product_key
WHERE d.year = 2026
```

```
GROUP BY p.category
ORDER BY total_sales DESC;
```

এটাই dimensional model-এর power।

---

28. Customer-wise Sales

```
SELECT
    c.customer_name,
    SUM(f.sales_amount) AS total_sales
FROM fact_sales f
JOIN dim_customer c
    ON f.customer_key = c.customer_key
GROUP BY
    c.customer_key,
    c.customer_name
ORDER BY total_sales DESC;
```

---

29. Monthly Sales

```
SELECT
    d.year,
    d.month,
    d.month_name,
    SUM(f.sales_amount) AS total_sales
FROM fact_sales f
JOIN dim_date d
    ON f.date_key = d.date_key
GROUP BY
    d.year,
    d.month,
    d.month_name
ORDER BY
    d.year,
    d.month;
```

---

30. Snowflake Schema

Star Schema-তে dimension table সাধারণত relatively denormalized থাকে।

কিন্তু dimension-কে আবার ছোট ছোট table-এ ভাগ করলে:

> **Snowflake Schema**

Example:

Star

```
fact_sales
    |
dim_product
    |
category
```

সব product information `dim_product` -এ থাকতে পারে।

Snowflake:

```
fact_sales
     |
dim_product
     |
dim_category
```

---

31. Star vs Snowflake

Star

```
Fact
 ├── Dim Customer
 ├── Dim Product
 ├── Dim Date
 └── Dim Store
```

Snowflake

```
Fact
 └── Dim Product
       └── Dim Category
             └── Dim Department
```

---

32. Star Schema vs Snowflake Schema

Star	Snowflake
Simpler	More complex
Dimension বেশি denormalized	Dimension more normalized
Fewer JOINs	More JOINs
Query সহজ	Query তুলনামূলক complex
BI-friendly	Storage/redundancy কমাতে useful হতে পারে

---

33. OLTP Database কি Star Schema?

সাধারণত না।

OLTP:

```
customers
orders
order_items
products
categories
payments
```

এখানে normalized design common।

Data Warehouse:

```
             dim_customer
                  |
dim_date — fact_sales — dim_product
                  |
              dim_store
```

এখানে dimensional model common।

---

34. OLTP → Data Warehouse

Real-world architecture:

```
Application DB
      ↓
   ETL / ELT
      ↓
Data Warehouse
      ↓
   BI / Reports
      ↓
Power BI / Tableau / Dashboard
```

তুমি যেহেতু Excel + SQL + Data Analysis শিখছ, এই pipeline খুব গুরুত্বপূর্ণ।

---

35. তোমার শেখার Journey-এর সাথে সম্পর্ক

তুমি এতদিন:

```
Excel
 ↓
SQL
 ↓
Database Design
```

শিখেছ।

এখন:

```
Database
 ↓
Data Warehouse
 ↓
Dimensional Modeling
 ↓
BI
 ↓
Data Engineering
```

এর দিকে যেতে পারবে।

⭐ Class 54 Golden Rules

Rule 1

```
Fact → What happened?
```

Rule 2

```
Dimension → Who / What / When / Where?
```

Rule 3

```
Fact → Measures + Foreign Keys
```

Rule 4

> **Fact table-এর Grain আগে define করো।**

Rule 5

```
Star Schema
=
Central Fact
+
Dimension Tables
```

Rule 6

```
M:N relationship
```

OLTP-তে যেমন junction table লাগে, dimensional model-এ business event অনুযায়ী fact table তৈরি হতে পারে।

🧠 Class 54 Cheat Sheet

```
Data Warehouse
      ↓
Analytics / Reporting

Fact Table
      ↓
Business event
      ↓
Measures + Dimension Keys

Dimension Table
```

```
      ↓
Business context
      ↓
Customer / Product / Date / Store

Grain
      ↓
What exactly does one fact row represent?

Star Schema
      ↓
Fact in center
Dimensions around it

Snowflake
      ↓
Normalized dimensions
```

📝 Assignment

একটি **Hotel Data Warehouse** design করো।

Business requirements:

- Customer hotel booking করে
- Customer-এর city আছে
- Room-এর room type আছে
- Hotel-এর location আছে
- Booking-এর date আছে
- Booking-এর number of nights আছে
- Room price আছে
- Discount আছে
- Total revenue calculate করা যাবে

তোমার কাজ:

1. কোনটি Fact Table?

2. কোন কোন Dimension Table হবে?

3. Fact table-এর **Grain** কী হবে?

4. কোনগুলো measure?

5. কোনগুলো dimension?

6. একটি Star Schema structure আঁকো/লিখো।

7. `fact_booking` table-এর সম্ভাব্য columns লিখো।

8. `dim_date`, `dim_customer`, `dim_room`, `dim_hotel`-এর columns লিখো।

🎯 MCQ

1. Fact Table মূলত কী রাখে?

A. শুধু customer information
B. Business events/measures
C. শুধু addresses
D. Database users

2. কোনটি Dimension Table-এর example?

A. FactSales
B. DimCustomer
C. TransactionAmount
D. Quantity

3. Fact Table-এর Grain কী?

A. Primary key-এর type
B. One fact row কী represent করে
C. Index-এর size
D. Database size

4. Star Schema-তে কী থাকে?

A. শুধু Dimension
B. শুধু Fact
C. Central Fact + Dimensions
D. শুধু normalized tables

5. কোনটি সাধারণত Additive Measure?

A. Sales Amount
B. Percentage
C. Conversion Rate
D. Ratio

6. `customer_key` **কী হতে পারে?**

A. Surrogate Key
B. Measure

C. Dimension attribute only
D. Aggregate function

7. Snowflake Schema-এর প্রধান বৈশিষ্ট্য কী?

A. Dimension আরও normalized
B. Fact table থাকে না
C. কোনো relationship থাকে না
D. শুধু একটি table থাকে

🔥 আজকের 10-second revision

```
Fact       → Event
Dimension  → Context
Measure    → Numeric value
Grain      → One row means what?
Star       → Fact + Dimensions
Snowflake  → Normalized Dimensions
```

পরের ক্লাস: Class 55 — SQL Interview Problems + Advanced Query Challenges — সেখানে এখন পর্যন্ত শেখা `JOIN + CTE + Window Function + Subquery + GROUP BY + CASE + Date + EXISTS + Index` একসাথে ব্যবহার করে বাস্তব interview-level problem solve করব।

📘 Class 55 — SQL Interview Problems + Advanced Query Challenges

আজকের ক্লাসে আমরা নতুন syntax কম শিখব। বরং **আগের Class 31–54-এর concepts একসাথে ব্যবহার করে interview-level SQL problem solve** করব।

আজকের লক্ষ্য:

```
SELECT
WHERE
GROUP BY
HAVING
JOIN
Subquery
CTE
Window Function
CASE
EXISTS
Date
NULL
```

```
Set Operation
Index/EXPLAIN
        ↓
Real SQL Problem Solving
```

---

1. Interview SQL Problem Solving Strategy

SQL problem দেখেই query লেখা শুরু করবে না।

এই 5-step process follow করবে:

```
Step 1 → What output is needed?
Step 2 → Which tables contain the data?
Step 3 → How are tables related?
Step 4 → Do I need filtering/grouping/ranking?
Step 5 → Write + test + optimize
```

Golden Rule ⭐

> **Question → Data relationship → Logic → SQL**

---

2. আমাদের Practice Database

আজকের examples-এর জন্য ধরো:

customers

```
customer_id
customer_name
city
```

products

```
product_id
product_name
category
price
```

orders

```
order_id
customer_id
product_id
order_date
quantity
```

Revenue:

```
quantity * price
```

Problem 1 — সব customer যারা order করেছে

Question:

> Find customers who have placed at least one order.

Solution

```sql
SELECT DISTINCT
    c.customer_id,
    c.customer_name
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id;
```

Alternative: EXISTS

```sql
SELECT
    c.customer_id,
    c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Interview Point

`JOIN` এবং `EXISTS` দুটোই ব্যবহার করা যায়, কিন্তু logical requirement যদি হয়:

> "related row exists কি?"

তাহলে `EXISTS` খুব natural।

Problem 2 — যেসব customer কোনো order করেনি

এটা interview-এর classic problem।

```
SELECT
    c.customer_id,
    c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Alternative

```
SELECT
    c.customer_id,
    c.customer_name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

এটা খুব গুরুত্বপূর্ণ pattern:

```
LEFT JOIN
+
WHERE right_table.id IS NULL
```

মানে:

> Matching record নেই।

Problem 3 — Customer-wise total sales

Revenue:

```
quantity × price
```

Query:

```sql
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.quantity * p.price) AS total_sales
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name;
```

এখানে ব্যবহার করেছি:

```
JOIN
+
SUM
+
GROUP BY
```

Problem 4 — Top 5 customers by sales

```sql
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.quantity * p.price) AS total_sales
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name
ORDER BY total_sales DESC
LIMIT 5;
```

Pattern

```
GROUP BY
→ calculate
```

```
→ ORDER BY DESC
→ LIMIT
```

Problem 5 — Customers whose sales are greater than 100,000

```sql
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.quantity * p.price) AS total_sales
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.customer_name
HAVING SUM(o.quantity * p.price) > 100000;
```

Important

Aggregate condition:

```sql
HAVING
```

not:

```sql
WHERE
```

কারণ:

```
WHERE → before grouping
HAVING → after grouping
```

Problem 6 — Second highest salary

ধরো:

```
employees
-----------
employee_id
name
salary
```

একটি simple solution:

```sql
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Logic:

```
Maximum salary
      ↓
Remove maximum
      ↓
Find maximum again
      ↓
Second highest
```

Problem 7 — Top 3 salaries

Window function ব্যবহার করি:

```sql
SELECT *
FROM (
    SELECT
        employee_id,
        name,
        salary,
        DENSE_RANK() OVER (
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
) x
WHERE salary_rank <= 3;
```

কেন `DENSE_RANK()`?

যদি:

```
100000
90000
90000
80000
```

তাহলে:

```
100000 → 1
90000  → 2
90000  → 2
80000  → 3
```

তাই top 3 salary levels পাওয়া যাবে।

Problem 8 — Highest salary in each department

এটা খুব common interview question।

```sql
SELECT *
FROM (
    SELECT
        employee_id,
        name,
        department,
        salary,
        RANK() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rnk
    FROM employees
) x
WHERE rnk = 1;
```

এখানে:

```
PARTITION BY department
```

মানে:

> প্রতিটি department আলাদাভাবে ranking হবে।

Problem 9 — Employees earning above department average

এটা একটু advanced।

প্রথমে department average বের করতে পারি:

```
SELECT
    e.employee_id,
    e.name,
    e.department,
    e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
```

এটা **correlated subquery**।

Problem 10 — একই problem Window Function দিয়ে

```
SELECT *
FROM (
    SELECT
        employee_id,
        name,
        department,
        salary,

        AVG(salary) OVER (
            PARTITION BY department
        ) AS dept_avg_salary

    FROM employees
) x
WHERE salary > dept_avg_salary;
```

এখানে:

```
department
     ↓
average salary
```

```
     ↓
compare employee salary
```

Problem 11 — Monthly sales

```sql
SELECT
    DATE_FORMAT(o.order_date, '%Y-%m') AS month,
    SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    DATE_FORMAT(o.order_date, '%Y-%m')
ORDER BY month;
```

Problem 12 — Month-over-month growth

প্রথমে monthly sales:

```sql
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
)
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month
    ) AS previous_sales
FROM monthly_sales;
```

এখন growth:

```sql
WITH monthly_sales AS (
    SELECT
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.quantity * p.price) AS sales
```

```
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY DATE_FORMAT(o.order_date, '%Y-%m')
),
sales_with_previous AS (
    SELECT
        month,
        sales,
        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales
    FROM monthly_sales
)
SELECT
    month,
    sales,
    previous_sales,
    ROUND(
        100 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percent
FROM sales_with_previous;
```

এখানে একসাথে:

```
CTE
+
GROUP BY
+
LAG
+
NULLIF
+
Calculation
```

ব্যবহার করেছি।

Problem 13 — Highest-selling product

```
SELECT
    p.product_id,
    p.product_name,
    SUM(o.quantity) AS total_quantity
```

```sql
FROM products p
JOIN orders o
    ON p.product_id = o.product_id
GROUP BY
    p.product_id,
    p.product_name
ORDER BY total_quantity DESC
LIMIT 1;
```

Problem 14 — Highest-selling product in each category

এটা advanced interview problem।

```sql
WITH product_sales AS (
    SELECT
        p.product_id,
        p.product_name,
        p.category,
        SUM(o.quantity) AS total_quantity
    FROM products p
    JOIN orders o
        ON p.product_id = o.product_id
    GROUP BY
        p.product_id,
        p.product_name,
        p.category
),
ranked_products AS (
    SELECT
        *,
        RANK() OVER (
            PARTITION BY category
            ORDER BY total_quantity DESC
        ) AS rnk
    FROM product_sales
)
SELECT *
FROM ranked_products
WHERE rnk = 1;
```

Pattern:

```
Aggregate
   ↓
```

```
CTE
   ↓
Window Function
   ↓
Filter Rank
```

⭐ এটা খুব important interview pattern।

Problem 15 — Customers with at least 3 orders

```sql
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.customer_name
HAVING COUNT(o.order_id) >= 3;
```

Problem 16 — Customers who bought every product in a category

এটা advanced logic।

ধরো category:

```
Electronics
```

আমরা জানতে চাই:

> কোন customer Electronics category-এর **সব product** কিনেছে?

এ ধরনের problem-এ `NOT EXISTS` খুব powerful।

Concept:

```sql
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
```

```
    SELECT 1
    FROM products p
    WHERE p.category = 'Electronics'
      AND NOT EXISTS (
          SELECT 1
          FROM orders o
          WHERE o.customer_id = c.customer_id
            AND o.product_id = p.product_id
      )
);
```

Logic:

```
Customer
   ↓
There does NOT exist
   ↓
a required product
   ↓
that customer did NOT buy
```

অর্থাৎ:

> এমন কোনো required product নেই যেটা customer কেনেনি।

Problem 17 — Duplicate emails

ধরো:

```
customers
-----------
customer_id
email
```

Duplicate email খুঁজতে:

```
SELECT
    email,
    COUNT(*) AS total
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Classic interview question।

Problem 18 — Find duplicate rows

ধরো:

```
name
city
```

একই name + city কতবার এসেছে:

```sql
SELECT
    name,
    city,
    COUNT(*) AS total
FROM customers
GROUP BY
    name,
    city
HAVING COUNT(*) > 1;
```

Problem 19 — Employees with same salary

```sql
SELECT
    salary,
    COUNT(*) AS employee_count
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

Problem 20 — First order of each customer

Window function:

```sql
SELECT *
FROM (
    SELECT
        o.*,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date
        ) AS rn
```

```
    FROM orders o
) x
WHERE rn = 1;
```

এখানে:

```
PARTITION BY customer_id
```

প্রতিটি customer-এর order আলাদা group হিসেবে ধরছে।

তারপর:

```
ORDER BY order_date
```

সবচেয়ে পুরোনো order:

```
ROW_NUMBER = 1
```

Problem 21 — Latest order of each customer

শুধু order direction পরিবর্তন:

```
SELECT *
FROM (
    SELECT
        o.*,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC
        ) AS rn
    FROM orders o
) x
WHERE rn = 1;
```

Pattern মনে রাখো:

```
First
→ ROW_NUMBER + ASC

Latest
→ ROW_NUMBER + DESC
```

Problem 22 — Running total

```sql
SELECT
    month,
    sales,
    SUM(sales) OVER (
        ORDER BY month
    ) AS running_total
FROM monthly_sales;
```

Example:

```
Month   Sales   Running
Jan     100     100
Feb     200     300
Mar     150     450
```

Problem 23 — Customers' percentage of total sales

```sql
WITH customer_sales AS (
    SELECT
        c.customer_id,
        c.customer_name,
        SUM(o.quantity * p.price) AS sales
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY
        c.customer_id,
        c.customer_name
)
SELECT
    customer_name,
    sales,
    ROUND(
        100 * sales / SUM(sales) OVER (),
        2
    ) AS sales_percentage
FROM customer_sales;
```

এখানে:

```
SUM(sales) OVER ()
```

মানে পুরো result-এর total।

Problem 24 — Customers whose latest order was more than 30 days ago

প্রথমে latest order:

```
WITH latest_orders AS (
    SELECT
        customer_id,
        MAX(order_date) AS latest_order_date
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    c.customer_name,
    l.latest_order_date
FROM customers c
JOIN latest_orders l
    ON c.customer_id = l.customer_id
WHERE l.latest_order_date < CURRENT_DATE - INTERVAL 30 DAY;
```

এটা customer retention / inactive customer analysis-এ useful।

Problem 25 — Category contribution to total sales

```
WITH category_sales AS (
    SELECT
        p.category,
        SUM(o.quantity * p.price) AS sales
    FROM orders o
    JOIN products p
        ON o.product_id = p.product_id
    GROUP BY p.category
)
SELECT
    category,
    sales,
    ROUND(
```

```
        100 * sales / SUM(sales) OVER (),
        2
    ) AS contribution_percent
FROM category_sales
ORDER BY sales DESC;
```

🔥 Interview-এর Most Important SQL Patterns

এখন পর্যন্ত শেখা problemগুলোকে pattern হিসেবে মনে রাখো।

Pattern 1 — Top N

```
ORDER BY value DESC
LIMIT N;
```

Pattern 2 — Top N per group

```
ROW_NUMBER()
OVER (
    PARTITION BY group
    ORDER BY value DESC
)
```

Pattern 3 — Rank with ties

```
RANK()
OVER (
    PARTITION BY group
    ORDER BY value DESC
)
```

Pattern 4 — Previous row

```
LAG(value)
OVER (
```

```
    ORDER BY date
)
```

Pattern 5 — Next row

```
LEAD(value)
OVER (
    ORDER BY date
)
```

Pattern 6 — Running total

```
SUM(value)
OVER (
    ORDER BY date
)
```

Pattern 7 — Group average comparison

```
AVG(value)
OVER (
    PARTITION BY group
)
```

Pattern 8 — No related record

```
NOT EXISTS (...)
```

অথবা:

```
LEFT JOIN ...
WHERE right.id IS NULL
```

Pattern 9 — Duplicate

```
GROUP BY column
HAVING COUNT(*) > 1
```

Pattern 10 — Previous period growth

```
LAG
+
calculation
+
NULLIF
```

🧠 SQL Interview Decision Tree

Question দেখলে নিজেকে জিজ্ঞেস করো:

```
Need individual rows?
        ↓
     SELECT

Need filtering?
        ↓
     WHERE

Need groups?
        ↓
   GROUP BY

Need aggregate filter?
        ↓
    HAVING

Need multiple tables?
        ↓
      JOIN

Need existence?
        ↓
     EXISTS

Need nested logic?
        ↓
```

```
    Subquery / CTE

Need ranking?
        ↓
 Window Function

Need previous/next?
        ↓
 LAG / LEAD

Need Top N per group?
        ↓
 RANK / DENSE_RANK / ROW_NUMBER

Need performance?
        ↓
 EXPLAIN + Index
```

⚠️ Common Interview Mistakes

Mistake 1

Aggregate condition-এ `WHERE` ব্যবহার করা।

❌

```
WHERE SUM(sales) > 100000
```

✅

```
HAVING SUM(sales) > 100000
```

Mistake 2

`COUNT(*)` এবং `COUNT(column)` -এর difference না বোঝা।

```
COUNT(*)       → rows
COUNT(column)  → non-NULL values
```

Mistake 3

`NOT IN` + NULL problem ভুলে যাওয়া।

Related-record existence problem-এ অনেক সময়:

```
NOT EXISTS
```

safer choice।

Mistake 4

Top N এবং Top N per group-কে একই মনে করা।

Overall Top 5:

```
ORDER BY salary DESC
LIMIT 5;
```

Each department-এর Top 5:

```
ROW_NUMBER()
OVER (
    PARTITION BY department
    ORDER BY salary DESC
)
```

Mistake 5

`RANK()` বনাম `ROW_NUMBER()` না বোঝা।

```
ROW_NUMBER
→ unique sequential number

RANK
→ ties get same rank, gaps possible

DENSE_RANK
→ ties get same rank, no gaps
```

Example:

```
Salary
100
90
90
80
```

```
ROW_NUMBER  → 1,2,3,4
RANK        → 1,2,2,4
DENSE_RANK  → 1,2,2,3
```

⭐ Class 55 Golden Rules

```
Simple filter
→ WHERE

Group filter
→ HAVING

Multiple tables
→ JOIN

Existence
→ EXISTS

No existence
→ NOT EXISTS

Top N
→ ORDER BY + LIMIT

Top N per group
→ Window Function

Previous row
→ LAG

Next row
→ LEAD

Running total
→ SUM() OVER()

Duplicate
→ GROUP BY + HAVING COUNT(*) > 1
```

```
Performance
→ EXPLAIN
```

📝 Class 55 Challenge Set

এগুলো নিজে solve করার চেষ্টা করো।

Challenge 1

প্রতিটি department-এর **second highest salary** বের করো।

Challenge 2

যেসব customer **কখনো order করেনি** তাদের বের করো।

Challenge 3

প্রতিটি customer-এর **latest order** বের করো।

Challenge 4

প্রতিটি category-এর **top 3 products by sales** বের করো।

Challenge 5

প্রতি মাসের sales এবং **previous month's sales** দেখাও।

Challenge 6

প্রতি মাসের **sales growth percentage** বের করো।

Challenge 7

যেসব customer-এর total spending **average customer spending-এর চেয়ে বেশি**, তাদের বের করো।

Challenge 8

Duplicate customer email বের করো।

Challenge 9

প্রতিটি department-এর employee count এবং average salary বের করো, কিন্তু শুধু যেসব department-এ **at least 5 employees** আছে।

Challenge 10 ⭐

প্রতিটি customer-এর:

```
customer_name
total_orders
total_sales
average_order_value
latest_order_date
customer_rank_by_sales
```

এক query-তে বের করার চেষ্টা করো।

🎯 Class 55 MCQ

1. Aggregate result filter করার জন্য কোনটি ব্যবহার করা হয়?

A. WHERE
B. HAVING
C. ORDER BY
D. JOIN

2. Top 5 overall records বের করতে সবচেয়ে simple pattern কোনটি?

A. `GROUP BY + HAVING`
B. `ORDER BY DESC + LIMIT 5`
C. `LAG()`
D. `EXISTS`

3. প্রতিটি department-এর top 3 employee বের করতে কোনটি সবচেয়ে suitable?

A. LIMIT 3
B. ROW_NUMBER/RANK/DENSE_RANK + PARTITION BY
C. DISTINCT
D. UNION

4. Previous row-এর value পেতে কোন function?

A. LEAD
B. LAG
C. RANK
D. FIRST_VALUE

5. Duplicate value খুঁজতে কোন pattern useful?

A. `GROUP BY + HAVING COUNT(*) > 1`
B. `ORDER BY + LIMIT`
C. `LAG()`
D. `EXISTS`

6. `ROW_NUMBER()` -এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য কী?

A. Tie হলে একই number দেয়
B. প্রতিটি row-কে unique sequence number দেয়
C. শুধু aggregate করে
D. শুধু date-এর জন্য কাজ করে

7. Query performance investigate করতে কোনটি সবচেয়ে useful?

A. DISTINCT
B. EXPLAIN
C. UNION
D. CASE

🔥 Final Revision

Class 31 থেকে Class 55 পর্যন্ত তোমার SQL journey এখন:

```
SQL Basics
   ↓
CRUD
   ↓
GROUP BY
   ↓
JOIN
   ↓
Subquery
   ↓
CTE
   ↓
Window Functions
   ↓
Date/String/NULL
   ↓
Views
   ↓
Procedures/Functions
   ↓
Transactions
   ↓
Triggers
   ↓
Indexes
   ↓
EXPLAIN
   ↓
```

```
Normalization
   ↓
Database Design
   ↓
Star Schema
   ↓
⭐ Interview Problem Solving
```

Class 56 হবে — Full SQL Data Analyst Project: একটি বাস্তব Sales Analytics database নিয়ে শুরু থেকে data analysis, KPI, customer analysis, product analysis, monthly trend, ranking এবং final dashboard-ready SQL তৈরি।

📘 Class 56 — Full SQL Data Analyst Project

আজ আমরা **একটা complete Sales Analytics Project** করব।
এটা শুধু SQL syntax practice না—এখানে তুমি একজন **Junior Data Analyst** হিসেবে business data থেকে answer বের করবে।

আজকের Project

> **E-Commerce Sales Analytics**

আমরা বের করব:

```
Sales KPI
↓
Monthly Sales
↓
Customer Analysis
↓
Product Analysis
↓
Category Analysis
↓
Top Customers
↓
Top Products
↓
Customer Ranking
↓
Growth Analysis
↓
Business Insights
```

---

1. Project Database Design

আমাদের 3টি main table থাকবে:

```
customers
products
orders
```

Relationship:

```
customers
    |
    | 1
    |
    | N
  orders
    |
    | N
    |
    | 1
 products
```

অর্থাৎ:

```
Customer 1 ——— N Orders
Product  1 ——— N Orders
```

---

2. Create Tables

Customers

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    city VARCHAR(50),
    email VARCHAR(100),
    signup_date DATE
);
```

Products

```sql
CREATE TABLE products (
    product_id INT PRIMARY KEY,
```

```
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2)
);
```

Orders

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    order_date DATE,
    quantity INT,

    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id),

    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

---

3. Sample Data

Customers

```
INSERT INTO customers
(customer_id, customer_name, city, email, signup_date)
VALUES
(1, 'Rahim', 'Dhaka', 'rahim@gmail.com', '2025-01-10'),
(2, 'Karim', 'Chittagong', 'karim@gmail.com', '2025-02-15'),
(3, 'Sadia', 'Dhaka', 'sadia@gmail.com', '2025-03-20'),
(4, 'Nadia', 'Sylhet', 'nadia@gmail.com', '2025-04-12'),
(5, 'Hasan', 'Dhaka', 'hasan@gmail.com', '2025-05-01'),
(6, 'Tanvir', 'Rajshahi', 'tanvir@gmail.com', '2025-06-10');
```

Products

```
INSERT INTO products
(product_id, product_name, category, price)
VALUES
(1, 'Laptop', 'Electronics', 80000),
(2, 'Phone', 'Electronics', 40000),
(3, 'Headphone', 'Electronics', 5000),
(4, 'Chair', 'Furniture', 10000),
```

```
(5, 'Table', 'Furniture', 20000),
(6, 'Keyboard', 'Accessories', 3000);
```

Orders

```
INSERT INTO orders
(order_id, customer_id, product_id, order_date, quantity)
VALUES
(1, 1, 1, '2026-01-05', 1),
(2, 2, 2, '2026-01-10', 2),
(3, 1, 3, '2026-01-15', 2),
(4, 3, 4, '2026-01-20', 1),
(5, 4, 5, '2026-02-05', 2),
(6, 2, 6, '2026-02-10', 3),
(7, 5, 2, '2026-02-15', 1),
(8, 1, 2, '2026-03-01', 1),
(9, 3, 1, '2026-03-10', 1),
(10, 5, 4, '2026-03-15', 2),
(11, 2, 3, '2026-03-20', 2),
(12, 4, 1, '2026-04-05', 1);
```

---

4. প্রথম কাজ — Understand the Data

প্রথমে কখনো complex query লিখবে না।

Check:

```
SELECT *
FROM customers;
```

```
SELECT *
FROM products;
```

```
SELECT *
FROM orders;
```

তারপর row count:

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

```sql
SELECT COUNT(*) AS total_products
FROM products;
```

```sql
SELECT COUNT(*) AS total_orders
FROM orders;
```

---

5. Revenue Calculation

Orders table-এ revenue নেই।

আমাদের calculate করতে হবে:

```
Revenue = quantity × product price
```

Query:

```sql
SELECT
    o.order_id,
    o.order_date,
    p.product_name,
    o.quantity,
    p.price,
    o.quantity * p.price AS revenue
FROM orders o
JOIN products p
    ON o.product_id = p.product_id;
```

এটা project-এর সবচেয়ে important calculation।

---

6. ⭐ Create Sales View

যেহেতু একই JOIN বারবার লাগবে, আমরা একটা View বানাতে পারি।

```sql
CREATE VIEW sales_data AS
SELECT
    o.order_id,
    o.order_date,

    c.customer_id,
    c.customer_name,
    c.city,
```

```
    p.product_id,
    p.product_name,
    p.category,
    p.price,

    o.quantity,

    o.quantity * p.price AS revenue

FROM orders o

JOIN customers c
    ON o.customer_id = c.customer_id

JOIN products p
    ON o.product_id = p.product_id;
```

এখন:

```
SELECT *
FROM sales_data;
```

আমাদের analysis সহজ হয়ে গেল।

---

7. KPI — Key Performance Indicators

একটি business dashboard-এর প্রথম অংশ সাধারণত KPI।

আমরা বের করব:

```
Total Revenue
Total Orders
Total Customers
Total Quantity
Average Order Value
```

Total Revenue

```
SELECT
    SUM(revenue) AS total_revenue
FROM sales_data;
```

Total Orders

```sql
SELECT
    COUNT(DISTINCT order_id) AS total_orders
FROM sales_data;
```

Total Quantity

```sql
SELECT
    SUM(quantity) AS total_quantity
FROM sales_data;
```

Average Order Value

```sql
SELECT
    SUM(revenue)
    / COUNT(DISTINCT order_id) AS average_order_value
FROM sales_data;
```

---

8. KPI এক Query-তে

Real dashboard-এর জন্য:

```sql
SELECT
    SUM(revenue) AS total_revenue,

    COUNT(DISTINCT order_id) AS total_orders,

    COUNT(DISTINCT customer_id) AS total_customers,

    SUM(quantity) AS total_quantity,

    ROUND(
        SUM(revenue)
        / COUNT(DISTINCT order_id),
        2
    ) AS average_order_value

FROM sales_data;
```

এটাই একটি basic **Sales KPI Query**।

---

9. Monthly Sales Analysis

Business জানতে চাইবে:

> কোন মাসে কত sales হয়েছে?

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    SUM(revenue) AS monthly_sales
FROM sales_data
GROUP BY
    DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Result concept:

```
Month       Sales
--------------------
2026-01     ...
2026-02     ...
2026-03     ...
2026-04     ...
```

---

10. Monthly Orders

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    COUNT(DISTINCT order_id) AS total_orders
FROM sales_data
GROUP BY
    DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

---

11. Monthly Average Order Value

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,

    SUM(revenue) AS sales,

    COUNT(DISTINCT order_id) AS orders,

    ROUND(
        SUM(revenue) /
```

```
        COUNT(DISTINCT order_id),
        2
    ) AS average_order_value

FROM sales_data

GROUP BY
    DATE_FORMAT(order_date, '%Y-%m')

ORDER BY month;
```

---

12. 📈 Month-over-Month Growth

এখন আমাদের analytics একটু advanced হবে।

```
WITH monthly_sales AS (

    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(revenue) AS sales

    FROM sales_data

    GROUP BY
        DATE_FORMAT(order_date, '%Y-%m')
),

sales_with_previous AS (

    SELECT
        month,
        sales,

        LAG(sales) OVER (
            ORDER BY month
        ) AS previous_sales

    FROM monthly_sales
)

SELECT
    month,
    sales,
    previous_sales,

    ROUND(
```

```
        100 * (sales - previous_sales)
        / NULLIF(previous_sales, 0),
        2
    ) AS growth_percent

FROM sales_with_previous;
```

এখানে একসাথে:

```
View
+
CTE
+
GROUP BY
+
LAG
+
NULLIF
+
Calculation
```

ব্যবহার হয়েছে।

---

13. Customer Sales Analysis

Business question:

> কোন customer সবচেয়ে বেশি টাকা spend করেছে?

```
SELECT
    customer_id,
    customer_name,
    SUM(revenue) AS total_spending
FROM sales_data
GROUP BY
    customer_id,
    customer_name
ORDER BY total_spending DESC;
```

---

14. Top 5 Customers

```sql
SELECT
    customer_id,
    customer_name,
    SUM(revenue) AS total_spending
FROM sales_data
GROUP BY
    customer_id,
    customer_name
ORDER BY total_spending DESC
LIMIT 5;
```

---

15. Customer Order Frequency

```sql
SELECT
    customer_id,
    customer_name,
    COUNT(DISTINCT order_id) AS total_orders
FROM sales_data
GROUP BY
    customer_id,
    customer_name
ORDER BY total_orders DESC;
```

---

16. Customer Average Order Value

```sql
SELECT
    customer_id,
    customer_name,

    SUM(revenue) AS total_sales,

    COUNT(DISTINCT order_id) AS total_orders,

    ROUND(
        SUM(revenue)
        / COUNT(DISTINCT order_id),
        2
    ) AS average_order_value

FROM sales_data

GROUP BY
```

```
    customer_id,
    customer_name

ORDER BY total_sales DESC;
```

---

17. Customer Ranking

এখন window function ব্যবহার করব।

```
WITH customer_sales AS (

    SELECT
        customer_id,
        customer_name,
        SUM(revenue) AS total_sales

    FROM sales_data

    GROUP BY
        customer_id,
        customer_name
)

SELECT
    customer_id,
    customer_name,
    total_sales,

    RANK() OVER (
        ORDER BY total_sales DESC
    ) AS sales_rank

FROM customer_sales;
```

Result:

```
Customer    Sales       Rank
-----------------------------
Rahim       ...          1
Karim       ...          2
Sadia       ...          3
...
```

---

18. Customer Contribution %

এখন জানতে চাই:

> Total sales-এর কত % কোন customer generate করেছে?

```
WITH customer_sales AS (

    SELECT
        customer_id,
        customer_name,
        SUM(revenue) AS total_sales

    FROM sales_data

    GROUP BY
        customer_id,
        customer_name
)

SELECT
    customer_name,
    total_sales,

    ROUND(
        100 * total_sales
        / SUM(total_sales) OVER (),
        2
    ) AS contribution_percent

FROM customer_sales

ORDER BY total_sales DESC;
```

---

19. Product Analysis

সবচেয়ে বেশি quantity কোন product বিক্রি হয়েছে?

```
SELECT
    product_id,
    product_name,
    SUM(quantity) AS total_quantity
FROM sales_data
GROUP BY
    product_id,
```

```
    product_name
ORDER BY total_quantity DESC;
```

---

20. Product Revenue Ranking

```
WITH product_sales AS (

    SELECT
        product_id,
        product_name,
        SUM(quantity) AS quantity_sold,
        SUM(revenue) AS total_revenue

    FROM sales_data

    GROUP BY
        product_id,
        product_name
)

SELECT
    *,
    RANK() OVER (
        ORDER BY total_revenue DESC
    ) AS revenue_rank

FROM product_sales;
```

---

21. Category Analysis

```
SELECT
    category,
    SUM(quantity) AS total_quantity,
    SUM(revenue) AS total_sales
FROM sales_data
GROUP BY category
ORDER BY total_sales DESC;
```

এতে business জানতে পারবে:

> কোন category সবচেয়ে profitable/revenue-generating?

Note: এখানে আমরা `profit` calculate করছি না—তাই technically এটা **sales/revenue analysis**, profit analysis নয়।

---

22. Category Contribution %

```sql
WITH category_sales AS (

    SELECT
        category,
        SUM(revenue) AS sales

    FROM sales_data

    GROUP BY category
)

SELECT
    category,
    sales,

    ROUND(
        100 * sales /
        SUM(sales) OVER (),
        2
    ) AS contribution_percent

FROM category_sales

ORDER BY sales DESC;
```

---

23. Top Product per Category ⭐

এটা interview + analyst দুটোতেই important।

```sql
WITH product_sales AS (

    SELECT
        category,
        product_id,
        product_name,
        SUM(revenue) AS sales

    FROM sales_data
```

```
    GROUP BY
        category,
        product_id,
        product_name
),

ranked_products AS (

    SELECT
        *,
        RANK() OVER (
            PARTITION BY category
            ORDER BY sales DESC
        ) AS rnk

    FROM product_sales
)

SELECT *
FROM ranked_products
WHERE rnk = 1;
```

Logic:

```
Product Sales
     ↓
Group by Category
     ↓
Rank products
     ↓
Take Rank = 1
```

---

24. City-wise Sales

```
SELECT
    city,
    SUM(revenue) AS total_sales,
    COUNT(DISTINCT customer_id) AS customers
FROM sales_data
GROUP BY city
ORDER BY total_sales DESC;
```

এটা geographical/business analysis।

25. Customer Segmentation

আমরা customer-দের spending অনুযায়ী segment করতে পারি।

```
WITH customer_sales AS (

    SELECT
        customer_id,
        customer_name,
        SUM(revenue) AS total_sales

    FROM sales_data

    GROUP BY
        customer_id,
        customer_name
)

SELECT
    customer_id,
    customer_name,
    total_sales,

    CASE

        WHEN total_sales >= 100000
            THEN 'VIP'

        WHEN total_sales >= 50000
            THEN 'High Value'

        WHEN total_sales >= 20000
            THEN 'Medium Value'

        ELSE 'Low Value'

    END AS customer_segment

FROM customer_sales;
```

Important

এখানে threshold example হিসেবে দেওয়া হয়েছে। Real business-এ threshold business requirement/data distribution অনুযায়ী ঠিক করতে হবে।

26. Inactive Customers

যারা কোনো order করেনি:

```
SELECT
    c.customer_id,
    c.customer_name
FROM customers c
WHERE NOT EXISTS (

    SELECT 1

    FROM orders o

    WHERE o.customer_id = c.customer_id
);
```

এটা **customer retention analysis**-এর basic অংশ।

---

27. Latest Order per Customer

```
SELECT
    customer_id,
    customer_name,
    MAX(order_date) AS latest_order_date
FROM sales_data
GROUP BY
    customer_id,
    customer_name;
```

---

28. Customer Activity Status

```
SELECT
    customer_id,
    customer_name,
    MAX(order_date) AS latest_order_date,

    CASE

        WHEN MAX(order_date)
             >= CURRENT_DATE - INTERVAL 30 DAY
        THEN 'Active'
```

```
        ELSE 'Inactive'

    END AS status

FROM sales_data

GROUP BY
    customer_id,
    customer_name;
```

Note: sample dataset-এর historical dates হলে current date-এর সাথে compare করলে সবাই inactive হতে পারে। Real project-এ analysis date বা reporting period carefully define করতে হবে।

---

29. Running Total of Sales

```
WITH monthly_sales AS (

    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(revenue) AS sales

    FROM sales_data

    GROUP BY
        DATE_FORMAT(order_date, '%Y-%m')
)

SELECT
    month,
    sales,

    SUM(sales) OVER (
        ORDER BY month
    ) AS cumulative_sales

FROM monthly_sales;
```

---

30. Moving Average

3-month moving average:

```sql
WITH monthly_sales AS (

    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,
        SUM(revenue) AS sales

    FROM sales_data

    GROUP BY
        DATE_FORMAT(order_date, '%Y-%m')
)

SELECT
    month,
    sales,

    AVG(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING
        AND CURRENT ROW
    ) AS moving_average

FROM monthly_sales;
```

---

31. Final Dashboard Dataset ⭐

একজন Data Analyst হিসেবে তুমি BI tool-এ dashboard বানাতে চাইলে এমন summary dataset useful হবে।

```sql
WITH customer_summary AS (

    SELECT
        customer_id,
        customer_name,

        COUNT(DISTINCT order_id) AS total_orders,

        SUM(revenue) AS total_sales,

        MAX(order_date) AS latest_order

    FROM sales_data

    GROUP BY
        customer_id,
        customer_name
```

```
)

SELECT
    customer_id,
    customer_name,
    total_orders,
    total_sales,

    ROUND(
        total_sales / total_orders,
        2
    ) AS average_order_value,

    latest_order,

    RANK() OVER (
        ORDER BY total_sales DESC
    ) AS sales_rank

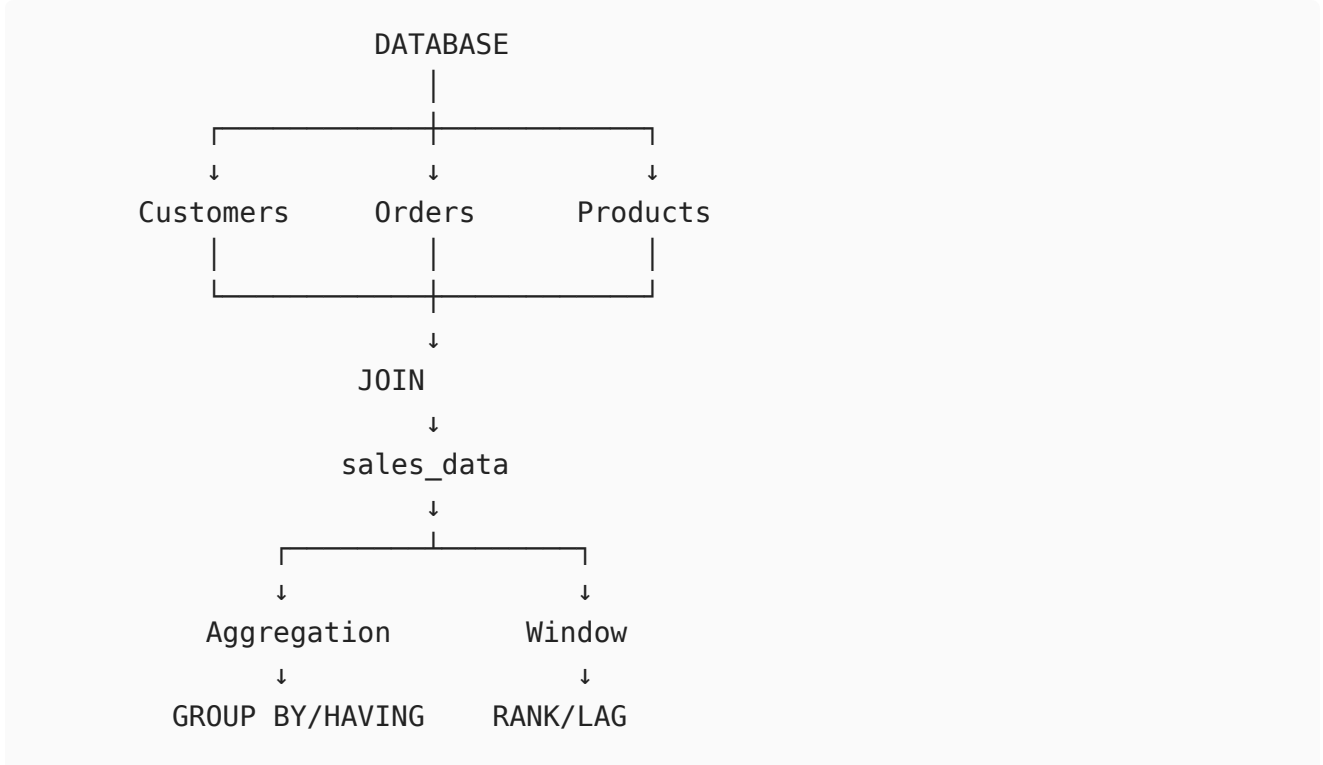
FROM customer_summary

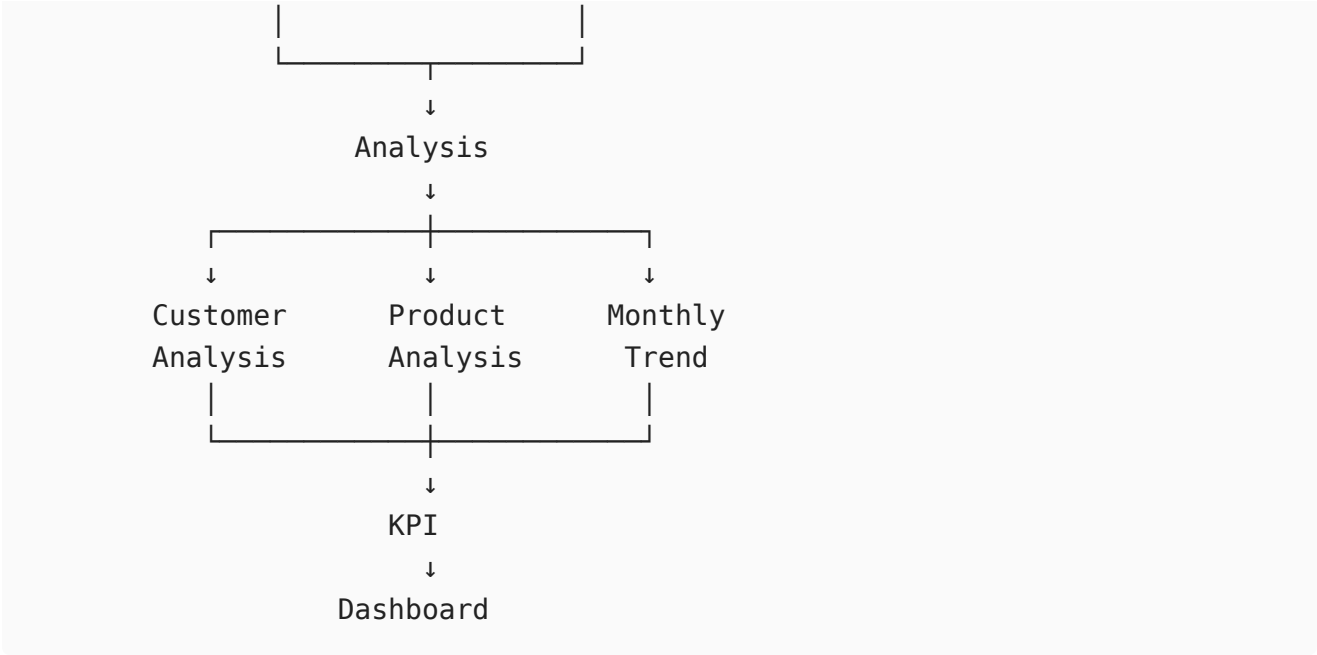
ORDER BY sales_rank;
```

এই output Excel / Power BI / Tableau-তে নিয়ে dashboard তৈরি করা যায়।

🧠 Project Architecture

আজকের project-এর পুরো flow:





⭐ Class 56 — Most Important Queries

তুমি এই 10টা pattern ভালোভাবে আয়ত্ত করলে অনেক Data Analyst SQL problem solve করতে পারবে:

```
1. JOIN
2. GROUP BY
3. HAVING
4. CTE
5. CASE
6. RANK()
7. ROW_NUMBER()
8. LAG()
9. SUM() OVER()
10. NOT EXISTS
```

⚠️ Common Mistakes

1. Revenue ভুলভাবে calculate করা

❌

```
SUM(price)
```

যদি multiple quantity থাকে।

✅

```
SUM(quantity * price)
```

---

2. Order count ভুল করা

যদি order table-এ multiple product row থাকে, তাহলে:

```
COUNT(*)
```

সবসময় unique order count দেবে না।

প্রয়োজনে:

```
COUNT(DISTINCT order_id)
```

---

3. Customer ranking-এর আগে aggregation না করা

❌ সরাসরি raw order-এর উপর customer ranking।

সঠিক:

```
Orders
 ↓
Customer-wise SUM
 ↓
RANK
```

---

4. `WHERE` বনাম `HAVING`

```
WHERE  → individual rows filter
HAVING → groups filter
```

---

5. Profit আর Revenue একই মনে করা

```
Revenue = Selling amount
```

```
Profit = Revenue - Cost
```

আমাদের বর্তমান database-এ `cost` নেই, তাই আমরা **sales/revenue analytics** করছি।

📝 Class 56 Assignment — Full Analyst Challenge

এবার তোমার জন্য বাস্তব assignment। **নিজে query লিখবে।**

Level 1

1. Total revenue বের করো।

2. Total number of orders বের করো।

3. প্রতিটি product-এর total quantity sold বের করো।

4. প্রতিটি category-এর total revenue বের করো।

Level 2

5. Top 3 customers by revenue বের করো।

6. প্রতিটি customer-এর total orders এবং total revenue বের করো।

7. প্রতিটি city-এর total revenue বের করো।

8. যেসব customer 2টির বেশি order করেছে তাদের বের করো।

Level 3

9. প্রতিটি category-এর highest-revenue product বের করো।

10. প্রতিটি customer-এর sales rank বের করো।

11. Monthly sales এবং previous month sales বের করো।

12. Monthly growth percentage বের করো।

🔥 Level 4 — Interview Challenge

13. প্রতিটি customer-এর:

```
customer_name
total_orders
total_sales
average_order_value
latest_order_date
sales_rank
```

একটি query-তে বের করো।

14. প্রতিটি category-এর:

```
category
total_sales
sales_percentage
rank
```

বের করো।

15. এমন customer বের করো যাদের spending **average customer spending-এর চেয়ে বেশি**।

🎯 Class 56 MCQ

1. Revenue সাধারণত কীভাবে calculate করা হচ্ছে?

A. `price + quantity`
B. `price - quantity`
C. `price * quantity`
D. `price / quantity`

2. Unique order count-এর জন্য কোনটি safer?

A. `COUNT(*)`
B. `COUNT(DISTINCT order_id)`
C. `SUM(order_id)`
D. `AVG(order_id)`

3. Customer ranking করার আগে সাধারণত কী করতে হবে?

A. Customer-wise aggregate
B. DELETE
C. INSERT
D. DROP

4. Previous month's sales পেতে কোন function?

A. LEAD
B. LAG
C. RANK
D. COUNT

5. Category-এর top product বের করতে কোন pattern useful?

A. `LIMIT 1`
B. `RANK() OVER(PARTITION BY category...)`
C. `COUNT(*)`
D. `DISTINCT`

6. Revenue এবং Profit-এর মধ্যে সঠিক সম্পর্ক কোনটি?

A. সবসময় একই
B. Profit = Revenue + Cost
C. Profit = Revenue − Cost
D. Revenue = Profit − Cost

🏆 Class 56 Golden Rule

```
Raw Tables
    ↓
JOIN
    ↓
Create Analysis Dataset/View
    ↓
Aggregate
    ↓
Window Functions
    ↓
KPI
    ↓
Business Questions
    ↓
Insights
    ↓
Dashboard
```

Class 57 হবে — Full Database Designer Project: requirement থেকে শুরু করে **ER Diagram → entities → relationships → normalization → PK/FK → constraints → indexes → SQL schema → production-ready database design**।

হ্যাঁ, **আরও class লাগবে**—তবে এখন তোমার SQL foundation অনেক strong হয়ে গেছে। Class 31–58 পর্যন্ত তুমি SQL + Database Design-এর বড় অংশ cover করেছ।

আমি এখন আর ২০–৩০টা random class না করে **আরও প্রায় 12টি focused class** করার পর একটি solid Database Designer/Junior Database Engineer level-এ যেতে বলব।

📚 Proposed Final Database Roadmap

Class	Topic
59	Advanced Indexing & Query Performance
60	EXPLAIN / EXPLAIN ANALYZE + Slow Query Optimization
61	Advanced SQL: Cursors, Recursive CTE, Advanced Window
62	Advanced Transactions & Isolation
63	Locking, Deadlock & Concurrency — Advanced
64	Database Partitioning
65	Database Security, Users, Roles & Privileges
66	Backup, Restore & Disaster Recovery
67	PostgreSQL Advanced Features
68	Database Scaling, Replication & Read/Write Architecture
69	Production Database Design & Best Practices
70	**Final Database Designer Project** ⭐

তারপর?

Class 70-এর project-এ আমরা একদম বাস্তব project design করব:

```
Business Requirement
        ↓
ER Diagram
        ↓
Normalization
        ↓
Schema
        ↓
PK / FK / Constraints
        ↓
Indexes
        ↓
Transactions
        ↓
Security
        ↓
```

```
Performance
        ↓
PostgreSQL
        ↓
Django/DRF Integration
        ↓
Production Database
```

🎯 আমার recommendation

তোমার লক্ষ্য যেহেতু **Database Designer** হওয়া, তাই **Class 70 পর্যন্ত যাওয়া worth it**।

এরপর আর শুধু SQL class চালিয়ে যাওয়ার দরকার নেই। বরং:

SQL + Database Design → PostgreSQL → Real Projects → Data Engineering basics

এই path-এ যাওয়া বেশি useful হবে।

তাই next হবে Class 59 — Advanced Indexing & Query Performance.

📚 Class 59 — Advanced Indexing & Query Performance

আজকের class খুব important, কারণ **Database Designer** হিসেবে শুধু query লিখলেই হবে না—query **কত দ্রুত execute হচ্ছে** সেটাও বুঝতে হবে।

আজ আমরা শিখব:

1. B-Tree / B+Tree
2. Composite Index
3. Leftmost Prefix Rule
4. Covering Index
5. Clustered vs Secondary Index
6. Index Selectivity
7. EXPLAIN ANALYZE
8. Slow Query
9. Filesort
10. Temporary Table
11. Pagination Optimization
12. Keyset Pagination

1️⃣ প্রথমে বুঝি — Index আসলে কেন দরকার?

ধরো `customers` table-এ:

```
10,000,000 customers
```

তুমি query করলে:

```sql
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

Index না থাকলে database-কে অনেক row পরীক্ষা করতে হতে পারে।

এটাকে সহজভাবে ভাবো:

```
No Index

Row 1  → check
Row 2  → check
Row 3  → check
...
Row 10,000,000 → check
```

Index থাকলে:

```
Index
  ↓
email = rahim@gmail.com
  ↓
matching row
```

অর্থাৎ:

> **Index হলো database-এর জন্য একটি দ্রুত lookup structure।**

2️⃣ B-Tree / B+Tree কী?

Relational database-এ সাধারণ index implementation হিসেবে **B-Tree family** খুব common।

একটা সহজ ধারণা:

```
                [50]
              /      \
        [20,30]      [70,80]
```

```
        /  |  \      / |  \
      ... ... ...   ... ... ...
```

Database সরাসরি সব record search না করে tree structure ব্যবহার করে দ্রুত target location খুঁজতে পারে।

কেন Tree?

কারণ data যদি sorted structure-এ থাকে:

```
10
20
30
40
50
60
70
80
90
```

তাহলে binary-search-এর মতো ধারণা ব্যবহার করে দ্রুত search করা যায়।

3️⃣ B+Tree ধারণা

Database indexing-এর ক্ষেত্রে **B+Tree** ধারণাটা বিশেষভাবে important।

Simplified structure:

```
            Root
             |
      ----------------
      |              |
  Internal       Internal
      |              |
 -----------    -----------
 | | | | | |    | | | | | |
 Leaf nodes → → → → → → → →
```

Leaf nodes-এ sorted key/value references থাকে এবং leaf level সাধারণত sequential traversal-এর জন্য linked থাকে।

তাই range query-এর জন্য খুব useful:

```sql
SELECT *
FROM products
```

```
WHERE price BETWEEN 1000 AND 5000;
```

Index সাহায্য করতে পারে:

```
1000 → 1200 → 1500 → 2000 → 2500 → ... → 5000
```

4️⃣ সব query-তে Index দরকার?

❌ না।

এটা খুব important।

ধরো table:

```
100 rows
```

এখানে full table scan অনেক সময় index ব্যবহারের চেয়েও সহজ/দ্রুত হতে পারে।

কিন্তু:

```
50 million rows
```

এখানে ভালো index huge difference করতে পারে।

Golden Rule

> **Index blindly create করবে না। Query pattern দেখে index design করবে।**

5️⃣ Composite Index

একাধিক column নিয়ে তৈরি index:

```
CREATE INDEX idx_customer_date
ON orders(customer_id, order_date);
```

এখানে:

```
customer_id
      +
order_date
```

একসাথে index করা হয়েছে।

6️⃣ Composite Index কেন দরকার?

ধরো query:

```
SELECT *
FROM orders
WHERE customer_id = 10
AND order_date >= '2026-01-01';
```

এই query-এর জন্য:

```
(customer_id, order_date)
```

index খুব useful হতে পারে।

কারণ index structure:

```
customer_id
     ↓
order_date
     ↓
matching rows
```

7️⃣ সবচেয়ে Important — Leftmost Prefix Rule ⭐

ধরো:

```
CREATE INDEX idx_orders
ON orders(customer_id, order_date, status);
```

Column order:

```
1. customer_id
2. order_date
3. status
```

এখন কোন query index-এর leading part ব্যবহার করতে পারবে?

✅ ভালো candidates

```
WHERE customer_id = 10
```

```
WHERE customer_id = 10
AND order_date = '2026-09-14'
```

```
WHERE customer_id = 10
AND order_date = '2026-09-14'
AND status = 'CONFIRMED'
```

কারণ তারা শুরু করছে:

```
customer_id
```

দিয়ে।

⚠️ শুধু order_date

```
WHERE order_date = '2026-09-14'
```

এই composite index-এর জন্য সাধারণত leading-column সুবিধা পাবে না।

কারণ:

```
(customer_id, order_date, status)
  ↑
প্রথম column বাদ
```

⚠️ শুধু status

```
WHERE status = 'CONFIRMED'
```

এটাও leading prefix নয়।

8️⃣ Column Order কেন এত important?

ধরো:

```
INDEX(customer_id, order_date)
```

এবং

```
customer_id
    ↓
order_date
```

Database প্রথমে customer অনুযায়ী organize করতে পারে, তারপর customer-এর ভিতরে date অনুযায়ী।

কিন্তু:

```
INDEX(order_date, customer_id)
```

হলে structure-এর priority বদলে যায়।

তাই:

> **Composite index-এর column order query pattern-এর উপর নির্ভর করে।**

9️⃣ Covering Index ⭐

এটা advanced কিন্তু খুব important।

ধরো:

```
CREATE INDEX idx_customer_email
ON customers(customer_id, email);
```

Query:

```
SELECT customer_id, email
FROM customers
WHERE customer_id = 10;
```

Query-এর জন্য দরকার:

```
customer_id
email
```

দুটোই index-এর মধ্যে আছে।

তাই database অনেক ক্ষেত্রে মূল table-এ গিয়ে row fetch না করেই index থেকেই result দিতে পারে।

এটাকে বলে:

> **Covering Index**

সহজভাবে

Normal:

```
Index
 ↓
Find row
 ↓
Go to table
 ↓
Get data
```

Covering:

```
Index
 ↓
Find + return required data
```

এতে I/O কমতে পারে।

🔟 Clustered Index

এখন খুব important distinction।

MySQL InnoDB-তে primary key clustered index-এর ভিত্তি হিসেবে কাজ করে।

Conceptually:

```
Primary Key Index
       ↓
actual table data
```

অর্থাৎ primary key-এর B-tree structure-এর leaf level-এ row data থাকে।

ধরো:

```sql
CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
```

এখানে InnoDB-তে:

```
PRIMARY KEY (id)
        ↓
Clustered Index
        ↓
Table data
```

1️⃣1️⃣ Secondary Index

Primary key ছাড়া অন্য index:

```sql
CREATE INDEX idx_email
ON users(email);
```

এটা:

```
Secondary Index
```

InnoDB-তে secondary index-এর leaf entries সাধারণত primary key value-ও ধরে।

Conceptually:

```
email index
     ↓
email → primary_key
             ↓
        clustered data
```

তাই secondary index দিয়ে search করার পরে প্রয়োজন হলে primary-key lookup করতে হতে পারে।

1️⃣2️⃣ Clustered vs Secondary

বিষয়	Clustered	Secondary
Data	leaf-এ table data	সাধারণত index entries
InnoDB PK	হ্যাঁ	না
Primary key	ভিত্তি	অন্য index
Table lookup	কম হতে পারে	প্রয়োজন হতে পারে
সংখ্যা	একটি clustered structure	একাধিক secondary index

Golden Rule

```
Primary Key → Clustered structure in InnoDB
Other indexes → Secondary indexes
```

1️⃣3️⃣ Index Selectivity ⭐

Selectivity মানে সহজভাবে:

> একটি column কতটা effectively rows আলাদা করতে পারে।

ধরো:

Gender

```
Male
Female
```

শুধু 2 ধরনের value।

Selectivity relatively low।

Email

```
rahim@gmail.com
karim@gmail.com
sadia@gmail.com
...
```

প্রায় প্রতিটি row আলাদা।

Selectivity high।

Example

10 million rows:

```
status:
ACTIVE     → 9,000,000
INACTIVE   → 1,000,000
```

এখানে:

```
WHERE status = 'ACTIVE'
```

index সবসময় huge benefit নাও দিতে পারে, কারণ matching rows অনেক বেশি।

অন্যদিকে:

```
WHERE email = 'x@example.com'
```

খুব selective।

1️⃣4️⃣ High vs Low Selectivity

```
High Selectivity
        ↓
Email
Phone
UUID
Unique ID

Low Selectivity
        ↓
Gender
Boolean
Status
```

তবে মনে রাখবে:

> **Low-selectivity column-এ index কখনোই useful নয়—এটা ভুল।**

Composite index, query pattern, table size, data distribution এবং optimizer decision সব matter করে।

1️⃣5️⃣ EXPLAIN — Database কী করছে দেখব কীভাবে?

Query:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

এতে database query execution plan সম্পর্কে information দেয়।

1️⃣6️⃣ EXPLAIN-এর গুরুত্বপূর্ণ অংশ

MySQL-এ তুমি এগুলো দেখবে:

```
id
select_type
table
type
possible_keys
key
key_len
ref
rows
filtered
Extra
```

সব মুখস্থ করার দরকার নেই। Importantগুলো আগে বুঝো।

`possible_keys`

Database কোন index ব্যবহার করতে **পারতে পারে**।

`key`

Database শেষ পর্যন্ত কোন index **choose করেছে**।

Example:

```
possible_keys → idx_email
key           → idx_email
```

Good sign হতে পারে।

কিন্তু:

```
possible_keys → idx_email
key           → NULL
```

মানে index available হলেও optimizer সেটা ব্যবহার করেনি।

1️⃣7️⃣ `rows`

Optimizer আনুমানিক কত rows examine করতে পারে।

Example:

```
rows = 5
```

সাধারণত:

```
rows = 5
```

better than:

```
rows = 5,000,000
```

কিন্তু এটা **estimate**, actual exact number নয়।

1️⃣8️⃣ EXPLAIN `type`

Important values:

```
const
eq_ref
ref
range
index
ALL
```

Roughly efficient → less efficient direction:

```
const
 ↓
eq_ref
 ↓
ref
 ↓
range
 ↓
index
 ↓
ALL
```

1️⃣9️⃣ `ALL` কী?

```
type = ALL
```

সাধারণত full table scan বোঝায়।

Example:

```
10 million rows
        ↓
check many/all rows
```

তবে ছোট table হলে:

```
100 rows
```

`ALL` দেখলেই panic করার দরকার নেই।

2️⃣0️⃣ `Using filesort`

ধরো:

```sql
SELECT *
FROM orders
ORDER BY order_date DESC;
```

যদি suitable index না থাকে, database-এর অতিরিক্ত sorting operation লাগতে পারে।

EXPLAIN-এর `Extra`-তে দেখতে পারো:

```
Using filesort
```

নাম দেখে ভাববে না যে অবশ্যই disk-এ file তৈরি হচ্ছে।

এটা MySQL-এর execution-plan terminology; sorting operation বোঝায়।

2️⃣1️⃣ `Using temporary`

কিছু query-তে intermediate result রাখার জন্য temporary structure ব্যবহার হতে পারে।

বিশেষ করে complex:

```
GROUP BY
ORDER BY
DISTINCT
JOIN
```

এর combination-এ দেখা যেতে পারে।

Example:

```sql
SELECT city, COUNT(*)
FROM customers
GROUP BY city
ORDER BY COUNT(*) DESC;
```

EXPLAIN-এ:

```
Using temporary
Using filesort
```

দুটোই দেখা যেতে পারে।

এটা দেখলেই query bad—এমন নয়।

2️⃣2️⃣ Query Optimization Workflow ⭐

এটা তোমার সবচেয়ে important workflow:

```
Slow Query
    ↓
EXPLAIN
```

```
    ↓
Check type
    ↓
Check possible_keys
    ↓
Check key
    ↓
Check rows
    ↓
Check Extra
    ↓
Optimize
    ↓
EXPLAIN Again
```

Golden Rule

> **Guess করে optimization নয় — EXPLAIN দিয়ে verify করবে।**

2️⃣3️⃣ Sargability

একটা গুরুত্বপূর্ণ performance concept।

ধরো:

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

এখানে indexed column-এর উপর function চালানো হচ্ছে।

অনেক ক্ষেত্রে এটা index-এর efficient use বাধাগ্রস্ত করতে পারে।

এর পরিবর্তে:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01';
```

এটা range-based এবং index-friendly হতে পারে।

2️⃣4️⃣ Pagination Problem

ধরো website-এ:

```
Page 1
Page 2
Page 3
...
Page 100000
```

Traditional pagination:

```sql
SELECT *
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 1000000;
```

Problem:

```
OFFSET 1,000,000
```

Database-কে অনেক preceding rows skip করতে হতে পারে।

Large dataset-এ এটা expensive হতে পারে।

2️⃣5️⃣ Traditional OFFSET Pagination

Example:

```sql
SELECT *
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 100;
```

Page 1:

```
OFFSET 0
```

Page 2:

```
OFFSET 20
```

Page 3:

```
OFFSET 40
```

Simple এবং common।

কিন্তু huge offset:

```
OFFSET 1,000,000
```

problematic হতে পারে।

2️⃣6️⃣ Keyset Pagination ⭐⭐⭐

Large dataset-এর জন্য powerful technique।

ধরো previous page-এর শেষ ID:

```
last_id = 100
```

Next page:

```
SELECT *
FROM products
WHERE product_id > 100
ORDER BY product_id
LIMIT 20;
```

Database সরাসরি:

```
id > 100
```

এর জায়গা থেকে continue করতে পারে।

2️⃣7️⃣ OFFSET vs Keyset

OFFSET

```
SELECT *
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 1000000;
```

Concept:

```
Skip 1,000,000
      ↓
Take 20
```

Keyset

```sql
SELECT *
FROM products
WHERE product_id > 1000000
ORDER BY product_id
LIMIT 20;
```

Concept:

```
Start after 1,000,000
        ↓
Take 20
```

2️⃣8️⃣ Keyset Pagination-এর সুবিধা

Large table:

```
100 million rows
```

Traditional:

```
OFFSET 90,000,000
```

খুব expensive হতে পারে।

Keyset:

```
WHERE id > last_seen_id
```

অনেক বেশি scalable হতে পারে, যদি appropriate index থাকে।

2️⃣9️⃣ Django/DRF-এ কেন Important?

তুমি যেহেতু Django/DRF করো, production API-তে এটা খুব useful।

Example:

```
GET /api/products/?cursor=...
```

DRF-এর cursor-based pagination এই ধরনের scalability concept ব্যবহার করে।

তাই database performance জানা থাকলে backend API design-ও better হবে।

3️⃣0️⃣ Hotel Project-এ Apply করি

তোমার hotel database মনে করো:

```
bookings
---------
booking_id
customer_id
room_id
check_in
check_out
status
```

আমরা আগে index দিয়েছিলাম:

```
CREATE INDEX idx_bookings_room_dates
ON bookings(room_id, check_in, check_out);
```

Availability query:

```
SELECT 1
FROM bookings
WHERE room_id = 101
AND status IN ('PENDING', 'CONFIRMED')
AND check_in < '2026-09-14'
AND check_out > '2026-09-12';
```

এখন প্রশ্ন:

> এই index আসলেই query-তে useful হচ্ছে?

Guess করবে না।

Check করবে:

```sql
EXPLAIN
SELECT 1
FROM bookings
WHERE room_id = 101
AND status IN ('PENDING', 'CONFIRMED')
AND check_in < '2026-09-14'
AND check_out > '2026-09-12';
```

তারপর দেখবে:

```
possible_keys
key
rows
type
Extra
```

3️⃣1️⃣ Production Database Designer Mindset

একজন beginner ভাবে:

> "Query কাজ করছে।"

একজন database designer ভাবে:

> "Query কাজ করছে, কিন্তু 10 million rows হলে কী হবে?"

আর একজন production engineer ভাবে:

```
Query
 ↓
Index
 ↓
Execution Plan
 ↓
Rows examined
 ↓
Concurrency
 ↓
Latency
 ↓
Scalability
```

এই mindset develop করাই Class 59-এর মূল লক্ষ্য।

🧠 আজকের Golden Rules

```
1. Index → Faster data retrieval

2. B-Tree/B+Tree → Common index structure

3. Composite Index → Multiple columns

4. Column order matters

5. Leftmost Prefix → Leading columns matter

6. Covering Index → Required columns available in index

7. InnoDB PK → Clustered index basis

8. Other indexes → Secondary indexes

9. High selectivity → Often better filtering

10. possible_keys ≠ actual key

11. EXPLAIN → Query execution plan

12. rows → Optimizer estimate

13. ALL → Usually full table scan

14. filesort → Sorting operation

15. temporary → Temporary intermediate structure

16. OFFSET → Large values can become expensive

17. Keyset → Better for large-scale pagination

18. EXPLAIN → Optimize → EXPLAIN again
```

📝 Class 59 Assignment

Part A — Index

এই table ধরো:

```sql
CREATE TABLE orders (
    order_id BIGINT PRIMARY KEY,
    customer_id BIGINT,
    order_date DATE,
    status VARCHAR(20),
    total_amount DECIMAL(10,2)
);
```

Task 1

`customer_id`-এর জন্য index তৈরি করো।

Task 2

`customer_id + order_date` composite index তৈরি করো।

Task 3

Explain করো:

```sql
WHERE customer_id = 10
```

এবং

```sql
WHERE order_date = '2026-09-14'
```

এর মধ্যে কোনটি `(customer_id, order_date)` index-এর জন্য বেশি natural fit এবং কেন।

Task 4

Covering index বলতে কী বোঝায়?

Task 5

Clustered এবং Secondary Index-এর difference লিখো।

🔥 Challenge

এই query optimize করার চেষ্টা করো:

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026
ORDER BY order_date;
```

তারপর তোমার optimized query লিখবে।

🎯 Final Challenge

ধরো:

```
products = 50 million rows
```

User শেষবার দেখেছে:

```
product_id = 9,500,000
```

Next 20 products আনতে তুমি কোনটা prefer করবে?

A

```
LIMIT 20 OFFSET 9500000;
```

B

```
WHERE product_id > 9500000
ORDER BY product_id
LIMIT 20;
```

👉 **A/B + কারণ** লিখে দাও।

তারপর আমরা **Class 60 — EXPLAIN / EXPLAIN ANALYZE + Slow Query Optimization**-এ যাব।

📚 Class 60 — EXPLAIN / EXPLAIN ANALYZE + Slow Query Optimization

আজকের class হলো **SQL Performance-এর practical side**।
Class 59-এ আমরা Index শিখেছি। আজ শিখব:

> **Database কেন একটি query slow করছে এবং কীভাবে সেটা identify + optimize করতে হয়।**

আজকের topics:

1. EXPLAIN কী?
2. EXPLAIN output পড়া

3. `type`
4. `possible_keys` vs `key`
5. `rows`
6. `filtered`
7. `Extra`
8. EXPLAIN ANALYZE
9. Slow Query
10. Full Table Scan
11. Index ব্যবহার না হওয়ার কারণ
12. Sargable vs Non-Sargable
13. ORDER BY optimization
14. JOIN optimization
15. GROUP BY optimization
16. Query optimization workflow
17. Real-world hotel example

1️⃣ EXPLAIN কী?

ধরো তুমি লিখলে:

```
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

তুমি জানো query-এর result কী হবে।

কিন্তু তুমি জানো না:

> Database internally **কীভাবে** result বের করছে?

এইটা দেখার জন্য:

```
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

EXPLAIN database-এর **execution plan** দেখায়।

সহজভাবে:

```
SQL Query
   ↓
EXPLAIN
   ↓
Database কীভাবে query execute করবে?
```

2️⃣ EXPLAIN কেন দরকার?

ধরো:

```
customers = 10 million rows
```

Query:

```sql
SELECT *
FROM customers
WHERE email = 'x@gmail.com';
```

তুমি ভাবলে:

> "email-এর index আছে, তাই fast."

কিন্তু database হয়তো index ব্যবহার করছে না।

EXPLAIN দিয়ে জানতে পারবে:

```
Index available?
Index used?
How many rows estimated?
Full scan?
Sorting?
Temporary table?
```

3️⃣ Basic EXPLAIN

```sql
EXPLAIN
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

Conceptually output:

```
type        ref
possible_keys idx_email
key         idx_email
rows        1
Extra       Using index condition
```

এখন একে একে বুঝি।

4️⃣ `table`

কোন table access করা হচ্ছে।

Example:

```
table
-----
customers
```

Multiple JOIN থাকলে multiple table-এর execution information দেখতে পারো।

5️⃣ `type` ⭐

`type` খুব important।

Common values:

```
system
const
eq_ref
ref
range
index
ALL
```

এগুলো broadly access method বোঝায়।

`const`

খুব efficient single-row lookup-এর ক্ষেত্রে দেখা যেতে পারে।

Example:

```sql
SELECT *
FROM users
WHERE id = 10;
```

যদি `id` primary key হয়:

```
type = const
```

হতে পারে।

6️⃣ `eq_ref`

JOIN-এর ক্ষেত্রে highly efficient lookup।

Example:

```sql
SELECT *
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id;
```

যদি `c.customer_id` unique/primary key হয়, relevant access-এর ক্ষেত্রে `eq_ref` দেখা যেতে পারে।

7️⃣ `ref`

Non-unique indexed lookup-এর ক্ষেত্রে common।

Example:

```sql
SELECT *
FROM orders
WHERE customer_id = 10;
```

যদি:

```sql
CREATE INDEX idx_customer
ON orders(customer_id);
```

থাকে, তাহলে:

```
type = ref
```

দেখা যেতে পারে।

8️⃣ range

Range condition-এর ক্ষেত্রে।

Example:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2026-02-01';
```

যদি `order_date` indexed হয়:

```
type = range
```

হতে পারে।

9️⃣ ALL ⚠️

এটা সাধারণত full table scan।

Example:

```sql
SELECT *
FROM customers
WHERE city = 'Dhaka';
```

যদি suitable index না থাকে:

```
type = ALL
```

হতে পারে।

মানে database-কে অনেক/all rows examine করতে হচ্ছে।

🔟 কিন্তু `ALL` মানেই bad নয়

ধরো:

```
customers = 50 rows
```

Full scan:

```
50 rows
```

এটা perfectly fine হতে পারে।

Problem:

```
customers = 50,000,000
```

এবং:

```
type = ALL
```

তখন investigation দরকার।

Golden Rule

> **EXPLAIN-এর কোনো একটি value দেখে query-কে automatically bad বলবে না।**

Context দেখতে হবে।

1️⃣1️⃣ `possible_keys`

Database কোন কোন index consider করতে পারে।

Example:

```
possible_keys
-------------
idx_email
```

মানে:

> `idx_email` potentially useful হতে পারে।

কিন্তু এখনো নিশ্চিত নয় যে database এটা ব্যবহার করবে।

1️⃣2️⃣ key ⭐

Database শেষ পর্যন্ত কোন index choose করেছে।

Example:

```
possible_keys → idx_email
key           → idx_email
```

Good indication।

কিন্তু:

```
possible_keys → idx_email
key           → NULL
```

মানে index available ছিল, কিন্তু optimizer সেটি choose করেনি।

1️⃣3️⃣ কেন Index থাকা সত্ত্বেও ব্যবহার নাও হতে পারে?

অনেক কারণ আছে।

কারণ ১ — Table ছোট

```
100 rows
```

Full scan cheaper হতে পারে।

কারণ ২ — Low selectivity

```
status = 'ACTIVE'
```

যদি 95% rows ACTIVE হয়, index খুব useful নাও হতে পারে।

কারণ ৩ — Function ব্যবহার

```
WHERE YEAR(order_date) = 2026
```

কারণ ৪ — Data distribution

Optimizer statistics অনুযায়ী অন্য strategy cheaper হতে পারে।

কারণ ৫ — Composite index-এর ভুল column usage

```
INDEX(customer_id, order_date)
```

কিন্তু query করছে শুধু:

```
order_date
```

1️⃣4️⃣ `rows` ⭐

`rows` হলো optimizer-এর estimate:

> Query execution-এর সময় আনুমানিক কত rows examine করতে হতে পারে।

Example:

```
rows = 5
```

vs

```
rows = 5,000,000
```

সাধারণত প্রথমটা better।

কিন্তু মনে রাখবে:

> `rows` exact result count নয়।

1️⃣5️⃣ `filtered`

MySQL EXPLAIN output-এ `filtered` আনুমানিক percentage বোঝাতে পারে যে examined rows-এর কত শতাংশ পরবর্তী condition-এ pass করবে।

Example:

```
rows = 100000
filtered = 1.00
```

Roughly:

```
100,000 examined
       ↓
~1% passes
```

অর্থাৎ filtering অনেক পরে হচ্ছে—এটা optimization investigation-এর clue হতে পারে।

1️⃣6️⃣ Extra ⭐⭐⭐

`Extra` খুব useful।

এখানে database-এর additional execution information পাওয়া যায়।

Common values:

```
Using where
Using index
Using filesort
Using temporary
```

1️⃣7️⃣ Using where

মানে WHERE condition apply হচ্ছে।

এটা নিজে problem নয়।

Example:

```
WHERE city = 'Dhaka'
```

EXPLAIN-এ:

```
Using where
```

দেখতে পারো।

1️⃣8️⃣ Using index ⭐

এটা সাধারণত **covering index**-এর মতো situation-এর indication হতে পারে।

Example:

```sql
CREATE INDEX idx_customer_email
ON customers(customer_id, email);
```

Query:

```sql
SELECT customer_id, email
FROM customers
WHERE customer_id = 10;
```

Required columns index-এর মধ্যে থাকলে table data access না করেই result পাওয়া যেতে পারে।

EXPLAIN:

```
Extra = Using index
```

দেখাতে পারে।

1️⃣9️⃣ Using filesort

এটা sorting operation-এর indication।

Example:

```sql
SELECT *
FROM products
ORDER BY price DESC;
```

যদি appropriate index না থাকে, MySQL-এর EXPLAIN-এ:

```
Using filesort
```

দেখতে পারো।

⚠️ `filesort` নাম দেখে ধরে নেবে না যে অবশ্যই disk-এর physical file ব্যবহার হচ্ছে।

এটা MySQL-এর sorting execution terminology।

2️⃣0️⃣ Using temporary

Database intermediate result রাখার জন্য temporary structure ব্যবহার করছে।

Example:

```
SELECT city, COUNT(*)
FROM customers
GROUP BY city
ORDER BY COUNT(*) DESC;
```

EXPLAIN-এ কখনো:

```
Using temporary
Using filesort
```

দেখতে পারো।

এগুলো automatically error নয়।

2️⃣1️⃣ EXPLAIN-এর একটা mental model

EXPLAIN দেখার সময় এই sequence follow করো:

```
1. কোন table?
        ↓
2. type কী?
        ↓
3. possible_keys কী?
        ↓
4. key কোনটা?
        ↓
5. rows কত?
        ↓
6. filtered কত?
        ↓
7. Extra কী?
```

2️⃣2️⃣ Slow Query কী?

যে query application-এর জন্য unacceptable time নেয় বা excessive resources ব্যবহার করে, তাকে practically slow query বলা যায়।

Example:

```
API request
   ↓
Django
   ↓
SQL
   ↓
Database
   ↓
5 seconds
```

User-এর জন্য:

```
🐌 Slow
```

2️⃣3️⃣ Slow Query-এর common কারণ

```
Missing Index
      ↓
Full Table Scan
      ↓
Huge Rows Examined
      ↓
Slow Query
```

আরও:

```
SELECT *
Large JOIN
Bad JOIN condition
Unnecessary sorting
Large OFFSET
Functions on indexed columns
Too many rows returned
Poor composite index
N+1 queries
```

2️⃣4️⃣ Example — Slow Query

ধরো:

```sql
SELECT *
FROM orders
WHERE customer_id = 50000;
```

Table:

```
100 million orders
```

Index নেই।

EXPLAIN:

```
type = ALL
rows = 100000000
```

এটা warning sign।

2️⃣5️⃣ Optimization

Create index:

```sql
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

তারপর:

```sql
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 50000;
```

এখন সম্ভবত access method indexed lookup-এর দিকে যাবে।

কিন্তু exact plan database statistics এবং data distribution-এর উপর নির্ভর করবে।

2️⃣6️⃣ Sargability ⭐

এটা interview-এ আসতে পারে।

Sargable condition হলো এমন predicate যা database-কে indexed column efficiently search করতে সাহায্য করতে পারে।

ধরো:

```
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
```

এটা সাধারণত index-friendly range condition।

অন্যদিকে:

```
WHERE YEAR(order_date) = 2026
```

এখানে column-এর উপর function apply করা হচ্ছে।

এতে সাধারণ B-tree index-এর efficient range access বাধাগ্রস্ত হতে পারে।

2️⃣7️⃣ Bad vs Better

❌

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

✅

```
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01';
```

এবং:

```
CREATE INDEX idx_order_date
ON orders(order_date);
```

থাকলে range query-এর জন্য index ব্যবহার করার সুযোগ তৈরি হয়।

2️⃣8️⃣ `SELECT *` কেন avoid করব?

ধরো:

```
customers
-------------
id
name
email
phone
address
city
country
created_at
...
```

তোমার দরকার শুধু:

```
id
name
email
```

কিন্তু তুমি লিখলে:

```sql
SELECT *
FROM customers;
```

Database অনেক unnecessary data retrieve/transfer করতে পারে।

Better:

```sql
SELECT id, name, email
FROM customers;
```

Golden Rule

> **যা দরকার শুধু সেটাই SELECT করো।**

2️⃣9️⃣ JOIN Optimization

ধরো:

```sql
SELECT
    c.customer_name,
    o.order_date
FROM customers c
```

```
JOIN orders o
    ON c.customer_id = o.customer_id;
```

Important columns:

```
customers.customer_id
orders.customer_id
```

`orders.customer_id` -এ suitable index query plan-এর জন্য useful হতে পারে।

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

তারপর:

```
EXPLAIN
SELECT ...
```

দিয়ে verify করবে।

3️⃣0️⃣ JOIN + Filter

ধরো:

```
SELECT
    c.customer_name,
    o.order_date
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
WHERE c.city = 'Dhaka';
```

এখানে:

```
city
customer_id
```

দুটোই query-এর অংশ।

কোন index best হবে সেটা workload ও data distribution-এর উপর নির্ভর করবে।

Possible candidates:

```
customers(city)
orders(customer_id)
```

তারপর EXPLAIN দিয়ে verify।

3️⃣1️⃣ ORDER BY Optimization

Query:

```sql
SELECT *
FROM products
ORDER BY product_id
LIMIT 20;
```

যদি `product_id` primary key হয়, database ordered index access ব্যবহার করতে পারে।

এটা বড় dataset-এ helpful।

3️⃣2️⃣ ORDER BY + LIMIT

এই pattern খুব common:

```sql
SELECT *
FROM products
ORDER BY product_id DESC
LIMIT 20;
```

Database-এর জন্য:

```
Need:
latest/highest 20
```

সব 50 million rows sort না করে suitable index ব্যবহার করার সুযোগ থাকতে পারে।

3️⃣3️⃣ Pagination Performance

Bad for huge offsets:

```sql
SELECT *
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 5000000;
```

Better scalable approach:

```sql
SELECT *
FROM products
WHERE product_id > 5000000
ORDER BY product_id
LIMIT 20;
```

এটাই **Keyset/Cursor-style pagination** concept।

3️⃣4️⃣ EXPLAIN ANALYZE ⭐⭐⭐

এখন আসল advanced topic।

`EXPLAIN`:

> Optimizer কী plan করেছে?

`EXPLAIN ANALYZE`:

> Query execute করার সময় বাস্তবে কী ঘটেছে?

MySQL 8+ এবং PostgreSQL-এ syntax/format আলাদা হতে পারে।

MySQL example

```sql
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 50000;
```

এটি query execute করে actual execution statistics দেখাতে পারে।

⚠️ তাই `EXPLAIN ANALYZE` read query হলেও **বাস্তবে query execute করে**—production environment-এ ব্যবহার করার আগে সতর্ক থাকবে।

3️⃣5️⃣ EXPLAIN vs EXPLAIN ANALYZE

বিষয়	EXPLAIN	EXPLAIN ANALYZE
Plan	✅	✅
Actual execution	সাধারণত না	✅
Estimated rows	✅	✅
Actual timing	না	✅
Actual rows	না	✅
Query execute	সাধারণ EXPLAIN-এ না	✅

3️⃣6️⃣ Estimated vs Actual

এটা খুব important।

ধরো optimizer বলল:

```
estimated rows = 10
```

কিন্তু actual execution:

```
actual rows = 500,000
```

Huge difference!

মানে optimizer-এর estimation ভুল হতে পারে।

এতে query plan খারাপ হতে পারে।

3️⃣7️⃣ কেন Estimate ভুল হতে পারে?

Possible reasons:

```
Outdated statistics
Data distribution changed
Skewed data
Correlated columns
Complex predicates
```

Database statistics optimizer-কে plan নির্বাচন করতে সাহায্য করে।

3️⃣8️⃣ Optimization কখন শুরু করবে?

এই mistake করবে না:

> "EXPLAIN-এ filesort আছে, তাই index বানাই।"

বরং:

```
User reports slow
        ↓
Measure
        ↓
EXPLAIN
        ↓
Understand bottleneck
        ↓
Create/change index or query
        ↓
Test
        ↓
EXPLAIN again
        ↓
Benchmark
```

3️⃣9️⃣ Real-World Hotel Example ⭐

আমাদের hotel database:

```
bookings
----------------
booking_id
customer_id
room_id
check_in
check_out
status
```

Availability query:

```sql
SELECT 1
FROM bookings
WHERE room_id = 101
AND status IN ('PENDING', 'CONFIRMED')
```

```
AND check_in < '2026-09-14'
AND check_out > '2026-09-12';
```

ধরো bookings:

```
20 million rows
```

এখন:

```
EXPLAIN
SELECT 1
FROM bookings
WHERE room_id = 101
AND status IN ('PENDING', 'CONFIRMED')
AND check_in < '2026-09-14'
AND check_out > '2026-09-12';
```

তুমি investigate করবে:

```
type
possible_keys
key
rows
Extra
```

4️⃣0️⃣ কোন index consider করা যায়?

একটি candidate:

```
CREATE INDEX idx_room_dates
ON bookings(room_id, check_in, check_out);
```

কিন্তু `status` -সহ আরও index বানানোর আগে workload এবং actual EXPLAIN plan দেখা উচিত।

কারণ:

> **Best index theoretically নয়—actual workload-এর জন্য effective index।**

4️⃣1️⃣ Database Designer-এর Optimization Checklist

Slow query পেলে:

Step 1

Query reproduce করো।

Step 2

```
EXPLAIN
```

চালাও।

Step 3

দেখো:

```
type
key
rows
Extra
```

Step 4

Check করো:

```
Missing index?
Wrong index?
Composite index order?
Too many rows?
Function on indexed column?
Unnecessary SELECT *?
Large OFFSET?
Expensive JOIN?
Sorting?
```

Step 5

Optimization করো।

Step 6

আবার:

```
EXPLAIN
```

Step 7

প্রয়োজনে:

```
EXPLAIN ANALYZE
```

দিয়ে actual behavior যাচাই করো।

4️⃣2️⃣ একটি Complete Example

ধরো:

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026
ORDER BY order_date DESC;
```

Problem

```
YEAR(order_date)
```

indexed column-এর উপর function।

Better query

```
SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
ORDER BY order_date DESC;
```

Index:

```
CREATE INDEX idx_orders_date
ON orders(order_date);
```

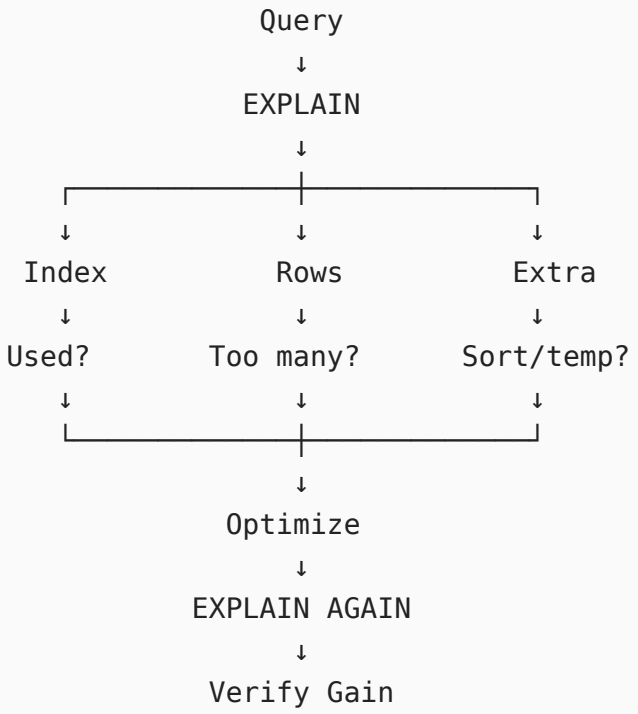
তারপর:

```
EXPLAIN
SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
ORDER BY order_date DESC;
```

তারপর plan compare করবে।

4️⃣3️⃣ সবচেয়ে গুরুত্বপূর্ণ Mental Model ⭐⭐⭐

Database optimization এভাবে চিন্তা করবে:



🧠 Class 60 Golden Rules

1. EXPLAIN → Execution plan বোঝার tool
2. possible_keys → সম্ভাব্য index
3. key → actually selected index
4. rows → estimated rows examined
5. ALL → Usually full table scan
6. ref/range → Often useful indexed access patterns
7. Extra → Additional execution information
8. Using filesort → Sorting operation
9. Using temporary → Temporary intermediate structure
10. EXPLAIN ≠ EXPLAIN ANALYZE

11. EXPLAIN ANALYZE → Actual execution information

12. Function on indexed column → Can hurt index usage

13. SELECT * → Unnecessary data retrieval হতে পারে

14. Large OFFSET → Pagination slow হতে পারে

15. Keyset pagination → Large dataset-এর জন্য scalable approach

16. Index blindly create করবে না

17. EXPLAIN first → Optimize → EXPLAIN again

18. Estimated ≠ Actual

📝 Class 60 Assignment

Task 1 — Basic EXPLAIN

এই query-এর জন্য EXPLAIN লিখো:

```sql
SELECT *
FROM customers
WHERE email = 'rahim@gmail.com';
```

Task 2 — Identify Problem

```sql
SELECT *
FROM orders
WHERE YEAR(order_date) = 2026;
```

কেন performance problem হতে পারে?

এটার optimized version লেখো।

Task 3 — Index

এই query-এর জন্য একটি suitable index propose করো:

```sql
SELECT *
FROM orders
WHERE customer_id = 100
AND order_date >= '2026-01-01'
AND order_date < '2026-02-01';
```

Task 4 — EXPLAIN Interpretation

ধরো EXPLAIN output:

```
type = ALL
possible_keys = idx_customer
key = NULL
rows = 10,000,000
```

এখানে কী বুঝবে?

Task 5 — Concept

Difference লেখো:

```
EXPLAIN
vs
EXPLAIN ANALYZE
```

🔥 Task 6 — Pagination Challenge

Products table-এ 50 million rows আছে।

কোনটা prefer করবে?

A

```sql
SELECT *
FROM products
ORDER BY product_id
LIMIT 20 OFFSET 5000000;
```

B

```sql
SELECT *
FROM products
WHERE product_id > 5000000
ORDER BY product_id
LIMIT 20;
```

A/B + কেন লিখবে।

🎯 Interview Challenge

একজন interviewer বলল:

> **"Your Django API is taking 5 seconds to return a list of customers. How would you investigate and optimize it?"**

তুমি যদি এই প্রশ্নের উত্তর **Database Designer/Backend Developer হিসেবে** দিতে পারো, তাহলে Class 59–60-এর concepts practically বুঝেছ।

Next: Class 61 — Advanced SQL: Recursive CTE, Advanced Window Functions & Cursors.

📚 Class 61 — Advanced SQL: Recursive CTE, Advanced Window Functions & Cursors

আজকের class-এ আমরা SQL-এর **advanced programming + analytical features** শিখব।

Class 38-এ আমরা সাধারণ CTE শিখেছিলাম। আজ সেটাকে আরও advanced level-এ নিয়ে যাব।

আজকের Topics

1. Recursive CTE
2. Hierarchical Data
3. Employee → Manager Tree
4. Recursive CTE-এর stopping condition
5. Advanced Window Functions
6. Window Frame
7. `FIRST_VALUE()`
8. `LAST_VALUE()`
9. `NTH_VALUE()`
10. `PERCENT_RANK()`
11. `CUME_DIST()`

12. `NTILE()`
13. Cursors
14. Cursor কখন ব্যবহার করা হয়
15. Cursor বনাম Set-based SQL
16. Real-world examples
17. Database Designer best practices

Part 1 — Recursive CTE

1️⃣ Recursive CTE কী?

সাধারণ CTE:

```
WITH sales AS (
    SELECT *
    FROM orders
)
SELECT *
FROM sales;
```

এটা একবার query করে result তৈরি করে।

কিন্তু **Recursive CTE নিজেকেই reference করতে পারে।**

অর্থাৎ:

```
CTE
 ↓
নিজেকে আবার call
 ↓
নতুন row
 ↓
আবার নিজেকে call
 ↓
...
```

এটা বিশেষভাবে useful:

- Organization hierarchy
- Employee → Manager
- Category → Subcategory
- Folder → Subfolder
- Parent → Child

- Tree structure
- Number/date sequence

2️⃣ Recursive CTE-এর Basic Structure

Structure:

```
WITH RECURSIVE cte_name AS (

    -- Anchor
    SELECT ...

    UNION ALL

    -- Recursive
    SELECT ...
    FROM cte_name
    WHERE ...

)

SELECT *
FROM cte_name;
```

এখানে দুইটা অংশ:

```
Anchor
   ↓
Starting rows

Recursive
   ↓
Next rows
```

3️⃣ Anchor Member কী?

Anchor হলো recursion-এর **starting point**।

Example:

```
SELECT 1 AS number
```

এখান থেকে শুরু:

```
1
```

4️⃣ Recursive Member কী?

Recursive part previous result-এর উপর কাজ করে।

```
SELECT number + 1
FROM numbers
WHERE number < 10
```

তাহলে:

```
1
↓
2
↓
3
↓
4
↓
...
↓
10
```

5️⃣ প্রথম Recursive CTE Example ⭐

1 থেকে 10 পর্যন্ত number:

```
WITH RECURSIVE numbers AS (

    SELECT 1 AS number

    UNION ALL

    SELECT number + 1
    FROM numbers
    WHERE number < 10
)
```

```
SELECT *
FROM numbers;
```

Output:

```
number
------
1
2
3
4
5
6
7
8
9
10
```

6️⃣ এটা কীভাবে কাজ করছে?

Step by step:

```
Anchor:
1

Recursive:
1 + 1 = 2

2 + 1 = 3

3 + 1 = 4

...

9 + 1 = 10
```

তারপর:

```
number = 10
```

এখন:

```
WHERE number < 10
```

false।

তাই recursion stop।

7️⃣ Stopping Condition ⭐⭐⭐

Recursive CTE-তে সবচেয়ে important বিষয়:

> **Recursion কখন stop করবে সেটা অবশ্যই design করতে হবে।**

Example:

```
WHERE number < 10
```

না থাকলে recursion চলতে থাকবে/DBMS recursion limit-এ গিয়ে error দিতে পারে।

Golden Rule

```
Recursive CTE
     ↓
Start
     ↓
Generate next
     ↓
Check condition
     ↓
Stop?
  ↙     ↘
No       Yes
↓         ↓
Repeat    Finish
```

8️⃣ Date Sequence তৈরি করা

ধরো:

```
2026-09-01
থেকে
2026-09-07
```

```
WITH RECURSIVE dates AS (
```

```sql
    SELECT DATE('2026-09-01') AS date_value

    UNION ALL

    SELECT DATE_ADD(date_value, INTERVAL 1 DAY)
    FROM dates
    WHERE date_value < '2026-09-07'
)

SELECT *
FROM dates;
```

Output:

```
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-05
2026-09-06
2026-09-07
```

এটা analytics-এ useful।

9️⃣ Hierarchical Data

এখন আসল real-world example।

ধরো company:

```
CEO
│
├── Manager A
│   ├── Employee 1
│   └── Employee 2
│
└── Manager B
    ├── Employee 3
    └── Employee 4
```

এখানে employee table:

```sql
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    name VARCHAR(100),
```

```
    manager_id INT
);
```

Data:

```
employee_id | name        | manager_id
-------------------------------------
1           | CEO         | NULL
2           | Manager A   | 1
3           | Employee 1  | 2
4           | Employee 2  | 2
5           | Manager B   | 1
6           | Employee 3  | 5
7           | Employee 4  | 5
```

এখানে:

```
manager_id
```

নিজের table-এর:

```
employee_id
```

কে reference করছে।

এটাকে **self-referencing relationship** বলে।

🔟 Recursive CTE দিয়ে hierarchy

CEO থেকে শুরু করে পুরো hierarchy বের করি:

```
WITH RECURSIVE employee_tree AS (

    -- Anchor
    SELECT
        employee_id,
        name,
        manager_id,
        0 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL
```

```
    -- Recursive
    SELECT
        e.employee_id,
        e.name,
        e.manager_id,
        et.level + 1
    FROM employees e
    JOIN employee_tree et
        ON e.manager_id = et.employee_id
)

SELECT *
FROM employee_tree
ORDER BY level, employee_id;
```

Output conceptually:

```
employee       level
--------------------
CEO               0
Manager A         1
Manager B         1
Employee 1        2
Employee 2        2
Employee 3        2
Employee 4        2
```

1️⃣1️⃣ `level` কেন রাখলাম?

```
0 AS level
```

CEO:

```
level = 0
```

Manager:

```
level = 1
```

Employee:

```
level = 2
```

এতে hierarchy depth বোঝা যায়।

1️⃣2️⃣ Recursive CTE-এর Real Applications

```
Organization
      ↓
Employee hierarchy

E-commerce
      ↓
Category hierarchy

File system
      ↓
Folder → Subfolder

Location
      ↓
Country → State → City

Comments
      ↓
Parent comment → Reply
```

Part 2 — Advanced Window Functions

Class 39–40-এ আমরা Window Functions শিখেছি।

আজ আরও advanced functions দেখব।

1️⃣3️⃣ `FIRST_VALUE()`

একটি partition/group-এর প্রথম value বের করে।

Example:

```sql
SELECT
    employee_id,
    department,
    salary,
    FIRST_VALUE(salary) OVER (
```

```
        PARTITION BY department
        ORDER BY salary DESC
    ) AS highest_salary
FROM employees;
```

Output concept:

```
department | salary | highest_salary
-------------------------------------
IT         | 90000  | 90000
IT         | 70000  | 90000
IT         | 50000  | 90000

HR         | 80000  | 80000
HR         | 60000  | 80000
```

1️⃣4️⃣ LAST_VALUE() ⚠️

এখানে common mistake আছে।

অনেকে লিখে:

```
LAST_VALUE(salary) OVER (
    PARTITION BY department
    ORDER BY salary
)
```

এবং আশা করে:

> পুরো department-এর lowest salary আসবে।

কিন্তু default window frame-এর কারণে result অনেক সময় প্রত্যাশামতো হবে না।

সঠিকভাবে পুরো partition-এর last value পেতে:

```
LAST_VALUE(salary) OVER (
    PARTITION BY department
    ORDER BY salary
    ROWS BETWEEN
        UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
) AS lowest_salary
```

Golden Rule ⭐

`LAST_VALUE()` ব্যবহার করলে **window frame explicitly বুঝতে হবে।**

1️⃣5️⃣ Window Frame

Window frame মানে:

> Current row-এর জন্য window-এর কোন rows consider হবে।

Example:

```
ROWS BETWEEN
2 PRECEDING
AND CURRENT ROW
```

মানে:

```
2 rows before
+
current row
```

1️⃣6️⃣ Moving Average

ধরো monthly sales:

```
Jan 100
Feb 200
Mar 300
Apr 400
```

3-month moving average:

```
SELECT
    month,
    sales,
    AVG(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg
FROM monthly_sales;
```

Concept:

```
Jan → 100

Feb → (100+200)/2

Mar → (100+200+300)/3

Apr → (200+300+400)/3
```

1️⃣7️⃣ NTH_VALUE()

Partition-এর N-th value বের করে।

Example:

```sql
SELECT
    employee_id,
    department,
    salary,
    NTH_VALUE(salary, 2) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN
            UNBOUNDED PRECEDING
            AND UNBOUNDED FOLLOWING
    ) AS second_highest
FROM employees;
```

এখানে:

```
N = 2
```

অর্থাৎ second value।

1️⃣8️⃣ NTILE()

Rows-কে প্রায় সমান সংখ্যক group/bucket-এ ভাগ করে।

Example:

```sql
SELECT
    employee_id,
    salary,
```

```
    NTILE(4) OVER (
        ORDER BY salary DESC
    ) AS salary_quartile
FROM employees;
```

Output:

```
salary | quartile
-----------------
90000  | 1
85000  | 1
80000  | 2
70000  | 2
60000  | 3
50000  | 3
40000  | 4
30000  | 4
```

এখানে:

```
1 → Top 25%
2 → Next 25%
3 → Next 25%
4 → Bottom 25%
```

Exact row distribution depends on number of rows.

1️⃣9️⃣ PERCENT_RANK()

একটি row-এর relative rank percentage হিসেবে দেখায়।

```
SELECT
    employee_id,
    salary,
    PERCENT_RANK() OVER (
        ORDER BY salary
    ) AS percentile_rank
FROM employees;
```

Conceptually:

```
Lowest → 0
Highest → 1
```

মাঝের rows-এর value 0 থেকে 1-এর মধ্যে।

2️⃣0️⃣ `CUME_DIST()`

Cumulative distribution।

```
SELECT
    employee_id,
    salary,
    CUME_DIST() OVER (
        ORDER BY salary
    ) AS cumulative_distribution
FROM employees;
```

এটা বলে:

> বর্তমান salary-এর চেয়ে **less than or equal** salary কত proportion rows-এর মধ্যে আছে।

2️⃣1️⃣ PERCENT_RANK vs CUME_DIST

Function	ধারণা
`PERCENT_RANK()`	Relative rank
`CUME_DIST()`	Cumulative distribution

সহজভাবে:

```
PERCENT_RANK → আমার rank কোথায়?
CUME_DIST    → আমার value পর্যন্ত কতজন?
```

2️⃣2️⃣ Advanced Ranking Example

ধরো employee salary:

```
100k
90k
80k
70k
```

```
60k
50k
```

আমরা একসাথে:

```sql
SELECT
    employee_id,
    salary,

    ROW_NUMBER() OVER (
        ORDER BY salary DESC
    ) AS row_num,

    RANK() OVER (
        ORDER BY salary DESC
    ) AS rank_num,

    DENSE_RANK() OVER (
        ORDER BY salary DESC
    ) AS dense_rank_num,

    NTILE(4) OVER (
        ORDER BY salary DESC
    ) AS quartile,

    PERCENT_RANK() OVER (
        ORDER BY salary DESC
    ) AS percent_rank

FROM employees;
```

এতে এক query-তে multiple analytical metrics পাওয়া যায়।

Part 3 — Cursors

এখন SQL-এর আরেকটা advanced topic।

2️⃣3️⃣ Cursor কী?

Cursor হলো database-এর result set-এর rows **একটি করে process করার mechanism**।

সাধারণ SQL:

```sql
UPDATE employees
SET salary = salary * 1.10
```

```
WHERE department = 'IT';
```

এটা **set-based operation**।

Database এক operation-এ matching rows process করতে পারে।

Cursor approach:

```
Row 1 → process
Row 2 → process
Row 3 → process
Row 4 → process
...
```

2️⃣4️⃣ Cursor কেন দরকার?

কিছু procedural logic-এ row-by-row processing প্রয়োজন হতে পারে।

যেমন:

```
Employee list
   ↓
Each employee
   ↓
Complex calculation
   ↓
Special business rule
   ↓
Generate result
```

কিন্তু গুরুত্বপূর্ণ কথা:

> **Cursor হলো advanced tool; default solution নয়।**

2️⃣5️⃣ Cursor-এর Basic Flow

MySQL stored procedure-এ সাধারণ flow:

```
DECLARE cursor
      ↓
OPEN cursor
      ↓
FETCH row
```

```
      ↓
Process row
      ↓
FETCH next
      ↓
No more rows?
      ↓
CLOSE cursor
```

2️⃣6️⃣ Cursor Example

ধরো employees:

```
employee_id
name
salary
```

Procedure-এর ভিতরে cursor:

```
DELIMITER //

CREATE PROCEDURE process_employees()
BEGIN

    DECLARE done INT DEFAULT FALSE;
    DECLARE emp_id INT;
    DECLARE emp_salary DECIMAL(10,2);

    DECLARE emp_cursor CURSOR FOR
        SELECT employee_id, salary
        FROM employees;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET done = TRUE;

    OPEN emp_cursor;

    read_loop: LOOP

        FETCH emp_cursor
        INTO emp_id, emp_salary;

        IF done THEN
            LEAVE read_loop;
        END IF;
```

```
        -- process current employee
    END LOOP;

    CLOSE emp_cursor;
END //

DELIMITER ;
```

এখানে অনেক syntax আছে, তাই concept আগে বুঝো।

2️⃣7️⃣ Cursor-এর Important Parts

Cursor declaration

```
DECLARE emp_cursor CURSOR FOR
SELECT employee_id, salary
FROM employees;
```

Open

```
OPEN emp_cursor;
```

Fetch

```
FETCH emp_cursor
INTO emp_id, emp_salary;
```

Close

```
CLOSE emp_cursor;
```

2️⃣8️⃣ NOT FOUND Handler

Cursor যখন আর কোনো row পায় না:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET done = TRUE;
```

তারপর:

```
IF done THEN
    LEAVE read_loop;
END IF;
```

মানে:

```
No more rows
      ↓
done = TRUE
      ↓
Exit loop
```

2️⃣9️⃣ Cursor vs Set-Based SQL ⭐⭐⭐

এটা খুব important।

Cursor

```
Row 1
 ↓
Row 2
 ↓
Row 3
 ↓
...
```

Set-based

```
Whole matching set
       ↓
One SQL operation
```

সাধারণভাবে:

> **Set-based SQL prefer করবে।**

3️⃣0️⃣ Example

ধরো সবাইকে 10% salary increase দিতে হবে।

❌ Cursor unnecessary

```
Employee 1 → update
Employee 2 → update
Employee 3 → update
...
```

✅ Set-based

```sql
UPDATE employees
SET salary = salary * 1.10;
```

এটা simpler এবং সাধারণত বেশি efficient।

3️⃣1️⃣ Cursor কখন ব্যবহার করব?

Cursor consider করা যেতে পারে যখন:

```
Complex row-specific logic
        +
Set-based SQL দিয়ে সহজে express করা যায় না
```

উদাহরণ:

```
Row
 ↓
Complex procedural calculation
 ↓
External-like sequencing/business logic
 ↓
Next row
```

তবে অনেক ক্ষেত্রে cursor-এর বদলে application code, batch processing, অথবা set-based SQL ভালো solution হতে পারে।

3️⃣2️⃣ Cursor-এর Disadvantages

```
Slower
More procedural
More resource usage
Harder to maintain
Large datasets-এ problematic হতে পারে
```

বিশেষ করে:

```
1 million rows
```

কে একে একে process করলে overhead হতে পারে।

3️⃣3️⃣ Database Designer হিসেবে Priority

তোমার priority:

```
1. Set-based SQL
        ↓
2. Optimized Query
        ↓
3. Batch Processing
        ↓
4. Application Logic
        ↓
5. Cursor — যখন সত্যিই দরকার
```

এটা absolute rule নয়, কিন্তু good engineering mindset।

3️⃣4️⃣ Recursive CTE vs Cursor

Feature	Recursive CTE	Cursor
Main purpose	Hierarchy/recursive query	Row-by-row processing
Style	Declarative	Procedural
Tree data	⭐⭐⭐	সাধারণত না
Sequence	⭐⭐⭐	সম্ভব
Large dataset	Query-dependent	Often expensive

Feature	Recursive CTE	Cursor
Preferred	যখন recursion দরকার	যখন row-by-row logic unavoidable

---

3️⃣5️⃣ Real-World Example — Category Tree

ধরো:

```
Electronics
├── Laptop
│   ├── Gaming Laptop
│   └── Business Laptop
└── Phone
    ├── Android
    └── iPhone
```

Table:

```
categories
---------------
category_id
name
parent_id
```

Recursive CTE:

```sql
WITH RECURSIVE category_tree AS (

    SELECT
        category_id,
        name,
        parent_id,
        0 AS level
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT
        c.category_id,
        c.name,
        c.parent_id,
        ct.level + 1
    FROM categories c
    JOIN category_tree ct
```

```
        ON c.parent_id = ct.category_id
)

SELECT *
FROM category_tree
ORDER BY level, category_id;
```

এটাই real-world application:

```
Django
   ↓
Category API
   ↓
Recursive SQL
   ↓
Nested category tree
```

3️⃣6️⃣ Recursive CTE-এর সবচেয়ে বড় Mistake

এই query:

```
WITH RECURSIVE numbers AS (
    SELECT 1

    UNION ALL

    SELECT number + 1
    FROM numbers
)
SELECT *
FROM numbers;
```

এখানে stopping condition নেই।

❌ Dangerous.

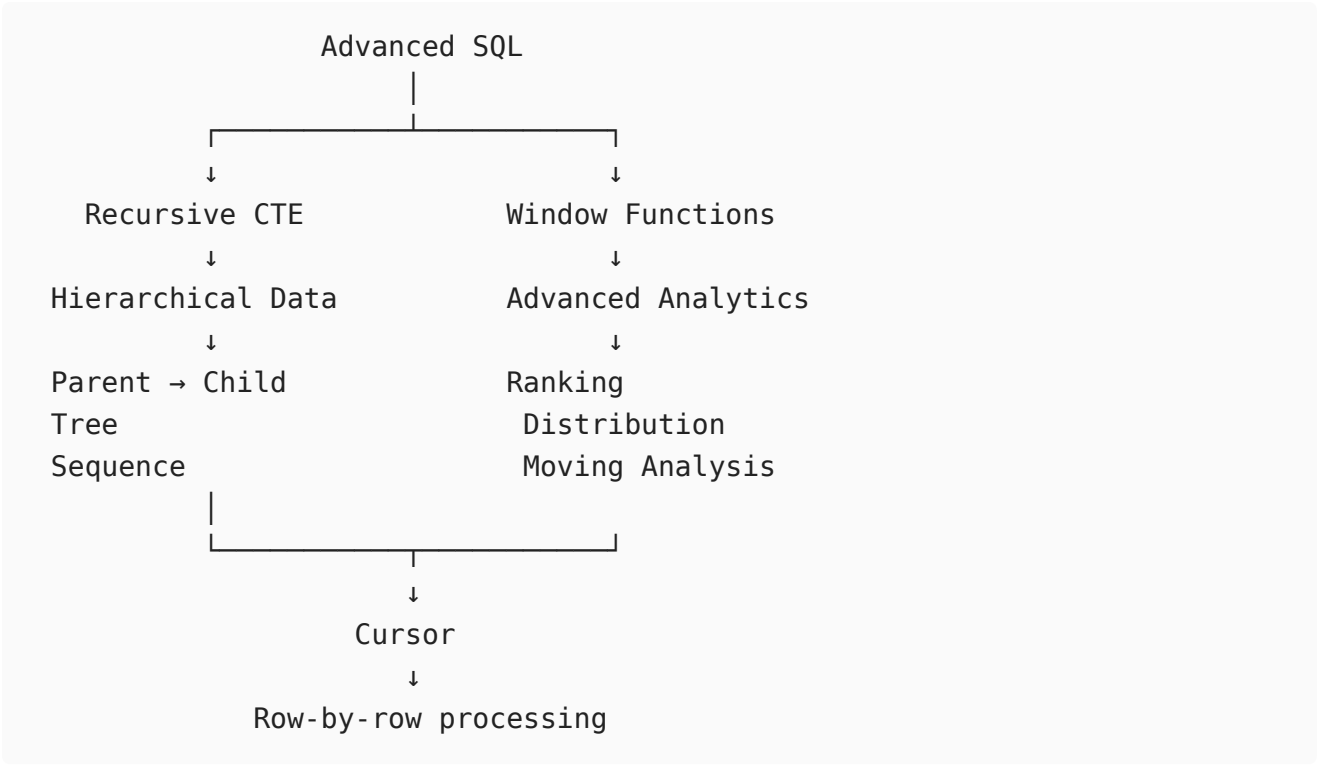
Better:

```
WHERE number < 100
```

Golden Rule

> **Recursive query-তে termination condition design করো।**

3️⃣7️⃣ Class 61 Complete Mental Model



🧠 Class 61 Golden Rules

1. Recursive CTE → CTE নিজেকে reference করতে পারে

2. Anchor → Starting point

3. Recursive member → Next rows generate করে

4. Recursive CTE → Termination condition দরকার

5. Hierarchical data → Recursive CTE-এর major use case

6. FIRST_VALUE → First value

7. LAST_VALUE → Window frame carefully define করতে হয়

8. NTH_VALUE → N-th value

9. NTILE → Rows into buckets

10. PERCENT_RANK → Relative rank

11. CUME_DIST → Cumulative distribution

12. Window frame → কোন rows consider হবে

```
13. Cursor → Row-by-row processing

14. OPEN → Cursor শুরু

15. FETCH → Row আনা

16. CLOSE → Cursor বন্ধ

17. NOT FOUND → Cursor শেষ

18. Set-based SQL → সাধারণত cursor-এর আগে prefer

19. Cursor → যখন row-by-row logic সত্যিই দরকার

20. Recursive CTE ≠ Cursor
```

📝 Class 61 Assignment

Task 1 — Recursive Number

Recursive CTE ব্যবহার করে:

```
1 → 20
```

generate করো।

Task 2 — Date

এই date range generate করো:

```
2026-09-01
to
2026-09-10
```

Recursive CTE দিয়ে।

Task 3 — Employee Hierarchy ⭐

এই table:

```
employees
----------------------------
id
name
manager_id
```

থেকে CEO থেকে সব employee-এর hierarchy বের করো এবং:

```
employee_id
name
manager_id
level
```

দেখাও।

Task 4 — Window Function

Employee table থেকে বের করো:

```
employee_id
department
salary
highest_salary_in_department
lowest_salary_in_department
```

`FIRST_VALUE()` এবং `LAST_VALUE()` ব্যবহার করে।

Task 5 — NTILE

Employees-কে salary অনুযায়ী 4 group-এ ভাগ করো:

```
1 → Top 25%
2 → 25–50%
3 → 50–75%
4 → Bottom 25%
```

`NTILE(4)` ব্যবহার করে।

Task 6 — Concept

Explain করো:

```
Recursive CTE
vs
Cursor
```

কখন কোনটা ব্যবহার করবে?

🔥 Interview Challenge

Interviewer জিজ্ঞেস করল:

> **"You have an employee table where every employee stores their manager_id. How would you retrieve the complete management hierarchy starting from the CEO?"**

Best answer-এর core হবে:

```
Self-referencing relationship
        ↓
Recursive CTE
        ↓
Anchor = CEO
        ↓
Recursive JOIN = manager_id → employee_id
        ↓
level/depth
        ↓
Termination
```

এটা বুঝতে পারলে তুমি SQL-এর **advanced querying level**-এ ঢুকে গেছ।

পরবর্তী: Class 62 — Advanced Transactions & Isolation Levels — এখানে আমরা real-world banking, hotel booking, inventory এবং concurrent transactions আরও গভীরভাবে দেখব।

📚 Class 62 — Advanced Transactions & Isolation Levels

আজকের class-এ আমরা **Transaction + Concurrency + Isolation Level**-কে আরও গভীরে বুঝব।

Class 48-এ আমরা ACID এবং basic transaction শিখেছিলাম। Class 58-এ locking-এর introduction পেয়েছিলে। আজকের লক্ষ্য হলো:

> **একই সময়ে অনেক user database ব্যবহার করলে data কীভাবে correct রাখা যায়?**

🎯 আজকে যা শিখব

1. Transaction আবার একটু গভীরভাবে
2. Concurrency কী?
3. Isolation কেন দরকার?
4. Dirty Read
5. Non-Repeatable Read
6. Phantom Read
7. Lost Update
8. Isolation Levels
9. READ UNCOMMITTED
10. READ COMMITTED
11. REPEATABLE READ
12. SERIALIZABLE
13. Snapshot/MVCC concept
14. `SELECT ... FOR UPDATE`
15. Optimistic vs Pessimistic concurrency
16. Hotel booking example
17. Inventory example
18. Django `transaction.atomic()`
19. Production best practices

1️⃣ Transaction কী?

Transaction হলো database operation-এর একটি **logical unit of work**।

ধরো bank transfer:

```
Account A → -1000
Account B → +1000
```

দুটো operation-ই সফল হতে হবে।

```
A -1000 ✅
B +1000 ✅
```

তাহলে:

```
COMMIT;
```

কিন্তু:

```
A -1000 ✅
B +1000 ❌
```

হলে পুরো transaction rollback করতে চাই:

```
ROLLBACK;
```

2️⃣ ACID আবার মনে করি

```
A → Atomicity
C → Consistency
I → Isolation
D → Durability
```

Atomicity

```
সব হবে
অথবা
কিছুই হবে না
```

Consistency

Database valid state-এ থাকবে।

Isolation

এক transaction-এর intermediate কাজ অন্য concurrent transaction-এর জন্য কীভাবে visible হবে তা নিয়ন্ত্রণ করে।

Durability

COMMIT-এর পর data persist করবে।

3️⃣ Concurrency কী?

ধরো hotel website-এ একই room:

```
Room 101
```

এবং একই সময়ে:

```
User A → Room 101 book করতে চায়
User B → Room 101 book করতে চায়
```

দুটো request একই সময়ে database-এ এলো।

```
User A ──────┐
             ├──→ Database
User B ──────┘
```

এটাই concurrency।

4️⃣ Problem কোথায়?

ধরো available room:

```
Room 101 = AVAILABLE
```

দুই transaction:

```
T1 → check room
T2 → check room
```

দুটোই দেখে:

```
AVAILABLE
```

তারপর:

```
T1 → booking
T2 → booking
```

ফলে:

```
❌ Double Booking
```

এটাই concurrency problem-এর classic example।

5️⃣ Isolation কেন দরকার?

এক transaction-এর কাজ অন্য transaction-এর সঙ্গে কীভাবে interact করবে সেটা control করতে হয়।

```
Transaction A
       ↕
   Isolation
       ↕
Transaction B
```

Isolation ঠিকভাবে না থাকলে বিভিন্ন anomaly হতে পারে।

6️⃣ Four Important Problems ⭐⭐⭐

মনে রাখবে:

```
1. Dirty Read
2. Non-Repeatable Read
3. Phantom Read
4. Lost Update
```

7️⃣ Dirty Read

Transaction A এমন data পড়ল যেটা Transaction B পরিবর্তন করেছে কিন্তু এখনো COMMIT করেনি।

Example:

T1

```sql
START TRANSACTION;

UPDATE accounts
SET balance = 5000
WHERE account_id = 1;
```

এখনো:

```
COMMIT হয়নি
```

T2

```
SELECT balance
FROM accounts
WHERE account_id = 1;
```

T2 যদি `5000` দেখে, কিন্তু T1 পরে:

```
ROLLBACK;
```

তাহলে T2 এমন data পড়েছিল যা শেষ পর্যন্ত database-এ থাকলই না।

এটাই:

> **Dirty Read**

8️⃣ Non-Repeatable Read

এক transaction একই row দুইবার পড়ে কিন্তু দুইবার different value পায়।

T1

```
SELECT balance
FROM accounts
WHERE account_id = 1;
```

Result:

```
10000
```

এর মধ্যে T2:

```
UPDATE accounts
SET balance = 8000
WHERE account_id = 1;

COMMIT;
```

T1 আবার:

```
SELECT balance
FROM accounts
WHERE account_id = 1;
```

Result:

```
8000
```

একই transaction-এর মধ্যে:

```
First read  → 10000
Second read → 8000
```

এটাই:

> **Non-Repeatable Read**

9️⃣ Phantom Read

এবার row-এর value বদলায়নি।

বরং **নতুন matching rows** এসেছে।

ধরো T1:

```sql
SELECT *
FROM orders
WHERE customer_id = 10;
```

Result:

```
3 rows
```

এর মধ্যে T2:

```sql
INSERT INTO orders (...)
VALUES (..., 10, ...);

COMMIT;
```

T1 আবার একই query চালালো:

```sql
SELECT *
FROM orders
WHERE customer_id = 10;
```

এবার:

```
4 rows
```

নতুন matching row যেন "phantom" হিসেবে দেখা দিল।

এটাই:

> **Phantom Read**

🔟 Lost Update

দুই transaction একই data read করে এবং পরে update করে।

ধরো:

```
stock = 10
```

T1

```
read → 10
```

T2

```
read → 10
```

T1:

```
10 - 3 = 7
```

T2:

```
10 - 5 = 5
```

তারপর:

```
T1 → stock = 7
T2 → stock = 5
```

T1-এর update effectively হারিয়ে গেল।

এটাই:

> **Lost Update**

1️⃣1️⃣ Isolation Levels ⭐⭐⭐

SQL standard-এ commonly চারটি isolation level:

```
READ UNCOMMITTED
        ↓
READ COMMITTED
        ↓
REPEATABLE READ
        ↓
SERIALIZABLE
```

সাধারণভাবে নিচের দিকে গেলে isolation stronger হয়, তবে concurrency/performance trade-off থাকে।

1️⃣2️⃣ READ UNCOMMITTED

সবচেয়ে weak standard isolation level।

এখানে dirty read সম্ভব।

Concept:

```
T1 changes data
        ↓
T2 may see uncommitted data
```

Advantage

Concurrency বেশি হতে পারে।

Disadvantage

Data consistency খুব weak।

সাধারণ business transaction-এর জন্য সাধারণত desirable নয়।

1️⃣3️⃣ READ COMMITTED

এখানে:

> অন্য transaction-এর **uncommitted changes** পড়বে না।

অর্থাৎ:

```
Dirty Read → ❌
```

কিন্তু:

```
Non-Repeatable Read → সম্ভব
Phantom Read → DBMS/implementation অনুযায়ী handling ভিন্ন হতে পারে
```

Example:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

1️⃣4️⃣ REPEATABLE READ ⭐

এক transaction-এর মধ্যে একই row বারবার পড়লে consistent view পাওয়ার লক্ষ্য থাকে।

```
First read  → 10000
Second read → 10000
```

এটি database implementation-এর উপর নির্ভর করে MVCC/snapshot mechanisms ব্যবহার করতে পারে।

MySQL InnoDB

MySQL InnoDB-এর default isolation level সাধারণত:

> **REPEATABLE READ**

Check:

```
SELECT @@transaction_isolation;
```

1️⃣5️⃣ SERIALIZABLE

সবচেয়ে strong standard isolation level।

Conceptually concurrent transactions এমনভাবে execute হওয়ার guarantee পেতে পারে যেন তারা serial order-এ execute হয়েছে।

```
T1
 ↓
T2
 ↓
T3
```

Concurrency কমে যেতে পারে।

Potential result:

```
More locking/contention
      ↓
Less concurrency
      ↓
Potentially lower throughput
```

তবে exact behavior DBMS-এর implementation-এর উপর নির্ভর করে।

1️⃣6️⃣ Isolation Comparison ⭐

Isolation	Dirty Read	Non-Repeatable	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented for consistent reads	DBMS-dependent
SERIALIZABLE	Prevented	Prevented	Prevented

এখানে একটা important point:

> SQL standard-এর anomaly table এবং বাস্তব DBMS-এর implementation একেবারে identical নয়।

বিশেষ করে **MySQL InnoDB** MVCC ও locking ব্যবহার করে।

1️⃣7️⃣ MVCC কী?

MVCC = **Multi-Version Concurrency Control**

সহজভাবে:

> Database একই data-এর বিভিন্ন version ব্যবহার করে concurrent reads/writes manage করতে পারে।

Concept:

```
Row
 ↓
Version 1
Version 2
Version 3
```

একজন transaction পুরনো consistent version দেখতে পারে, আর অন্য transaction নতুন committed version নিয়ে কাজ করতে পারে।

এতে read/write concurrency improve করা যায়।

1️⃣8️⃣ MVCC-এর সহজ উদাহরণ

ধরো:

```
balance = 10000
```

T1 update করছে:

```
balance = 8000
```

একই সময়ে T2 consistent read করছে।

MVCC implementation অনুযায়ী T2 তার transaction-এর জন্য appropriate committed version দেখতে পারে।

এতে সব read-এর জন্য blocking lock দরকার হয় না।

1️⃣9️⃣ `SELECT ... FOR UPDATE` ⭐⭐⭐

এখন খুব important।

ধরো hotel booking:

```
START TRANSACTION;

SELECT *
FROM rooms
WHERE room_id = 101
FOR UPDATE;
```

`FOR UPDATE` transactional context-এ selected rows-এর উপর update-oriented lock নিতে বলে।

তারপর:

```
-- check availability

INSERT INTO bookings (...);

COMMIT;
```

এখানে অন্য concurrent transaction একই locked resource update করতে চাইলে অপেক্ষা করতে হতে পারে।

2️⃣0️⃣ Hotel Booking Example

ধরো:

```
Room 101
```

দুই user:

```
User A → Book
User B → Book
```

T1

```
START TRANSACTION;

SELECT *
FROM rooms
WHERE room_id = 101
FOR UPDATE;
```

এখন relevant row lock হয়েছে।

তারপর:

```
Check availability
      ↓
Create booking
      ↓
COMMIT
```

T2

একই room-এর locked row access/update করতে গেলে তাকে T1-এর transaction শেষ হওয়ার জন্য অপেক্ষা করতে হতে পারে।

এতে race condition control করা সহজ হয়।

2️⃣1️⃣ কিন্তু একটা গুরুত্বপূর্ণ বিষয় ⚠️

শুধু:

```
SELECT ...
FOR UPDATE
```

লিখলেই hotel double-booking problem magically solved হবে না।

তোমাকে পুরো transaction এবং overlap logic সঠিকভাবে design করতে হবে।

যেমন:

```
BEGIN
 ↓
Lock relevant resource
 ↓
Check overlapping booking
 ↓
If available
    ↓
Create booking
 ↓
COMMIT
```

2️⃣2️⃣ Inventory Example

ধরো:

```
stock = 10
```

দুই user:

```
User A buys 7
User B buys 5
```

Naive approach:

```
A reads 10
B reads 10

A → 3
B → 5
```

Incorrect result হতে পারে।

Better atomic update:

```sql
UPDATE products
SET stock = stock - 7
WHERE product_id = 10
AND stock >= 7;
```

তারপর application check করবে:

```
affected rows = 1?
```

হলে সফল।

তারপর B:

```sql
UPDATE products
SET stock = stock - 5
WHERE product_id = 10
AND stock >= 5;
```

যদি stock insufficient হয়:

```
affected rows = 0
```

2️⃣3️⃣ Atomic UPDATE কেন Powerful?

কারণ:

```
Read stock
   ↓
Calculate
   ↓
Write stock
```

এর বদলে database-এর condition-এর ভিতরেই:

```
UPDATE
WHERE stock >= requested_quantity
```

দেওয়া হয়েছে।

এতে race condition reduce করার ভালো সুযোগ তৈরি হয়।

2️⃣4️⃣ Optimistic Concurrency

এখানে আমরা ধরে নিই:

> Conflict খুব frequent নয়।

তাই আগে lock না করে version check করি।

Table:

```
id
stock
version
```

Current:

```
stock = 10
version = 5
```

Application update করে:

```
UPDATE products
SET
    stock = 7,
    version = version + 1
```

```
WHERE product_id = 10
AND version = 5;
```

যদি:

```
affected rows = 1
```

Success।

যদি:

```
affected rows = 0
```

তাহলে অন্য transaction আগে update করেছে।

এটাই conflict detection।

2️⃣5️⃣ Pessimistic Concurrency

এখানে assumption:

> Conflict হতে পারে, তাই আগে থেকেই lock করি।

Example:

```
SELECT *
FROM products
WHERE product_id = 10
FOR UPDATE;
```

তারপর update।

2️⃣6️⃣ Optimistic vs Pessimistic

	Optimistic	Pessimistic
Lock early	❌	✅
Version check	সাধারণত ✅	সাধারণত না
Conflict handling	পরে detect	আগে prevent/block
High contention	সবসময় ideal নয়	useful হতে পারে

	Optimistic	Pessimistic
Low contention	Often good	May be unnecessary
Example	version column	`FOR UPDATE`

2️⃣7️⃣ Deadlock

ধরো:

T1

```
Lock A
 ↓
Waiting for B
```

T2

```
Lock B
 ↓
Waiting for A
```

```
T1 → B-এর জন্য অপেক্ষা
T2 → A-এর জন্য অপেক্ষা
```

কেউ এগোতে পারছে না।

এটাই:

> **Deadlock**

2️⃣8️⃣ Deadlock Prevention

Rule 1 — Consistent Lock Order

সব transaction:

```
A → B
```

এই order-এ lock করবে।

কেউ:

```
B → A
```

করবে না।

Rule 2 — Short Transactions

❌

```
BEGIN
 ↓
5 minutes processing
 ↓
COMMIT
```

Better:

```
BEGIN
 ↓
minimum DB work
 ↓
COMMIT
```

Rule 3 — Proper Indexes

Poor index-এর কারণে বেশি rows lock/search করতে হতে পারে।

Rule 4 — Retry

Application layer-এ deadlock হলে transaction retry strategy রাখা যেতে পারে।

2️⃣9️⃣ Django-তে Transaction

Django:

```python
from django.db import transaction

with transaction.atomic():
    # create booking
```

```
    # create payment
    # other related updates
    pass
```

Block-এর ভিতরের database operations একটি atomic unit হিসেবে কাজ করে।

Exception হলে transaction rollback হতে পারে।

3️⃣0️⃣ Django + `select_for_update()`

Hotel example:

```
from django.db import transaction

with transaction.atomic():

    room = (
        Room.objects
        .select_for_update()
        .get(id=101)
    )

    # check booking overlap

    # create booking
```

এটা database-level row locking-এর Django abstraction।

3️⃣1️⃣ Isolation Level কখন Change করব?

Default isolation level blindly change করবে না।

প্রথমে বুঝবে:

```
Business requirement
        ↓
Consistency requirement
        ↓
Concurrency requirement
        ↓
Performance
        ↓
Isolation choice
```

Example:

```
Simple reporting
→ lower isolation may be sufficient

Financial transaction
→ stronger consistency may be needed

Hotel booking
→ proper locking + transaction design important
```

এখানে exact choice workload এবং DBMS-এর behavior-এর উপর নির্ভর করবে।

3️⃣2️⃣ Isolation Level Set করা

MySQL example:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

তারপর transaction:

```
START TRANSACTION;

...

COMMIT;
```

Current setting:

```
SELECT @@transaction_isolation;
```

3️⃣3️⃣ Important — Isolation ≠ Locking

এটা interview-এ খুব useful।

```
Isolation Level
      +
Locks
      +
MVCC
```

```
      +
Transaction Design
```

সব মিলে concurrency behavior তৈরি হয়।

শুধু isolation level জানলেই পুরো concurrency বুঝে ফেলা যায় না।

3️⃣4️⃣ Hotel Booking — Production Flow ⭐⭐⭐

একটা ভালো transaction flow:

```
User requests Room 101
        ↓
START TRANSACTION
        ↓
Lock relevant room/resource
        ↓
Check room status
        ↓
Check overlapping bookings
        ↓
Available?
   ↙          ↘
 YES           NO
 ↓              ↓
Create         ROLLBACK
Booking
 ↓
Create Payment
 ↓
COMMIT
```

যদি মাঝখানে error:

```
ROLLBACK
```

3️⃣5️⃣ সবচেয়ে Common Mistake

এইটা:

```
if room_available():
    create_booking()
```

যদি `room_available()` এবং `create_booking()` আলাদা transaction/context-এ হয়, তাহলে দুই concurrent request একই সময়ে available দেখে booking করতে পারে।

এটাই race condition।

Better:

```
availability check
       +
booking creation
       ↓
same transaction
       +
proper locking/concurrency control
```

3️⃣6️⃣ Isolation Level Selection — Simple Mental Model

```
READ UNCOMMITTED
       ↓
Fast / weak consistency

READ COMMITTED
       ↓
No dirty reads

REPEATABLE READ
       ↓
Stable consistent reads

SERIALIZABLE
       ↓
Strongest standard isolation
```

কিন্তু:

```
Stronger isolation
       ↓
Potentially more contention
       ↓
Potentially lower concurrency
```

3️⃣7️⃣ Database Designer-এর Real Mindset

Beginner:

> "আমার query correct."

Database designer:

> "Concurrent users থাকলে correct থাকবে?"

Production engineer:

> "Concurrent users + failure + retry + deadlock + high load—সব অবস্থায় কী হবে?"

এই level-এর চিন্তা develop করাই Class 62-এর মূল লক্ষ্য।

🧠 Class 62 Golden Rules ⭐⭐⭐

```
1. Transaction → Logical unit of work

2. Isolation → Concurrent transactions-এর interaction control

3. Dirty Read → Uncommitted data read

4. Non-Repeatable Read → Same row, different value

5. Phantom Read → New matching rows appear

6. Lost Update → One update overwrites another

7. READ UNCOMMITTED → Weakest

8. READ COMMITTED → No dirty reads

9. REPEATABLE READ → Stable consistent reads

10. SERIALIZABLE → Strongest standard isolation

11. MVCC → Multiple row versions help concurrency

12. FOR UPDATE → Lock rows for transactional update

13. Optimistic → Version/conflict check

14. Pessimistic → Lock early

15. Deadlock → Transactions waiting on each other

16. Short transactions → Better concurrency
```

```
17. Consistent lock order → Helps prevent deadlocks

18. Atomic UPDATE → Useful for inventory

19. Django atomic() → Transaction boundary

20. Isolation level alone ≠ complete concurrency solution
```

📝 Class 62 Assignment

Task 1 — Concepts

নিজের ভাষায় explain করো:

```
Dirty Read
Non-Repeatable Read
Phantom Read
Lost Update
```

Task 2 — Isolation

এই table পূরণ করো:

Isolation	Dirty Read	Non-Repeatable Read	Phantom
READ UNCOMMITTED	?	?	?
READ COMMITTED	?	?	?
REPEATABLE READ	?	?	?
SERIALIZABLE	?	?	?

Task 3 — Inventory ⭐

Current stock:

```
stock = 10
```

User A:

```
buy 7
```

User B:

```
buy 5
```

একটি **atomic UPDATE query** লেখো যাতে stock negative না হয়।

Task 4 — Hotel Booking ⭐⭐⭐

দুই user একই সময়ে:

```
Room 101
2026-09-20 → 2026-09-25
```

book করতে চায়।

একটি transaction flow লেখো:

```
BEGIN
↓
?
↓
?
↓
?
↓
COMMIT / ROLLBACK
```

Task 5 — Optimistic Concurrency

Current:

```
stock = 10
version = 5
```

একজন user 3 units কিনছে।

`version` ব্যবহার করে UPDATE query লেখো।

🔥 Task 6 — Interview Challenge

Interviewer:

> **"Two users try to book the same hotel room at exactly the same time. How would you prevent double booking?"**

তোমার answer-এ ideally থাকবে:

```
Transaction
+
Lock / appropriate concurrency control
+
Availability check
+
Overlap detection
+
Create booking
+
COMMIT
+
ROLLBACK on failure
```

🎯 Class 62-এর সবচেয়ে গুরুত্বপূর্ণ ৫টি জিনিস

যদি আজ সবকিছু মনে না থাকে, অন্তত এগুলো solid করো:

```
1. Isolation Level
2. Dirty / Non-repeatable / Phantom
3. Lost Update
4. SELECT ... FOR UPDATE
5. Optimistic vs Pessimistic Concurrency
```

পরবর্তী: Class 63 — Locking, Deadlock & Concurrency — Advanced। সেখানে আমরা **Shared Lock, Exclusive Lock, Row Lock, Gap Lock, Next-Key Lock, Deadlock Analysis, Lock Order এবং real concurrent transaction scenarios** আরও গভীরভাবে করব।

📚 Class 63 — Locking, Deadlock & Concurrency — Advanced

আজকের class হলো **Database Concurrency-এর advanced level**।

Class 62-এ আমরা Isolation Level, Dirty Read, Lost Update, `FOR UPDATE`, Optimistic/Pessimistic Concurrency শিখেছি।

আজ আমরা আরও গভীরে যাব:

1. Lock কী?
2. Shared Lock
3. Exclusive Lock
4. Row Lock
5. Table Lock
6. Gap Lock
7. Next-Key Lock
8. Record Lock
9. Lock Compatibility
10. Deadlock কীভাবে তৈরি হয়
11. Deadlock analysis
12. Lock order
13. Real concurrent transaction
14. Hotel booking
15. Inventory
16. Bank transfer
17. Django concurrency
18. Production best practices

1️⃣ Lock কী?

সহজভাবে:

> **Lock হলো database-এর mechanism যা concurrent transactions-এর conflicting access control করে।**

ধরো:

```
Account A
balance = 10000
```

দুই transaction একই সময়ে balance update করতে চায়:

```
T1 ─────→ Account A
T2 ─────→ Account A
```

Database-কে decide করতে হবে:

```
কে আগে access করবে?
কে wait করবে?
কখন দ্বিতীয় transaction access পাবে?
```

এখানে lock সাহায্য করে।

2️⃣ Lock-এর প্রধান দুই ধরনের ধারণা

আজকের সবচেয়ে important:

```
Shared Lock (S)
       ↓
Read

Exclusive Lock (X)
       ↓
Write
```

3️⃣ Shared Lock — S Lock

Shared lock সাধারণত **read-এর জন্য** ব্যবহৃত হয়।

Concept:

```
T1 → S Lock → Row A
```

আরেক transaction-ও compatible shared lock পেতে পারে:

```
T1 → S Lock
T2 → S Lock
```

দুজনেই read করতে পারে।

কিন্তু অন্য transaction যদি conflicting exclusive lock চায়:

```
T1 → S
T2 → X
```

T2-কে wait করতে হতে পারে।

4️⃣ Exclusive Lock — X Lock

Exclusive lock সাধারণত update/delete-এর মতো write operation-এর জন্য ব্যবহৃত হয়।

```
T1 → X Lock → Row A
```

এখন অন্য transaction একই row-তে conflicting access চাইলে অপেক্ষা করতে হতে পারে।

Concept:

```
X Lock
  ↓
Other conflicting lock
  ↓
WAIT
```

5️⃣ Shared vs Exclusive

	Shared (S)	**Exclusive (X)**
Read	✅	সম্ভব/implementation-specific context
Write	❌	✅
Multiple transactions	Shared locks compatible হতে পারে	Generally exclusive
Main purpose	Read protection	Write protection

Simplified compatibility:

Existing	**New S**	**New X**
S	✅	❌
X	❌	❌

এটা conceptual compatibility table; exact locking behavior DBMS ও isolation-এর উপর নির্ভর করে।

6️⃣ Row Lock ⭐

Row lock মানে পুরো table lock না করে নির্দিষ্ট row-এর উপর lock।

ধরো:

```
customers

id | name
---------
1  | Rahim
2  | Karim
3  | Sadia
4  | Nadia
```

T1:

```
SELECT *
FROM customers
WHERE id = 2
FOR UPDATE;
```

Relevant row:

```
Karim
```

transaction-এর context-এ update-oriented row lock নেওয়া হতে পারে।

অন্য transaction:

```
UPDATE customers
SET name = 'Karim Ahmed'
WHERE id = 2;
```

প্রয়োজনে wait করতে পারে যতক্ষণ না প্রথম transaction lock release করে।

7️⃣ Row Lock কেন ভালো?

ধরো 1 million rows-এর table।

যদি একজন user শুধু:

```
Row 500
```

update করে, তাহলে ideally অন্য user যেন:

```
Row 700
```

update করতে পারে।

Row-level locking এই ধরনের concurrency support করে।

```
T1 → Row 500 🔒
T2 → Row 700 🔒

Both can potentially proceed
```

8️⃣ Table Lock

Table-level lock:

```
Entire Table
     🔒
```

তখন অন্য transaction-এর access বেশি restricted হতে পারে।

Large concurrent applications-এ unnecessary broad locks problematic হতে পারে।

তাই:

```
Row lock
   ↓
Usually finer-grained

Table lock
   ↓
Broader
```

তবে কোন lock ব্যবহার হবে তা DBMS এবং statement-এর উপর নির্ভর করে।

9️⃣ `SELECT ... FOR UPDATE` আবার দেখি ⭐⭐⭐

```sql
START TRANSACTION;

SELECT *
FROM products
WHERE product_id = 10
```

```
FOR UPDATE;

-- update stock

COMMIT;
```

এখানে উদ্দেশ্য:

```
Read row
   ↓
Lock for transactional update
   ↓
Perform logic
   ↓
COMMIT
```

🔟 Important: Lock কতক্ষণ থাকে?

সাধারণ transactional row lock transaction শেষ হওয়া পর্যন্ত থাকতে পারে:

```
BEGIN
 ↓
Lock
 ↓
Queries
 ↓
COMMIT / ROLLBACK
 ↓
Lock release
```

তাই:

> **Long transaction = Long lock duration = More contention**

1️⃣1️⃣ Lock Contention

ধরো:

```
T1 → Row 100 🔒
```

T2:

```
T2 → Row 100
      ↓
    WAIT
```

এটাকে বলা যায়:

> **Lock contention**

অর্থাৎ multiple transactions একই resource-এর জন্য compete করছে।

1️⃣2️⃣ Lock Wait

Concept:

```
T1
 ↓
Lock Row A
 ↓
Processing...
 ↓
COMMIT

T2
 ↓
Needs Row A
 ↓
WAIT ⏳
 ↓
T1 commits
 ↓
T2 continues
```

যদি T1 unnecessarily অনেক সময় ধরে transaction open রাখে, T2-এর wait time বাড়তে পারে।

1️⃣3️⃣ Gap Lock ⭐⭐⭐

এটা MySQL InnoDB-এর advanced locking concept।

ধরো indexed values:

```
10
20
```

```
30
40
```

এখানে values-এর মাঝেও gaps আছে:

```
(-∞,10)
(10,20)
(20,30)
(30,40)
(40,+∞)
```

Gap lock কোনো existing row-কে নয়, index-এর **gap/range between records**-কে lock করতে পারে।

এর উদ্দেশ্যের একটি গুরুত্বপূর্ণ অংশ হলো concurrent inserts-এর behavior control করা।

1️⃣4️⃣ Gap Lock কেন দরকার?

ধরো T1 একটি range scan করছে:

```
SELECT *
FROM orders
WHERE order_id BETWEEN 10 AND 20
FOR UPDATE;
```

তারপর অন্য transaction যদি একই locked range-এর মধ্যে নতুন row insert করতে চায়, InnoDB-এর isolation/locking behavior অনুযায়ী তাকে block করা হতে পারে।

এখানে:

```
10 ———————— 20
   🔒 GAP
```

Gap lock concept relevant।

1️⃣5️⃣ Record Lock vs Gap Lock

Record Lock

Existing index record-এর উপর lock।

```
10
🔒
```

Gap Lock

দুই record-এর মাঝের gap-এর উপর lock।

```
10      20
  🔒🔒🔒
```

1️⃣6️⃣ Next-Key Lock ⭐⭐⭐

এটা একটু advanced।

Next-key lock = Record Lock + Gap Lock

Concept:

```
Gap + Record
```

অর্থাৎ:

```
(previous value, current record]
```

ধরো:

```
10
20
30
```

20-এর next-key locking conceptually:

```
(10, 20]
```

অর্থাৎ:

```
Gap between 10 and 20
        +
Record 20
```

1️⃣7️⃣ Gap Lock vs Next-Key Lock

	Gap Lock	Next-Key Lock
Existing record lock	❌	✅
Gap lock	✅	✅
Main idea	Gap protect	Gap + record protect
MySQL InnoDB	Important concept	Important concept

1️⃣8️⃣ কেন এইগুলো জানা দরকার?

Database designer হিসেবে তোমাকে প্রতিদিন:

```
Gap Lock
Next-Key Lock
```

ব্যবহার করতে হবে না।

কিন্তু production debugging-এ যদি:

```
"Why is my INSERT waiting?"
```

এই ধরনের problem আসে, তখন এই concepts helpful।

1️⃣9️⃣ Deadlock ⭐⭐⭐

এখন সবচেয়ে important অংশ।

Deadlock:

> **দুই বা তার বেশি transaction একে অন্যের locked resource-এর জন্য অপেক্ষা করছে।**

Classic example:

```
T1 locks A
T2 locks B

T1 wants B
T2 wants A
```

Diagram:

```
T1 ──locks──→ A
 ↑             ↓
 │           wants
 │             ↓
 B ←──locks── T2
```

দুজনেই wait করছে।

2️⃣0️⃣ Deadlock Example

ধরো:

```
Account 1
Account 2
```

T1

```
START TRANSACTION;

SELECT *
FROM accounts
WHERE account_id = 1
FOR UPDATE;
```

T1:

```
Account 1 🔒
```

তারপর T1:

```
SELECT *
FROM accounts
WHERE account_id = 2
FOR UPDATE;
```

একই সময়ে T2

```
START TRANSACTION;

SELECT *
FROM accounts
WHERE account_id = 2
FOR UPDATE;
```

T2:

```
Account 2 🔒
```

তারপর:

```
SELECT *
FROM accounts
WHERE account_id = 1
FOR UPDATE;
```

এখন:

```
T1 owns Account 1
T2 owns Account 2

T1 wants Account 2
T2 wants Account 1
```

💥 **Deadlock**

2️⃣1️⃣ Deadlock Graph

এভাবে মনে রাখো:

```
T1
 |
 | holds
 ↓
 A
 |
 | wants
 ↓
 B
 ↑
 | holds
```

```
 |
T2
 |
 | wants
 ↓
 A
```

অর্থাৎ:

```
T1 → T2 → T1
```

একটা cycle তৈরি হয়েছে।

2️⃣2️⃣ Database কী করে?

Modern DBMS সাধারণত deadlock detect করতে পারে।

তারপর একটি transaction victim হিসেবে rollback করে:

```
T1 ──┐
     ├── Deadlock
T2 ──┘
      ↓
DB detects
      ↓
One transaction rollback
      ↓
Other continues
```

অর্থাৎ:

> Deadlock হলে database necessarily permanently stuck হয়ে থাকবে না।

2️⃣3️⃣ Deadlock কেন সবসময় Application Bug নয়?

Concurrent system-এ deadlock ঘটতে পারে।

Important হলো:

```
Prevent where possible
+
Detect
```

```
+
Retry safely
```

2️⃣4️⃣ Deadlock Prevention — Lock Order ⭐⭐⭐

সবচেয়ে practical technique:

> **সব transaction একই order-এ resource lock করবে।**

ধরো:

```
Account 1
Account 2
```

সব transaction:

```
1 → 2
```

এই order-এ lock করবে।

তাহলে:

```
T1 → 1 → 2
T2 → 1 → 2
```

T2 হয়তো 1-এর জন্য wait করবে।

কিন্তু:

```
T2 → 2 → 1
```

এর মতো reverse order না হলে cycle তৈরির chance কমে।

2️⃣5️⃣ Lock Order Golden Rule ⭐

```
Multiple resources
       ↓
Always lock
       ↓
Same deterministic order
```

Example:

```
Customer ID ascending
Product ID ascending
Account ID ascending
```

যে domain-এর resource হোক, consistent order গুরুত্বপূর্ণ।

2️⃣6️⃣ Bank Transfer — Correct Design

ধরো:

```
Account A → Account B
```

Transaction:

```
BEGIN
 ↓
Lock smaller account ID first
 ↓
Lock larger account ID
 ↓
Check balance
 ↓
Debit
 ↓
Credit
 ↓
COMMIT
```

যদি transfer reverse direction হয়, তবুও:

```
smaller ID → larger ID
```

order maintain করবে।

এতে deadlock risk কমে।

2️⃣7️⃣ Hotel Booking — Advanced Concurrency ⭐⭐⭐

আমাদের hotel system:

```
Room 101
```

দুই user:

```
A → Sept 20–25
B → Sept 20–25
```

Transaction:

```
BEGIN
 ↓
Lock relevant room/resource
 ↓
Check existing bookings
 ↓
Check overlap
 ↓
Available?
 ↓
Create booking
 ↓
COMMIT
```

2️⃣8️⃣ Overlap Condition

Existing booking:

```
check_in  = Sept 20
check_out = Sept 25
```

New booking:

```
check_in  = Sept 22
check_out = Sept 24
```

Overlap:

```
new_check_in < existing_check_out
AND
new_check_out > existing_check_in
```

SQL:

```sql
SELECT 1
FROM bookings
WHERE room_id = 101
AND status IN ('PENDING', 'CONFIRMED')
AND check_in < '2026-09-25'
AND check_out > '2026-09-20'
FOR UPDATE;
```

তবে এখানে একটা subtle point আছে:

> Exact locking behavior নির্ভর করবে index, isolation level, query plan এবং DBMS-এর implementation-এর উপর।

শুধু query দেখে locking behavior assume করা উচিত নয়।

2️⃣9️⃣ Inventory — Concurrent Purchase

Stock:

```
100
```

User A:

```
buy 60
```

User B:

```
buy 50
```

Naive:

```
A reads 100
B reads 100
```

দুজনই ধরে নেয় stock available।

Problem:

```
100 - 60 - 50
= -10
```

3️⃣0️⃣ Better — Atomic UPDATE

```
UPDATE products
SET stock = stock - 60
WHERE product_id = 10
AND stock >= 60;
```

তারপর:

```
affected_rows = 1
```

হলে success।

B:

```
UPDATE products
SET stock = stock - 50
WHERE product_id = 10
AND stock >= 50;
```

যদি insufficient stock হয়:

```
affected_rows = 0
```

3️⃣1️⃣ কেন এটা ভালো?

কারণ condition + update একই database operation-এর মধ্যে:

```
stock >= quantity
       +
stock = stock - quantity
```

এতে application-level read-modify-write race কমানো যায়।

3️⃣2️⃣ Locking + Indexing ⭐

এখানে Class 59–60-এর knowledge দরকার।

ধরো:

```sql
SELECT *
FROM bookings
WHERE room_id = 101
AND check_in < '2026-09-25'
AND check_out > '2026-09-20'
FOR UPDATE;
```

Appropriate indexes না থাকলে database-কে বেশি rows inspect করতে হতে পারে।

ফলে:

```
Poor index
 ↓
More rows examined
 ↓
More locking work/contention
 ↓
Slower transaction
 ↓
Higher deadlock/wait risk
```

তাই:

> **Index শুধু query speed-এর জন্য নয়; concurrency behavior-এর উপরও প্রভাব ফেলতে পারে।**

3️⃣3️⃣ Long Transaction কেন Dangerous?

❌

```
BEGIN
 ↓
Lock
 ↓
API call
 ↓
External processing
 ↓
User input
 ↓
More processing
 ↓
COMMIT
```

এটা avoid করা উচিত।

কারণ lock অনেকক্ষণ ধরে থাকতে পারে।

Better:

```
Prepare data
 ↓
BEGIN
 ↓
Lock
 ↓
DB operations
 ↓
COMMIT
```

Golden Rule

> **Transaction যত short করা যায় তত ভালো।**

3️⃣4️⃣ Application-এর ভিতরে External API Call

ধরো:

```
with transaction.atomic():
    # lock room

    call_payment_gateway()

    # create booking
```

এটা risky হতে পারে যদি payment API 5 seconds নেয়।

কারণ:

```
DB Lock
   ↓
Payment API waits 5 sec
   ↓
Lock held 5 sec
```

এতে অন্য requests block হতে পারে।

Production design-এ payment workflow সাধারণত carefully designed হয় যাতে unnecessary DB lock external network call-এর জন্য ধরে রাখা না হয়।

3️⃣5️⃣ Django + Row Lock

```
from django.db import transaction

with transaction.atomic():

    room = (
        Room.objects
        .select_for_update()
        .get(id=101)
    )

    # check booking conflicts

    Booking.objects.create(
        room=room,
        ...
    )
```

Concept:

```
atomic()
   +
select_for_update()
   +
validation
   +
write
```

3️⃣6️⃣ Django + Optimistic Locking

ধরো model:

```
product_id
stock
version
```

Update:

```
updated = Product.objects.filter(
    id=10,
    version=5
).update(
    stock=7,
```

```
    version=6
)
```

যদি:

```
updated == 1
```

Success।

যদি:

```
updated == 0
```

Conflict হয়েছে।

3️⃣7️⃣ Pessimistic vs Optimistic — আবার

Pessimistic

```
Lock first
   ↓
Then work
```

Example:

```
.select_for_update()
```

Optimistic

```
Work assuming no conflict
        ↓
Check version
        ↓
Conflict?
```

3️⃣8️⃣ Deadlock Debugging Mindset

Production-এ deadlock error এলে:

Step 1

কোন দুই transaction involved?

Step 2

কোন resource lock করেছে?

Step 3

কোন resource চাইছে?

Step 4

Lock order কী?

Step 5

দুই transaction কি opposite order-এ lock করছে?

Step 6

Transaction কি unnecessarily long?

Step 7

Index কি appropriate?

3️⃣9️⃣ Deadlock Analysis Example

ধরো logs:

```
Transaction A:
Lock Product 10
Waiting for Product 20

Transaction B:
Lock Product 20
Waiting for Product 10
```

Diagram:

```
A ──holds──→ 10
A ──wants──→ 20

B ──holds──→ 20
B ──wants──→ 10
```

Cycle:

```
A → B → A
```

Solution

সব transaction:

```
Lock 10
then
Lock 20
```

4️⃣0️⃣ Deadlock বনাম Lock Wait

এ দুটো এক না।

Lock Wait

```
T1 → holds A
T2 → waits A
```

T1 commit করলে:

```
T2 continues
```

Deadlock

```
T1 → waits B
T2 → waits A
```

এবং:

```
T1 → T2 → T1
```

cycle।

4️⃣1️⃣ Lock Wait বনাম Deadlock

	Lock Wait	Deadlock
Waiting	✅	✅
Cycle	❌	✅
Eventually resolve	Usually yes	Requires intervention by DB
DB rollback victim	Not necessarily	Typically one transaction
Main solution	Short transactions	Lock ordering + design

4️⃣2️⃣ Production Best Practices ⭐⭐⭐

1. Transaction short রাখো

```
BEGIN
 ↓
minimum DB work
 ↓
COMMIT
```

2. Consistent lock order

```
Always:
A → B
```

3. Proper indexes

যাতে unnecessary rows scan/lock না হয়।

4. Atomic operations

Inventory-এর মতো ক্ষেত্রে:

```
UPDATE ... WHERE stock >= quantity
```

5. Deadlock retry

Application layer-এ safe retry strategy রাখো।

6. External API transaction-এর ভিতরে unnecessarily call করো না

7. Lock everything করো না

যেখানে প্রয়োজন সেখানে lock।

4️⃣3️⃣ Complete Concurrency Architecture

একজন Database Designer হিসেবে চিন্তা করবে:

```
User Request
     ↓
Django API
     ↓
Transaction
     ↓
Appropriate Lock
     ↓
Read / Validate
     ↓
Business Logic
     ↓
Write
     ↓
COMMIT
```

যদি conflict:

```
Conflict
   ↓
Wait / Retry / Rollback
```

যদি deadlock:

```
Deadlock
   ↓
DB detects
   ↓
One transaction rollback
   ↓
Application may retry
```

🧠 Class 63 Golden Rules ⭐⭐⭐

```
1. Lock → Concurrent access control

2. Shared Lock → Read-oriented lock
```

```
3. Exclusive Lock → Write-oriented lock

4. Row Lock → Specific row/resource

5. Table Lock → Broad table-level lock

6. Lock Contention → Multiple transactions compete for same resource

7. Gap Lock → Index gap protection

8. Record Lock → Existing index record protection

9. Next-Key Lock → Record Lock + Gap Lock

10. Deadlock → Transactions waiting for each other

11. Lock Wait ≠ Deadlock

12. Consistent Lock Order → Deadlock risk কমায়

13. Short Transaction → Better concurrency

14. Proper Index → Less unnecessary scanning/locking

15. Atomic UPDATE → Race condition কমাতে useful

16. FOR UPDATE → Transactional update locking

17. Optimistic → Version check

18. Pessimistic → Lock first

19. Deadlock → Application retry strategy useful হতে পারে

20. Transaction-এর ভিতরে unnecessary external API call avoid

21. Lock only what you need

22. Database concurrency = Transaction + Isolation + Locking + Good Design
```

📝 Class 63 Assignment

Task 1 — Lock

Explain করো:

```
Shared Lock
Exclusive Lock
Row Lock
Table Lock
```

Task 2 — Lock Compatibility

এই table পূরণ করো:

Existing	New Shared	New Exclusive
Shared	?	?
Exclusive	?	?

Task 3 — Deadlock ⭐

এই situation explain করো:

```
T1 → locks Account 1
T2 → locks Account 2

T1 → wants Account 2
T2 → wants Account 1
```

কেন deadlock হবে?

Task 4 — Deadlock Prevention

উপরের problem কীভাবে **consistent lock order** দিয়ে solve করবে?

Task 5 — Inventory ⭐

Stock:

```
stock = 10
```

User A:

```
buy = 7
```

User B:

```
buy = 5
```

একটি atomic SQL UPDATE লেখো যাতে stock negative না হয়।

Task 6 — Hotel Booking ⭐⭐⭐

দুই user একই room একই date-এ book করছে।

একটি complete flow লেখো:

```
BEGIN
↓
?
↓
?
↓
?
↓
?
↓
COMMIT / ROLLBACK
```

🔥 Interview Challenge

Interviewer জিজ্ঞেস করল:

> **"Your application is experiencing frequent database deadlocks. How would you diagnose and reduce them?"**

তোমার answer-এর structure:

```
1. Identify conflicting transactions
        ↓
2. Check lock order
        ↓
3. Find opposite lock ordering
        ↓
4. Make lock order consistent
```

```
        ↓
5. Shorten transactions
        ↓
6. Review indexes
        ↓
7. Avoid unnecessary locks
        ↓
8. Add safe retry strategy
        ↓
9. Monitor and test again
```

🎯 Class 63-এর সবচেয়ে গুরুত্বপূর্ণ 6টি বিষয়

```
Shared Lock
Exclusive Lock
Row Lock
Gap/Next-Key Lock
Deadlock
Consistent Lock Order
```

এই 6টা ভালোভাবে বুঝলে তুমি **basic SQL → advanced SQL → database engineering** transition-এর একটা গুরুত্বপূর্ণ অংশ পার করে ফেলেছ।

Next: Class 64 — Database Partitioning। সেখানে আমরা শিখব **Partition কী, Range/List/Hash Partition, Partition Pruning, বড় table কীভাবে ভাগ করা হয়, কখন partition ব্যবহার করা উচিত/উচিত নয়, এবং orders/logs/booking table-এ practical design**।

📚 Class 64 — Database Partitioning

আজকের topic **Database Engineering-এর খুব গুরুত্বপূর্ণ একটি বিষয়**।

এখন পর্যন্ত আমরা শিখেছি:

```
SQL
→ JOIN
→ Subquery
→ CTE
→ Window Function
→ View
→ Transaction
→ Locking
→ Index
```

```
→ EXPLAIN
→ Query Optimization
```

এখন প্রশ্ন:

> **একটা table যদি 10 কোটি বা 50 কোটি row হয়ে যায়, তখন কীভাবে manage করব?**

এখানেই আসে **Database Partitioning**।

1️⃣ Partition কী?

সহজ ভাষায়:

> **একটি বড় table-কে logicalভাবে কয়েকটি ছোট অংশে ভাগ করাকে Partitioning বলে।**

ধরো:

```
orders
100 million rows
```

একটা বিশাল table:

```
orders
──────────────────────
100,000,000 rows
```

আমরা এটাকে বছর অনুযায়ী ভাগ করলাম:

```
orders
│
├── orders_2024
├── orders_2025
├── orders_2026
└── orders_2027
```

Logicalভাবে application-এর কাছে এটি এখনও:

```
orders
```

কিন্তু database ভিতরে data partition অনুযায়ী রাখে।

2️⃣ Partition ≠ Multiple Tables ⭐

এটা খুব গুরুত্বপূর্ণ।

অনেকে মনে করে:

```
orders_2024
orders_2025
orders_2026
```

তাহলে এগুলো manually আলাদা table।

Partitioning-এ সাধারণত:

```
Parent Table
     ↓
Partitions
```

Database partitioning mechanism এগুলোকে একটি partitioned table-এর অংশ হিসেবে manage করে।

3️⃣ Partition কেন ব্যবহার করব?

ধরো:

```
orders = 500 million rows
```

Query:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01';
```

যদি table partitioned হয়:

```
orders
│
├── 2024
├── 2025
├── 2026  ← relevant
└── 2027
```

Database অনেক ক্ষেত্রে শুধু relevant partition scan করতে পারে।

এটাকে বলা হয়:

⭐ Partition Pruning

4️⃣ Partition Pruning ⭐⭐⭐

Partition pruning হলো:

> Query-এর condition দেখে database unnecessary partitions বাদ দিয়ে শুধু relevant partitionগুলো access করা।

ধরো:

```
P1 → 2024
P2 → 2025
P3 → 2026
P4 → 2027
```

Query:

```
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01'
```

তাহলে:

```
P1 ❌
P2 ❌
P3 ✅
P4 ❌
```

এটাই partition pruning।

5️⃣ Partitioning-এর প্রধান ধরন

আজ আমরা 3টি প্রধান type শিখব:

```
1. RANGE
2. LIST
3. HASH
```

আর কিছু DBMS-এ আরও partitioning strategy আছে।

6️⃣ RANGE Partitioning ⭐⭐⭐

সবচেয়ে common practical use case:

> **Date / time অনুযায়ী data ভাগ করা।**

যেমন:

```
2024
2025
2026
2027
```

Example

ধরো:

```
orders
```

আমরা order date অনুযায়ী partition করব।

Conceptually:

```
2024 → Partition 1
2025 → Partition 2
2026 → Partition 3
2027 → Partition 4
```

MySQL example:

```
CREATE TABLE orders (
    order_id BIGINT NOT NULL,
    customer_id BIGINT NOT NULL,
    order_date DATE NOT NULL,
    amount DECIMAL(12,2)
)
PARTITION BY RANGE (YEAR(order_date)) (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p2026 VALUES LESS THAN (2027),
```

```
    PARTITION p2027 VALUES LESS THAN (2028),
    PARTITION pmax VALUES LESS THAN MAXVALUE
);
```

এখানে:

```
YEAR(order_date)
```

দেখে partition select হবে।

7️⃣ `VALUES LESS THAN` বুঝি

এই অংশ:

```
PARTITION p2025 VALUES LESS THAN (2026)
```

মানে:

```
2025 সালের data
```

কারণ 2026-এর আগের year।

Example:

```
2025-01-01 → p2025
2025-07-15 → p2025
2025-12-31 → p2025
```

8️⃣ RANGE Partition Visualization

```
orders
│
├── p2024
│     └── 2024 data
│
├── p2025
│     └── 2025 data
│
├── p2026
│     └── 2026 data
│
```

```
└── p2027
     └── 2027 data
```

Query:

```sql
SELECT *
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2027-01-01';
```

Potentially:

```
Partition Pruning
       ↓
p2026 only
```

9️⃣ Monthly RANGE Partition

বড় logs/orders table হলে yearly partition যথেষ্ট নাও হতে পারে।

তখন:

```
2026-01
2026-02
2026-03
2026-04
...
```

Concept:

```
p2026_01
p2026_02
p2026_03
...
```

এতে পুরনো data archive/drop করাও সহজ হতে পারে।

🔟 LIST Partitioning

LIST partitioning ব্যবহার করি যখন নির্দিষ্ট category/value অনুযায়ী data ভাগ করতে চাই।

ধরো:

```
country
```

বা:

```
region
```

Example:

```
Dhaka
Chittagong
Rajshahi
Sylhet
```

Conceptually:

```
PARTITION BY LIST (region)
```

তারপর:

```
p_dhaka
p_chittagong
p_other
```

1️⃣1️⃣ LIST Example

MySQL-style example:

```
CREATE TABLE customers (
    customer_id BIGINT NOT NULL,
    name VARCHAR(100),
    region INT NOT NULL
)
PARTITION BY LIST (region) (
    PARTITION p_dhaka VALUES IN (1),
    PARTITION p_chittagong VALUES IN (2),
    PARTITION p_rajshahi VALUES IN (3),
    PARTITION p_sylhet VALUES IN (4)
);
```

ধরো:

```
region = 1
```

তাহলে:

```
p_dhaka
```

1️⃣2️⃣ LIST Partition কখন useful?

যখন values discrete এবং business meaning আছে।

যেমন:

```
Region
Country
Business Unit
Tenant Group
Status category
```

কিন্তু একটা গুরুত্বপূর্ণ বিষয়:

> High-cardinality column দিয়ে LIST partition করা সবসময় ভালো design নয়।

যেমন:

```
customer_id
```

এর millions of unique values থাকলে millions partition বানানো absurd।

1️⃣3️⃣ HASH Partitioning ⭐⭐⭐

Hash partitioning-এ database একটি hash function ব্যবহার করে row-কে partition-এ distribute করে।

Concept:

```
customer_id
     ↓
   HASH
     ↓
Partition
```

ধরো:

```
4 partitions
```

তাহলে data roughly distribute হতে পারে:

```
P0
P1
P2
P3
```

1️⃣4️⃣ HASH Example

```
CREATE TABLE orders (
    order_id BIGINT NOT NULL,
    customer_id BIGINT NOT NULL,
    amount DECIMAL(12,2)
)
PARTITION BY HASH(customer_id)
PARTITIONS 4;
```

Database `customer_id` থেকে hash value তৈরি করে row partition-এ রাখে।

1️⃣5️⃣ RANGE vs LIST vs HASH

Type	ভাগ করার ভিত্তি	Common Use
RANGE	Range	Date/time
LIST	Specific values	Region/category
HASH	Hash distribution	Even distribution

সহজভাবে:

```
Date → RANGE
Category/Region → LIST
Distribution → HASH
```

1️⃣6️⃣ Partitioning বনাম Indexing ⭐⭐⭐

এটা interview-এর জন্য খুব important।

Index

```
একটি table
     ↓
Index
     ↓
দ্রুত specific rows খুঁজে পাওয়া
```

Partition

```
একটি বড় table
     ↓
Multiple partitions
     ↓
Unnecessary partitions বাদ দেওয়া
```

অর্থাৎ:

> **Index এবং Partition একে অপরের replacement নয়।**

একসাথে ব্যবহার করা যায়।

1️⃣7️⃣ Example

ধরো:

```
orders = 500 million rows
```

Partition:

```
2024
2025
2026
2027
```

আর প্রতিটি partition-এ appropriate indexes থাকতে পারে।

Query:

```
WHERE order_date >= ...
AND customer_id = 100;
```

তখন:

```
Partition pruning
       ↓
Relevant partition
       ↓
Index
       ↓
Matching rows
```

1️⃣8️⃣ Partitioning বনাম Sharding

দুটো এক জিনিস নয়।

Partitioning

একই database-এর ভিতরে table ভাগ:

```
One DB
│
└── orders
    ├── P1
    ├── P2
    └── P3
```

Sharding

Data multiple database/server-এ ভাগ করা:

```
DB Server 1 → customers 1–1M
DB Server 2 → customers 1M–2M
DB Server 3 → customers 2M–3M
```

সহজভাবে:

```
Partitioning → One DB/table structure split
Sharding → Multiple database/server split
```

1️⃣9️⃣ কখন Partition ব্যবহার করব?

Partitioning সাধারণত useful হতে পারে যখন:

✅ 1. Table অনেক বড়

যেমন:

```
100M+
500M+
1B+ rows
```

তবে **row count একাই partition করার কারণ নয়**।

✅ 2. Queries naturally partition key filter করে

যেমন:

```
WHERE order_date >= ...
AND order_date < ...
```

Date-based queries হলে RANGE partitioning meaningful হতে পারে।

✅ 3. Historical data আছে

যেমন:

```
2020
2021
2022
2023
2024
2025
2026
```

Old partitions archive/manage করা সহজ হতে পারে।

✅ 4. Data lifecycle আছে

ধরো:

```
Keep logs for 12 months
```

তাহলে পুরনো partition drop/archive করা efficient হতে পারে।

2️⃣0️⃣ কখন Partition ব্যবহার করব না? ❌

এটা খুব গুরুত্বপূর্ণ।

❌ Small table

```
10,000 rows
```

এখানে partition করার প্রয়োজন সাধারণত নেই।

❌ শুধু performance-এর আশায়

অনেকে ভাবে:

> Partition করলেই query fast হবে।

না।

Partition key query-তে ব্যবহার না হলে pruning নাও হতে পারে।

❌ Too many partitions

যেমন:

```
10 million partitions
```

খুব খারাপ design।

Partition-এরও management overhead আছে।

❌ Wrong partition key

ধরো business query:

```
WHERE customer_id = 100
```

কিন্তু partition:

```
order_status
```

তাহলে partitioning হয়তো query-তে খুব বেশি সাহায্য করবে না।

2️⃣1️⃣ Partition Key কী?

যে column/expression দিয়ে table partition করা হয় সেটাই:

> **Partition Key**

Example:

```
PARTITION BY RANGE (YEAR(order_date))
```

এখানে:

```
Partition Key = order_date / YEAR(order_date) expression
```

2️⃣2️⃣ Good Partition Key

Good partition key-এর বৈশিষ্ট্য:

```
Frequently filtered
Predictable
Meaningful ranges
Manageable number of partitions
Good data distribution
```

Date খুব common কারণ:

```
orders
logs
events
bookings
transactions
```

সবগুলোতেই time dimension থাকে।

2️⃣3️⃣ Orders Table — Practical Design ⭐⭐⭐

ধরো:

```
orders
1 billion rows
```

Fields:

```
order_id
customer_id
order_date
status
amount
```

Business query:

```sql
SELECT SUM(amount)
FROM orders
WHERE order_date >= '2026-01-01'
AND order_date < '2026-02-01';
```

এখানে:

```
RANGE(order_date)
```

ভালো candidate হতে পারে।

2️⃣4️⃣ Logs Table — Best Practical Example

Logs naturally time-based।

```
application_logs
```

Data:

```
2026-01
2026-02
```

```
2026-03
...
```

তাই:

```
RANGE(date/time)
```

খুব natural।

Architecture:

```
application_logs
│
├── p2026_01
├── p2026_02
├── p2026_03
├── p2026_04
└── ...
```

2️⃣5️⃣ Log Retention ⭐⭐⭐

ধরো requirement:

> Keep logs only 12 months.

Partitioning থাকলে conceptual workflow:

```
Current
│
├── Jan
├── Feb
├── ...
└── Dec
```

পুরনো partition:

```
archive/drop
```

করার মাধ্যমে lifecycle management সহজ হতে পারে।

এটা huge log table-এ বিশেষভাবে useful।

2️⃣6️⃣ Booking Table — Practical Design

আমাদের hotel system:

```
bookings
```

Columns:

```
booking_id
customer_id
room_id
check_in
check_out
status
created_at
```

ধরো:

```
500 million bookings
```

Business queries:

```
Recent bookings
Monthly bookings
Yearly revenue
Historical booking analysis
```

এখানে time-based partitioning candidate হতে পারে।

Possible key:

```
created_at
```

অথবা business requirement অনুযায়ী:

```
check_in
```

কিন্তু কোনটা নেব তা query pattern-এর উপর নির্ভর করবে।

2️⃣7️⃣ `created_at` vs `check_in`

এখানে database designer হিসেবে চিন্তা করতে হবে।

যদি query:

```
WHERE created_at BETWEEN ...
```

তাহলে:

```
created_at
```

logical partition key।

যদি query:

```
WHERE check_in BETWEEN ...
```

হয়, তাহলে:

````
Database ideally:

```text
p2024 ❌
p2025 ❌
p2026 ✅
p2027 ❌
````

তারপর p2026-এর relevant rows search করবে।

এটাই:

Partition Pruning

2️⃣9️⃣ Query যদি Partition Key না ব্যবহার করে?

ধরো partition:

```
RANGE(order_date)
```

কিন্তু query:

```
SELECT *
FROM orders
WHERE customer_id = 100;
```
```

এখানে database customer_id খুঁজতে multiple partitions consider করতে হতে পারে।

তাই:

```
Partitioning
≠
Automatically faster
```

3️⃣0️⃣ Partition + Index

Strong production design অনেক সময়:

```
Partition
   +
Index
   +
Good Query
```

Example:

```
orders
│
├── p2025
│     └── indexes
│
├── p2026
│     └── indexes
│
└── p2027
      └── indexes
```

Exact index/partition behavior অবশ্য DBMS-specific।

3️⃣1️⃣ Partition Maintenance

Large systems-এ future partitions আগে থেকে plan করতে হয়।

ধরো:

```
2026
2027
2028
```

কিন্তু 2029 এ data insert করতে হবে।

তাহলে partition strategy-তে future range handle করতে হবে।

কিছু DBMS-এ:

```
ADD PARTITION
SPLIT PARTITION
MERGE PARTITION
DROP PARTITION
```

এর মতো operations আছে।

Syntax DBMS অনুযায়ী আলাদা।

3️⃣2️⃣ Partition Drop বনাম DELETE ⭐⭐⭐

ধরো January 2025-এর সব logs remove করতে হবে।

Traditional:

```
DELETE FROM logs
WHERE log_date >= '2025-01-01'
AND log_date < '2025-02-01';
```

Huge dataset হলে অনেক rows individually process করতে হতে পারে।

Partitioned design-এ পুরো partition lifecycle operation ব্যবহার করা যেতে পারে:

```
p2025_01
    ↓
DROP / archive
```

এটি retention management-এ অনেক বেশি efficient হতে পারে।

তবে DROP PARTITION destructive operation—production-এ আগে backup/retention policy নিশ্চিত করতে হবে।

3️⃣3️⃣ Partitioning-এর Cost

Partitioning-এর সুবিধা আছে, কিন্তু free নয়।

Potential drawbacks:

```
More complexity
More administration
Partition management
Query planning overhead
Wrong design → little/no benefit
Too many partitions → overhead
Constraints/index behavior can be DBMS-specific
```

তাই:

> **Partition because you have a problem it solves—not because partitioning sounds advanced.**

3️⃣4️⃣ Partitioning Decision Flow ⭐⭐⭐

Database designer হিসেবে:

```
Table কি খুব বড়?
        ↓
       YES
        ↓
Queries কি naturally কোনো range/value filter করে?
        ↓
       YES
        ↓
Partition pruning সম্ভব?
        ↓
       YES
        ↓
Partition key নির্বাচন
        ↓
Partition count design
        ↓
Indexes design
        ↓
EXPLAIN / benchmark
        ↓
Production
```

3️⃣5️⃣ Partitioning + Query Optimization

আমাদের আগের workflow:

```
Slow Query
   ↓
EXPLAIN
   ↓
Index
   ↓
Optimize
```

Large table-এ এখন:

```
Slow Query
   ↓
EXPLAIN
   ↓
Partition pruning হচ্ছে?
   ↓
Index ব্যবহার হচ্ছে?
   ↓
Rows examined?
   ↓
Optimize
```

3️⃣6️⃣ Partitioning + Database Designer

তুমি যদি ভবিষ্যতে database designer হও, তাহলে তোমাকে এই প্রশ্নগুলো করতে হবে:

```
1. Table কত বড় হবে?
2. Data growth rate কত?
3. Query pattern কী?
4. Historical data কতদিন থাকবে?
5. Date-based filtering আছে?
6. Partition pruning সম্ভব?
7. কত partition হবে?
8. Index কোথায় লাগবে?
9. Archive policy কী?
10. Backup/restore strategy কী?
```

3️⃣7️⃣ Real Architecture

ধরো একটি e-commerce company:

```
orders
500M rows
```

Design:

```
orders
│
├── 2024
│
├── 2025
│
├── 2026
│
└── 2027
```

প্রতিটি partition-এ appropriate indexing:

```
order_date
customer_id
```

Query:

```
2026 orders
      ↓
Partition pruning
      ↓
p2026
      ↓
Index
      ↓
Rows
```

3️⃣8️⃣ Partitioning vs Normalization

এগুলো completely different concepts।

Normalization

Data structure/design:

```
Reduce redundancy
Improve consistency
```

Partitioning

Large table management:

```
Split physical/logical data access
Improve manageability
Enable pruning
```

অর্থাৎ:

```
Normalization → Data design
Partitioning → Large-scale data organization
```

একসাথে ব্যবহার করা যায়।

3️⃣9️⃣ Partitioning vs Sharding vs Index

Feature	Index	Partitioning	Sharding
Main purpose	Faster lookup	Split large table	Distribute data
Scope	Table access structure	Table partitions	Multiple DB/server
Multiple servers	❌	Usually ❌	✅
Pruning	❌	✅	Routing/shard pruning
Complexity	Low–Medium	Medium	High
Typical scale	Small → Huge	Huge tables	Very large distributed systems

---

4️⃣0️⃣ Interview Questions ⭐⭐⭐

Q1. What is database partitioning?

Answer:

> Database partitioning is the technique of dividing a large table into smaller logical partitions based on a partition key, while presenting the data as one logical table.

Q2. What is partition pruning?

> Partition pruning is the optimizer's ability to skip irrelevant partitions and access only partitions that can contain rows matching the query predicate.

Q3. RANGE vs LIST vs HASH?

```
RANGE → ranges, especially dates
LIST → predefined values
HASH → hash-based distribution
```

Q4. Does partitioning replace indexing?

No.

```
Partitioning ≠ Indexing
```

They solve different problems and can work together.

Q5. When should you avoid partitioning?

```
Small table
No useful partition key
No pruning benefit
Too many partitions
Complexity not justified
```

🧠 Class 64 Golden Rules

```
1. Partitioning → Large table ভাগ করা

2. Partition Key → যে value/expression দিয়ে partition করা হয়

3. RANGE → Range/date-based data

4. LIST → Specific values
```

```
5. HASH → Hash-based distribution

6. Partition Pruning → Unnecessary partitions skip করা

7. Partitioning ≠ Indexing

8. Partitioning ≠ Sharding

9. Large table ≠ Automatically partition করতে হবে

10. Query pattern দেখে partition key নির্বাচন করো

11. Date/time → Common RANGE partition candidate

12. Logs → Time-based partition খুব common

13. Orders → Date-based partition useful হতে পারে

14. Booking → created_at/check_in query pattern অনুযায়ী নির্বাচন

15. Too many partitions → Bad

16. Small table → Usually unnecessary

17. Partition + Index একসাথে ব্যবহার করা যায়

18. Retention management-এ partition useful

19. DROP partition → powerful but destructive

20. EXPLAIN/benchmark দিয়ে verify করো

21. Partitioning solves a specific scale/management problem

22. Good database design = Requirement + Query Pattern + Data Growth
```

📝 Class 64 Assignment

Task 1 — Basic

Explain:

```
Partition
Partition Key
Partition Pruning
```

Task 2 — Type

নিচের ক্ষেত্রে কোন partitioning সবচেয়ে logical?

```
A. Orders by year
B. Customers by region
C. Data evenly distributed using customer_id
```

Choose:

```
RANGE
LIST
HASH
```

Task 3 — Orders Design ⭐

তোমার `orders` table:

```
order_id
customer_id
order_date
amount
```

Data:

```
500 million rows
```

এটাকে 2024, 2025, 2026, 2027 অনুযায়ী partition করার design লেখো।

Task 4 — Partition Pruning ⭐⭐⭐

Partitions:

```
p2024
p2025
p2026
p2027
```

Query:

```sql
SELECT SUM(amount)
FROM orders
WHERE order_date >= '2026-05-01'
AND order_date < '2026-06-01';
```

কোন partition(s) relevant হতে পারে এবং কেন?

Task 5 — Hotel Booking

Hotel system:

```
500 million bookings
```

Columns:

```
booking_id
customer_id
room_id
check_in
check_out
created_at
status
```

প্রশ্ন:

`created_at` নাকি `check_in` —কোনটাকে partition key হিসেবে নেবে?

শুধু answer নয়, **query pattern দিয়ে justification** দাও।

Task 6 — Interview Challenge 🔥

একজন interviewer বলল:

> **"Our orders table has 800 million rows. Should we partition it?"**

তুমি সরাসরি `YES` বলবে না।

তোমার answer হওয়া উচিত:

```
First:
→ Query patterns
→ Data growth
→ Retention policy
```

```
→ Partition key
→ Partition pruning possibility
→ Existing indexes
→ EXPLAIN / benchmark
→ Operational complexity

Then:
→ Decide whether partitioning is justified
```

🎯 আজকের সবচেয়ে গুরুত্বপূর্ণ Concept

এক লাইনে মনে রাখো:

```
Huge Table
    ↓
Partition Key
    ↓
Partitions
    ↓
Query Filter
    ↓
Partition Pruning
    ↓
Only relevant data
    ↓
Better performance / management
```

আর **Database Designer-এর Golden Rule**:

> **“Partitioning করব কারণ table বড়—এটা যথেষ্ট কারণ নয়; partitioning করব কারণ data/query/retention pattern থেকে measurable benefit পাওয়া যাবে।”**

Next → Class 65: Database Replication & High Availability
সেখানে আমরা শিখব **Primary/Replica, Read Replica, Replication, Failover, High Availability, Backup বনাম Replication, এবং Django application-এ read/write database routing**।

📚 Class 65 — Database Replication & High Availability

আজ আমরা Database Engineering-এর আরেকটি **production-level topic** শিখব।

Class 64-এ আমরা বড় table কীভাবে **Partition** করা যায় শিখেছি।

আজকের প্রশ্ন:

> **Database server যদি down হয়ে যায়, তাহলে application কী করবে?**

অথবা:

> **একসাথে হাজার হাজার user database থেকে read করলে কীভাবে load কমাব?**

এখানেই আসে:

```
Replication
High Availability
Read Replica
Failover
Backup
Database Routing
```

1️⃣ Replication কী?

সহজভাবে:

> **একটি database-এর data অন্য database server-এ কপি/replicate করে রাখাকে Database Replication বলে।**

ধরো আমাদের primary database:

```
id="p7m3x1"
Primary DB
    │
    │ Replication
    ↓
Replica DB
```

আরও replica থাকতে পারে:

```
id="k4q8n2"
              ┌──→ Replica 1
Primary DB ──┼──→ Replica 2
              └──→ Replica 3
```

2️⃣ Primary এবং Replica কী?

Primary

যে database সাধারণত application-এর **write operations** গ্রহণ করে।

```
INSERT
UPDATE
DELETE
```

উদাহরণ:

```
id="x8r2m5"
Primary DB
   ↑
INSERT order
UPDATE stock
DELETE user
```

Replica

Primary থেকে data replicate করে রাখা database server।

অনেক architecture-এ replica মূলত **read traffic** handle করে।

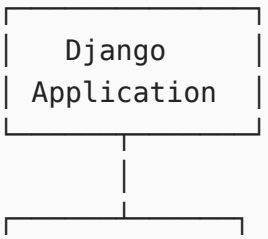
```
id="v3q7k1"
Replica
   ↑
SELECT
SELECT
SELECT
```

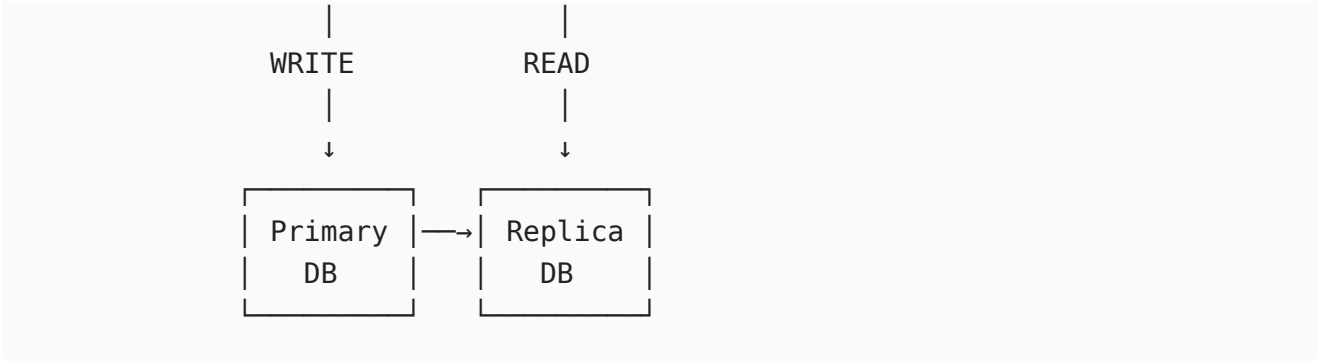
তাই একে বলা হয়:

Read Replica

3️⃣ Basic Architecture ⭐⭐⭐

একটা simple production architecture:





4️⃣ Replication কীভাবে কাজ করে?

ধরো:

```
Primary
orders = 100
```

User নতুন order দিল:

```
INSERT INTO orders (...);
```

Primary:

```
100
 ↓
101
```

Replication mechanism-এর মাধ্যমে replica-তেও পরিবর্তন পৌঁছায়:

```
Primary = 101
Replica = 101
```

Conceptually:

```
Application
     ↓
Primary
     ↓
Replication
     ↓
Replica
```

বাস্তব replication mechanism DBMS অনুযায়ী আলাদা।

5️⃣ Primary → Replica

একাধিক replica:

```
                    ┌── Replica 1
                    │
Primary ────────────┼── Replica 2
                    │
                    └── Replica 3
```

এখন read workload distribute করা যায়।

যেমন:

```
100,000 SELECT requests
```

একটা server-এর বদলে:

```
Replica 1 → 33k
Replica 2 → 33k
Replica 3 → 34k
```

এটা architecture/load balancing strategy-এর উপর নির্ভর করবে।

6️⃣ Read Replica কেন দরকার?

ধরো application-এ:

```
10% → WRITE
90% → READ
```

যেমন:

```
INSERT order
UPDATE profile
```

কিন্তু:

```
SELECT products
SELECT categories
SELECT orders
```

```
SELECT dashboard
SELECT reports
```

অনেক বেশি।

তখন:

```
Primary
  ↓
Writes

Replicas
  ↓
Reads
```

করলে primary-এর read workload কমানো যেতে পারে।

7️⃣ Read Scaling ⭐⭐⭐

ধরো:

```
Primary
CPU = 90%
```

কারণ:

```
READ + WRITE
```

আমরা read replicas যোগ করলাম:

```
Primary
  ↓
Write

Replica 1 → Read
Replica 2 → Read
Replica 3 → Read
```

এখন read workload scale করা সহজ হতে পারে।

8️⃣ কিন্তু Replica কি সবসময় Up-to-Date?

না।

এটাই খুব important।

Replication-এ কিছু ক্ষেত্রে:

```
Primary
   ↓
   101
```

Replica হয়তো:

```
Replica
   ↓
   100
```

কিছু সময়ের জন্য।

এটাকে বলা হয়:

Replication Lag

9️⃣ Replication Lag ⭐⭐⭐

ধরো:

```
T1:
INSERT order 500
```

Primary immediately:

```
order = 500
```

কিন্তু replica-তে change পৌঁছাতে একটু সময় লাগতে পারে।

তখন:

```
Primary → 500
Replica → 499
```

এই difference হলো:

> **Replication Lag**

🔟 Real Problem

User order create করল:

```
POST /orders
```

Write গেল:

```
Primary
```

তারপর immediately:

```
GET /orders
```

যদি GET replica-তে যায়:

```
Primary → order exists
Replica → order not arrived yet
```

User দেখতে পারে:

```
"No order"
```

যদিও order successfully তৈরি হয়েছে।

এটাকে বলা যায়:

Read-after-write consistency problem

1️⃣1️⃣ Read-after-write Solution

Critical read-এর ক্ষেত্রে:

```
WRITE
 ↓
Primary
 ↓
Immediately READ
 ↓
Primary
```

অথবা system অনুযায়ী এমন consistency strategy ব্যবহার করতে হবে যাতে replica lag-এর কারণে incorrect UX না হয়।

1️⃣2️⃣ Primary/Replica Routing

Concept:

```
INSERT
UPDATE
DELETE
    ↓
Primary
```

আর:

```
SELECT
    ↓
Replica
```

কিন্তু সব SELECT blindly replica-তে পাঠানো ঠিক নয়।

1️⃣3️⃣ কোন Read Primary-তে যাবে?

যেমন:

```
User just updated profile
```

তারপর:

```
GET profile
```

Immediate consistency দরকার হলে primary ব্যবহার করা ভালো হতে পারে।

অন্যদিকে:

```
Product catalog
```

যেখানে কয়েক milliseconds/seconds lag acceptable:

```
Replica
```

ব্যবহার করা যেতে পারে।

1️⃣4️⃣ Failover কী?

ধরো:

```
Primary DB
      ↓
      💥 DOWN
```

Application আর write করতে পারছে না।

তখন একটি healthy replica-কে নতুন primary করা হতে পারে।

```
Before:

Primary
   ↓
Replica


After:

Old Primary 💥

Replica
   ↓
New Primary
```

এটাকে বলে:

Failover

1️⃣5️⃣ High Availability — HA ⭐⭐⭐

High Availability-এর উদ্দেশ্য:

> **System যেন failure হলেও যতটা সম্ভব available থাকে।**

Architecture:

```
Application
      ↓
```

```
DB layer
   ┌──┴──┐
   ↓     ↓
Primary Replica
```

Primary fail করলে:

```
Replica
   ↓
Promote
   ↓
New Primary
```

তারপর application নতুন primary-তে write করবে।

1️⃣6️⃣ HA ≠ Replication

এটা interview-এর জন্য খুব important।

Replication:

```
Data copies
```

High Availability:

```
System remains available despite failures
```

Replication HA-এর একটি component হতে পারে।

কিন্তু শুধু replication থাকলেই complete HA architecture হয় না।

1️⃣7️⃣ Failover Flow

```
Primary
   ↓
Health Check
   ↓
Failure detected
   ↓
Choose healthy replica
   ↓
Promote replica
```

```
   ↓
Update routing
   ↓
Application reconnects
```

1️⃣8️⃣ Automatic vs Manual Failover

Manual

Admin manually:

```
Replica → Primary
```

করবে।

Automatic

Monitoring/orchestration system:

```
Failure detected
       ↓
Automatic promotion
       ↓
Routing updated
```

করতে পারে।

Production systems-এ automation useful, তবে incorrect automatic failover-এর risk-ও আছে।

1️⃣9️⃣ Split Brain ⭐

Advanced HA problem।

ধরো ভুলভাবে দুই server-ই মনে করছে:

```
"I am Primary"
```

তাহলে:

```
DB1 → Primary
DB2 → Primary
```

দু জায়গায় conflicting writes হতে পারে।

এটা:

Split Brain

HA system design-এ এটি prevent করা অত্যন্ত গুরুত্বপূর্ণ।

2️⃣0️⃣ Backup বনাম Replication ⭐⭐⭐

অনেক beginner ভাবে:

> Replica আছে → Backup দরকার নেই।

❌ ভুল।

Replication এবং Backup-এর উদ্দেশ্য আলাদা।

2️⃣1️⃣ Replication

```
Primary
   ↓
Replica
```

যদি Primary-তে ভুল করে:

```
DELETE FROM customers;
```

হয় এবং সেই change replicate হয়ে যায়:

```
Primary → DELETE
Replica → DELETE
```

Replica-ও data হারাতে পারে।

2️⃣2️⃣ Backup

Backup হলো:

> নির্দিষ্ট সময়ের database copy/snapshot/data recovery source রাখা।

যেমন:

```
Monday Backup
Tuesday Backup
Wednesday Backup
```

যদি ভুল করে:

```
DELETE all customers
```

হয়ে যায়, backup থেকে recovery করা সম্ভব হতে পারে।

2️⃣3️⃣ Backup বনাম Replication

	Backup	Replication
Main purpose	Recovery	Availability/read scaling
Historical recovery	✅	❌
Server failure	সাহায্য করতে পারে	দ্রুত failover-এ useful
Accidental DELETE	Recovery source হতে পারে	DELETE replicate হতে পারে
Read scaling	❌	✅
HA	নিজে complete HA নয়	HA architecture-এর অংশ হতে পারে

Golden Rule ⭐

> **Replication is not a backup.**

2️⃣4️⃣ RPO এবং RTO ⭐⭐⭐

আজকের class-এর important production concepts।

RPO — Recovery Point Objective

প্রশ্ন:

> Failure হলে সর্বোচ্চ কত data loss tolerate করতে পারি?

ধরো:

```
RPO = 5 minutes
```

মানে business এমন architecture চায় যাতে recovery scenario-তে প্রায় 5 মিনিটের বেশি data loss না হয়।

2️⃣5️⃣ RTO — Recovery Time Objective

প্রশ্ন:

> Failure-এর পর কত সময়ের মধ্যে system restore করতে হবে?

ধরো:

```
RTO = 10 minutes
```

মানে:

```
Failure
 ↓
Recovery
 ↓
≤ 10 minutes target
```

2️⃣6️⃣ RPO vs RTO

মনে রাখো:

```
RPO → How much data can we lose?
RTO → How much time can we be down?
```

2️⃣7️⃣ Django Multiple Database

Django multiple database support করে।

`settings.py` :

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
```

```
        "NAME": "app_db",
        "USER": "postgres",
        "PASSWORD": "password",
        "HOST": "primary-db",
        "PORT": "5432",
    },

    "replica": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "app_db",
        "USER": "postgres",
        "PASSWORD": "password",
        "HOST": "replica-db",
        "PORT": "5432",
    },
}
```

এখানে:

```
default → Primary
replica → Read Replica
```

2️⃣8️⃣ Django Direct Database Selection

Primary:

```
Order.objects.using("default").create(
    customer_id=1,
    amount=500
)
```

Replica:

```
orders = Order.objects.using("replica").all()
```

2️⃣9️⃣ Django Database Router ⭐⭐⭐

প্রতিবার `.using()` manually লেখা inconvenient।

তাই Django-তে:

> **Database Router**

ব্যবহার করা যায়।

Concept:

```
SELECT
 ↓
Replica

INSERT
UPDATE
DELETE
 ↓
Primary
```

3️⃣0️⃣ Basic Router

```python
class DatabaseRouter:

    def db_for_read(self, model, **hints):
        return "replica"

    def db_for_write(self, model, **hints):
        return "default"
```

তারপর:

```python
DATABASE_ROUTERS = [
    "myapp.routers.DatabaseRouter"
]
```

3️⃣1️⃣ Router-এর সমস্যা

এই simple router production-ready নয়।

কারণ:

```
WRITE → Primary
READ → Replica
```

কিন্তু write-এর পর immediate read করলে replication lag problem হতে পারে।

Example:

```
POST /order
   ↓
Primary

GET /order
   ↓
Replica

Replica lag
   ↓
Order not found
```

তাই real production architecture-এ routing rules আরও sophisticated হতে পারে।

3️⃣2️⃣ Better Django Strategy

Conceptually:

```
Normal read
    ↓
Replica

Critical read after write
    ↓
Primary
```

আর failover হলে:

```
Current Primary
      ↓
Failure
      ↓
New Primary
      ↓
Routing configuration/service discovery
```

3️⃣3️⃣ Read Replica Load Balancing

যদি 3 replicas থাকে:

```
Primary
 ├── Replica 1
```

```
  ├── Replica 2
  └── Replica 3
```

Read traffic:

```
SELECT
 ↓
Load Balancer / DB proxy / routing layer
 ↓
Replica 1/2/3
```

এতে application-কে প্রতিটি replica individually manage করতে নাও হতে পারে।

3️⃣4️⃣ Database Proxy Concept

Production architecture:

```
Django
   ↓
DB Proxy
   ↓
┌──────────┬──────────┐
│          │          │
Primary   Replica1    Replica2
```

Proxy:

```
WRITE → Primary
READ → Replica
```

করতে পারে এবং failover/routing সহজ করতে পারে—specific technology অনুযায়ী behavior আলাদা।

3️⃣5️⃣ Replication Types — Basic Concept

Replication বিভিন্নভাবে হতে পারে।

Synchronous

```
Primary
  ↓
wait
```

```
  ↓
Replica confirms
  ↓
Commit/ack
```

Data consistency বেশি strong হতে পারে, কিন্তু latency বাড়তে পারে।

Asynchronous

```
Primary
  ↓
Commit
  ↓
Replica receives later
```

Performance/latency ভালো হতে পারে, কিন্তু replication lag হতে পারে।

3️⃣6️⃣ Sync vs Async

	Synchronous	Asynchronous
Replica confirmation	অপেক্ষা করে	সাধারণত অপেক্ষা করে না
Consistency	Stronger সম্ভাবনা	Lag possible
Latency	বেশি হতে পারে	কম হতে পারে
Complexity	Higher	তুলনামূলক সহজ
Data loss risk on failure	architecture অনুযায়ী কম হতে পারে	recent unreplicated data থাকতে পারে

বাস্তব DBMS implementation অনেক বেশি nuanced।

3️⃣7️⃣ Hotel Management Example ⭐⭐⭐

আমাদের hotel system:

```
Django
  ↓
Database
```

Production-এ:

```
                    ┌── Replica 1
                    │
Django → Primary ───┼── Replica 2
                    │
                    └── Replica 3
```

Booking create

```
POST /bookings
       ↓
Primary
```

Available rooms search

```
GET /rooms/available
       ↓
Replica
```

কিন্তু final booking confirmation-এর critical availability check:

```
Transaction
   ↓
Primary
   ↓
Lock
   ↓
Check availability
   ↓
Create booking
```

এখানে consistency গুরুত্বপূর্ণ।

3️⃣8️⃣ Hotel Failure Scenario

Primary:

```
💥 DOWN
```

System:

```
Replica 1
    ↓
```

```
Promote
    ↓
New Primary
```

Routing:

```
Django
   ↓
New Primary
```

Read replicas architecture অনুযায়ী পুনরায় configure হতে পারে।

3️⃣9️⃣ E-commerce Example

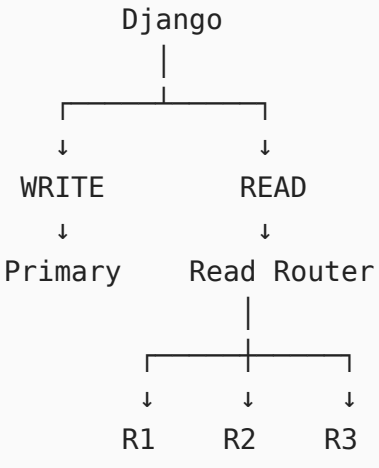
Traffic:

```
1000 requests/sec
```

ধরো:

```
900 reads
100 writes
```

Architecture:



এটাই read scaling-এর basic idea।

4️⃣0️⃣ Replication + Partitioning

দুটো একসাথে ব্যবহার করা যায়।

```
Database
  │
  ├── Partitioning
  │       ↓
  │   Large table split
  │
  └── Replication
          ↓
      DB copies
```

Different problems solve করে:

```
Partitioning → Large table/data organization
Replication → Availability/read scaling
```

4️⃣1️⃣ Replication + Indexing

এগুলোকেও combine করা যায়:

```
Application
    ↓
Replica
    ↓
Partition
    ↓
Index
    ↓
Rows
```

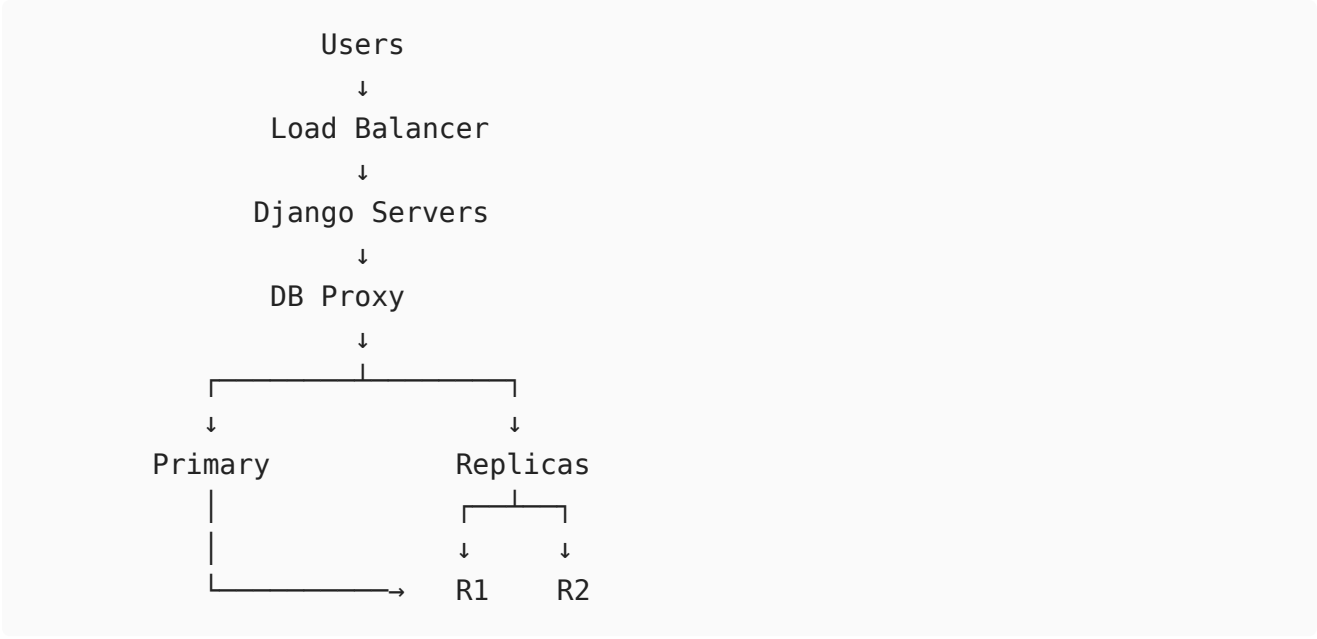
Database architecture এখন:

```
Index
+
Partition
+
Replication
+
Transactions
+
Locking
```

সবগুলো আলাদা সমস্যার solution।

4️⃣2️⃣ Full Production Architecture ⭐⭐⭐

একটি mature application:



আর:

```
Backup System
      ↓
Object Storage / Backup Storage
```

Monitoring:

```
Monitoring
   ↓
DB health
Replication lag
CPU
Memory
Connections
Disk
Errors
```

4️⃣3️⃣ Monitoring কী কী?

Production database-এ monitor করতে পারো:

```
CPU
Memory
Disk usage
Connections
Query latency
Slow queries
Replication lag
Lock waits
Deadlocks
Replication status
Backup status
```

4️⃣4️⃣ Database Failure-এর সময় কী হবে?

ধরো:

```
Primary 💥
```

Without HA

```
Application
   ↓
Primary ❌
   ↓
Application down
```

With HA

```
Primary ❌
   ↓
Failover
   ↓
New Primary
   ↓
Application continues
```

Recovery speed architecture অনুযায়ী নির্ভর করবে।

4️⃣5️⃣ Backup Strategy

একটি production database-এর জন্য সাধারণভাবে ভাবতে হবে:

```
Primary
   +
Replica(s)
   +
Backup
   +
Monitoring
   +
Failover Strategy
```

একটা জিনিসের উপর depend করা উচিত নয়।

4️⃣6️⃣ Common Mistakes ❌

❌ Mistake 1

> Replica = Backup

ভুল।

❌ Mistake 2

> Replication থাকলে database কখনো down হবে না।

ভুল।

❌ Mistake 3

> সব SELECT replica-তে পাঠাব।

ভুল।

Consistency requirements বুঝতে হবে।

❌ Mistake 4

> Failover মানেই application automatically ঠিক হয়ে যাবে।

সবসময় না।

Routing, connection handling, health checks, promotion, DNS/proxy/service discovery ইত্যাদি properly design করতে হয়।

❌ Mistake 5

> অনেক replica = unlimited scaling

ভুল।

Replication নিজেও network, storage এবং write/replication overhead তৈরি করে।

🧠 Class 65 Golden Rules ⭐⭐⭐

```
1. Replication → Database data copy রাখা

2. Primary → Usually handles writes

3. Replica → Copy of primary data

4. Read Replica → Read workload handle করতে পারে

5. Replication Lag → Primary ও Replica data-এর temporary difference

6. Read-after-write → Replica lag-এর কারণে সমস্যা হতে পারে

7. Failover → Failed primary-এর বদলে another DB promote করা

8. High Availability → Failure-এর পর service available রাখার goal

9. Replication ≠ High Availability

10. Replication ≠ Backup

11. Backup → Disaster / accidental deletion recovery

12. RPO → Acceptable data loss

13. RTO → Acceptable recovery time

14. Synchronous → Stronger consistency, potentially higher latency

15. Asynchronous → Lower latency, replication lag possible

16. Django → Multiple DB support করে
```

```
17. Database Router → Read/write routing করতে পারে

18. Write → Primary

19. Read → Replica, when consistency allows

20. Critical read → Primary হতে পারে

21. Deadlock/locking → Concurrency control

22. Partitioning → Large table management

23. Replication → Availability/read scaling

24. Indexing → Query performance

25. Production DB → Backup + Replication + Monitoring + Failover
```

📝 Class 65 Assignment

Task 1 — Basic

Explain:

```
Primary
Replica
Read Replica
Replication
Replication Lag
Failover
High Availability
```

Task 2 — Architecture ⭐

এই architecture complete করো:

```
Django
  ↓
?
  ↓
┌──────────────┐
│              │
?              ?
```

```
|                |
Write            Read
```

Task 3 — Backup vs Replication ⭐⭐⭐

এই scenario:

> একজন admin ভুল করে `customers` table-এর সব rows delete করে ফেলেছে। সেই DELETE replica-তেও replicate হয়েছে।

প্রশ্ন:

কেন replica থেকে সহজে recovery করা যাবে না এবং এখানে backup কেন দরকার?

Task 4 — Replication Lag

Timeline:

```
10:00:00 → User creates order
10:00:00 → Primary has order
10:00:01 → Replica still hasn't received it
```

User যদি `GET /orders` করে এবং request replica-তে যায়, কী সমস্যা হতে পারে?

Task 5 — Django Router ⭐⭐⭐

এই requirement:

```
INSERT → Primary
UPDATE → Primary
DELETE → Primary
SELECT → Replica
```

একটি basic Django `DatabaseRouter` লেখো।

Task 6 — Hotel System 🔥

আমাদের hotel application-এর জন্য architecture design করো:

```
Django
Primary
Replica 1
Replica 2
Backup
Failover
```

তারপর explain করো:

1. Booking কোথায় write হবে?
2. Room search কোথায় read হবে?
3. Final availability check কোথায় হবে?
4. Primary down হলে কী হবে?
5. Backup কেন আলাদা রাখতে হবে?

🎯 Interview Challenge

Interviewer জিজ্ঞেস করল:

> **"We have a Django e-commerce application. The database receives 90% reads and 10% writes. How would you scale the database?"**

Strong answer:

```
Current DB
    ↓
Analyze workload
    ↓
Primary + Read Replicas
    ↓
Writes → Primary
Reads → Replicas
    ↓
Monitor replication lag
    ↓
Critical read → Primary when needed
    ↓
Load balancing / routing
    ↓
Backup
    ↓
Failover strategy
    ↓
Monitoring
```

আর যদি জিজ্ঞেস করে:

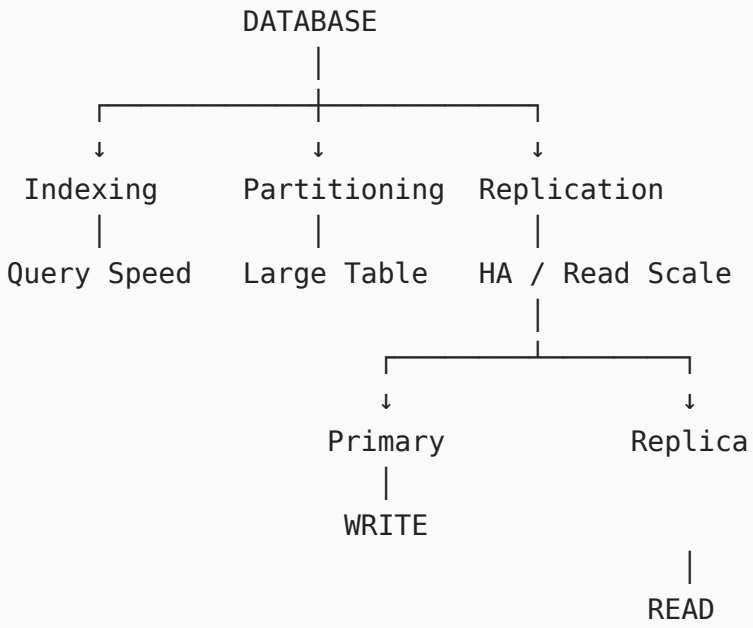
> **"Is replication a backup?"**

তোমার এক লাইনের answer:

> **No. Replication improves availability/read scaling, while backups are primarily for recovery and historical restore.**

🏆 আজকের সবচেয়ে গুরুত্বপূর্ণ ধারণা

পুরো Class 65-কে এভাবে মনে রাখো:



এবং পুরো production mindset:

```
Database Engineering
        ↓
Transactions
        ↓
Locking
        ↓
Indexing
        ↓
Partitioning
        ↓
Replication
        ↓
Backup
        ↓
Failover
```

```
        ↓
High Availability
```

Next → Class 66: Database Security & Access Control — সেখানে আমরা শিখব **Database User/Role, GRANT, REVOKE, Least Privilege, Authentication vs Authorization, SQL Injection, Django DB security, sensitive data protection এবং production database security design**।

📚 Class 66 — Database Security & Access Control

আজকের Class 66 খুবই গুরুত্বপূর্ণ, কারণ **ভালো database design শুধু fast হওয়া যথেষ্ট নয়—secure-ও হতে হবে।**

আমরা শিখব:

1. Database Security কী
2. User & Role
3. Authentication vs Authorization
4. `GRANT`
5. `REVOKE`
6. Least Privilege
7. SQL Injection
8. Django Database Security
9. Sensitive Data Protection
10. Password Security
11. Secrets Management
12. Production Database Security Architecture
13. Common Security Mistakes
14. Database Security Checklist

1️⃣ Database Security কী?

সহজভাবে:

> **Database-এর data এবং database operations-কে unauthorized access, modification, deletion এবং leakage থেকে protect করাকে Database Security বলে।**

ধরো hotel application-এর database:

```
customers
rooms
bookings
payments
```

সব user-এর সব data access করা উচিত নয়।

যেমন:

```
Customer
   ↓
নিজের booking দেখতে পারবে

Receptionist
   ↓
Booking manage করতে পারবে

Finance
   ↓
Payment-related data দেখতে পারবে

DB Admin
   ↓
Database administration
```

এটাই access control-এর ধারণা।

2️⃣ Database Security-এর Main Goals ⭐

```
Confidentiality
Integrity
Availability
```

Confidentiality

Unauthorized person যেন data দেখতে না পারে।

Integrity

Unauthorized/invalid modification যেন না হয়।

Availability

Authorized user যেন প্রয়োজনের সময় database ব্যবহার করতে পারে।

এগুলোকে অনেক সময় **CIA Triad** বলা হয়:

```
C → Confidentiality
I → Integrity
A → Availability
```

3️⃣ Authentication vs Authorization ⭐⭐⭐

এটা অবশ্যই পরিষ্কার করতে হবে।

Authentication

> **তুমি কে?**

Example:

```
Username
Password
```

User login করল।

```
Rahim
   ↓
Login
   ↓
Identity verified
```

Authorization

> **তুমি কী করতে পারবে?**

ধরো Rahim authenticated।

কিন্তু সে:

```
DELETE customers
```

করতে পারবে কি?

এটা authorization-এর বিষয়।

4️⃣ সহজে মনে রাখো

```
Authentication
      ↓
WHO are you?

Authorization
      ↓
WHAT can you do?
```

Example

```
Login successful
        ↓
Authentication ✅
        ↓
Can delete users?
        ↓
Authorization
```

5️⃣ Database User কী?

Database server-এ আলাদা user account থাকতে পারে।

যেমন:

```
root
app_user
report_user
readonly_user
admin_user
```

প্রত্যেক user-এর permission আলাদা হতে পারে।

6️⃣ কেন Application-এর জন্য `root` ব্যবহার করা উচিত নয়? ❌

ধরো Django application:

```
Django
   ↓
```

```
root database user
```

যদি application-এর কোনো security vulnerability হয় এবং attacker database credential/access পেয়ে যায়:

```
Attacker
   ↓
root
   ↓
Full DB control
```

খুব dangerous।

Better:

```
Django
   ↓
app_user
   ↓
Only required permissions
```

7️⃣ Role কী?

Role হলো permissions-এর একটি logical collection।

ধরো:

```
role_readonly
role_app
role_reporting
role_admin
```

একজন user-কে role দেওয়া যায়।

Concept:

```
User
 ↓
Role
 ↓
Permissions
```

8️⃣ User বনাম Role

```
User
= কে database access করছে

Role
= সে কী ধরনের permission পাবে
```

Example:

```
Rahim
   ↓
reporting_role
   ↓
SELECT permission
```

9️⃣ GRANT ⭐⭐⭐

`GRANT` দিয়ে permission দেওয়া হয়।

MySQL example:

```sql
GRANT SELECT
ON hotel_db.*
TO 'report_user'@'localhost';
```

মানে:

```
report_user
     ↓
SELECT
     ↓
hotel_db
```

🔟 Multiple Permission

```sql
GRANT SELECT, INSERT, UPDATE
ON hotel_db.bookings
TO 'booking_user'@'localhost';
```

এই user:

```
SELECT   ✅
INSERT   ✅
UPDATE   ✅
DELETE   ❌
```

1️⃣1️⃣ All Privileges

```
GRANT ALL PRIVILEGES
ON hotel_db.*
TO 'admin_user'@'localhost';
```

এটা powerful permission।

Production application user-কে blindly `ALL PRIVILEGES` দেওয়া উচিত নয়।

1️⃣2️⃣ REVOKE ⭐⭐⭐

Permission remove করতে:

```
REVOKE UPDATE
ON hotel_db.bookings
FROM 'booking_user'@'localhost';
```

এখন:

```
SELECT → ✅
INSERT → ✅
UPDATE → ❌
```

1️⃣3️⃣ GRANT বনাম REVOKE

Command	কাজ
`GRANT`	Permission দেওয়া
`REVOKE`	Permission সরানো

মনে রাখো:

```
GRANT  → Give
REVOKE → Remove
```

1️⃣4️⃣ Least Privilege ⭐⭐⭐

আজকের সবচেয়ে গুরুত্বপূর্ণ security principle:

> **একজন user/application-কে তার কাজের জন্য যতটুকু permission দরকার, শুধু ততটুকুই দেওয়া উচিত।**

এটাই:

Principle of Least Privilege

1️⃣5️⃣ Example

ধরো reporting application শুধু report read করবে।

❌ Bad:

```
report_user
→ SELECT
→ INSERT
→ UPDATE
→ DELETE
→ DROP
→ ALTER
```

এটা unnecessary।

✅ Better:

```
report_user
→ SELECT
```

1️⃣6️⃣ Django Application User

ধরো Django application শুধু:

```
SELECT
INSERT
UPDATE
```

করবে।

তাহলে application database user-এর:

```
DELETE database
DROP table
ALTER database
```

permission দেওয়ার দরকার নেই—যদি application-এর প্রয়োজন না থাকে।

1️⃣7️⃣ Why Least Privilege Important?

ধরো application vulnerable হলো।

Attacker somehow application DB credential-এর access পেল।

Full privilege:

```
Attacker
   ↓
DB user
   ↓
DROP
DELETE
ALTER
UPDATE
```

Damage huge হতে পারে।

Limited privilege:

```
Attacker
   ↓
Limited DB user
   ↓
Limited operations
```

Blast radius কমে।

1️⃣8️⃣ SQL Injection ⭐⭐⭐⭐⭐

এটা database security-এর সবচেয়ে famous vulnerability-গুলোর একটি।

SQL Injection হলো:

> **Untrusted user input এমনভাবে SQL query-তে ঢুকে যায় যে attacker query-এর intended meaning পরিবর্তন করতে পারে।**

1️⃣9️⃣ Vulnerable SQL Example

ধরো application username input নিচ্ছে:

```
rahim
```

Developer ভুলভাবে string concatenate করল:

```
query = (
    "SELECT * FROM users "
    "WHERE username = '" + username + "'"
)
```

এটা unsafe pattern।

কারণ user input SQL syntax-এর অংশ হয়ে যেতে পারে।

2️⃣0️⃣ কেন Dangerous?

Database query হওয়া উচিত:

```
SQL structure
      +
User data
```

কিন্তু unsafe concatenation করলে:

```
SQL structure
      +
User input treated as SQL
```

হয়ে যেতে পারে।

2️⃣1️⃣ SQL Injection Prevention ⭐⭐⭐

সবচেয়ে গুরুত্বপূর্ণ:

> **Parameterized Queries / Prepared Statements ব্যবহার করো।**

Concept:

```
SQL structure
      ↓
Parameter
      ↓
User input treated as data
```

2️⃣2️⃣ Django কি SQL Injection থেকে Safe?

Django ORM সাধারণত parameterized queries ব্যবহার করে।

Example:

```
User.objects.filter(
    username=username
)
```

এটা raw string concatenation-এর চেয়ে নিরাপদ।

2️⃣3️⃣ Dangerous Django Pattern

❌ Avoid:

```
query = f"""
    SELECT *
    FROM users
    WHERE username = '{username}'
"""
```

এখানে user input directly SQL string-এর মধ্যে ঢুকছে।

2️⃣4️⃣ Django Raw SQL

Raw SQL দরকার হলে parameterized form ব্যবহার করো।

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT * FROM users WHERE username = %s",
        [username]
    )
```

এখানে:

```
SQL structure → fixed
username → parameter
```

2️⃣5️⃣ Django ORM — Preferred

যেখানে সম্ভব:

```
User.objects.filter(
    email=email
)
```

অথবা:

```
Booking.objects.filter(
    room_id=room_id,
    status="CONFIRMED"
)
```

ORM ব্যবহার করলে query construction safer এবং maintainable হয়।

তবে ORM ব্যবহার করলেই application-এর সব security problem magically চলে যায় না।

2️⃣6️⃣ Input Validation ⭐

User input validate করা উচিত।

যেমন:

```
Email
Phone
Date
Quantity
ID
```

কিন্তু মনে রাখবে:

> **Validation alone SQL Injection prevention-এর replacement নয়।**

Primary defense:

```
Parameterized query
```

2️⃣7️⃣ Sensitive Data কী?

Database-এ কিছু data বিশেষভাবে protect করতে হয়।

যেমন:

```
Passwords
API secrets
Authentication tokens
Personal information
Payment-related sensitive data
Private business information
```

2️⃣8️⃣ Password কখনো Plain Text রাখবে না ❌

Bad:

```
users

username | password
-------------------
rahim    | 123456
```

যদি database leak হয়:

```
Attacker
 ↓
```

```
Password directly visible
```

2️⃣9️⃣ Password Hashing ⭐⭐⭐

Password-এর জন্য **secure password hashing algorithm** ব্যবহার করতে হয়।

Concept:

```
Password
   ↓
Password Hashing
   ↓
Hash
   ↓
Database
```

Login:

```
Entered Password
      ↓
Verify against stored hash
      ↓
Match?
```

Django-এর built-in password hashing system ব্যবহার করা উচিত।

3️⃣0️⃣ Hashing vs Encryption

এটা important।

Hashing

সাধারণভাবে one-way transformation।

```
Password
   ↓
Hash
```

মূল password থেকে hash তৈরি হয় এবং verification করা হয়।

Encryption

Data encrypt করা হয়:

```
Plaintext
   ↓
Encryption + Key
   ↓
Ciphertext
```

Appropriate key থাকলে decrypt করা যায়।

সহজে:

```
Password → Hash

Recoverable sensitive data → Encryption may be appropriate
```

3️⃣1️⃣ Encryption at Rest

Database/storage-এর data disk-এ encrypted রাখা।

```
Database
   ↓
Encrypted Storage
```

যদি storage বা disk improperly accessed হয়, plaintext data exposure কমাতে সাহায্য করতে পারে।

3️⃣2️⃣ Encryption in Transit ⭐

Application → Database connection-ও secure হওয়া উচিত।

```
Django
   |
   | TLS/SSL
   ↓
Database
```

এতে network-এর মাধ্যমে transmitted data protect করা যায়।

3️⃣3️⃣ Database Credentials কোথায় রাখবে? ⭐⭐⭐

❌ Bad:

```
DATABASES = {
    "PASSWORD": "my_super_secret_password"
}
```

Source code-এ secret commit করলে repository leak হলে credential exposed হতে পারে।

3️⃣4️⃣ Environment Variables

Better:

```
import os

DATABASE_PASSWORD = os.environ["DB_PASSWORD"]
```

Environment:

```
DB_PASSWORD=********
```

Production-এ আরও ভালোভাবে dedicated **secret manager** ব্যবহার করা যেতে পারে।

3️⃣5️⃣ `.env` File

Development-এ:

```
.env
```

এ রাখা হতে পারে:

```
DB_NAME=hotel_db
DB_USER=app_user
DB_PASSWORD=secret
```

এবং:

```
.env
```

কে Git repository-তে commit করা যাবে না।

`.gitignore`-এ:

```
.env
```

রাখা উচিত।

3️⃣6️⃣ Production Secret Management

Production environment-এ:

```
Django
   ↓
Secret Manager
   ↓
DB credentials
```

Cloud/platform অনুযায়ী managed secret storage ব্যবহার করা যায়।

মূল principle:

> **Secrets source code-এর মধ্যে hard-code করো না।**

3️⃣7️⃣ Database Network Security

Database ideally public internet-এর জন্য unnecessarily exposed থাকা উচিত নয়।

Bad architecture:

```
Internet
   ↓
Database
```

Better:

```
Internet
   ↓
Django/API
   ↓
Private Network
```

```
   ↓
Database
```

3️⃣8️⃣ Firewall

Database server-এর access সীমিত করা যায়:

```
Allowed:
Django Server → Database

Blocked:
Random Internet → Database
```

Database port publicly open রাখার প্রয়োজন আছে কি না carefully evaluate করতে হবে।

3️⃣9️⃣ Database Access Layers

Production architecture:

```
User
 ↓
Frontend
 ↓
API
 ↓
Django
 ↓
Database User
 ↓
Database
```

User ideally সরাসরি database-এ connect করবে না।

4️⃣0️⃣ Application vs Database Authorization

ধরো:

```
Customer
Receptionist
Admin
```

Django application-level authorization:

```
Customer
 → own booking

Receptionist
 → booking management

Admin
 → system management
```

এটা application layer-এর authorization।

Database permissions:

```
app_user
report_user
admin_user
```

এগুলো database-level access control।

দুই layer-ই useful।

4️⃣1️⃣ Defense in Depth ⭐⭐⭐

Security-এর একটি গুরুত্বপূর্ণ principle:

> **একটি security control-এর উপর depend না করে multiple security layers ব্যবহার করা।**

Architecture:

```
Authentication
      ↓
Authorization
      ↓
Input Validation
      ↓
Parameterized Queries
      ↓
Database Permissions
      ↓
Network Security
      ↓
Encryption
      ↓
```

```
Monitoring
      ↓
Backup
```

4️⃣2️⃣ Django Security Settings

Production Django application-এ security-related settings carefully configure করতে হয়।

যেমন:

```
DEBUG = False
```

Production-এ:

```
ALLOWED_HOSTS = [
    "example.com"
]
```

আর HTTPS ব্যবহার করলে secure cookie settings-এর মতো configuration বিবেচনা করতে হয়।

4️⃣3️⃣ Django Authentication

Django authentication system ব্যবহার করলে password hashing, sessions ইত্যাদি অনেক security mechanism framework-level support পায়।

Custom authentication বানালেও security principles maintain করতে হবে।

বিশেষ করে:

```
Password hashing
Session/token security
Authorization
CSRF protection where applicable
HTTPS
```

4️⃣4️⃣ SQL Injection ছাড়াও কী কী Risk আছে?

Database security শুধু SQL Injection না।

আরও আছে:

```
Broken access control
Weak passwords
Credential leakage
Excessive database privileges
Unencrypted connections
Public database exposure
Sensitive data leakage
Poor backup security
Insufficient logging
Insecure raw SQL
```

4️⃣5️⃣ Audit Logging ⭐⭐⭐

Important database actions track করা যায়:

```
Who?
What?
When?
```

Example:

```
user_id = 25
action = UPDATE_BOOKING
booking_id = 101
time = 10:30
```

এতে suspicious activity investigate করা সহজ হয়।

4️⃣6️⃣ Audit Log Example

```sql
CREATE TABLE audit_logs (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT,
    action VARCHAR(100),
    table_name VARCHAR(100),
    record_id BIGINT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Example:

```
25 | UPDATE_BOOKING | bookings | 101
```

4️⃣7️⃣ Sensitive Payment Data

একটা important rule:

> Application database-এ unnecessary raw payment-card data store করা উচিত নয়।

বিশেষ করে:

```
CVV
Full card details
```

নিজের database-এ casually store করা উচিত নয়।

Payment provider-এর secure tokenization/payment system ব্যবহার করা সাধারণ production approach।

4️⃣8️⃣ Data Minimization ⭐

যে data দরকার নেই:

```
Don't collect
```

যে data দরকার:

```
Collect minimum necessary
```

কারণ:

```
Less sensitive data
        ↓
Less exposure risk
```

4️⃣9️⃣ Production Database Security Architecture ⭐⭐⭐

একটি secure architecture:

```
                Internet
                   ↓
             Load Balancer
                   ↓
              Django API
                   ↓
        Authentication/Authorization
                   ↓
           Application DB User
                   ↓
           Private DB Network
                   ↓
           Primary Database
                   ↓
            Read Replica
                   ↓
             Encrypted Backup
```

এখানে:

```
Authentication
Authorization
Least Privilege
TLS
Private Network
Parameterized Queries
Encryption
Backup
Monitoring
```

সব একসাথে কাজ করছে।

5️⃣0️⃣ Hotel System Security Design ⭐⭐⭐

আমাদের hotel project-এ:

```
Customer
Receptionist
Manager
Admin
```

Application-level:

```
Customer
 ↓
Own bookings

Receptionist
 ↓
Booking management

Manager
 ↓
Reports

Admin
 ↓
System administration
```

Database-level:

```
Django app
 ↓
app_user
```

এবং reporting:

```
Reporting service
 ↓
report_user
 ↓
SELECT only
```

5️⃣1️⃣ Hotel Payment Security

Database:

```
payments
----------------
payment_id
booking_id
amount
payment_method
status
```

Store করা যেতে পারে:

```
amount
payment_method
transaction/reference ID
status
```

কিন্তু unnecessary sensitive payment credentials/card security data নিজের DB-তে রাখবে না।

5️⃣2️⃣ Database Security Checklist ⭐⭐⭐

Production database deploy করার আগে:

```
☑ Strong authentication
☑ Least privilege
☑ Separate DB users
☑ No root for application
☑ Parameterized queries
☑ Django ORM where appropriate
☑ DEBUG=False
☑ HTTPS/TLS
☑ Private DB network
☑ Secrets outside source code
☑ Password hashing
☑ Sensitive data protection
☑ Audit logging
☑ Monitoring
☑ Secure backups
☑ Tested restore procedure
```

5️⃣3️⃣ Most Common Mistakes ❌

Mistake 1

```
Django → root DB user
```

❌

Mistake 2

```
query = f"SELECT ... '{user_input}'"
```

❌

Mistake 3

Source code:

```
DB_PASSWORD=123456
```

❌

Mistake 4

সব user-কে:

```
Admin
```

করে দেওয়া।

❌

Mistake 5

Password:

```
123456
```

plain text database-এ রাখা।

❌

Mistake 6

Database:

```
0.0.0.0/0
```

থেকে publicly accessible করে রাখা।

❌

Mistake 7

Replica আছে বলে backup বন্ধ করা।

❌

5️⃣4️⃣ Security Architecture Mindset

Database designer হিসেবে এখন তোমার চিন্তা হবে:

```
Who can access?
       ↓
What can they access?
       ↓
What can they modify?
       ↓
What if credentials leak?
       ↓
What if application is hacked?
       ↓
What if data is deleted?
       ↓
Can we detect it?
       ↓
Can we recover?
```

এটাই production security mindset।

🧠 Class 66 Golden Rules ⭐⭐⭐⭐⭐

```
1. Authentication → Who are you?

2. Authorization → What can you do?

3. User → Database identity

4. Role → Collection of permissions

5. GRANT → Give permission
```

```
6. REVOKE → Remove permission

7. Least Privilege → Minimum required permission

8. Application should not use DB root/admin unnecessarily

9. SQL Injection → Untrusted input changes SQL meaning

10. Parameterized Query → Primary SQL injection defense

11. Django ORM → Generally uses parameterized queries

12. Password → Never store plain text

13. Password → Use secure password hashing

14. Hashing ≠ Encryption

15. TLS → Protect data in transit

16. Encryption at rest → Protect stored data

17. Secrets → Never hard-code in source code

18. Database → Prefer private network

19. Audit Log → Who + What + When

20. Data Minimization → Don't store unnecessary sensitive data

21. Replication ≠ Backup

22. Defense in Depth → Multiple security layers

23. Database security ≠ Only SQL Injection

24. Application Authorization + Database Permissions
    should work together

25. Production DB → Security + Monitoring + Backup + Recovery
```

📝 Class 66 Assignment

Task 1 — Authentication vs Authorization

Explain:

```
Authentication
Authorization
```

একটা hotel example দাও।

Task 2 — GRANT / REVOKE

`report_user`-কে শুধু:

```
hotel_db
```

এর `bookings` table-এ `SELECT` permission দেওয়ার SQL লেখো।

তারপর permission remove করার SQL লেখো।

Task 3 — Least Privilege ⭐

একটি Django application-এর জন্য নিচের permissions:

```
SELECT
INSERT
UPDATE
DELETE
DROP
ALTER
```

এর মধ্যে কোনগুলো প্রয়োজন হতে পারে এবং কোনগুলো unnecessarily dangerous?

Reason করো—সব application-এর একই permission set হয় না।

Task 4 — SQL Injection ⭐⭐⭐

এই code-এর security problem identify করো:

```
query = f"""
SELECT *
FROM customers
WHERE email = '{email}'
"""
```

তারপর Django-তে safer approach লেখো।

Task 5 — Password

Explain:

```
Password
   ↓
Hash
   ↓
Database
```

এবং:

Hashing আর Encryption-এর মধ্যে difference কী?

Task 6 — Production Security ⭐⭐⭐

এই architecture-কে secure করো:

```
Internet
   ↓
Django
   ↓
Database
```

কমপক্ষে **8টি security improvement** লিখো।

🔥 Interview Challenge

Interviewer জিজ্ঞেস করল:

> **"How would you secure a production Django application database?"**

একটা strong answer:

```
1. Use a dedicated application DB user
2. Apply least privilege
3. Never use root unnecessarily
4. Use Django ORM / parameterized queries
5. Prevent SQL injection
```

```
6. Hash passwords securely
7. Keep DB credentials out of source code
8. Use environment/secret management
9. Use TLS for DB connections where appropriate
10. Keep database on a private network
11. Protect sensitive data
12. Enable audit/monitoring
13. Maintain secure backups
14. Test restoration
15. Control application-level authorization
```

🏆 এখন পর্যন্ত Database Engineering Journey

তুমি এখন এই level পর্যন্ত এসেছ:

```
Class 31
SQL Fundamentals
      ↓
32–40
Advanced SQL
      ↓
41–46
SQL Analytics + Views
      ↓
47–49
Procedures + Transactions + Triggers
      ↓
50–51
Indexing + Query Optimization
      ↓
52–53
Normalization + Database Design
      ↓
54
Data Warehouse
      ↓
55–57
Interview + Full Database Projects
      ↓
58–63
Concurrency + Locking + Deadlock
      ↓
64
Partitioning
      ↓
65
```

```
Replication + High Availability
      ↓
66
Security + Access Control
```

🎯 Database Designer হিসেবে এখন তোমার core stack:

```
SQL
+
Database Design
+
Normalization
+
ERD
+
Indexing
+
Query Optimization
+
Transactions
+
Concurrency
+
Partitioning
+
Replication
+
Security
```

এখন তুমি শুধু **SQL query writer** level-এ নেই; আমরা ধীরে ধীরে **Database Engineer / Database Designer mindset**-এর দিকে যাচ্ছি।

Next → Class 67: Database Backup, Recovery & Disaster Recovery — সেখানে আমরা শিখব **Full/Incremental/Differential Backup, Point-in-Time Recovery (PITR), RPO/RTO practical design, Restore, Disaster Recovery, Backup strategy এবং production database recovery architecture**।

📚 Class 67 — Database Backup, Recovery & Disaster Recovery

আজকের class-এ আমরা **Database Reliability-এর সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি** শিখব।

আগের classes-এ:

- Class 58–63 → Transaction, Concurrency, Locking, Deadlock

- Class 64 → Partitioning
- Class 65 → Replication & High Availability
- Class 66 → Database Security

আজ:

> **Database নষ্ট/ডিলিট/corrupt হলে কীভাবে data recover করব?**

আমরা শিখব:

```
1. Full Backup
2. Incremental Backup
3. Differential Backup
4. Restore
5. Point-in-Time Recovery (PITR)
6. RPO
7. RTO
8. Disaster Recovery
9. Backup Strategy
10. Production Recovery Architecture
```

1️⃣ Backup কী?

সহজ ভাষায়:

> **Database-এর data-এর একটি recoverable copy নিরাপদ জায়গায় সংরক্ষণ করাই Backup।**

ধরো:

```
Production Database
        |
        ↓
     Backup
        |
        ↓
Backup Storage
```

যদি database নষ্ট হয়:

```
Database ❌
   |
   ↓
Restore
   |
   ↓
```

```
Backup
   │
   ↓
Recovered Database
```

2️⃣ কেন Backup প্রয়োজন?

Production database-এ অনেক ধরনের সমস্যা হতে পারে:

```
❌ Accidental DELETE
❌ Accidental UPDATE
❌ Application bug
❌ Database corruption
❌ Disk failure
❌ Server failure
❌ Ransomware
❌ Human error
❌ Cloud/Infrastructure failure
❌ Natural disaster
```

Example:

```
DELETE FROM bookings;
```

ভুল করে execute হলো।

শুধু replication থাকলে:

```
Primary
   │
   ↓
DELETE
   │
   ↓
Replica
```

Replica-তেও সেই change replicate হতে পারে।

কিন্তু backup থাকলে:

```
Backup
   ↓
Restore
```

```
    ↓
Recover
```

3️⃣ Backup বনাম Replication ⭐⭐⭐

এটা খুব ভালোভাবে বুঝতে হবে।

Backup	Replication
Recovery-এর জন্য	Availability/read scaling-এর জন্য
Historical copy রাখা যায়	সাধারণত current state replicate করে
Accidental deletion recover করতে useful	Delete replicate হতে পারে
Restore করা যায়	Failover/read-এর জন্য useful
Long-term retention করা যায়	Continuous copy
Disaster Recovery-এর গুরুত্বপূর্ণ অংশ	HA-এর গুরুত্বপূর্ণ অংশ

Golden Rule

> **Replication ≠ Backup**

4️⃣ Full Backup ⭐⭐⭐

Full Backup মানে:

> **পুরো database-এর backup নেওয়া।**

ধরো:

```
Database:

A
B
C
D
E
```

Full backup:

```
Backup
 ↓
```

```
A B C D E
```

5️⃣ Full Backup Example

ধরো Sunday:

```
Sunday
↓
Full Backup
↓
A B C D E
```

Monday-তে নতুন data এসেছে:

```
F G
```

পরের full backup:

```
Monday
↓
Full Backup
↓
A B C D E F G
```

অর্থাৎ প্রত্যেক full backup পুরো database capture করে।

6️⃣ Full Backup-এর সুবিধা/অসুবিধা

সুবিধা

```
✅ Restore সহজ
✅ Backup chain কম
✅ Recovery straightforward
```

অসুবিধা

```
❌ Storage বেশি লাগে
❌ Large DB হলে backup নিতে বেশি সময় লাগে
❌ Network bandwidth বেশি লাগতে পারে
```

7️⃣ Incremental Backup ⭐⭐⭐

Incremental Backup:

> **শেষ backup-এর পর যে data পরিবর্তিত হয়েছে, শুধু সেটার backup।**

ধরো:

```
Sunday
Full → A B C
```

Monday:

```
New → D E
Incremental → D E
```

Tuesday:

```
New → F
Incremental → F
```

Wednesday:

```
New → G H
Incremental → G H
```

Architecture:

```
Sunday
Full
 A B C
   │
   ├── Monday → D E
   │
   ├── Tuesday → F
   │
   └── Wednesday → G H
```

8️⃣ Incremental Restore

Wednesday পর্যন্ত recover করতে হলে:

```
Full Backup
     ↓
Monday Incremental
     ↓
Tuesday Incremental
     ↓
Wednesday Incremental
```

সব relevant backup chain ঠিকভাবে apply করতে হবে।

তাই:

```
Backup size ↓
Restore complexity ↑
```

9️⃣ Differential Backup ⭐⭐⭐

Differential Backup:

> **শেষ Full Backup-এর পর থেকে যত changes হয়েছে, সেগুলোর backup।**

ধরো:

Sunday

```
Full
A B C
```

Monday

নতুন:

```
D E
```

Differential:

```
D E
```

Tuesday

নতুন:

```
F
```

Tuesday differential:

```
D E F
```

Wednesday

নতুন:

```
G
```

Wednesday differential:

```
D E F G
```

🔟 Incremental বনাম Differential

ধরো:

```
Sunday = Full
Monday = 10 GB changes
Tuesday = 5 GB changes
Wednesday = 8 GB changes
```

Incremental

```
Monday    → 10 GB
Tuesday   → 5 GB
Wednesday → 8 GB
```

Differential

```
Monday    → 10 GB
Tuesday   → 15 GB
Wednesday → 23 GB
```

কারণ differential প্রতিবার **শেষ Full Backup থেকে** হিসাব করে।

1️⃣1️⃣ Final Comparison ⭐⭐⭐⭐⭐

বিষয়	Full	Incremental	Differential
কী backup করে	সব	Last backup-এর পর changes	Last full-এর পর changes
Backup size	বড়	ছোট	মাঝারি/ধীরে বড়
Backup speed	ধীর	দ্রুত	মাঝারি
Restore	সহজ	বেশি steps	তুলনামূলক সহজ
Storage	বেশি	কম	মাঝারি
Restore chain	কম	বেশি	কম

মনে রাখো:

```
Incremental
→ Since LAST BACKUP

Differential
→ Since LAST FULL BACKUP
```

1️⃣2️⃣ Restore কী?

Backup নেওয়া হলো:

```
Database
   ↓
Backup
```

এখন backup থেকে database পুনরায় তৈরি করা:

```
Backup
   ↓
Restore
   ↓
Database
```

এটাই Restore।

1️⃣3️⃣ Backup আছে মানেই Recovery নিশ্চিত? ❌

না।

এটা production-এর খুব common mistake।

ধরো:

```
Backup: SUCCESS ✅
```

কিন্তু disaster-এর সময়:

```
Restore ❌
Backup corrupted ❌
Wrong credentials ❌
Missing backup files ❌
Incomplete backup ❌
```

তাই:

> **Backup success ≠ Recovery success**

1️⃣4️⃣ Restore Testing ⭐⭐⭐⭐⭐

Regularly test করতে হবে:

```
Backup
   ↓
Restore
   ↓
Database starts?
   ↓
Data correct?
   ↓
Application works?
```

এটাকে বলা যায়:

> **Recovery Drill / Restore Test**

Production database-এর জন্য এটি অত্যন্ত গুরুত্বপূর্ণ।

1️⃣5️⃣ Point-in-Time Recovery — PITR ⭐⭐⭐⭐⭐

এখন সবচেয়ে গুরুত্বপূর্ণ topic।

PITR মানে:

> **Database-কে একটি নির্দিষ্ট সময়ের অবস্থায় recover করার capability।**

ধরো:

```
10:00 → Normal
10:30 → Booking created
10:40 → Payment created
10:45 → Application bug
10:46 → Important records deleted ❌
```

আমরা চাই:

```
10:45:59
```

অর্থাৎ deletion-এর ঠিক আগের state।

এটাই:

Point-in-Time Recovery

1️⃣6️⃣ PITR-এর Basic Architecture

Conceptually:

```
Full/Base Backup
       +
Transaction Logs
       ↓
Recovery
       ↓
Desired Time
```

Database system অনুযায়ী log mechanism আলাদা হতে পারে।

PostgreSQL

```
Base Backup
+
WAL
```

MySQL

```
Full Backup
+
Binary Log
```

1️⃣7️⃣ PITR Example 🔥

ধরো:

```
02:00 → Full Backup
```

এরপর:

```
02:10 → Booking 101
02:20 → Booking 102
02:30 → Payment 201
02:40 → Booking 103
02:45 → ❌ Accidental DELETE
```

আমরা recover করতে চাই:

```
02:44:59
```

Recovery concept:

```
02:00 Full Backup
       |
       +
02:00 → 02:44:59 Logs
       |
       ↓
Database at 02:44:59
```

1️⃣8️⃣ PITR কেন গুরুত্বপূর্ণ?

শুধু daily backup থাকলে:

```
Monday 02:00
       ↓
Tuesday 02:00
```

এর মধ্যে disaster হলে অনেক hours-এর data loss হতে পারে।

PITR ব্যবহার করলে:

```
Full Backup
+
Continuous Log Information
```

থেকে much finer recovery point পাওয়া সম্ভব।

1️⃣9️⃣ RPO — Recovery Point Objective ⭐⭐⭐

RPO প্রশ্ন করে:

> **Disaster হলে maximum কত data loss acceptable?**

ধরো:

```
RPO = 15 minutes
```

এর অর্থ business-এর recovery target হলো approximately:

```
Maximum acceptable data loss ≈ 15 minutes
```

এটি architecture-এর achievable target; exact result backup/logging design-এর উপর নির্ভর করে।

2️⃣0️⃣ RTO — Recovery Time Objective ⭐⭐⭐

RTO প্রশ্ন করে:

> **Disaster হওয়ার পর কত সময়ের মধ্যে system আবার চালু করতে হবে?**

ধরো:

```
RTO = 1 hour
```

মানে target:

```
Failure
  ↓
Recovery
```

```
   ↓
Service restored within ~1 hour
```

2️⃣1️⃣ RPO vs RTO ⭐⭐⭐⭐⭐

এটা interview-এ খুব common।

```
RPO → DATA LOSS
RTO → DOWNTIME
```

আরও সহজ:

> **RPO = How much data can we lose?**

> **RTO = How long can we be down?**

2️⃣2️⃣ Practical Example

আমাদের hotel application:

```
RPO = 15 minutes
RTO = 1 hour
```

এর অর্থ:

```
Data loss target
→ ~15 minutes-এর বেশি না

Downtime target
→ ~1 hour-এর মধ্যে recovery
```

এখন daily backup যথেষ্ট হবে?

না, কারণ daily backup দিয়ে 15-minute RPO reliably meet করা কঠিন।

তাই architecture-এ frequent backups এবং/or transaction-log based recovery দরকার হতে পারে।

2️⃣3️⃣ RPO/RTO Business অনুযায়ী

সব application-এর requirement এক নয়।

Personal Blog

```
RPO = 24 hours
RTO = 24 hours
```

হতে পারে acceptable।

E-commerce

```
RPO → Low
RTO → Low
```

Banking / Critical System

```
RPO → Very Low
RTO → Very Low
```

ফলে:

```
Business Criticality ↑
        ↓
Recovery Requirement ↑
        ↓
Infrastructure Complexity ↑
```

2️⃣4️⃣ Disaster Recovery — DR ⭐⭐⭐⭐⭐

Disaster Recovery হলো:

> **Major failure/disaster-এর পর system, database এবং services recover করার পরিকল্পনা ও architecture।**

Disaster হতে পারে:

```
Database failure
Server failure
Disk failure
Datacenter failure
Cloud outage
Cyber incident
Human error
Natural disaster
```

2️⃣5️⃣ Backup বনাম Disaster Recovery

```
Backup
→ Data copy

Disaster Recovery
→ Complete recovery strategy
```

DR-এর মধ্যে থাকতে পারে:

```
Backup
+
Restore
+
Replication
+
Failover
+
Standby infrastructure
+
Monitoring
+
Recovery procedures
```

2️⃣6️⃣ Backup কোথায় রাখবে? ❌

Bad architecture:

```
Server
 ├── Production DB
 └── Backup
```

Server নষ্ট হলে:

```
Database ❌
Backup ❌
```

তাই backup আলাদা infrastructure/storage-এ রাখা উচিত।

2️⃣7️⃣ Better Architecture

```
Production DB
      |
      ↓
Backup System
      |
      ↓
Separate Storage
```

আরও resilient:

```
Production Region
       |
       ↓
Backup
       |
       ↓
Different Storage / Region
```

2️⃣8️⃣ 3-2-1 Backup Rule ⭐⭐⭐

Classic backup principle:

```
3 copies
2 different storage/media
1 offsite copy
```

Example:

```
Original Database
       +
Backup Storage 1
       +
Offsite Backup Storage 2
```

বাস্তব environment-এ business requirement অনুযায়ী policy পরিবর্তিত হতে পারে।

2️⃣9️⃣ Immutable Backup ⭐⭐⭐

Immutable backup হলো এমন backup যা নির্দিষ্ট retention period-এর মধ্যে সহজে পরিবর্তন/মুছে ফেলা যায় না।

Architecture:

```
Production DB
      ↓
Backup
      ↓
Immutable Storage
```

এটি ransomware বা malicious deletion-এর বিরুদ্ধে useful protection দিতে পারে।

3️⃣0️⃣ Backup Encryption

Backup-এর মধ্যেও sensitive information থাকতে পারে।

তাই:

```
Production Data
      ↓
Encrypted Backup
      ↓
Secure Storage
```

এখানে encryption key management-ও অত্যন্ত গুরুত্বপূর্ণ।

3️⃣1️⃣ Backup Retention

সব backup চিরকাল রাখতে হয় না।

Example policy:

```
Daily   → 30 days
Weekly  → 3 months
Monthly → 1 year
```

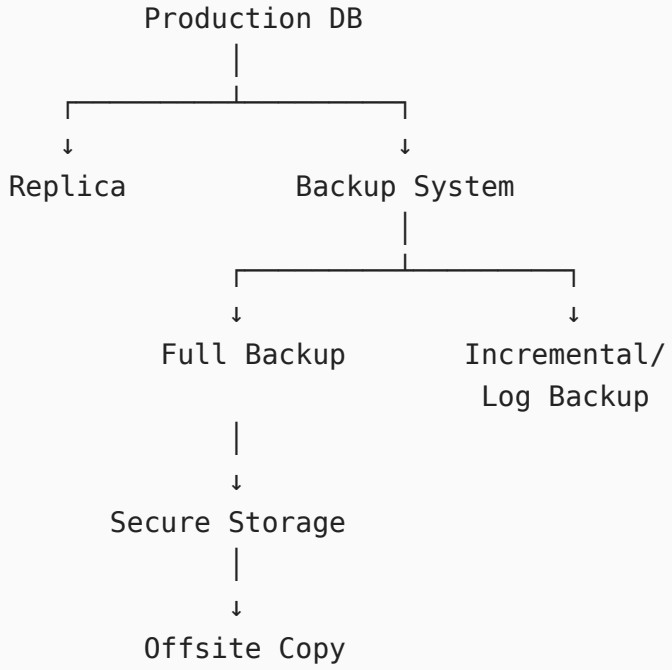
এগুলো শুধু example।

Actual retention নির্ভর করবে:

```
Business
Legal requirements
Compliance
Storage cost
Recovery needs
```

3️⃣2️⃣ Production Backup Strategy ⭐⭐⭐⭐⭐

একটি practical design:



3️⃣3️⃣ Hotel Management System Example 🔥

আমাদের database:

```
customers
rooms
room_types
bookings
payments
```

ধরো:

```
02:00 → Full Backup
```

তারপর:

```
02:10 → Booking 101
02:20 → Payment 201
02:30 → Booking 102
02:40 → Payment 202
02:45 → Application bug
```

আমরা চাই:

```
02:44:59
```

তাহলে conceptually:

```
Full Backup
     +
WAL/Binlog
     ↓
PITR
     ↓
02:44:59
```

3️⃣4️⃣ Complete Disaster Scenario

ধরো:

```
Primary DB Server ❌
```

Recovery process:

```
Detect Failure
      ↓
Assess Incident
      ↓
Choose Recovery Target
      ↓
Provision/Prepare DB
      ↓
Restore Full Backup
      ↓
Apply Incremental/Logs
      ↓
Verify Data
```

```
      ↓
Reconnect Application
      ↓
Monitor System
```

3️⃣5️⃣ Recovery Verification

Database restore করার পর শুধু:

```
Database started ✅
```

বললেই হবে না।

Check করতে হবে:

```
✅ Database connection
✅ Tables
✅ Row counts
✅ Indexes
✅ Constraints
✅ Recent transactions
✅ Application login
✅ Critical API
✅ Booking creation
✅ Payment history
```

Hotel application-এ:

```
Can customer create booking?
Can receptionist view booking?
Can room availability work?
Can payment history load?
```

3️⃣6️⃣ Recovery Drill ⭐⭐⭐⭐⭐

Production team periodically simulate করে:

```
"Primary database is unavailable."
```

তারপর:

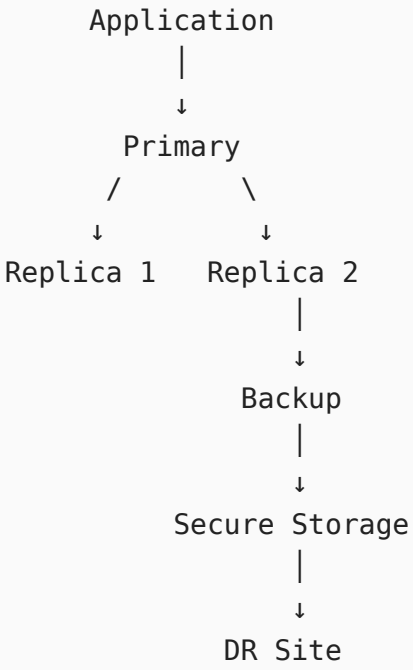
```
Backup
 ↓
Restore
 ↓
Recovery
 ↓
Application Test
 ↓
Measure Time
```

এতে বোঝা যায়:

> **আমাদের stated RTO সত্যিই achievable কিনা।**

3️⃣7️⃣ Replication + Backup + DR

Class 65-এর replication-এর সঙ্গে আজকের backup combine করি:



এখানে:

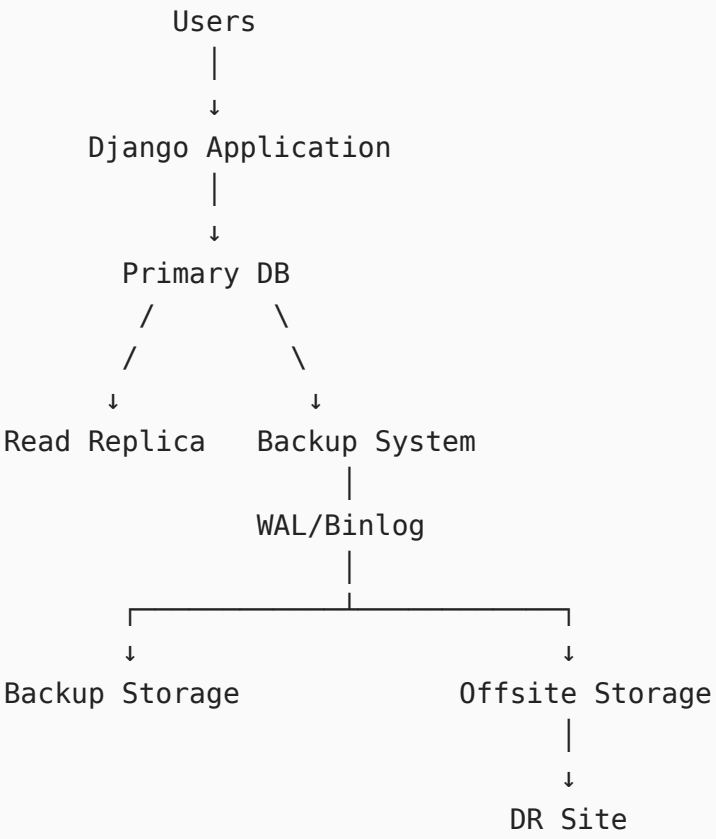
```
Replica
→ Availability / Read Scaling

Backup
→ Recovery

DR
→ Major Disaster Recovery
```

3️⃣8️⃣ Production Database Recovery Architecture ⭐⭐⭐⭐⭐

একটি mature architecture:



3️⃣9️⃣ Recovery Strategy Design

Database designer হিসেবে শুধু:

> "Backup নেব"

বললে হবে না।

এই প্রশ্নগুলো করতে হবে:

1. Database কত বড়?
2. প্রতিদিন কত data change হয়?
3. RPO কত?
4. RTO কত?
5. Backup কতদিন রাখতে হবে?
6. Backup কোথায় রাখা হবে?
7. Offsite copy দরকার?
8. PITR দরকার?

```
9. Encryption দরকার?
10. Immutable backup দরকার?
11. Restore কত দ্রুত করতে হবে?
12. Restore test কত ঘন ঘন হবে?
13. Disaster হলে কে recovery করবে?
```

4️⃣0️⃣ Small Project বনাম Production

Small Django Project

```
Daily Backup
+
Separate Storage
+
Periodic Restore Test
```

অনেক ক্ষেত্রে যথেষ্ট হতে পারে।

Production System

```
Full Backup
+
Incremental / Log Backup
+
PITR
+
Offsite Backup
+
Encryption
+
Retention
+
Monitoring
+
Restore Testing
+
DR Plan
+
RPO/RTO
```

🧠 Class 67 Golden Rules ⭐⭐⭐⭐⭐

```
FULL
→ Entire database

INCREMENTAL
→ Changes since last backup

DIFFERENTIAL
→ Changes since last full backup

RESTORE
→ Backup থেকে database পুনরায় তৈরি

PITR
→ Specific point in time-এ recovery

RPO
→ Acceptable data loss

RTO
→ Acceptable downtime

Replication
→ Availability / Read Scaling

Backup
→ Recovery

DR
→ Complete recovery strategy

Replication ≠ Backup

Backup ≠ Disaster Recovery

Backup success ≠ Recovery success

Backup should be stored separately

Offsite backup → Major infrastructure failure protection

Immutable backup → Tampering/deletion protection

Encryption → Protect backup data

Retention → কতদিন backup রাখা হবে

Restore Test → Backup সত্যিই recoverable কিনা যাচাই

Recovery Drill → পুরো recovery process practice
```

```
RPO/RTO → Backup architecture design-এর key requirements
```

📝 Class 67 Assignment

Task 1

ধরো:

```
Sunday → Full Backup
Monday → 10 GB changes
Tuesday → 5 GB changes
Wednesday → 8 GB changes
```

বল:

Incremental backup-এ প্রতিদিন কত GB backup হবে?

এবং:

Differential backup-এ প্রতিদিন কত GB backup হবে?

Task 2 — RPO/RTO

একটি hotel application:

```
RPO = 15 minutes
RTO = 1 hour
```

Explain:

1. RPO 15 minutes কী?
2. RTO 1 hour কী?
3. Daily-only backup কেন এই requirement-এর জন্য যথেষ্ট নাও হতে পারে?

Task 3 — PITR 🔥

Timeline:

```
01:00 → Full Backup
01:10 → Booking created
01:20 → Payment created
01:30 → Booking created
01:40 → Payment created
01:45 → Important data accidentally deleted
```

Question:

> **01:44:59 পর্যন্ত database recover করতে কীভাবে PITR ব্যবহার করবে?**

Task 4 — Replication

Explain:

> Primary database থেকে Replica-তে accidental DELETE replicate হয়ে গেলে কেন Replica-কে backup বলা যাবে না?

Task 5 — Production Architecture ⭐⭐⭐⭐⭐

Complete করো:

```
Django
   ↓
Primary DB
   ↓
Replica
   ↓
?
   ↓
?
   ↓
Offsite / DR
```

প্রতিটি component-এর কাজ লিখবে।

🔥 Interview Challenge

Interviewer:

> **"Your production database was accidentally modified at 2:45 PM. The last full backup was at 2:00 AM. How would you recover the database to 2:44 PM?"**

Strong answer:

```
02:00 AM Full Backup
        ↓
WAL / Binary Logs
        ↓
Replay changes
        ↓
Stop at desired recovery time
        ↓
02:44 PM state
        ↓
Verify data
        ↓
Make recovered DB available
        ↓
Reconnect application
```

তারপর interviewer জিজ্ঞেস করলে:

> **"How do you know your backup strategy actually works?"**

Answer:

```
Regular restore testing
+
Recovery drills
+
Backup monitoring
+
Documented recovery procedure
+
Measured RPO/RTO
```

🏆 এখন পর্যন্ত Database Engineering Journey

```
SQL
 ↓
Advanced SQL
 ↓
Database Design
 ↓
```

```
Normalization
 ↓
ERD
 ↓
Indexing
 ↓
Query Optimization
 ↓
Transactions
 ↓
Concurrency
 ↓
Locking
 ↓
Deadlock
 ↓
Partitioning
 ↓
Replication
 ↓
High Availability
 ↓
Security
 ↓
Backup
 ↓
PITR
 ↓
Disaster Recovery
```

এখন তোমার সবচেয়ে গুরুত্বপূর্ণ mental model:

```
                    DATABASE
                       │
        ┌──────────────┼──────────────┐
        ↓              ↓              ↓
  Performance     Reliability    Security
        │              │              │
    Indexing       Transaction    Access Control
    Partition      Replication    Encryption
    Optimize       Backup         SQL Injection
                   PITR
                   Failover
                   DR
```

⭐ পাঁচটি concept এক লাইনে

```
Indexing
→ Query দ্রুত করে

Partitioning
→ Large table manage করতে সাহায্য করে

Replication
→ Availability / Read Scaling

Backup
→ Data Recovery

Disaster Recovery
→ Major disaster-এর পর পুরো system recover
```

Next → Class 68: Database Monitoring, Logging & Observability — সেখানে আমরা শিখব **Database Metrics, Slow Query Log, Error Log, Audit Log, Connection Monitoring, CPU/Memory/Disk, Replication Lag, Lock/Deadlock Monitoring, Alerting এবং Production Database Observability Architecture**।

📚 Class 68 — Database Monitoring, Logging & Observability

আজ আমরা Database-এর **production monitoring** শিখব।

আগের class-এ আমরা শিখেছি:

```
Class 65 → Replication & High Availability
Class 66 → Security
Class 67 → Backup, Recovery & Disaster Recovery
```

এখন প্রশ্ন:

> **Production database-এ আসলে কী ঘটছে, সেটা কীভাবে বুঝব?**

যেমন:

- Database slow কেন?
- CPU কেন 95%?
- Memory শেষ হয়ে যাচ্ছে কেন?
- কোন query 10 seconds নিচ্ছে?
- কতগুলো connection আছে?
- Deadlock হচ্ছে কি?

- Replica কতটা পিছিয়ে আছে?
- Database error হচ্ছে কি?
- কোন user sensitive data access করছে?
- কখন alert পাঠাব?

এসবের জন্য দরকার:

Database Observability

1️⃣ Monitoring কী?

Monitoring মানে:

> **Database-এর health এবং performance নিয়মিত measure করা এবং সমস্যা হলে detect করা।**

সহজভাবে:

```
Database
   ↓
Metrics
   ↓
Monitoring
   ↓
Problem Detection
   ↓
Alert
   ↓
Action
```

2️⃣ Observability কী?

Monitoring বলে:

> **"Problem আছে কি?"**

Observability আরও বড় প্রশ্ন করে:

> **"কেন problem হচ্ছে?"**

Example:

```
API slow
   ↓
Database slow
   ↓
Query slow
   ↓
Missing index
```

Monitoring:

```
DB latency = 5 sec ❌
```

Observability:

```
Which query?
Why slow?
How many rows scanned?
Which index?
What happened at that time?
```

Golden Rule ⭐

```
Monitoring
→ What is happening?

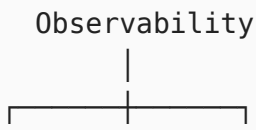
Observability
→ Why is it happening?
```

3️⃣ Database Observability-এর 3 Pillars ⭐⭐⭐⭐⭐

সাধারণ observability model:

```
Metrics
Logs
Traces
```

Database-এর ক্ষেত্রে:



```
      ↓      ↓      ↓
   Metrics  Logs  Traces
```

Metrics

Numeric measurements:

```
CPU = 80%
Connections = 150
Query latency = 300 ms
Disk = 75%
```

Logs

Events:

```
ERROR
WARNING
Slow Query
Authentication event
```

Traces

একটি request-এর পুরো journey:

```
User Request
    ↓
Django
    ↓
Service
    ↓
Database
    ↓
Query
```

কোন অংশ কত সময় নিল তা বোঝাতে tracing useful।

4️⃣ Database-এর গুরুত্বপূর্ণ Metrics

Production DB monitor করার সময় অনেক metrics থাকে।

সবচেয়ে important:

```
CPU
Memory
Disk
Connections
Query Latency
Queries/sec
Transactions/sec
Cache Hit Ratio
Locks
Deadlocks
Replication Lag
Disk I/O
```

5️⃣ CPU Monitoring ⭐⭐⭐

ধরো:

```
CPU = 95%
```

সম্ভাব্য কারণ:

```
Heavy Query
Large JOIN
Missing Index
Sort
Aggregation
Too many concurrent queries
```

কিন্তু:

> **CPU 95% দেখলেই database problem বলা যাবে না।**

Context দেখতে হবে।

Example:

```
CPU = 95%
Query latency = 20 ms
System healthy
```

হতে পারে workload intentionally high।

অন্যদিকে:

```
CPU = 95%
Query latency = 10 sec
Requests failing
```

এটা serious problem।

6️⃣ Memory Monitoring

Database memory-এর জন্য গুরুত্বপূর্ণ।

যদি memory pressure হয়:

```
Memory
 ↓
Cache efficiency ↓
 ↓
Disk I/O ↑
 ↓
Query latency ↑
```

Possible symptoms:

```
Slow queries
Swapping
High disk I/O
Application timeout
```

7️⃣ Disk Monitoring ⭐⭐⭐⭐⭐

Database-এর জন্য disk অত্যন্ত গুরুত্বপূর্ণ।

Monitor:

```
Disk Usage
Disk IOPS
Read latency
Write latency
Disk throughput
```

ধরো:

```
Disk = 98%
```

এটা critical হতে পারে।

কারণ database-এর জন্য প্রয়োজন হতে পারে:

```
Data files
Indexes
Logs
Temporary files
WAL/Binary logs
```

Disk full হলে database/application severe সমস্যা করতে পারে।

8️⃣ Connection Monitoring ⭐⭐⭐

Database-এ application কতগুলো connection খুলছে?

ধরো:

```
max_connections = 200
current_connections = 195
```

Danger zone হতে পারে।

কারণ নতুন request:

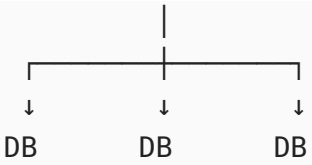
```
Django
  ↓
DB connection
  ↓
❌ Connection unavailable
```

9️⃣ Connection Pool

Modern applications সাধারণত connection management করে।

Concept:

```
Django
 | | | | |
 └─┴─┴─┴──→ Connection Pool
```



Pool-এর সুবিধা:

```
Connection reuse
Less connection overhead
Controlled concurrency
```

কিন্তু pool size ভুল হলে সমস্যা হতে পারে।

🔟 Too Many Connections

Symptoms:

```
Connection refused
Timeout
Database overloaded
Application errors
```

Possible causes:

```
Connection leak
Pool too large
Too many application instances
Long-running connections
Database max connection limit too low
```

1️⃣1️⃣ Query Latency ⭐⭐⭐⭐⭐

Query কত সময় নিচ্ছে?

Example:

```
Query A → 5 ms
Query B → 20 ms
Query C → 8 sec ❌
```

Query C investigate করতে হবে।

Monitor:

```
Average latency
P95 latency
P99 latency
Maximum latency
```

1️⃣2️⃣ P95 / P99 কেন?

ধরো 100 requests:

```
95 requests → fast
5 requests → very slow
```

Average দেখে problem লুকিয়ে যেতে পারে।

P95:

> 95% requests এই latency-এর মধ্যে complete করেছে।

P99:

> 99% requests এই latency-এর মধ্যে complete করেছে।

Example:

```
Average = 100 ms
P95 = 300 ms
P99 = 5 sec
```

মানে কিছু request অত্যন্ত slow।

1️⃣3️⃣ Queries Per Second — QPS

Database কত query process করছে?

```
QPS = Queries Per Second
```

Example:

```
10:00 → 1,000 QPS
```

```
10:05 → 10,000 QPS
```

হঠাৎ workload বেড়েছে।

তার সঙ্গে:

```
CPU ↑
Latency ↑
Connections ↑
```

হলে investigation দরকার।

1️⃣4️⃣ Slow Query Log ⭐⭐⭐⭐⭐

Slow Query Log database-এর অত্যন্ত গুরুত্বপূর্ণ tool।

এর উদ্দেশ্য:

> **যেসব query নির্দিষ্ট সময়ের বেশি নেয়, সেগুলো record করা।**

Example:

```
Query
↓
Execution time = 7.2 sec
↓
Slow Query Log
```

1️⃣5️⃣ Slow Query Example

ধরো:

```sql
SELECT *
FROM bookings
WHERE customer_id = 100;
```

Execution:

```
7.5 seconds ❌
```

Slow query log বলছে:

```
Query: SELECT * FROM bookings ...
Time: 7.5 sec
```

এখন:

```
EXPLAIN
```

দিয়ে investigate করব।

1️⃣6️⃣ Slow Query → EXPLAIN

আমাদের আগের Class 60 মনে করো:

```
Slow Query
    ↓
EXPLAIN
    ↓
type
key
rows
Extra
    ↓
Optimization
```

Example:

```sql
EXPLAIN
SELECT *
FROM bookings
WHERE customer_id = 100;
```

যদি:

```
type = ALL
key = NULL
rows = 5,000,000
```

হয়, index missing হতে পারে।

তখন candidate:

```sql
CREATE INDEX idx_bookings_customer
```

```
ON bookings(customer_id);
```

তারপর আবার:

```
EXPLAIN
```

1️⃣7️⃣ Error Log ⭐⭐⭐⭐⭐

Database error হলে error log useful।

Example:

```
ERROR
Connection failed
Disk full
Database crash
Authentication failure
Permission denied
Recovery error
```

Error log বলে:

> **Database system-এর ভিতরে কী ধরনের error ঘটছে।**

1️⃣8️⃣ Error Log বনাম Slow Query Log

Slow Query Log	Error Log
Slow query	Database errors
Performance investigation	Failure investigation
Query-centric	System/database-centric
Optimization-এর জন্য	Troubleshooting-এর জন্য

মনে রাখো:

```
Slow Query Log
→ "কোন query slow?"

Error Log
→ "Database-এ কী error হচ্ছে?"
```

1️⃣9️⃣ Audit Log ⭐⭐⭐⭐⭐

Audit Log-এর উদ্দেশ্য:

> **কে কী database operation করেছে তা record করা।**

Example:

```
User: admin
Action: UPDATE
Table: customers
Time: 10:35
```

আরেকটি:

```
User: report_user
Action: SELECT
Table: payments
Time: 11:02
```

2️⃣0️⃣ Audit Log কেন দরকার?

Security:

```
Who?
What?
When?
```

মানে:

```
Who accessed data?
What did they do?
When did they do it?
```

Useful for:

```
Security investigation
Compliance
Troubleshooting
Accountability
Sensitive data monitoring
```

2️⃣1️⃣ Audit Log বনাম Error Log

```
Audit Log
→ User activity

Error Log
→ Database/system error

Slow Query Log
→ Slow SQL
```

এই তিনটি confuse করবে না।

2️⃣2️⃣ Replication Lag ⭐⭐⭐⭐⭐

Class 65-এ আমরা replication শিখেছিলাম।

ধরো:

```
Primary
  ↓
Replica
```

Primary:

```
Order #100
```

Replica এখনো:

```
Order #99
```

তাহলে replica পিছিয়ে আছে।

এটাই:

> **Replication Lag**

2️⃣3️⃣ Replication Lag কেন হয়?

সম্ভাব্য কারণ:

```
High write workload
Network latency
Replica overloaded
Slow disk
Long-running transaction
Replication bottleneck
```

2️⃣4️⃣ Replication Lag Problem

User:

```
POST /orders
```

Write হলো:

```
Primary
```

তারপর immediately:

```
GET /orders
```

যদি read যায়:

```
Replica
```

তাহলে নতুন order নাও দেখা যেতে পারে।

কারণ:

```
Primary
  ↓
Replication Lag
  ↓
Replica
```

2️⃣5️⃣ Read-after-write Consistency

এই problem-কে আমরা Class 65-এ দেখেছিলাম।

Flow:

```
User creates booking
       ↓
Primary DB
       ↓
Success
       ↓
User immediately views booking
       ↓
Replica
       ↓
Booking missing ❌
```

Solution:

Critical read:

```
Primary
```

অথবা consistency-aware routing।

2️⃣6️⃣ Lock Monitoring ⭐⭐⭐⭐⭐

Class 63-এর locking মনে করো।

Database-এ transaction একে অপরকে block করতে পারে।

Example:

```
Transaction A
   ↓
Locks Row 10

Transaction B
   ↓
Needs Row 10
   ↓
WAIT
```

Monitoring-এ দেখতে চাই:

```
Who is waiting?
Who is blocking?
Which table?
```

```
Which query?
How long?
```

2️⃣7️⃣ Lock Wait

Example:

```
T1 → Row 100 locked
T2 → waiting
T3 → waiting
```

যদি T1 অনেকক্ষণ ধরে transaction open রাখে:

```
Waiting transactions ↑
      ↓
Latency ↑
      ↓
Requests timeout
```

2️⃣8️⃣ Deadlock Monitoring ⭐⭐⭐⭐⭐

Class 63:

```
T1 locks A
T2 locks B

T1 → wants B
T2 → wants A
```

Result:

```
T1 ↔ T2
Waiting forever
```

Database সাধারণত deadlock detect করে এবং একটি transaction abort/rollback করতে পারে।

Monitoring-এর কাজ:

```
Deadlock count
Deadlock frequency
```

```
Affected queries
Affected tables
```

2️⃣9️⃣ Deadlock Log

Production investigation:

```
Deadlock detected
   ↓
Find transactions
   ↓
Find queries
   ↓
Find lock order
   ↓
Fix application/database logic
```

Common fix:

```
Consistent lock order
Short transactions
Proper indexes
Avoid unnecessary locking
Retry aborted transaction where appropriate
```

3️⃣0️⃣ Database Disk I/O

Database শুধু CPU ব্যবহার করে না।

Disk I/O অত্যন্ত গুরুত্বপূর্ণ।

Monitor:

```
Read IOPS
Write IOPS
Read latency
Write latency
Throughput
```

যদি:

```
Disk latency ↑
```

হয়:

```
Query latency ↑
```

হতে পারে।

3️⃣1️⃣ Cache Hit Ratio

Database frequently accessed data memory-তে রাখার চেষ্টা করে।

Cache hit হলে:

```
Memory
 ↓
Fast
```

Miss হলে:

```
Disk
 ↓
Relatively slower
```

তাই cache efficiency monitor করা useful।

তবে:

> **একটি metric-এর threshold blindly apply করা যাবে না।**

Database engine, workload এবং workload pattern বুঝতে হবে।

3️⃣2️⃣ Database Health Dashboard

Production dashboard-এ আমরা দেখতে পারি:

```
┌──────────────────────────────────┐
│       DATABASE DASHBOARD         │
├──────────────────────────────────┤
│ CPU              65%             │
│ Memory           72%             │
│ Disk             68%             │
```

```
| Connections      120 / 500       |
| Query Latency    85 ms           |
| P95 Latency      210 ms          |
| QPS              4,500           |
| Slow Queries     12              |
| Deadlocks        0               |
| Lock Waits       3               |
| Replica Lag      2 sec           |
└──────────────────────────────────┘
```

এক নজরে database health বোঝা যায়।

3️⃣3️⃣ Alerting ⭐⭐⭐⭐⭐

Monitoring আছে কিন্তু alert নেই?

তাহলে engineer manually dashboard দেখছে।

Production-এ দরকার:

```
Metric
 ↓
Threshold / Condition
 ↓
Alert
 ↓
Engineer
```

3️⃣4️⃣ Example Alerts

CPU

```
CPU > 90%
for 10 minutes
```

→ Alert

Disk

```
Disk > 85%
```

→ Warning

```
Disk > 95%
```

→ Critical

Connections

```
Connections > 90% of limit
```

→ Alert

Replication

```
Replica lag > acceptable threshold
```

→ Alert

Query

```
P95 latency > target
```

→ Alert

Deadlock

```
Deadlock detected
```

→ Alert depending on severity/frequency.

3️⃣5️⃣ Warning বনাম Critical

সব alert একই priority না।

```
INFO
 ↓
WARNING
 ↓
CRITICAL
```

Example:

```
Disk = 80%
→ WARNING

Disk = 95%
→ CRITICAL
```

3️⃣6️⃣ Alert Fatigue ❌

অনেক বেশি alert দিলে:

```
Engineer
 ↓
100 alerts/day
 ↓
সব ignore
```

এটা dangerous।

তাই:

> **Alert only when an engineer needs to take action.**

3️⃣7️⃣ Good Alert Example

Bad:

```
CPU = 70%
ALERT!!!
```

Better:

```
CPU > 90%
AND
```

```
Query latency > 2 sec
for 10 minutes
```

Context-aware alert অনেক useful।

3️⃣8️⃣ Django + Database Monitoring

আমাদের architecture:

```
User
 ↓
React / Next.js
 ↓
Django / DRF
 ↓
Database
```

Monitoring করা যায়:

```
Frontend
API
Database
Infrastructure
```

Database level:

```
Connections
Queries
Latency
Locks
Deadlocks
CPU
Memory
Disk
Replication
```

3️⃣9️⃣ Request → Database Observability

একটি request:

```
POST /api/bookings/
        ↓
```

```
Django
        ↓
Service Logic
        ↓
SQL Query
        ↓
Database
```

ধরো API response:

```
3.5 sec
```

Tracing/metrics দিয়ে দেখা গেল:

```
Django logic → 100 ms
Database → 3.3 sec
```

এখন বোঝা গেল bottleneck database।

4️⃣0️⃣ Database Bottleneck Investigation ⭐⭐⭐⭐⭐

Production API slow:

```
API Slow
   ↓
Is Django slow?
   ↓
Is DB slow?
   ↓
Which query?
   ↓
Slow Query Log
   ↓
EXPLAIN
   ↓
Rows?
Index?
Lock?
I/O?
```

আরও systematically:

```
Observe
 ↓
```

```
Measure
 ↓
Identify
 ↓
Investigate
 ↓
Fix
 ↓
Verify
```

4️⃣1️⃣ Slow Query Investigation Example 🔥

Hotel API:

```
GET /api/available-rooms/
```

Latency:

```
6 seconds ❌
```

Investigation:

```
API
 ↓
Django = 100ms
 ↓
DB = 5.9 sec
```

Slow Query Log:

```
SELECT ...
FROM bookings
WHERE room_id = ...
```

তারপর:

```
EXPLAIN
SELECT ...
FROM bookings
WHERE room_id = 101
  AND check_in < '2026-09-20'
  AND check_out > '2026-09-18';
```

দেখলাম:

```
type = ALL
rows = 10,000,000
```

Potential optimization:

```
CREATE INDEX idx_booking_room_dates
ON bookings(room_id, check_in, check_out);
```

তারপর আবার:

```
EXPLAIN
```

এবং actual performance benchmark করতে হবে।

4️⃣2️⃣ Monitoring ≠ Optimization

খুব important distinction।

```
Monitoring
→ Problem detect

Optimization
→ Problem fix
```

Example:

```
Monitoring
→ Query latency = 8 sec

Optimization
→ EXPLAIN
→ Index
→ Query rewrite
→ Verify
```

4️⃣3️⃣ Logging Strategy

Production-এ বিভিন্ন log:

```
Application Log
Database Error Log
Slow Query Log
Audit Log
Access Log
Security Log
```

প্রতিটির purpose আলাদা।

4️⃣4️⃣ Centralized Logging ⭐⭐⭐

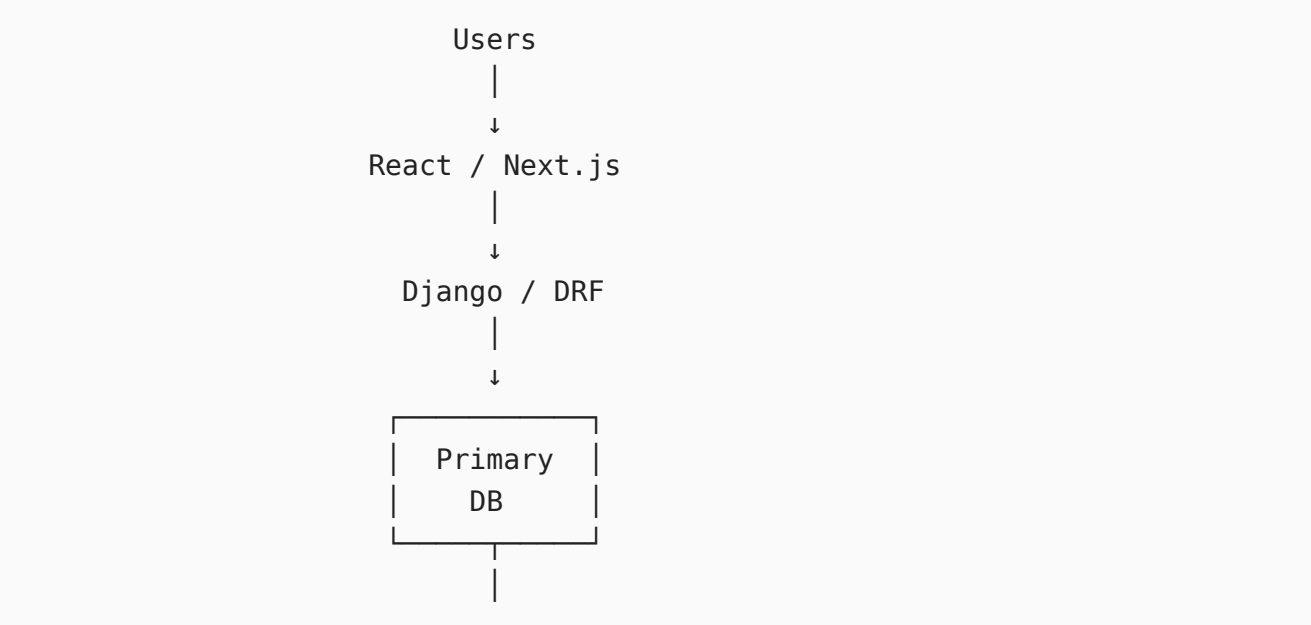
Production-এ শুধু server-এর local file-এ log রেখে দেওয়া সবসময় ideal নয়।

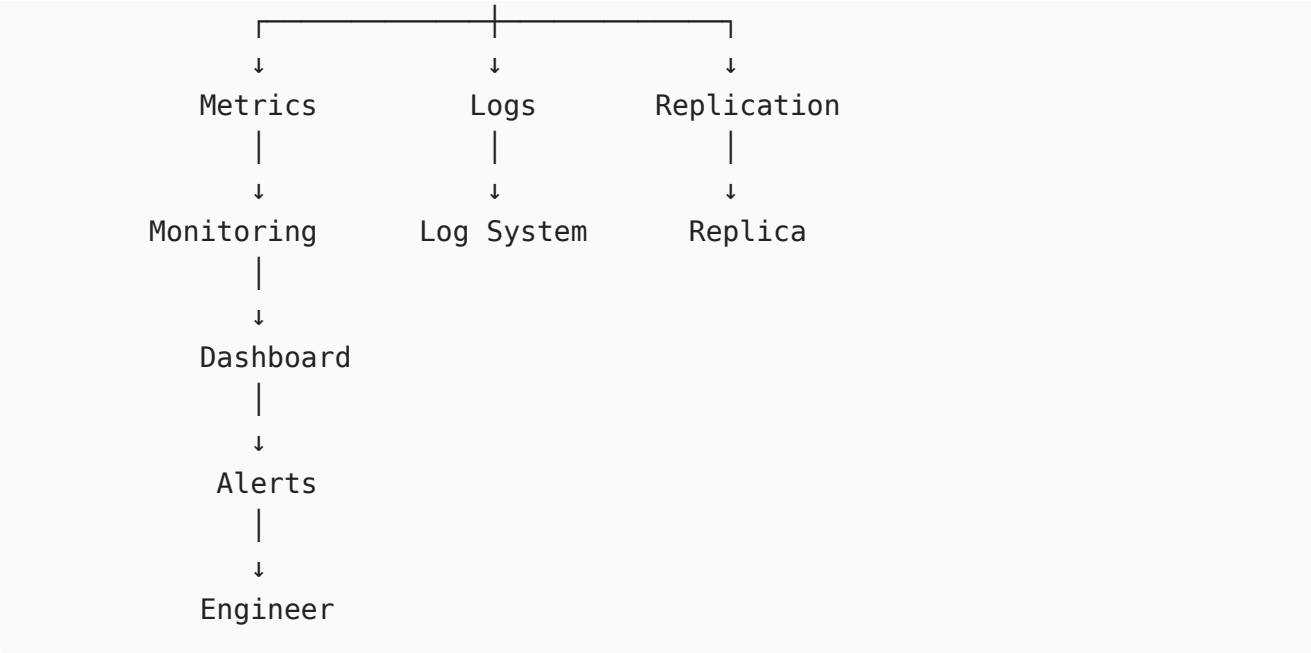
Concept:

```
Database
   ↓
Logs
   ↓
Central Log System
   ↓
Search / Dashboard / Alert
```

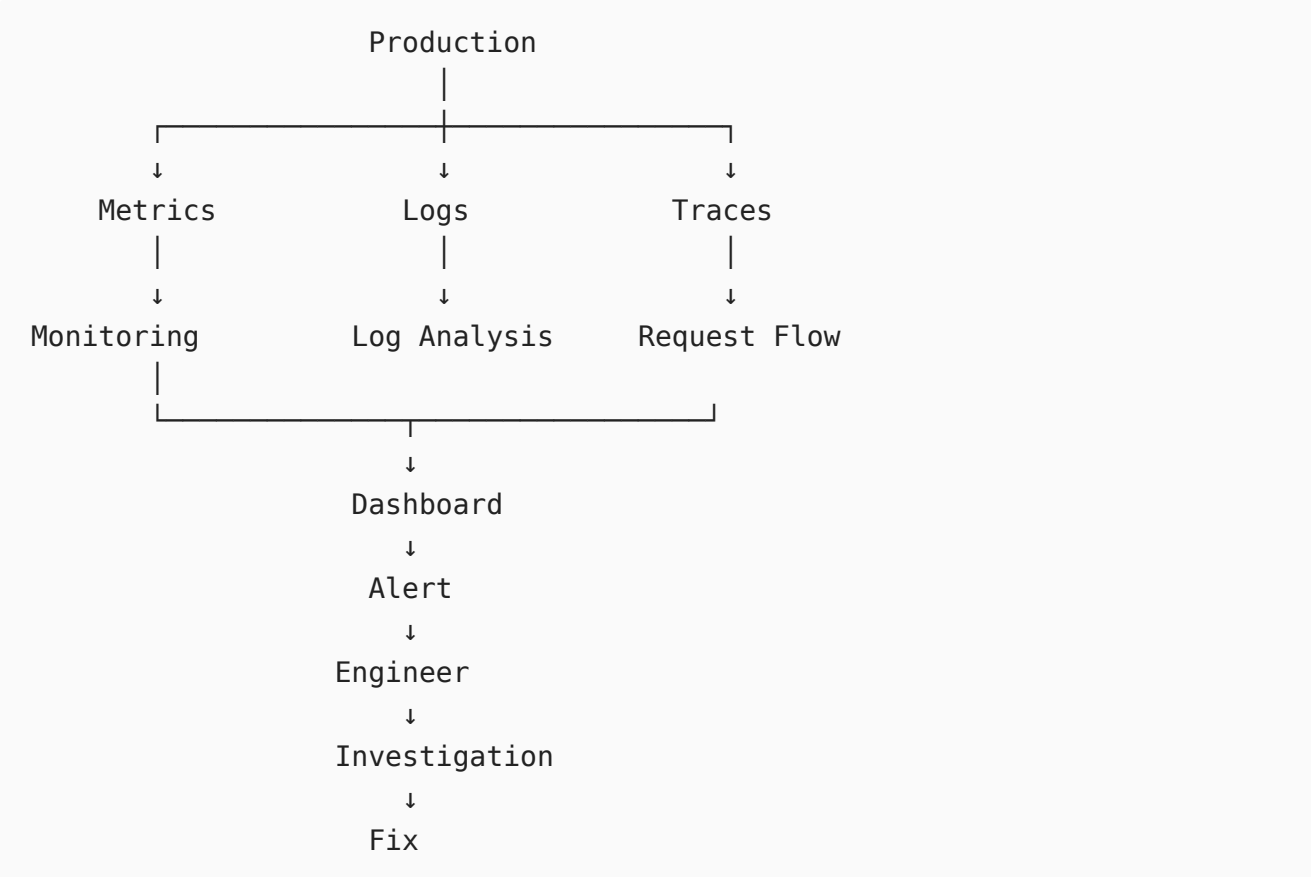
তাতে multiple server-এর logs এক জায়গায় investigate করা সহজ হয়।

4️⃣5️⃣ Production Observability Architecture ⭐⭐⭐⭐⭐





4️⃣6️⃣ Full Production Flow



4️⃣7️⃣ What Should We Monitor?

Database engineer হিসেবে minimum checklist:

```
Infrastructure
├── CPU
├── Memory
├── Disk
└── Network

Database
├── Connections
├── Query latency
├── QPS
├── Transactions
├── Cache
├── Locks
├── Deadlocks
└── Errors

Replication
├── Lag
├── Replica health
└── Replication status

Queries
├── Slow queries
├── Query frequency
├── Execution time
└── EXPLAIN plans

Security
├── Authentication
├── Authorization
└── Audit events
```

4️⃣8️⃣ Monitoring-এর 5টি প্রশ্ন

যেকোনো production database-এ এই ৫টি প্রশ্ন করতে পারবে:

1. Is it healthy?

```
CPU
Memory
Disk
Errors
```

2. Is it fast?

```
Latency
P95
P99
Slow queries
```

3. Is it overloaded?

```
Connections
QPS
CPU
I/O
```

4. Is it blocked?

```
Locks
Lock waits
Deadlocks
```

5. Is replication healthy?

```
Lag
Replica status
Replication errors
```

4️⃣9️⃣ Common Mistakes ❌

Mistake 1

শুধু CPU monitor করা।

❌

Database performance অনেক কিছু depend করে।

Mistake 2

শুধু average latency দেখা।

❌

P95/P99-ও গুরুত্বপূর্ণ।

Mistake 3

Slow query log না রাখা।

❌

Mistake 4

Replica আছে কিন্তু replication lag monitor করি না।

❌

Mistake 5

Deadlock হচ্ছে কিন্তু log/metrics নেই।

❌

Mistake 6

প্রতিটি ছোট metric-এর জন্য alert।

❌

Alert fatigue হবে।

Mistake 7

Monitoring আছে কিন্তু action plan নেই।

❌

Alert:

```
Disk 95%
```

তারপর কী?

```
Expand disk?
Delete old logs?
Investigate growth?
Failover?
```

Runbook দরকার।

5️⃣0️⃣ Monitoring + Runbook ⭐⭐⭐⭐⭐

Good production system:

```
Alert
 ↓
Runbook
 ↓
Investigation
 ↓
Action
```

Example:

```
Alert:
Replica lag > 60 sec
```

Runbook:

```
1. Check replica status
2. Check network
3. Check replica CPU/I/O
4. Check long-running queries
5. Check primary write rate
6. Check replication errors
7. Decide mitigation
```

🧠 Class 68 Golden Rules ⭐⭐⭐⭐⭐

```
Monitoring
→ Database health measure করা

Observability
→ System কেন এমন behave করছে তা বোঝা
```

Metrics
→ Numeric measurements

Logs
→ Events/details

Traces
→ Request journey

CPU
→ Compute pressure

Memory
→ Memory pressure/cache

Disk
→ Capacity + I/O health

Connections
→ Database connection load

Query Latency
→ Query কত দ্রুত/ধীরে চলছে

P95/P99
→ Tail latency বোঝায়

QPS
→ Queries per second

Slow Query Log
→ Slow SQL identify

Error Log
→ Database/system errors

Audit Log
→ Who + What + When

Replication Lag
→ Replica কতটা পিছিয়ে

Lock Monitoring
→ Who is waiting/blocking

Deadlock Monitoring
→ Circular lock conflict detect

Alert

```
→ Problem হলে engineer-কে notify

Alert Fatigue
→ Too many useless alerts

Monitoring ≠ Optimization

Slow Query
→ EXPLAIN
→ Investigate
→ Optimize
→ Verify

Backup
→ Recovery

Replication
→ Availability / Read Scaling

Monitoring
→ Know what is happening

Observability
→ Understand why
```

📝 Class 68 Assignment

Task 1 — Metrics

একটি production DB:

```
CPU = 92%
Memory = 85%
Disk = 91%
Connections = 480 / 500
P95 Query Latency = 4 sec
Deadlocks = 15/hour
Replica Lag = 120 sec
```

প্রতিটির জন্য বলো:

Normal / Warning / Critical — এবং কেন?

Task 2 — Slow Query 🔥

Query:

```
SELECT *
FROM bookings
WHERE customer_id = 100;
```

যদি:

```
Execution time = 8 seconds
rows examined = 10,000,000
key = NULL
```

তাহলে investigation-এর steps লিখো।

Task 3 — Replication

Scenario:

```
POST /api/orders/
        ↓
Primary
        ↓
Success

GET /api/orders/
        ↓
Replica
        ↓
New order missing
```

কেন এমন হলো?

এবং কীভাবে solve করবে?

Task 4 — Deadlock

```
T1:
LOCK A
WAIT B

T2:
```

```
LOCK B
WAIT A
```

Explain:

1. এটা কী?
2. কেন হচ্ছে?
3. কীভাবে prevent করবে?
4. Monitoring-এ কী দেখবে?

Task 5 — Production Architecture ⭐⭐⭐⭐⭐

এই architecture complete করো:

```
Django
   ↓
Primary DB
   ↓
Replica

Primary DB
   ↓
?
   ↓
Dashboard
   ↓
Alert
   ↓
Engineer
```

এখানে:

- Metrics
- Logs
- Slow Query
- Replication Lag
- Deadlock

কোথায় যাবে সেটা দেখাও।

🔥 Interview Challenge

Interviewer:

> **"Your Django API suddenly became 8 seconds slower. How would you determine whether the database is responsible?"**

Strong answer:

```
API latency
    ↓
Check Django/application metrics
    ↓
Measure DB query time
    ↓
Check database CPU / memory / disk I/O
    ↓
Check connections
    ↓
Check slow query log
    ↓
Identify expensive query
    ↓
EXPLAIN / EXPLAIN ANALYZE
    ↓
Check index / rows / locks
    ↓
Check deadlocks / lock waits
    ↓
Optimize
    ↓
Measure again
```

আর interviewer যদি জিজ্ঞেস করে:

> **"How will you know whether your production database is healthy?"**

বলবে:

```
CPU
Memory
Disk
Connections
Query latency
P95/P99
QPS
Slow queries
Errors
Locks
Deadlocks
```

```
Replication lag
Backup status
```

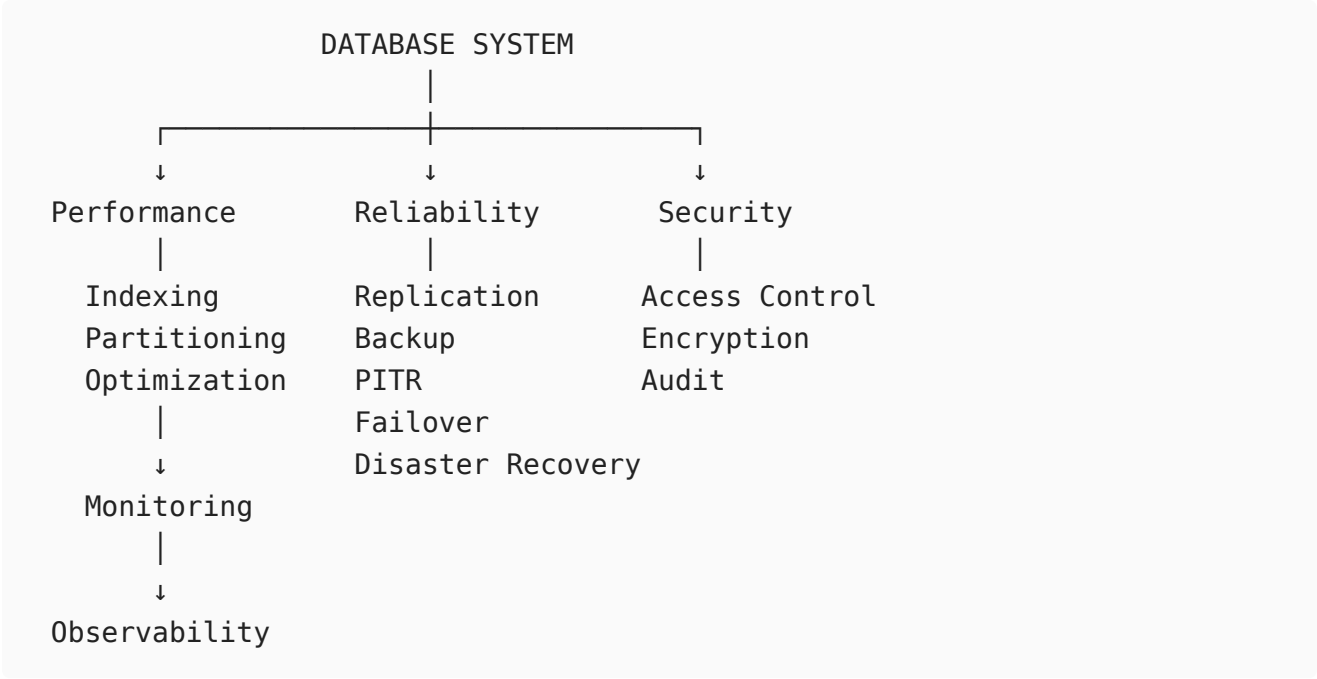
🏆 Database Engineering এখন কোথায়?

তোমার learning progression এখন:

```
SQL
 ↓
Advanced SQL
 ↓
Database Design
 ↓
Normalization
 ↓
ERD
 ↓
Indexing
 ↓
Query Optimization
 ↓
Transactions
 ↓
Concurrency
 ↓
Locking
 ↓
Deadlock
 ↓
Partitioning
 ↓
Replication
 ↓
High Availability
 ↓
Security
 ↓
Backup
 ↓
PITR
 ↓
Disaster Recovery
 ↓
Monitoring
 ↓
Logging
```

```
 ↓
Observability
```

এখন database-কে শুধু **"table + SQL"** হিসেবে না দেখে এভাবে চিন্তা করবে:



⭐ সবচেয়ে গুরুত্বপূর্ণ distinction

```
Indexing
→ Faster Query

Partitioning
→ Large Data Management

Replication
→ Availability / Read Scaling

Backup
→ Recovery

DR
→ Major Failure Recovery

Monitoring
→ Detect Problems

Observability
→ Understand Problems
```

Next → Class 69: Database High Availability, Failover & Disaster Recovery Architecture — এখানে আমরা আগের **Replication + Backup + PITR + Monitoring** concepts একসাথে নিয়ে **Single Point of Failure, Failover, Automatic Failover, Primary/Replica**

Architecture, Read/Write Routing, Health Checks, RPO/RTO-based HA design এবং production-grade database architecture design করব।

📚 Class 69 — Database High Availability, Failover & Disaster Recovery Architecture

আজকের class-টা খুব গুরুত্বপূর্ণ, কারণ আগের কয়েকটি class-এর concept একসাথে ব্যবহার করব:

```
Class 65 → Replication
Class 67 → Backup + PITR + DR
Class 68 → Monitoring + Logging + Observability
```

আজকের লক্ষ্য:

> **একটা Production Database কীভাবে এমনভাবে design করব, যাতে Primary DB নষ্ট হলেও application যত দ্রুত সম্ভব আবার চালু হতে পারে?**

আমরা শিখব:

```
1. Single Point of Failure (SPOF)
2. High Availability (HA)
3. Primary/Replica Architecture
4. Failover
5. Automatic Failover
6. Read/Write Routing
7. Health Checks
8. RPO/RTO-based HA Design
9. Backup + PITR + HA একসাথে
10. Production-grade Database Architecture
```

1️⃣ High Availability — HA কী?

High Availability মানে:

> **System failure হলেও service যতটা সম্ভব available রাখা।**

ধরো আমাদের application:

```
User
 ↓
Django
```

```
  ↓
Database
```

যদি database server নষ্ট হয়:

```
Database ❌
  ↓
Django ❌
  ↓
Application ❌
```

এটা low availability।

HA architecture:

```
                Django
                  |
                  ↓
              DB System
             /          \
            ↓            ↓
        Primary       Replica
```

Primary failure হলে:

```
Primary ❌
   ↓
Failover
   ↓
Replica becomes Primary
```

Application আবার database-এর সঙ্গে কাজ করতে পারে।

2️⃣ Single Point of Failure — SPOF ⭐⭐⭐⭐⭐

SPOF মানে:

> **System-এর এমন একটি component যার failure পুরো service বন্ধ করে দিতে পারে।**

Example:

```
Django
  ↓
ONE Database
```

এখানে:

```
Database = SPOF
```

কারণ:

```
DB failure
   ↓
Application unavailable
```

3️⃣ SPOF Example

ধরো:

```
                Application
                     |
                     ↓
                Primary DB
```

এখানে একটাই database।

Primary:

```
Healthy → Application works
Failed  → Application stops
```

তাই:

> **Single DB architecture = database-level SPOF**

4️⃣ SPOF কীভাবে কমাব?

একটি replica যোগ করি:

```
                Application
                     |
                     ↓
                 Primary
                     |
                     ↓
                  Replica
```

এখন Primary failure হলে:

```
Primary ❌
   ↓
Replica
   ↓
Promote
   ↓
New Primary
```

এটাই failover-এর basic idea।

5️⃣ Primary + Replica Architecture ⭐⭐⭐⭐⭐

Basic:

```
                Application
                     |
                     ↓
                  Primary
                     |
              Replication
                     |
                     ↓
                  Replica
```

Primary সাধারণত:

```
WRITE
```

Replica:

```
READ
```

তবে exact routing architecture-এর উপর নির্ভর করে।

6️⃣ Read/Write Routing ⭐⭐⭐⭐⭐

আমাদের Django application:

```
                Django
              /        \
          WRITE         READ
            ↓             ↓
         Primary       Replica
```

Example:

Create order

```
Order.objects.using("default").create(...)
```

→ Primary

Read orders

```
Order.objects.using("replica").all()
```

→ Replica

7️⃣ কিন্তু একটা সমস্যা আছে ⚠️

Replication asynchronous হলে:

```
Primary
  ↓
Replication Lag
  ↓
Replica
```

Primary-তে data এসেছে কিন্তু replica এখনো পায়নি।

Example:

```
POST /orders
      ↓
Primary
      ↓
Order #100 created ✅
```

তারপর:

```
GET /orders
      ↓
Replica
      ↓
Order #100 missing ❌
```

এটাকে বলে:

Read-after-write consistency problem

8️⃣ Solution: Critical Read → Primary

যদি user刚 write করেছে এবং immediately সেই data দরকার:

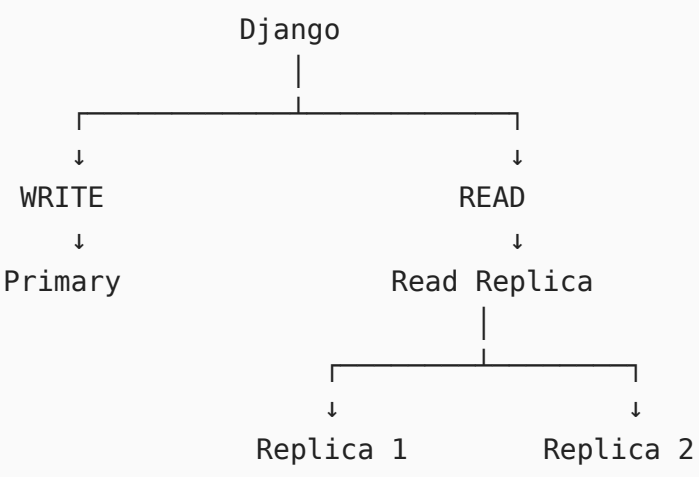
```
WRITE
 ↓
Primary

Immediate READ
 ↓
Primary
```

আর normal read:

```
READ
 ↓
Replica
```

9️⃣ Practical Routing Strategy



এতে:

```
Writes → Primary
Reads → Replicas
```

হলে read workload scale করা যায়।

🔟 Failover কী?

Failover:

> **Primary database failure হলে অন্য healthy database server-কে primary হিসেবে ব্যবহার করা।**

Before:

```
Primary
   ↓
Replica
```

After Primary failure:

```
Primary ❌

Replica
   ↓
PROMOTE
   ↓
New Primary
```

1️⃣1️⃣ Manual Failover

Engineer manually:

```
Detect Primary Failure
        ↓
Verify Replica
        ↓
Promote Replica
        ↓
Update DB Routing
```

```
        ↓
Application reconnect
```

Advantages:

```
More control
```

Disadvantages:

```
Human intervention
Recovery takes longer
Human error possible
```

1️⃣2️⃣ Automatic Failover ⭐⭐⭐⭐⭐

Automatic failover:

```
Primary
   ↓
Health Check
   ↓
❌ Failure detected
   ↓
Failover Controller
   ↓
Promote Replica
   ↓
Update Routing
   ↓
Application
```

Human manually intervene না করেও system failover করতে পারে।

1️⃣3️⃣ Automatic Failover-এর Risk ⚠️

Automatic failover powerful হলেও dangerous হতে পারে।

ধরো network সমস্যা:

```
Primary
   |
   X
```

```
    |
Monitoring system
```

Monitoring system ভাবল:

> Primary dead!

কিন্তু আসলে Primary alive।

তখন যদি Replica-কে primary বানায়:

```
Old Primary → thinks it's Primary
New Primary → thinks it's Primary
```

তাহলে:

Split Brain

হতে পারে।

1️⃣4️⃣ Split Brain ⭐⭐⭐⭐⭐

Split brain:

> **একই সময়ে দুই বা তার বেশি node নিজেকে Primary হিসেবে মনে করা।**

Example:

```
        Primary A
        ↙      ↘
     Writes  Writes
        ↘      ↙
        Primary B
```

এটা data consistency-এর জন্য extremely dangerous।

তাই production failover architecture-এ:

```
Leader election
+
Quorum / consensus mechanisms
+
Fencing
+
Health checks
```

জাতীয় mechanism প্রয়োজন হতে পারে।

1️⃣5️⃣ Health Check ⭐⭐⭐⭐⭐

Failover system কীভাবে জানবে database healthy?

Health checks।

Check করা যেতে পারে:

```
Database reachable?
Connection possible?
Accepting queries?
Replication healthy?
Replication lag?
Disk available?
CPU overloaded?
Read/write working?
```

1️⃣6️⃣ Simple Health Check

Concept:

```
Health Checker
     ↓
SELECT 1
     ↓
Database
```

যদি response:

```
Success → Healthy
Failure → Investigate
```

কিন্তু production health check শুধু `SELECT 1` হওয়া যথেষ্ট নয়।

1️⃣7️⃣ Liveness বনাম Readiness

এটি production architecture-এ গুরুত্বপূর্ণ।

Liveness

> Process/system alive কিনা?

```
Is DB process responding?
```

Readiness

> Traffic নেওয়ার জন্য DB actually ready কিনা?

```
Can this node safely serve application traffic?
```

একটি node alive হতে পারে কিন্তু:

```
Replication broken
Disk full
Read-only
Recovery চলছে
```

তখন সে healthy-looking হলেও traffic নেওয়ার জন্য ready নাও হতে পারে।

1️⃣8️⃣ Failover-এর সময় কী কী check করবে?

```
1. Primary really failed?
2. Replica healthy?
3. Replica sufficiently caught up?
4. Any replication lag?
5. Any transactions in flight?
6. Promote candidate?
7. Prevent old primary from writing?
8. Update routing?
9. Application reconnect?
10. Verify writes?
```

1️⃣9️⃣ RPO + Failover ⭐⭐⭐⭐⭐

ধরো:

```
Primary
   ↓
```

```
Replica
```

Replication asynchronous।

Primary failure-এর সময়:

```
Primary → 100 orders
Replica → 97 orders
```

তাহলে latest 3 orders replica-তে নেই।

Potential data loss:

```
3 orders
```

এখানেই RPO গুরুত্বপূর্ণ।

2️⃣0️⃣ RPO-based Architecture

RPO = 24 hours

Simple system:

```
Daily Backup
```

RPO = 1 hour

```
Frequent Backup
```

RPO = minutes

```
Frequent Backup
+
Transaction Logs / PITR
+
Replication
```

Very low RPO

Stronger replication/recovery architecture প্রয়োজন হতে পারে।

2️⃣1️⃣ RTO + Failover ⭐⭐⭐⭐⭐

ধরো:

```
RTO = 10 minutes
```

Manual process:

```
Detect
 ↓
Engineer wakes up
 ↓
Investigate
 ↓
Promote
 ↓
Routing
 ↓
Test
```

সময় 10 মিনিট ছাড়িয়ে যেতে পারে।

তাই low RTO চাইলে:

```
Automatic Detection
+
Automatic/fast Failover
+
Automated Routing
+
Pre-tested Recovery
```

প্রয়োজন হতে পারে।

2️⃣2️⃣ RPO/RTO Relationship

মনে রাখো:

```
RPO → Data Recovery Requirement
RTO → Time Recovery Requirement
```

Architecture:

```
Lower RPO
   ↓
More frequent/continuous data protection

Lower RTO
   ↓
Faster failover/recovery infrastructure
```

2️⃣3️⃣ Backup + PITR + Replication

এগুলো একে অপরের replacement নয়।

```
Replication
→ Fast availability/failover

Backup
→ Historical recovery

PITR
→ Recover to a specific time

DR
→ Major infrastructure disaster
```

2️⃣4️⃣ কেন Replica যথেষ্ট নয়?

ধরো:

```
10:00 → Good
10:30 → Good
11:00 → Application bug
11:01 → Bad UPDATE
```

Replication:

```
Primary
  ↓
Bad UPDATE
  ↓
Replica
```

Replica-তেও bad update চলে যেতে পারে।

কিন্তু PITR:

```
Backup
 +
Logs
 ↓
10:59:59
```

এখন আগের state recover করা সম্ভব হতে পারে।

2️⃣5️⃣ Production Architecture — Level 1

Small production:

```
                Django
                  |
                  ↓
               Primary
                  |
                  ↓
               Replica
                  |
                  ↓
               Backup
```

এখানে:

```
Primary → Writes
Replica → Reads / Failover
Backup → Recovery
```

2️⃣6️⃣ Production Architecture — Level 2

আরও robust:

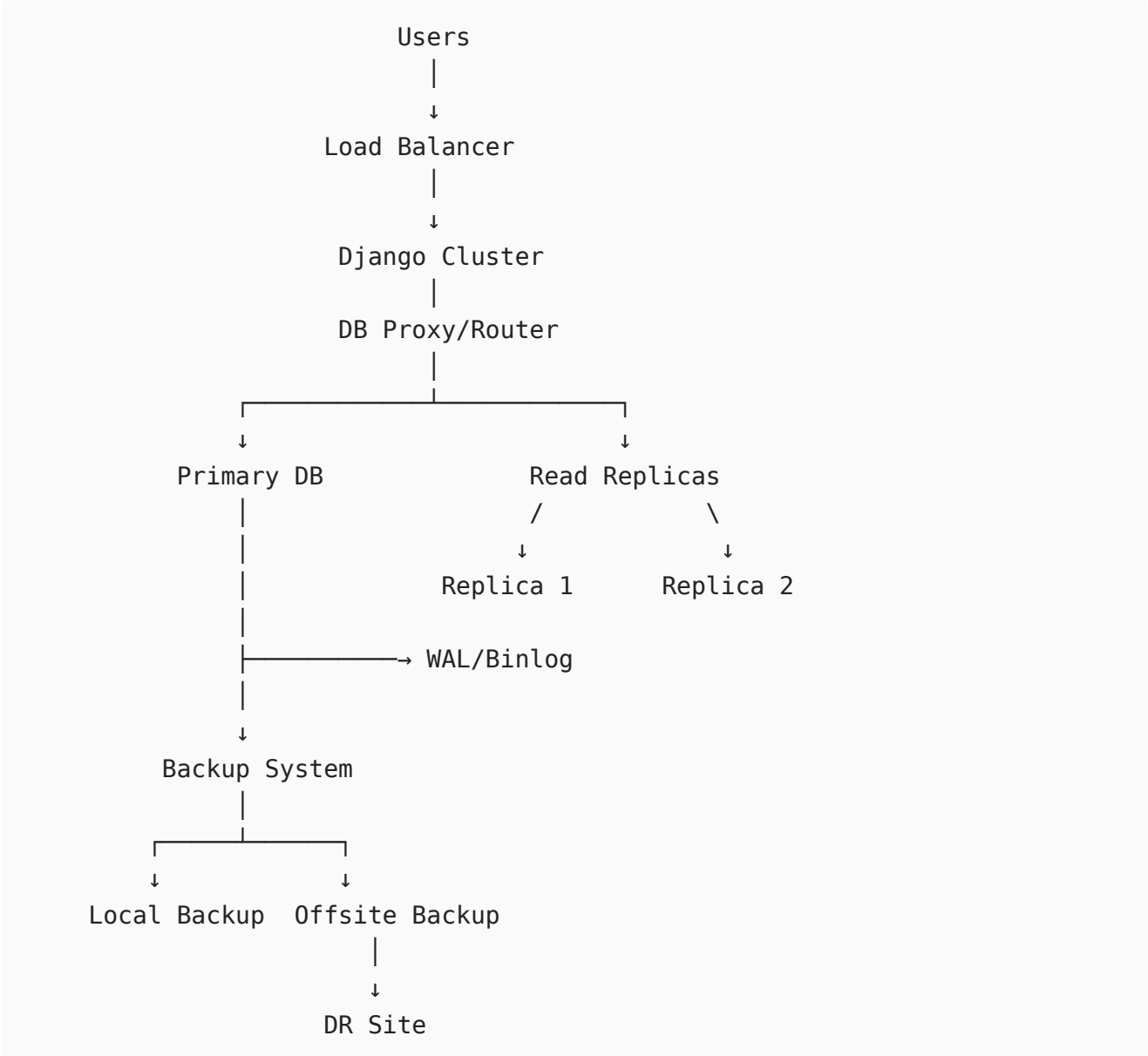
```
                  Django
                    |
               DB Router
               /       \
              ↓         ↓
```

```
          Primary      Replicas
                       /       \
                      ↓         ↓
                  Replica 1  Replica 2

Primary
   ↓
Backup
   ↓
Offsite Storage
```

2️⃣7️⃣ Production Architecture — Level 3 ⭐⭐⭐⭐⭐

এখন সব concepts combine করি:



Monitoring:

```
All components
      ↓
Metrics + Logs
      ↓
Monitoring
      ↓
Alerts
```

2️⃣8️⃣ Database Proxy/Router

Application সরাসরি database node-গুলোর address জানলে failover management complicated হতে পারে।

Instead:

```
Django
   ↓
DB Proxy / Router
   ↓
Primary / Replica
```

Proxy layer করতে পারে:

```
Connection routing
Health checking
Failover routing
Read/write separation
Connection management
```

2️⃣9️⃣ Failover Flow ⭐⭐⭐⭐⭐

Normal:

```
Django
 ↓
Proxy
 ↓
Primary
```

Failure:

```
Primary ❌
 ↓
Health Check
 ↓
Failover Controller
 ↓
Replica Promotion
 ↓
New Primary
 ↓
Proxy updates route
 ↓
Django
```

Application-এর perspective থেকে ideally:

```
Django
 ↓
Same DB endpoint
```

থাকে।

3️⃣0️⃣ Why Stable DB Endpoint?

Bad design:

```
Django
 ↓
primary-db-1
```

Primary change হলে application configuration change করতে হতে পারে।

Better:

```
Django
 ↓
db.example.internal
 ↓
Current Primary
```

Failover হলে:

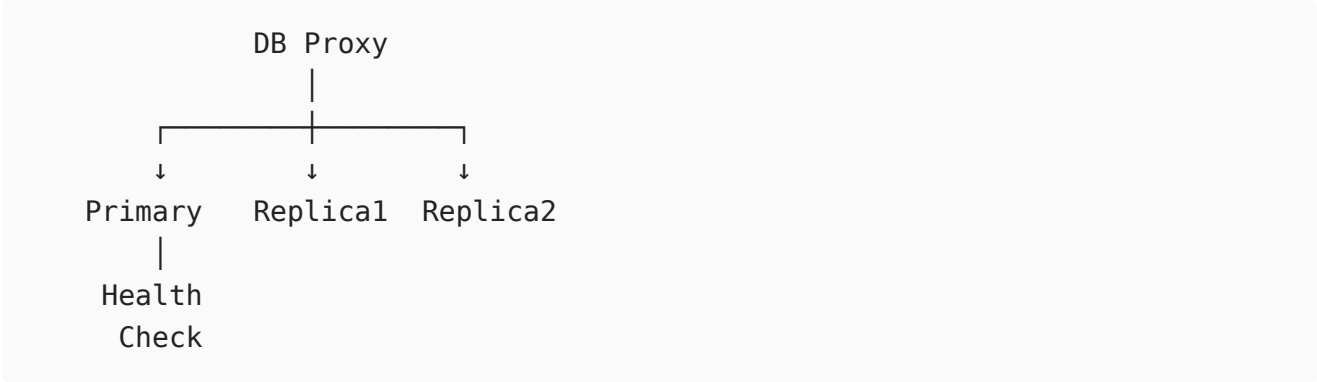
```
db.example.internal
       ↓
```

```
New Primary
```

Routing layer update করে।

3️⃣1️⃣ Health Check + Routing

Architecture:



Primary healthy:

```
WRITE → Primary
```

Primary failed:

```
WRITE → New Primary
```

Read:

```
READ → Healthy Replicas
```

3️⃣2️⃣ Automatic Failover-এর গুরুত্বপূর্ণ components

```
Health Checker
+
Failover Controller
+
Leader Election
+
Fencing
+
DB Proxy
+
```

```
Replication
+
Monitoring
```

সব system-এ exact implementation একই হবে না।

3️⃣3️⃣ Fencing কী?

Advanced concept।

ধরো old primary-এর network issue হয়েছে এবং system তাকে inactive ধরে ফেলেছে।

Failover করার আগে নিশ্চিত করতে হবে:

> **Old primary আর writes করতে পারবে না।**

এটাকে broadly fencing বলা হয়।

Concept:

```
Old Primary
     ↓
FENCE / isolate
     ↓
Cannot write

New Primary
     ↓
Accept writes
```

এটি split-brain risk কমাতে সাহায্য করে।

3️⃣4️⃣ Quorum Concept

Multiple nodes থাকলে decision নেওয়ার জন্য majority concept ব্যবহার করা যায়।

ধরো:

```
Node A
Node B
Node C
```

Majority:

```
2 out of 3
```

এতে এক node-এর ভুল সিদ্ধান্তে পুরো cluster বিভ্রান্ত হওয়ার risk কমে।

Exact quorum/consensus implementation depends on the HA technology.

3️⃣5️⃣ DR Site ⭐⭐⭐⭐⭐

ধরো পুরো primary datacenter:

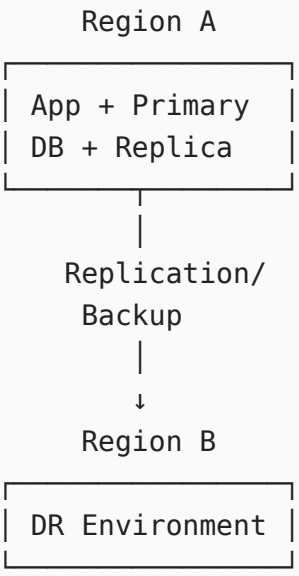
```
Region A ❌
```

তখন:

```
Region B
   ↓
DR Infrastructure
```

ব্যবহার করা যেতে পারে।

Architecture:



3️⃣6️⃣ Regional Disaster

ধরো:

```
Region A
   ❌
```

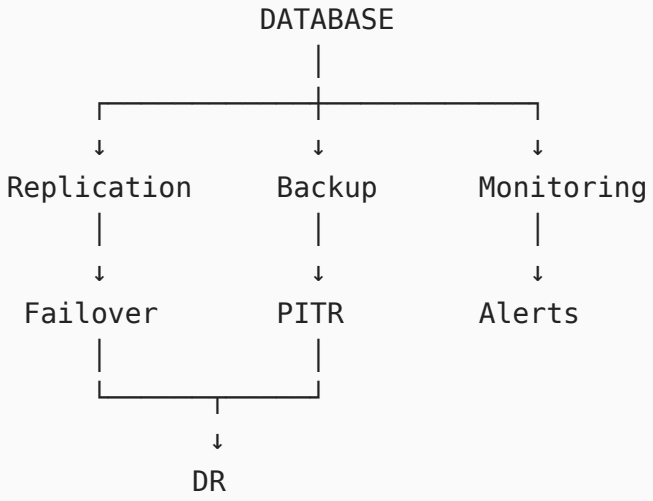
তাহলে recovery:

```
DR Region
   ↓
Promote / Restore
   ↓
Application
   ↓
Users
```

RTO/RPO অনুযায়ী architecture-এর complexity নির্ধারিত হবে।

3️⃣7️⃣ Full Recovery Strategy

Production database-এর জন্য:



3️⃣8️⃣ Hotel System — Production Example 🔥

আমাদের hotel system:

```
customers
rooms
room_types
bookings
payments
```

Traffic:

```
Customers
    ↓
Django
    ↓
DB Proxy
```

Writes:

```
Booking
Payment
Customer update
     ↓
Primary
```

Reads:

```
Room search
Booking history
Reports
     ↓
Read Replicas
```

Critical booking availability:

```
Primary
+
Transaction
+
Lock
```

Backup:

```
Primary
 ↓
Backup
 ↓
Offsite Storage
```

Monitoring:

```
CPU
Memory
Disk
```

```
Latency
Connections
Locks
Deadlocks
Replication Lag
Backup Status
```

3️⃣9️⃣ Hotel Double Booking + HA

একটি গুরুত্বপূর্ণ distinction:

HA থাকা মানেই double booking problem solve হবে না।

Double booking prevent করতে:

```
Transaction
+
Correct overlap logic
+
Locking
+
Proper concurrency control
```

দরকার।

HA solve করে:

```
Database availability
```

Business correctness solve করে:

```
Transaction + Concurrency Design
```

4️⃣0️⃣ HA বনাম Backup বনাম DR

Technology	Main Purpose
Index	Query performance
Partition	Large data management
Replication	Availability/read scaling
Failover	Continue service after node failure

Technology	Main Purpose
Backup	Data recovery
PITR	Recover specific time
DR	Major disaster recovery
Monitoring	Detect problems

4️⃣1️⃣ RPO/RTO-Based Architecture

Scenario A

```
RPO = 24h
RTO = 24h
```

Possible:

```
Daily Full Backup
+
Simple Restore
```

Scenario B

```
RPO = 1h
RTO = 4h
```

Possible:

```
Frequent Backup
+
Offsite Backup
+
Restore Procedure
```

Scenario C

```
RPO = 15 min
```

```
RTO = 1h
```

Possible architecture:

```
Replication
+
Frequent Backup
+
PITR
+
Monitoring
+
Fast Failover
```

Scenario D

```
Very Low RPO
Very Low RTO
```

এক্ষেত্রে:

```
Redundant infrastructure
+
Strong replication
+
Automated failover
+
PITR
+
Offsite/DR
+
Continuous monitoring
+
Regular recovery drills
```

প্রয়োজন হতে পারে।

4️⃣2️⃣ Production Readiness Checklist ⭐⭐⭐⭐⭐

Database production-এ যাওয়ার আগে:

```
□ Primary + Replica
□ Replication monitoring
□ Replication lag monitoring
□ Backup
□ PITR capability
□ Offsite backup
□ Backup encryption
□ Restore test
□ Failover test
□ Health checks
□ Stable DB endpoint
□ Connection management
□ Slow query monitoring
□ Lock monitoring
□ Deadlock monitoring
□ CPU monitoring
□ Memory monitoring
□ Disk monitoring
□ Alerts
□ Runbooks
□ RPO defined
□ RTO defined
□ DR plan
```

4️⃣3️⃣ Common Mistakes ❌

❌ 1. Replica আছে মানেই HA complete

না।

Need:

```
Failover
Health checks
Routing
Monitoring
Testing
```

❌ 2. Replica = Backup

না।

❌ 3. Automatic failover = always safe

না।

Split brain risk আছে।

❌ 4. শুধু `SELECT 1` health check

এটা basic liveness check হতে পারে; production readiness বোঝার জন্য আরও signals দরকার।

❌ 5. RPO/RTO define না করা

Architecture blindly design করা যাবে না।

❌ 6. Failover কখনো test না করা

Disaster-এর দিন প্রথমবার failover test করা উচিত নয়।

❌ 7. Backup restore test না করা

Backup আছে বললেই হবে না।

❌ 8. Read replica-তে critical immediate read পাঠানো

Replication lag থাকলে stale data দেখা যেতে পারে।

🧠 Class 69 Golden Rules ⭐⭐⭐⭐⭐

```
SPOF
→ Single failure point

HA
→ Keep service available despite failures

Primary
→ Usually handles writes
```

Replica
→ Read scaling / failover candidate

Replication
→ Keeps database copies synchronized

Failover
→ Move service to another DB node

Automatic Failover
→ Failure detection + automated promotion/routing

Health Check
→ Determine node health/readiness

Read/Write Routing
→ Write → Primary
→ Read → Replica when appropriate

Read-after-write
→ Critical reads may need Primary

Replication Lag
→ Replica may temporarily be behind

Split Brain
→ Multiple nodes think they are Primary

Fencing
→ Prevent old Primary from writing

Quorum
→ Majority-based decision concept

RPO
→ Acceptable data loss

RTO
→ Acceptable recovery time

Backup
→ Historical recovery

PITR
→ Recover to a specific time

DR
→ Major disaster recovery

```
Monitoring
→ Detect system problems

Failover ≠ Backup

Replication ≠ Backup

HA ≠ Disaster Recovery

Low RTO
→ Faster/more automated recovery needed

Low RPO
→ More continuous data protection needed

Failover must be tested

Restore must be tested

Production HA
→ Replication + Failover + Monitoring + Backup + DR
```

📝 Class 69 Assignment

Task 1 — SPOF

এই architecture:

```
Django
  ↓
Primary DB
```

Identify:

1. SPOF কোনটি?
2. Primary fail করলে কী হবে?
3. কীভাবে architecture improve করবে?

Task 2 — Read/Write Routing

এই operations:

```
1. Create Booking
2. Search Rooms
3. View Booking History
4. Create Payment
5. Generate Report
6. Immediately after booking → Show booking
```

প্রতিটির জন্য বলো:

```
Primary
নাকি
Replica
```

এবং **কেন**।

Task 3 — Failover 🔥

এই architecture:

```
Primary
   ↓
Replica
```

Primary suddenly fails।

পুরো failover process step-by-step লিখো।

Task 4 — RPO/RTO

একটি hotel system-এর requirement:

```
RPO = 15 minutes
RTO = 30 minutes
```

Design করো:

```
Backup
Replication
PITR
Monitoring
```

```
Failover
DR
```

Task 5 — Split Brain ⭐⭐⭐⭐⭐

Explain:

```
Why can automatic failover cause split brain?

How can we reduce this risk?
```

তোমার answer-এ ব্যবহার করবে:

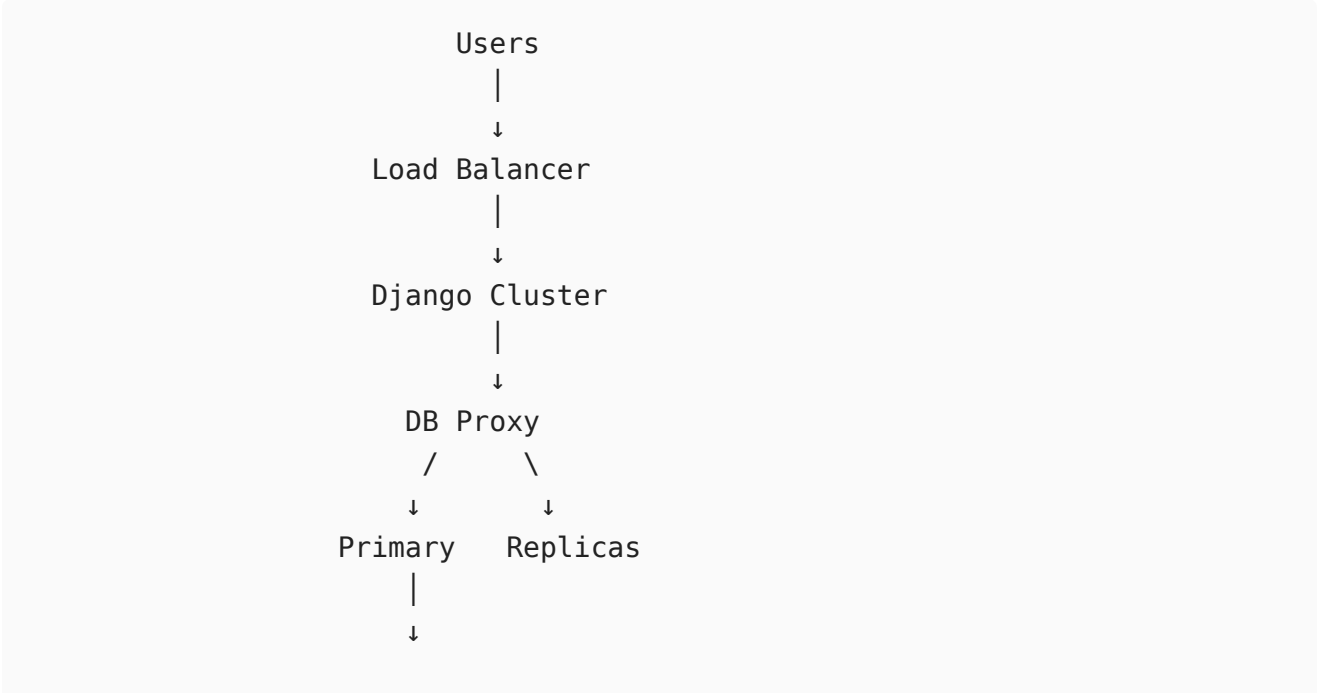
```
Health Check
Leader Election
Quorum
Fencing
```

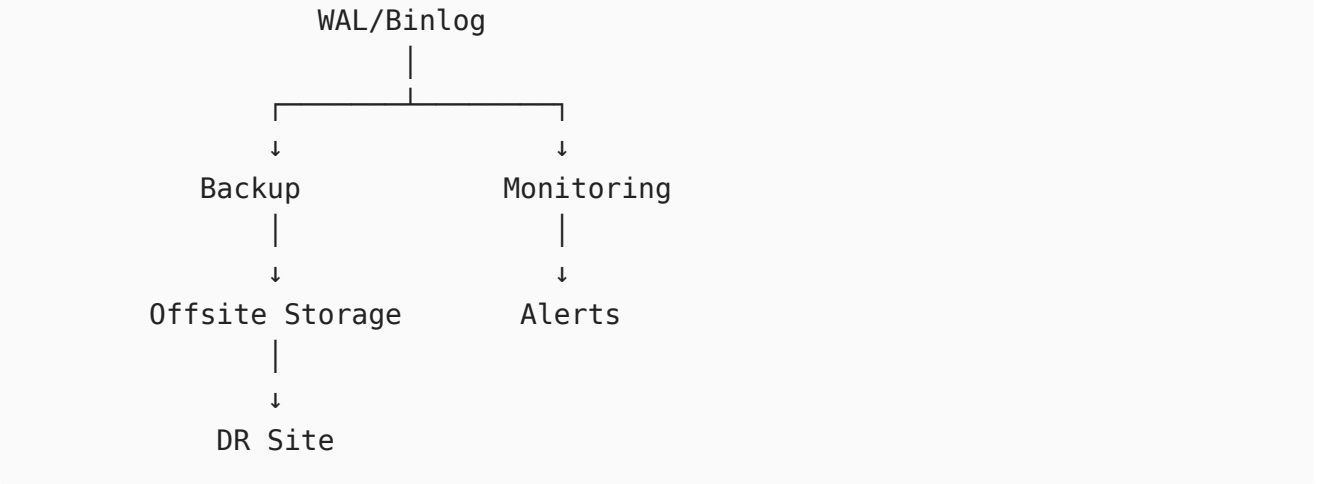
🔥 Interview Challenge

Interviewer:

> **"Design a production database architecture for a Django hotel booking system. The system requires RPO ≤ 15 minutes and RTO ≤ 30 minutes."**

একটি strong high-level answer এমন হতে পারে:





তারপর explain করবে:

```
Primary
→ Writes

Replicas
→ Read scaling + failover candidate

DB Proxy
→ Routing + failover abstraction

Replication
→ Low RPO / availability

PITR
→ Recover from accidental changes

Backup
→ Historical recovery

Monitoring
→ Detect failures

Automatic/fast failover
→ Meet RTO

Offsite DR
→ Regional disaster recovery
```

সবশেষে বলবে:

> **"I would validate the design with regular failover drills, restore tests, replication-lag monitoring, and measured RPO/RTO."**

এটাই একজন **Database Designer / Data Engineer-এর production-level চিন্তা**।